# Logic Rondo

## An Introduction to Logic for Humans and Computers

Henning Basold

Base revision 1205b91, (HEAD, tag: 2.13.9, origin/main, origin/HEAD, main) from 2025-05-15

DRAFT
Do not distribute

Copyright © 2020–2025 Henning Basold, under a Creative Commons Attribution-ShareAlike 4.0 International License: https://creativecommons.org/licenses/by-sa/4.0/.

For updates see https://gitlab.com/hbasold/logic-rondo/-/releases

This book has been written at the Leiden Institute for Advanced Computer Science of the Leiden University.

Document typeset with XƎLaTeX. Figures and diagrams drawn with Ti*k*Z/PGF.

"Robot" on page 58 by William Hollowell licensed under Creative Commons BY, URL: https://thenounproject.com/term/r2d2/10452/
"obstacle" on page 58 by Annette Spithoven licensed under Creative Commons BY, URL: https://thenounproject.com/term/obstacle/211167
"Heart" on page 58 by Bohdan Burmich licensed under Creative Commons BY, URL: https://thenounproject.com/term/heart/396287

# Contents

**Prologue** **1**

**I. Humans and Robots, Express Thyselves!** **5**

**1. Introduction to Propositional Logic** **7**
1.1. Syntax of Propositional Logic . . . . . 10
1.2. Parse Trees and Subformulas . . . . . 15

**2. Propositional Logic Deduction** **21**
2.1. Deductive Systems . . . . . 23
2.2. Natural Deduction . . . . . 24

**3. Fitch-Style Deduction and Classical Truth** **37**
3.1. Fitch-Style Natural Deduction . . . . . 38
3.2. Truth Values . . . . . 43
3.3. Satisfiability and Validity . . . . . 48
3.4. Soundness and Consistency . . . . . 50
3.5. Classical Logic and Completeness . . . . . 51

**4. Introduction to First-Order Predicate Logic** **57**
4.1. The Need for a Richer Language . . . . . 58
4.2. The Language of First-Order Logic . . . . . 63

**5. Proof Theory of First-Order Predicate Logic** **77**
5.1. The Difficulty of Names and Variables . . . . . 79
5.2. Axiomatising Terms and Substitutions . . . . . 81
5.3. Natural Deduction for FOL . . . . . 86
5.3.1. The Intuitionistic System $\mathbf{ND}_1$ . . . . . 87
5.3.2. The Classical System $\mathbf{cND}_1$ . . . . . 91

**6. Machine-Checked Proofs** **97**
6.1. Proof-Checking with Explicit Substitutions . . . . . 99

6.2. Proof-Checking with Implicit Substitutions . . . . . 100

**7. First-Order Horn Clauses and Automatic Deduction 111**
7.1. Automatic Deduction and the Cut-Rule . . . . . 113
7.2. First-Order Horn Clauses and Logic Programming . . . . . 117
7.3. Uniform Proofs . . . . . 122

**Intermezzo 135**

**II. Humans and Robots, Know Your Limits! 137**

**8. First-Order Logic with Equality 139**
8.1. Synactic Formulas with Equality . . . . . 139
8.2. Natural Deduction for FOL with Equality . . . . . 148

**9. Trees and Induction 161**
9.1. Trees and Induction . . . . . 163
9.2. Terms and Parameterised Trees . . . . . 172

**10. Boolean Semantics of First-Order Logic 177**
10.1. Models of First-Order Logic . . . . . 177
10.2. Valuations and the Interpretion of FOL . . . . . 182
10.3. Entailment and Satisfiability for FOL . . . . . 189
10.4. Soundness of Natural Deduction for FOL . . . . . 193

**11. Completeness and Limits of First-Order Logic 201**
11.1. Completeness . . . . . 201
11.2. Compactness and Expressiveness of First-Order Logic . . . . . 202
11.3. Primer on Model Theory . . . . . 205
11.4. Further Reading . . . . . 212

**12. Logic and Computation 215**
12.1. Computability . . . . . 215
12.2. Axiomatisations . . . . . 220
12.3. Primitive Recursion . . . . . 222
12.4. Further Reading . . . . . 230

**13. Gödel's Incompleteness Theorems 233**
13.1. Signature Maps and Theory Inclusions . . . . . 233
13.2. Primitive Recursive Arithmetic . . . . . 236
13.3. Diagonalisation and Incompleteness . . . . . 239

## Appendix 247

### A. Greek Letters 249

### B. Tools 251
B.1. Maps . . . . . 251
B.2. Sets . . . . . 254
B.3. Families of Sets and Maps . . . . . 258
B.4. Induction on Natural Numbers . . . . . 259
B.5. Formal Languages . . . . . 261

### C. Three-Valued Logic 265

### D. De Bruijn Trees 267

### E. Proof Rondo Formalisations 269
E.1. Train Crossing . . . . . 269
E.2. Drinker's Paradox . . . . . 269

### F. Logic Programming 271

## References, Notes & Indices 273

### Notes 273

### References 275

### Symbols 285

### Index 289

### Solutions 293

# Prologue

*Tok-tok-tok*, *Tok-tok-tok*. — The sound of the pen hitting the table drills into Clara's head.

"Too slow!", exclaims Professor Czerny.

Clara starts sweating.

"You will fail to finish the exam on time at that pace. You have to practice more!".

"But professor, I have been practising to no end!", Clara responds in despair. "Every time you give me an exercise, it is more difficult and you expect me to be even faster."

Czerny raises his eyebrows. "How do you think you will get better at logic, if not through exercises?"

Clara removes from her forehead the sweat with the arm of her jumper.

"Not through mind-numbing exercises! How can I understand anything about logic if we never get past repetitive exercises?"

Czerny bursts into laughter: "Do you believe that you know better than me what there is to learn? Do you believe that you can acquire, let alone appreciate, any complex subjects in logic if you haven't even learned how to do the simplest tasks in your sleep? How dare you challenging me! For as long as you are under my thumb, I determine what and how you learn!"

Only now Clara notices the sterile white room. She is sitting at a single desk on a hard wooden chair. Suddenly, the room starts stretching in front and behind her. Endless sequences of logical symbols appear on the side of the room and vanish into the distance with the stretching room.

"You see", Czerny exclaims, "I can make you do these exercises for as long as I wish and you will not go anywhere, if you don't obey my rule."

Clara feels the room expanding in all directions and sees that the floor as no bottom, no end, any longer. Her chest contracts and a scream bursts out.

# The Visitor

Clara opens her eyes, gasps for air and sits up. The light of the dawning days comes in through the light curtains. She finds herself in her own bed and her own small room. Cold sweat drops collect on her forehead and her pillow has a wet silhouette of her head. “Phew, what I dream! I should not prepare so late for my exams any more.”, she says to herself.

Suddenly, Clara realises that she is not alone. She slowly turns the head and suppresses a scream upon putting her eyes on the metallic object next to her bed. It looks like a can with cylindrical arms and legs sticking out, and a metal sphere for a head with something that looks like closed eyes and mouth. At the end of the hanging arms are metallic hands with fingers and the legs even have feet, making the object look like a can-shaped, clumsy human. Now awake, Clara realises that there is a robot next to her bed. She is hesitant to move, afraid to bring the it to life. Who knows if it is dangerous!

A red lamp on the chest of the robot starts flashing in regular pulses and buzzing noises come out of its interior. Clara jumps out of her bed, sprints to the other end of the room and hides behind her desk. The robot comes to life, his arms stretch under tension. Its empty eyes are opening, and Clara feels that she stares into ever-expanding space. Crackling noises are followed by a metallic voice: “Initial startup finished. Human presence detected. I am *Isaac*, please identify yourself.”

“Human presence?”, Clara thinks to herself, horrified by the voice, “Does it know that it I am here?”

Life comes into the eyes of the robot. Its pupils are growing, the iris changes colour from black to violet to green. It looks at Clara. “I am Isaac. Do not fear, I follow the three laws of robotics. Please identify yourself.”

Clara is unsure if the robot just wants to trick her but she takes courage and responds: “My name is Clara.”

The mimic of the robot lightens up, it almost seems to smile. “It is a pleasure to meet you Clara!”

Clara is baffled by the friendliness the robot displays. “How did you get into my room and what do you want here, Isaac?”

Isaac turns the head around to scan his environment. “I do not know. According to my log files, I was just activated. However, my self-scan shows a problem with the memory unit. May I ask you to take a look? I open the hatch at the back of my head.”

The robot turns the head and a small hatch opens, giving view on electronic parts that look nothing like what Clara has ever seen. She gets out from her hiding spot and cautiously approaches the robot. “Nothing seems to be damaged from what I can see, but I am no expert.”, she says.

The hatch closes and Isaac turns his head back. ‘A further self-scan shows that the memory unit is responding to requests, but the responses appear scrambled to me. I can only read the initial segment. It says: “`Deduction system v2.2, 2060`”. Which year is it?’

Clara is puzzled. “It is the year 2024. Are you saying that your memory system is from the year 2060?”

Isaac looks at Clara: “It appears so. My production date, however, is 2050. This means that this is not my first activation. But I don’t have any recollection of how I ended up here.”

Both fall silent. After some minutes of thinking, Clara has a realisation.

‘Did you say “Deduction System”?’, she asks Isaac.

“I did. Do you know anything about that?” Isaac is surprised.

“I am not sure, but I am currently taking a course on logic and deduction systems play a big part in that course.”, Clara explains. “You see, they allow the inference of new knowledge from existing knowledge in a way that is completely formal and can be understood by computers.”

Isaac is intrigued. “Computers like my brain, is that what you are saying?” His eyes and face seem to express curiosity.

“I don’t know. It might be that you are working with the same form of deduction that appear in my course. Maybe if you learn about logic and deductions we can access your memory and find out who you are. After all, I suppose that even the computers of the future still follow the same logical principles.”

# Part I.

# Humans and Robots, Express Thyselves!

# 1. Introduction to Propositional Logic

"OK, it's worth a try. What do you know about logic that could help?"

Clara lifts her shoulders: "The problem is that I don't know much, yet."

"Do you know someone who could help us?"

Clara thinks deeply: "I'm not sure. Your problem may need some great minds to solve."

"Who would that be?"

"I don't think that we are able to meet them." declares Clara disappointedly.

Isaac smiles: "Let me take care of that! Just tell me who comes to your mind."

"Well, this is perhaps a bit far-fetched. We could go to the cradle of philosophy and mathematics in Europe."

Clara just spoke out a name, when the light on Isaac starts flashing faster and faster. Suddenly everything is dark and Clara feels like she is floating in the air. After a few seconds, which felt like centuries, there is light again.

"By Zeus! Who are you, entering my house without any invitation!" A man plants himself in front of Clara and Isaac.

Clara goes a few steps backwards: "E… e… Excuse us, we did not mean to intrude your home!", Clara stutters. "Where are we?"

"I am Aristotle and you are in my home in Athens. Now, who are you and why are you here?"

Clara takes courage and responds: "I am Clara and this is Isaac. He is a robot and we try to recover his memory."

"A robot? What is that supposed to mean?", Aristotle seems to be getting angry again.

Clara hastens to respond: "You may think of an αὐτόματον (automaton) as Hephaestus created them, only that a robot can make decisions for itself and act like a human, all within limits certainly."

"Ha, ha, ha! Good joke!", Aristotle shakes with laughter. "So what do yo really want?"

Isaac hits Clara in the side and murmurs: "Don't tell him about that, you may have an uncontrollable effect on history!"

Clara turns to Aristotle: "In fact, we are here to learn about logic and we are not sure where to start learning. That's why we are here with you, one of the founders of formal logic."

"I see, I see.", Aristotle seems flattered. "If that is so, then please follow me and we will see what we can do for you."

Aristotle directs them to a room with walls full of rulers and compasses, on the floor stacks of wax tables and a table with a chair under a small window.

Clara whispers to Isaac: "Isaac, what happened? How did we get here and how come I can talk to him?"

"My Trans-O-Matic enables translation between any language known to me, including ancient Greek, for everyone in my surrounding."

"But did we really travel back in time to Aristotle?"

"Yes, we did. So be careful to not speak about anything from the future."

"Please come into my studies.", Aristotle makes an inviting gesture.

'Let us start where all logic starts: with simple deduction. Take, for example, the following deduction. "If it rains and Socrates has no umbrella, then he gets wet. Socrates has no umbrella and is not wet. Therefore, it does not rain."'

"Do you always have to use old male Philosophers as examples?", Clara asks a bit upset.

"Will you continue interrupting me for that? These are very illustrative examples, are they not?", Aristotle responds impatiently.

"Well... Never mind! I will not criticise this example further. The point is thus, that we infer knowledge from hypotheses and facts?"

"Yes, exactly."

We shall begin with the study of a simple logic that allows us to express the relation between propositions and infer knowledge from these relations. This logic is called *propositional logic*, and comprises *propositional variables* and *logical connectives* that allow us to unambiguously express propositions and their relations. These propositions will be formalised as logical *formulas*, whose *syntax* we will give in section 1.1.

For the time being, let us analyse Aristotle's example. We **highlight** in the example the propositions that can be either true or false.

> "If **it rains** and **Socrates has no umbrella**, then **he gets wet**. **Socrates has no umbrella** and **he does not get wet**. Therefore, **it does not rain**."

If we write instead

- $r$ for **it rains**,
- $u$ for **Socrates has no umbrella**, and
- $w$ for **he gets wet**,

then we can easily rewrite the first sentence to

> "If $r$ and $u$, then $w$."

and can read it still in exactly the same way by replacing $r$, $u$ and $w$ by the corresponding phrases. We call $r$, $u$ and $w$ *propositional variables*, as they stand for propositions that can be either true or false. This is very important to keep in mind: we reason about such variables independently of their truth, and arguments need to be able to account for any possibility!

We notice that the second sentence cannot be directly written with the three propositional variables because it says "he does *not* get wet", while we only have the variable $w$ that stands for "he gets wet". We will allow ourselves to write "not" in front of a variable to negate what it says, that is, "not $w$" stands for "he does *not* get wet". Natural language is full of ambiguities, caused by different ways of reading and interpreting sentences. One of the goals of formal logic is to prevent such ambiguities. With this in mind, we can write the second and third sentence as

> "$u$ and not $w$. Therefore, not $r$."

What is left of the original sentence are only the words "if", "then", "and", "therefore" and "not". These are *logical connectives*, and we will introduce a formal notation for them in section 1.1.

Let us appreciate for a moment that we have replaced in the original example certain phrases by variables and obtained an argument that relies only on the *structure* of the sentences, rather than their *meaning*. For instance, we could reinterpret the variables like so:

- $r$ — **Isaac's battery is empty**,
- $u$ — **no charger is in reach**, and
- $w$ — **Isaac stops working**,

This gives us another argument that is clearly valid:

> "If **Isaac's battery is empty** and **no charger is in reach**, then **Isaac stops working**. **No charger is in reach** and **Isaac does not stop working**. Therefore, **Isaac's battery is not empty**."

Understanding the steps of such arguments, or *proofs*, in general is the second goal of logic. The third goal is to understand the limits of logic.

## 1.1. Syntax of Propositional Logic

Syntax is the pillar of formal logic that allows us to express propositions *unambiguously*. As we saw earlier, it consists for propositional logic of two parts: propositional variables as atomic assertions and logical connectives to build complex formulas. Propositional variables are syntactic entities that represent propositions, but have *no intrinsic meaning* and only serve as *placeholders*. We shall assume to be given a fixed set of such variables.

**Assumption 1.1.** Assume that PVar is a countable set of **propositional variables**. Elements of PVar are denoted by lower-case letters, possibly with index: $p, q, \dots, p_0, p_1, \dots$

It is important to realise that propositional variables are an abstraction of natural language, just as the real numbers can be seen as an abstraction of 1-dimensional space or time, where propositions and proofs are unambiguous. This means that the transition between natural language and formal logic is not perfect. As a rule of thumb, we can consider something as **proposition** or **propositional variable** **if we can sensibly say that it is true or false in any given situation**. For instance, consider the following phrases.

a) Is the chair black?
b) Socrates has no umbrella.
c) Molecules consist of atoms.
d) $x^2 + 1 = 0$ has no solutions.
e) 5 is green.

Of these, items a) and e) are definitely *not* propositions in the understanding from above. Clearly, a) is a question and thus not a proposition; b) can be true or false depending on the situation, although it is debatable if it makes sense as proposition after Socrates' death; c) is true within our current physical models; d) is a proposition, but its truth depends on the domain of $x$. Finally, e) does not make sense because numbers are a mental abstraction and thus have no colours, although it would make sense to consider "This representation of 5 is green." as proposition.

To combine propositions, we will introduce symbolic logical connectives that allow us to unambiguously express propositions as formulas. For instance, we

will write $\wedge$ for "and" and $\neg$ for "not". With this notation and the variables $u$ and $w$ from above, the phrase

"Socrates has no umbrella and he does not get wet."

becomes

$$u \wedge \neg w\,.$$

Note that there are cases, where it is not clear how a sentence has to be read and we may have to use parentheses to disambiguate the reading. For example, we will write $\rightarrow$ for "if ...then ...". Then the sentence

"If it rains and Socrates has no umbrella, then he gets wet."

can be written as

$$(r \wedge u) \rightarrow w\,.$$

To avoid having too many parentheses, we introduce some reading conventions. We read, for example, the proposition $u \wedge \neg w$ intuitively already as $u \wedge (\neg w)$. Thus, part of the definition of formulas will be a reading convention that allows us to unambiguously determine the structure of formulas.

**Definition 1.2**

The **propositional formulas** (or **formulas**) $\varphi$ of propositional logic are generated by the following context-free grammar, in which $p \in \mathrm{PVar}$.

$$\varphi ::= p \mid \bot \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid \varphi \rightarrow \varphi \mid (\varphi)$$

The following reading conventions can be used to leave out parentheses.

- The connectives $\wedge$ and $\vee$ have precedence over $\rightarrow$.
- All connectives associate to the right.

The set of all propositional formulas is denoted by $\boxed{\mathrm{PForm}}$ and its elements by small Greek letters $\varphi, \psi, \gamma, \ldots$, possibly with a subscript index.

| Connective | Name | Pronunciation | Intuitive Meaning |
|---|---|---|---|
| $\boxed{\bot}$ | **absurdity** | bottom | $\bot$ never holds |
| $\boxed{\wedge}$ | **conjunction** | $\varphi$ and $\psi$ | both hold |
| $\boxed{\vee}$ | **disjunction** | $\varphi$ or $\psi$ | $\varphi$ or $\psi$ or both hold |
| $\boxed{\rightarrow}$ | **implication** | $\varphi$ implies $\psi$ | if $\varphi$ holds, then $\psi$ holds |

Table 1.1.: Logical connectives of propositional logic

As reading and understanding formulas may be difficult in the beginning, let me provide some help. In table 1.1, all the connectives with their name, pro-

nunciation and intuitive meaning are gathered. The name is how we will refer to a connective by itself, outside of formulas. To pronunciation we will use, of course, the pronunciation column. If you have trouble with Greek letters, then have a look at appendix A. The last column in the table also indicates how the connectives can be understood intuitively. Keep in mind though that this is only an intuition and the interpretation can vary radically for different applications and semantics, one of which we will discuss in chapter 3.

So far, we are missing negation. How can we then express Aristotle's example as a formula? Our intuition would dictate that the formula $\neg\varphi$ should hold only if $\varphi$ does not hold. Or, in other words, whenever $\varphi$ holds, something went wrong and we discovered an absurd situation. We can express this by the formula $\varphi \to \bot$, saying that $\varphi$ implies absurdity. Similarly, we can also express other logical connectives in terms of the basic connectives.

**Definition 1.3: Derived connectives**

We define three derived connectives as short-hand notation for the formula in second column of the following table.

| Connective | Definition | Name |
|---|---|---|
| $\boxed{\neg}$ | $\neg\varphi := \varphi \to \bot$ | **negation** |
| $\boxed{\top}$ | $\top := \neg\bot$ | **truth** or Top |
| $\boxed{\leftrightarrow}$ | $\varphi \leftrightarrow \psi := (\varphi \to \psi) \land (\psi \to \varphi)$ | **bi-implication** |

We adopt the following conventions: negation has precedence over $\lor$ and $\land$, thus also over $\to$; and bi-implication has the same precedence as $\to$.

Let us now come back to the original example.

**Example 1.4**

The deduction that it did not rain by experimenting with Socrates' misery consists of the following three formulas.

$$r \land u \to w \quad \text{and} \quad u \land \neg w \quad \text{and} \quad \neg r$$

Replacing the corresponding phrase in the original sentence, we obtain

"$r \land u \to w$. $u \land \neg w$. Therefore, $\neg r$."

We may read this as a deduction, where the sentence starting with "Therefore" signifies the *conclusion* of the deduction, and everything before are the made *assumptions*. This leaves us with two issues: First, how do we know that this is a valid deduction and, second, how can we decide what

| Formula | With parentheses |
|---|---|
| $p \to q \to r$ | $p \to (q \to r)$ |
| $p \wedge q \wedge r$ | $p \wedge (q \wedge r)$ |
| $p \vee q \vee r$ | $p \vee (q \vee r)$ |
| $p \wedge q \to r$ | $(p \wedge q) \to r$ |
| $p \vee q \to r$ | $(p \vee q) \to r$ |
| $p \to q \wedge r$ | $p \to (q \wedge r)$ |

Table 1.2.: Leaving out parentheses by using the precedences of connectives

the assumptions and conclusions are? The second question does not have a general answer due to the ambiguities of natural language. Indeed, the whole point of propositional logic is to provide a formal tool form *modelling* knowledge. One way to phrase the above sentence is as one formula:

$$(r \wedge u \to w) \wedge (u \wedge \neg w) \to \neg r$$

However, we need to realise that this formula only expresses *hypothetical* relations between propositions, whereas the deduction is intended to be read as a valid relation. This brings us to the first question: What is a valid deduction? One answer is that deductions themselves are constructed through some "obviously correct" rules, which will be pursued in chapter 2. Another answer is that the formula above should be true in some sense. Figuring out which formulas are true is the quest of *semantics*, which is investigated in chapter 3 and will be the main concern of part II.

Let us appreciate that we can leave out parentheses by using our reading conventions. Without them, the formula in example 1.4 would look like this:

$$\Big(\Big((r \wedge u) \to w\Big) \wedge \Big(u \wedge (\neg w)\Big)\Big) \to (\neg r)$$

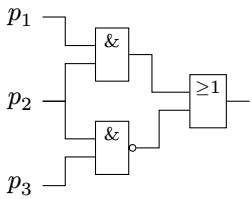
Neither Aristotle nor Clara would be happy to use such unwieldy formulas!

Table 1.2 shows more example, in which the reading conventions allow us to leave out parentheses. Note that there is no convention for mixing $\wedge$ and $\vee$ to avoid confusions. For example, "$p \wedge q \vee r$" is considered to be ambiguous and should be written, depending on the intent, as $(p \wedge q) \vee r$ or $p \wedge (q \vee r)$.

Logic in general, and propositional logic in particular, has of course many applications in mathematics, philosophy, sciences and, perhaps most interesting to you, computer science. Let us look at two such examples.

**Example 1.5: Boolean circuits as propositional formulas**

The following picture shows a logic circuit in the IEC (International Electrotechnical Commission) style with inputs $p_1$, $p_2$ and $p_3$. This circuit comprises one and-gate (top), one nand-gate (bottom) and an or-gate (right).




We easily describe this circuit compactly as one formula, by viewing the inputs $p_k$ as propositional variables:

$$(p_1 \wedge p_2) \vee \neg(p_2 \wedge p_3)$$

Most hardware description languages will use such formulas, possibly with a different syntax, to represent logic circuits and to optimise them.

In example 1.4, we represented knowledge about what may happen to Socrates if it rains. By **knowledge** we mean, both, assertions that are assumed to be unconditionally true (**facts**) and conditional assertions that allow us to deduce something from facts. For instance, we assumed $\neg r$ to be true in example 1.4, while the formula $r \wedge u \rightarrow w$ is an assertion that depends on $r \wedge u$. The same technique can be applied to other forms of knowledge, which may arise from **expert systems**, **automated reasoning** and **artificial intelligence**. This allows the application of all techniques available to propositional logic to these areas, including the possibility to store and independently verify proofs, see chapter 6. However, not all knowledge can be (efficiently) represented in propositional logic. Examples of this are **databases** and **routing problems**, which require first-order logic, see chapter 4. For the time being, let us look at one more elaborate example of how to represent propositional knowledge.

**Example 1.6: Representing propositional knowledge**

A typical use of logic is to analyse possible faults in a system and how they can be prevented. Consider a train crossing with a barrier that is triggered to close whenever a train arrives. The communication with the barrier may fail, however, and therefore the barrier may not close even when the train arrives. With a signal, a train driver may be notified whether the barrier

is closed and, if it is not, brake. Also those brakes may fail. The following propositional formulas model this situation.

$$\begin{aligned}
&\text{TrainArrives} \to \text{ClosingSignal}\\
&\neg\text{CommunicationFailure} \land \text{ClosingSignal} \to \text{BarrierClosed}\\
&\neg\text{TrainStops} \land \text{TrainArrives} \to \text{TrainOnCrossing}\\
&\neg\text{BarrierClosed} \land \text{TrainArrives} \to \text{TrainBrakes}\\
&\neg\text{BrakeFailure} \land \text{TrainBrakes} \to \text{TrainStops}
\end{aligned}$$

This list forms a model by considering implicitly the *conjunction* of these five formulas, that is, if we refer to the five formulas as $\varphi_1, \dots, \varphi_5$, then the model is $\varphi_1 \land \cdots \land \varphi_5$. However, we will see in chapter 2 how assumptions can be handled as lists. Given this model, we can analyse it by, for instance, declaring situations to be unsafe and analysing when they occur:

$$\text{TrainOnCrossing} \land \text{BarrierOpen} \to \text{Unsafe}$$

In chapter 2, will deduce the circumstances under which Unsafe can occur.

The model in example 1.6 to analyse faults is very crude, as it does not take the passing of time into account and because it handle failure probabilities. These require either extensions of propositional logic to **modal logic** or **probabilistic logic**, or the use of first-order logic that we will see in chapter 4.

## 1.2. Parse Trees and Subformulas

For a computer, like that in Isaac's mind, there might still be too much ambiguity. However, general parsing techniques for context-free grammars [Aho+07] can be applied to implement the precedence rules. The resulting of the parsing process are unambiguous two-dimensional trees.

| Formula | Top-Level Connective | Direct Subformulas |
|---|---|---|
| $p$ | $p$ | – |
| $\bot$ | $\bot$ | – |
| $\varphi \land \psi$ | $\land$ | $\varphi, \psi$ |
| $\varphi \lor \psi$ | $\lor$ | $\varphi, \psi$ |
| $\varphi \to \psi$ | $\to$ | $\varphi, \psi$ |

Table 1.3.: Top-Level Connectives and Direct Subformulas

**Definition 1.7**

The **top-level connective** and **direct subformulas** of formulas are given as in table 1.3. A formula is called **atomic** if it has no direct subformulas, that is, if it is of the shape $\bot$ or $p$ for $p \in \mathrm{PVar}$.

Given a formula $\varphi$, the **parse tree** of $\varphi$ is a tree, in which

i) the root is labelled by the top-level connective of $\varphi$, and
ii) its the children are parse trees of the direct subformulas of $\varphi$.

We write $\boxed{\mathrm{Sub}(\varphi)}$ for the set of all formulas that appear in the parse tree of $\varphi$, and call an element of $\mathrm{Sub}(\varphi)$ a **subformula** of $\varphi$.

When we picture such trees, we typically draw a circle for every node and write the label inside this node. This allows to picture, for example, the for-

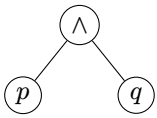



Figure 1.1.: Parse tree of formula $p \wedge q$

mula $p \wedge q$ as the tree in fig. 1.1. Three formulas occur in this tree: $p$, $q$ and $p \wedge q$. Each of these is thus a subformula of $p \wedge q$ and we have

$$\mathrm{Sub}(p \wedge q) = \{p, q, p \wedge q\}\,.$$

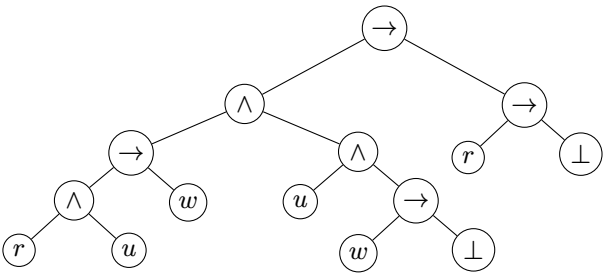



Figure 1.2.: Parse tree for Aristotle's formula

Our formula from earlier can serve as a more elaborate example.

**Example 1.8**

Recall the formula

$$(r \wedge u \to w) \wedge (u \wedge \neg w) \to \neg r$$

from example 1.4. The parse tree of this formula is given in fig. 1.2. Note that the parse tree does not contain negations because this is a derived connective. Instead, it is represented by the defining formula. For instance, $\neg r$ becomes $r \to \bot$ in the parse tree.
It is now easy to read off all subformulas from the tree:

$$\begin{aligned}
&\mathrm{Sub}\big((r \land u \to w) \land (u \land \neg w) \to \neg r\big) = \{ \\
&\quad r, u, r \land u, w, r \land u \to w, \\
&\quad \bot, w \to \bot, u \land (w \to \bot) \\
&\quad r \to \bot, (r \land u \to w) \land (u \land (w \to \bot)) \\
&\quad (r \land u \to w) \land (u \land (w \to \bot)) \to (r \to \bot) \\
&\}
\end{aligned}$$

Parse trees provide an unambiguous representation of formulas. They can form the basis of the implementation of propositional logic on computers and for formal proofs and semantics that you will meet in part II of the journey.

"These are a lot of formalities for expressing very simple things! Is all of this really necessary?", exclaims Clara exasperated.

Aristotle is getting impatient: "You should appreciate the clarity of representing formulas as parse trees! From my perspective, the formalisation of knowledge as formulas is a great leap forward. History is full of misunderstandings caused by ambiguities of natural language. The formal language of propositional logic does away with this, as there is no ambiguity in formulas and their elaboration into parse trees."

Isaac tries to rescue the situation: "Fantastic! Now we can represent knowledge clearly and may even think about automata to calculate with."

"What a fascinating idea...", Aristotle briefly falls into deep thoughts.

"However, I feel we are missing something."

Clara whispers to him: "Feelings? How ...?"

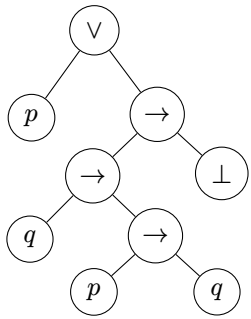
Isaac sends a mysterious look back and continues: "How can I do anything with this knowledge?"

"An excellent question, but I will not be able to answer it for you. I have now other things to attend. Let me just say that you need to learn how to *deduce* knowledge formally. May Athena guide your way in finding answers!"

# Exercises

## Exercise 1

Give the formula corresponding to the following parse tree. Remember that negation is defined in terms of implication to absurdity: $\neg\varphi = \varphi \to \bot$.




## Exercise 2

Decide which of the following strings is a propositional formula.

a) $p$

b) $()p$

c) $p \to q$

d) $p \to \wedge q$

e) $\neg p$

f) $\bot$

g) $p \to ()$

h) $\vee p$

i) $\top \to \bot$

j) $p \wedge q \to p, q$

## Exercise 3

Formalise the following two sentences as a propositional formula. Indicate clearly which part of the sentence each propositional variable refers to.

1. If it rains today, then it won't rain tomorrow.
2. If Dick met Jane yesterday, they had a cup of coffee together, or they took a walk in the park.

## Exercise 4

For each of the following formulas of PL, draw its parse tree and list all its subformulas.

a) $\varphi_0 = \neg(p \to q \wedge r) \vee q \to p$

b) $\varphi_1 = (p \vee q \to r) \wedge \neg q \to p \to r$

c) $\varphi_3 = p \to q \vee (r \wedge \neg s) \to q$

Hint: Before drawing the parse trees, change all $\neg\varphi$ to $\varphi \to \bot$, for any formula $\varphi$ and add parenthesis if needed to the formulas by following the reading conventions.

## Exercise 5

Formalise the following two sentences as a propositional formulas. Indicate clearly which part of the sentence each propositional variable refers to.

1. Today it will rain or shine, but not both.
2. De eerste getuige spreekt de waarheid of de tweede getuige spreekt niet de waarheid.

# 2. Propositional Logic Deduction

Isaac and Clara have left the house of Aristotle, and wander through the city.

After a few minutes of silence, Clara expresses her confusion: "I do understand now what propositional formulas are and how they can be used to formalise knowledge, but I do not at all understand what is going on. How did we get to Athens and to the time of Aristotle? This is impossible!"

"This is made possible by my Voyag-O-Matic unit, which enables travelling through time-space."

"Isn't this dangerous? What if we alter the past and change the future?"

"This is why I cautioned you to be careful. However, when we left the building, another of my units made sure that Aristotle's memory of our visit faded."

"And if we change anything else?"

"I will prevent this, as I have to follow the laws of robotics. Indeed, we should not loose any time here to avoid any incident. Where can I bring us next?"

"I think that we should learn more about formal proofs and I have an idea where to go!"

Clara tells Isaac a name and closes her eyes in anticipation. The sensation of floating sets in. She hears Isaac's voice, first muffled and then coming closer. Clara opens the eyes and she feels her weight coming back.

Isaac is concerned: "What is going on here? Who are all these people sitting in the fields, on the hills and in the garden of this villa?"

"Welcome to my home! These are the women working in the cotton mills of Torino. They are asking for their rights: limiting working days to 10 hours, lunch breaks, coverage of costs for working material by the factory owner etc."

"Professor Peano[1], what a pleasure to meet you! These are indeed very reasonable requests." Clara is already at ease in meeting historical people.

Isaac whispers to Clara: "I don't want to scare you, but some of these rights have been lost in our time."

"Perhaps one could use logic and computers to prove that rights will remain unstable, unless workers control their workspace and students their educational institution?" Clara stares into the distance, thinking.

Peano is getting excited: "What a wonderful idea! Although, students used to be in charge at the Universities of Bologna and Paris in the Middle Ages. In any case, what brings you two to my home?"

"Our visit to Aristotle has taught us how to formalise propositions as formulas. However, this left us dissatisfied.", Clara explains.

"Visit to Aristotle?", Peano frowns.

"Well, ahm…", Clara rushes to correct herself. "I mean our visit to the works of Aristotle."

"Ah, yes. Even though limited in scope, it contains many important developments on mathematical reasoning, such as equality [Pea16]. But be that as it may, in which way were you left dissatisfied?"

Clara explains: "One can write down whatever formula and has no assurance that it is valid. How do we know that some formula is a theorem of arithmetic or a valid deduction of other kind of knowledge?"

"In short, you are looking for proofs, rules of deductions. The Greeks developed already such rules, which you certainly know from Euclidean geometry. But I suppose you are looking for something more… modern, perhaps?"

"Indeed, we do."

"Formal proofs are what has guided my *Formulario* project." Peano makes a wide ranging gesture. "Please, come to my terrace and enjoy the warm evening with me over a glass of wine. I will tell you about a system that was conceived by Gerhard Gentzen [Gen35], with whom I disagree politically, but who paved the way for natural approaches to deduction."

Let us begin with what an informal, but rigorous, proof consists of. One of the main issues, which had been identified already by the Indian logicians [Gan04, Sec. 3.6 and Chap. 4], is that we need to be absolutely clear about the assumptions that we make and what we want to prove. Otherwise, we can easily prove anything we like, justify any dogma we wish, without providing actual evidence. Once this is stated, we can go state the proof method and steps. You may enumerate the ingredients of a proofs thus as follows, but be careful that this list is only an informal statement and may vary [Sol13].

I) Fix the background theory (axioms, definitions, deduction rules etc.)
II) State all the assumptions of the proposition ("Suppose that …")
III) State the proposition to be proved
IV) Provide the proof steps ("by induction", "by contraposition", etc.) that lead to the proposition, using the assumptions and background theory

You have perhaps seen informal proofs before, in which case you should be

able to match the above scheme to those proofs. Only that it will be difficult to pin down the exact background theory used in those proof. How can we, therefore, be sure that those proofs are correct? What we need is an unambiguous syntax for proofs, just like for formulas, in which all steps can be checked. If nothing went wrong during the check, the we could say with certainty that a proof is correct. The challenge that then remains is to design the proof syntax so that it allows humans to understand and communicate proofs.

## 2.1. Deductive Systems

Over time, logicians have developed a vast range of deduction methods. What I would like to focus on here are deductive systems that formalise proofs of syntactic formulas. Here is a list, certainly incomplete, of such methods, in which the methods **marked** are based on tree representations of proofs. Trees make proofs easy to understand, while still being suitable for automation.

- Axiomatic proof theory
- **Natural deduction**
- **Sequent calculus**
- **Tableaux methods**
- **Uniform/focalised proofs**
- Algebraic proof theory
- **Type theory**
- Category theory

In order to use a deductive systems in the formalisation of proofs, as outlined above, we have to

I) fix a deductive system $I$ with background theory,
II) list our assumptions $\Gamma$,
III) represent the proposition as formula $\varphi$, and
IV) construct a deduction of $\varphi$ from $\Gamma$ in $I$, which is written as

$$\Gamma \vdash_I \varphi\,.$$

We call this relation a *(hypothetical) judgement* [PD01].

One may wonder about the "hypothetical" predicate. The idea is that the formula $\varphi$ would be proven under the hypotheses $\Gamma$. We may read $\Gamma \vdash_I \varphi$ as "$\varphi$ is provable under the hypotheses $\Gamma$", but only once a deduction for this judgement has been found.

Before we get into devising deductive systems, we need to keep in mind that not any deductive system may be useful. For example, we may find a deductive system questionable if it allows us to prove that all cats fly or $1 + 1 = 1$ in

the natural numbers. As a first step, we may require that it is not possible to prove anything absurd in $I$, that is,

$$\vdash_I \bot \text{ is not provable in } I\,. \tag{Consistency}$$

This requirement is, unfortunately, often to weak or maybe even undesirable in some situations [Gan04; PTW18]. Thus, we usually give a reference that determines which judgements may be provable. This, however, will have to be deferred to since you currently have to further reference of when a deduction of formula should be valid.

## 2.2. Natural Deduction

Now we know what deductive systems are and what we desire of them, but how does such a system look like *concretely*? We will start with a deductive system that is very natural [Pra06] in the sense that deductions in that system correspond to intuitive reasoning. This system is based on Gentzen's system of *natural deduction* and judgements of the form $\Gamma \vdash \varphi$, where $\Gamma$ is a list of (propositional) formulas and $\varphi$ is a single formula. In our case, we call $\Gamma \vdash \varphi$ a **sequent** and $\Gamma$ a **context**.

**Conjunction** Let us begin with the case of conjunction and suppose that we want to prove $\varphi \wedge \psi$. Intuitively, we would expect this proposition to be true if $\varphi$ and $\psi$ are separately true. In the terminology of deductive systems, we should thus be able to derive from the sequents

$$\Gamma \vdash \varphi \quad \text{and} \quad \Gamma \vdash \psi$$

the following sequent.

$$\Gamma \vdash \varphi \wedge \psi$$

As it is such an natural step, we will make it a **deduction rule** or just **rule**:

$$\frac{\Gamma \vdash \varphi \qquad \Gamma \vdash \psi}{\Gamma \vdash \varphi \wedge \psi}\ (\wedge\mathrm{I})$$

This rules consists of hypotheses, the conclusion and a label to name the rule. The hypotheses above the line are what we have to prove before we can apply the rule. Once we have proved the hypotheses, we can deduce the conclusion using the rule. We use labels to identify in proofs the rules that we use, which helps both readability and allows us to verify proofs. In the case of the rule

above, we used the label ($\wedge$I) and read it as "conjunction introduction". The conclusion of this rule is thus a conjunction that has been introduced. You will have noticed that we used formulas $\varphi$ and $\psi$ but did not say what these formulas exactly are. In fact, the rule works for any formula and we can see deduction rules as *schemes* that are agnostic to the structure of $\varphi$ and $\psi$. With the rule ($\wedge$I), we are now able to prove conjunctions.

More generally, deduction rules are of the form

$$\frac{J_1 \quad J_2 \quad \cdots \quad J_n}{J}\ (\mathrm{L})$$

where all the hypotheses $J_1, \ldots, J_n$ and the conclusion $J$ are judgements, and L is a rule label. The label indicates whether we are **introducing** a connective, like ($\wedge$I) above, or are **eliminating** a connective, which means that the connective appears among the hypotheses.

Why would we need to eliminate a connective? Suppose that we know, for example from the assumptions of our proposition, that $\varphi \wedge \psi$ holds. In this case, we also know that $\varphi$ and $\psi$ hold separately. This leads us to the following two rules.

$$\frac{\Gamma \vdash \varphi \wedge \psi}{\Gamma \vdash \varphi}\ (\wedge\mathrm{E}_1) \qquad \frac{\Gamma \vdash \varphi \wedge \psi}{\Gamma \vdash \psi}\ (\wedge\mathrm{E}_2)$$

In these rules, the "E" stands for elimination and the number indicate which of the conjuncts we would like to access.

**Implication** Suppose you know that some formula, say $\varphi$, implies another formula $\psi$. Recall that implication is written as $\varphi \to \psi$. We may read this formula as "if we know $\varphi$, then we also know $\psi$". This gives us actually a good intuition for how to use, or *eliminate*, an implication. The intuition is that if we can prove the condition (antecedent) of an implication, then we know that the conclusion of the implication holds. To use this in natural deduction proofs, we turn this idea into a rule:

$$\frac{\Gamma \vdash \varphi \to \psi \qquad \Gamma \vdash \varphi}{\Gamma \vdash \psi}\ (\to\mathrm{E})$$

But how can we prove an implication? Also here we can use our intuition: if we can prove $\psi$ under the assumption that $\varphi$ holds, then this means that "if we know $\varphi$, then we also know $\psi$". Since this is our intuitive reading of the implication $\varphi \to \psi$, the deducing, or *introducing*, an implication adds the

$$\frac{\varphi \textbf{ in } \Gamma}{\Gamma \vdash \varphi}\ (\text{Assum}) \qquad \frac{\Gamma \vdash \bot}{\Gamma \vdash \varphi}\ (\bot\text{E})$$

$$\frac{\Gamma \vdash \varphi \wedge \psi}{\Gamma \vdash \varphi}\ (\wedge\text{E}_1) \qquad \frac{\Gamma \vdash \varphi \wedge \psi}{\Gamma \vdash \psi}\ (\wedge\text{E}_2) \qquad \frac{\Gamma \vdash \varphi \qquad \Gamma \vdash \psi}{\Gamma \vdash \varphi \wedge \psi}\ (\wedge\text{I})$$

$$\frac{\Gamma \vdash \varphi}{\Gamma \vdash \varphi \vee \psi}\ (\vee\text{I}_1) \qquad \frac{\Gamma \vdash \psi}{\Gamma \vdash \varphi \vee \psi}\ (\vee\text{I}_2)$$

$$\frac{\Gamma \vdash \varphi \vee \psi \qquad \Gamma, \varphi \vdash \delta \qquad \Gamma, \psi \vdash \delta}{\Gamma \vdash \delta}\ (\vee\text{E})$$

$$\frac{\Gamma, \varphi \vdash \psi}{\Gamma \vdash \varphi \to \psi}\ (\to\text{I}) \qquad \frac{\Gamma \vdash \varphi \to \psi \qquad \Gamma \vdash \varphi}{\Gamma \vdash \psi}\ (\to\text{E})$$

Figure 2.1.: Deduction Rules of the natural deduction system **ND**

antecedent $\varphi$ to the list of assumptions $\Gamma$ and asks then for a proof of $\psi$ from $\Gamma, \varphi$. As a rule, that looks like this:

$$\frac{\Gamma, \varphi \vdash \psi}{\Gamma \vdash \varphi \to \psi}\ (\to\text{I})$$

We know how to add assumptions, but not yet how to use them. Let us write $\boxed{\varphi \textbf{ in } \Gamma}$ if the formula $\varphi$ appears in the list $\Gamma$. The rule for using an assumption allows us to pick any formula from $\Gamma$ and use it as proven proposition.

$$\frac{\varphi \textbf{ in } \Gamma}{\Gamma \vdash \varphi}\ (\text{Assum})$$

In order to understand how to use these individual rules to deduce anything complex, let us think about what a deduction is: a combination of reasoning steps. This means that a deduction will combine several rules together and, since some rules have more than one hypothesis, and we will get something that looks like a tree.

**Definition 2.1: Natural deduction for propositional logic**

A (propositional) **context** $\Gamma$ is a list of formulas. Let $\Gamma$ be a context and $\varphi$ a formula. We write $\boxed{\Gamma, \varphi}$ for the context given by **extending** $\Gamma$ by $\varphi$, which means that $\varphi$ is appended to $\Gamma$. We call the pair $\boxed{\Gamma \vdash \varphi}$ a **(propositional) sequent**. We abbreviate the sequent $\Gamma \vdash \varphi$ to $\vdash \varphi$ if $\Gamma$ is empty. The rules of the **system ND** of natural deduction for propositional logic are in fig. 2.1. A **deduction** or **proof tree** for $\Gamma \vdash \varphi$ is a finite tree, such that

i) each node is labelled with a sequent and a rule label, such that the children of the node are labelled with the hypotheses of that rule;
ii) the root of the tree is labelled with $\Gamma \vdash \varphi$; and
iii) the leaves of the tree are labelled with (Assum).

We say that $\varphi$ is **provable** in context $\Gamma$ in $\boxed{\textbf{ND}}$, if there is a deduction for $\Gamma \vdash \varphi$. A formula $\varphi$ is a **theorem** if $\vdash \varphi$ holds.

Figure 2.1 contains the rules for the two connectives that we did not discuss, yet. The rule ($\bot$E) allows us to deduce *anything* from absurdity. This rule is sometimes also referred to as the **principal of explosion** because, once absurdity has been proven, anything is possible. Finally, there are the rules for disjunction. It should be rather obvious why the introduction rules ($\vee I_1$) and ($\vee I_2$) make sense, as $\varphi \vee \psi$ holds whenever $\varphi$ or $\psi$ holds. The elimination rule ($\vee$E) is a bit more complex. Underlying it is the idea that we want to prove $\delta$, knowing that $\varphi \vee \psi$ holds. As we cannot know which of the two it will be, we have to make a case distinction and prove that $\delta$ holds in either case. If we succeed, then $\delta$ can be deduced from $\varphi \vee \psi$.

Another way of thinking about disjunction elimination is if-then-else branching in programs. We can think of it this way: the rule builds a proof of $\delta$ that takes a proof of $\varphi \vee \psi$ as input. To proceed, we make a case distinction on this input and check whether it proves $\varphi$ or $\psi$. We can then use this information to prove $\delta$ for both cases separately. One may even go as far as suggesting to write proofs as programs. Indeed, you may be delighted to hear that *type theory* [NG14] concerns itself with interpreting proofs as programs, which originates in the *Brouwer-Heyting-Kolmogorov interpretation* [TvD88a]. This is very fascinating, but we continue for the moment with natural deduction.

Definition 2.1 may look pedantic at first sight, but it just formalises our intuition of proofs. To illustrate this, let us go through some simple examples.

**Example 2.2**

In this example, we show how the (Assum)-rule and (→I)-rule can be used together to deduce $\vdash p \to q \to p$. The formula $p \to q \to p$ also allows us to "store" knowledge and is one of the building blocks of combinatory logic in form of the K-combinator. Here is a deduction for this theorem:

$$\dfrac{\dfrac{\dfrac{p \textbf{ in } p, q}{p, q \vdash p}\,(\text{Assum})}{p \vdash q \to p}\,(\to\text{I})}{\vdash p \to q \to p}\,(\to\text{I})$$

How does this proof tree relate to the definition? Let us redraw it in a way that can be more easily recognised as a tree.

$$\begin{array}{c} \boxed{p, q \vdash p}\ (\text{Assum}) \\ | \\ \boxed{p \vdash q \to p}\ (\to\text{I}) \\ | \\ \boxed{\vdash p \to q \to p}\ (\to\text{I}) \end{array}$$

Even though this looks more like a list, we can still see everything that make a proof tree according to definition 2.1. First of all, the root at the bottom of the tree is labelled by the sequent $\vdash p \to q \to p$ that we aim to prove and the rule (→I) that is used to prove it. This rule demands that we move the antecedent $p$ of the implication to the context and then prove the sequent $p \vdash q \to p$, which in turn is how the child of the root is labelled. Similarly, we can check the labelling of this child. Finally, the leaf node is labelled by (Assum). This shows that the tree indeed fulfils all the requirements demanded by definition 2.1 to make it a valid proof tree.

Clearly, the second representation of the tree in example 2.2 is somewhat sparse in information and less intuitive than assembling rules into a proof. This is why we will work for now with the first presentation.

?

Is the requirement iii) of definition 2.1 necessary?

As writing $\varphi$ **in** $\Gamma$ in (Assum) is tedious, we allow ourselves to leave this out whenever $\varphi$ **in** $\Gamma$ obviously holds. Moreover, as the label (Assum) can be

inferred, we will typically shorten the rule to

$$\overline{\Gamma \vdash \varphi}$$

if the application of (Assum) is obvious. In non-obvious cases, we will still indicate the occurrence of $\varphi$ in $\Gamma$.

**Example 2.3**

We deduce $p \wedge q \to r \vdash p \to q \to r$. This process is also known as *Currying*. As the sequents can get quite lengthy, let us name the assumptions throughout the proof by defining $\Gamma = p \wedge q \to r$.

$$\dfrac{\dfrac{\dfrac{\overline{\Gamma, p, q \vdash p \wedge q \to r} \qquad \dfrac{\overline{\Gamma, p, q \vdash p} \quad \overline{\Gamma, p, q \vdash q}}{\Gamma, p, q \vdash p \wedge q}\,(\wedge\mathrm{I})}{\Gamma, p, q \vdash r}\,(\to\mathrm{E})}{\Gamma, p \vdash q \to r}\,(\to\mathrm{I})}{\Gamma \vdash p \to q \to r}\,(\to\mathrm{I})$$

We have simplified the application of (Assum) in the two right leaves, but added some information on the left leaf to make the proof easier to read.

"Aristotle gave us... I mean we found an example in Aristotle's work, of a deduction. Can we justify that in natural deduction?", Clara wonders. "That example used negation and, if I remember correctly, negation was defined in terms of implication and absurdity. Can we thus use the deduction rules for those connectives to reason about negation and prove the said example?"

Absolutely! But I would advise to make life simpler and first establish some specific rules for negation inside the system **ND**. These rules would not be part of the system itself but can be derived from it and can be used *like* proof rules: they are admissible.

**Definition 2.4**

A deduction rule

$$\dfrac{J_1 \quad J_2 \quad \cdots \quad J_n}{J}\,(\mathrm{L})$$

is **admissible**, if it is possible to construct from any deduction of the sequents $J_1, \ldots, J_n$ a deduction for the sequent $J$.

We can make life easier by providing admissible rules for negation.

**Theorem 2.5**

The following two rules are admissible in **ND**.

$$\frac{\Gamma, \varphi \vdash \bot}{\Gamma \vdash \neg\varphi}\,(\neg\mathrm{I}) \qquad \frac{\Gamma \vdash \varphi \qquad \Gamma \vdash \neg\varphi}{\Gamma \vdash \psi}\,(\neg\mathrm{E})$$

*Proof.* Let $\varphi$ and $\psi$ be formulas, and suppose that $\Gamma, \varphi \vdash \bot$ holds. We obtain the following proof and thus a proof tree for $(\neg\mathrm{I})$ because $\neg\varphi = \varphi \to \bot$.

$$\frac{\Gamma, \varphi \vdash \bot}{\Gamma \vdash \varphi \to \bot}\,(\to\mathrm{I})$$

Next, suppose that there are proof trees for $\Gamma \vdash \varphi$ and $\Gamma \vdash \neg\varphi$. We can build the following proof tree by using the definition of $\neg\varphi$ in terms of $\varphi \to \bot$.

$$\frac{\dfrac{\Gamma \vdash \varphi \qquad \Gamma \vdash \neg\varphi}{\Gamma \vdash \bot}\,(\to\mathrm{E})}{\Gamma \vdash \psi}\,(\bot\mathrm{E})$$

This shows that both, $(\neg\mathrm{I})$ and $(\neg\mathrm{E})$, are admissible in **ND**. □

Using these two rules, we can easily prove sequents that involve negation. In particular, we can prove the statement by Aristotle!

$$\frac{\dfrac{\dfrac{\dfrac{\dfrac{}{\Gamma, r \vdash r \wedge u \to w} \qquad \dfrac{\dfrac{}{\Gamma, r \vdash r} \qquad \dfrac{\dfrac{}{\Gamma, r \vdash u \wedge \neg w}}{\Gamma, r \vdash u}\,(\wedge\mathrm{E}_1)}{\Gamma, r \vdash r \wedge u}\,(\wedge\mathrm{I})}{\Gamma, r \vdash w}\,(\to\mathrm{E}) \qquad \dfrac{\dfrac{}{\Gamma, r \vdash u \wedge \neg w}}{\Gamma, r \vdash \neg w}\,(\wedge\mathrm{E}_2)}{\Gamma, r \vdash \bot}\,(\neg\mathrm{E})}{\Gamma \vdash \neg r}\,(\neg\mathrm{I})$$

Figure 2.2.: Proof in **ND** for excluding rain on Socrates

**Example 2.6: It does not rain for Socrates**

Let $\Gamma = r \wedge u \to w, u \wedge \neg w$. The in fig. 2.2 is a proof for $\Gamma \vdash \neg r$.

Let us also go over some proofs that involve the elimination of disjunction. Here is an interesting example that shows that, if we can exclude on alternative of a disjunction, then the other necessarily has to hold.

**Example 2.7**

We put $\Gamma = p \vee q, \neg q$ and prove $\Gamma \vdash p$ by means of the following deduction.

$$\dfrac{\dfrac{}{\Gamma \vdash p \vee q} \qquad \dfrac{}{\Gamma, p \vdash p} \qquad \dfrac{\dfrac{}{\Gamma, q \vdash q} \qquad \dfrac{}{\Gamma, q \vdash \neg q}}{\Gamma, q \vdash p}\,(\neg\mathrm{E})}{\Gamma \vdash p}\,(\vee\mathrm{E})$$

Note that ($\vee$E) acts like a case distinction and we show in the second, were we suppose that $q$ holds, that this case is absurd. This allows us to show that $\Gamma \vdash p$ must hold.

Another common proof method, found in mathematics but also in deducing from negative knowledge such as parts of the train model in example 1.6, involves turning an implication around using negation. This proof method is called **contraposition** and we can prove that it can be also used in **ND**.

**Example 2.8: Contraposition**

Let $\varphi$ and $\psi$ be formulas. For convenience, set $\Gamma = \varphi \to \psi, \neg\psi, \varphi$. The following is a proof that shows that if $\varphi \to \psi$ holds, then also $\neg\psi \to \neg\varphi$.

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\Gamma \vdash \neg\psi} \qquad \dfrac{\dfrac{}{\Gamma \vdash \varphi \to \psi} \qquad \dfrac{}{\Gamma \vdash \varphi}}{\Gamma \vdash \psi}\,(\to\mathrm{E})}{\varphi \to \psi, \neg\psi, \varphi \vdash \bot}\,(\neg\mathrm{E})}{\varphi \to \psi, \neg\psi \vdash \neg\varphi}\,(\neg\mathrm{I})}{\varphi \to \psi \vdash \neg\psi \to \neg\varphi}\,(\to\mathrm{I})$$

It is important to note that the other direction is not provable in **ND** because $\neg\neg p \vdash p$ is not provable in **ND**. This will be discussed further in chapter 3.

The next example shows that negation distributes over disjunction, thereby turning the disjunction into a conjunction. This is known as one of *de Morgan's laws*. Formally, we prove the bi-implication $\neg(\varphi \vee \psi) \leftrightarrow (\neg\varphi \wedge \neg\psi)$.

**Example 2.9**

Let $\varphi$ and $\psi$ be arbitrary formulas. We first prove $\neg(\varphi \vee \psi) \vdash \neg\varphi$. To avoid writing too much, let us define $\Gamma = \neg(\varphi \vee \psi), \varphi$. Then we have the

following derivation.

$$\dfrac{\dfrac{\dfrac{}{\Gamma \vdash \neg(\varphi \vee \psi)} \quad \dfrac{\dfrac{}{\Gamma \vdash \varphi}}{\Gamma \vdash \varphi \vee \psi}\,(\vee\mathrm{I}_1)}{\Gamma \vdash \bot}\,(\neg\mathrm{E})}{\neg(\varphi \vee \psi) \vdash \neg\varphi}\,(\neg\mathrm{I}) \qquad (D_1)$$

Similarly, we can find a derivation $D_2$ for $\neg(\varphi \vee \psi) \vdash \neg\psi$ and thus, by putting these derivations together, we get the following derivation.

$$\dfrac{\dfrac{D_1}{\neg(\varphi \vee \psi) \vdash \neg\varphi} \quad \dfrac{D_2}{\neg(\varphi \vee \psi) \vdash \neg\psi}}{\neg(\varphi \vee \psi) \vdash \neg\varphi \wedge \neg\psi}\,(\wedge\mathrm{I}) \qquad (D_3)$$

For the other direction, we use $\Gamma = \neg\varphi \wedge \neg\psi, \varphi \vee \psi$ and define $\Gamma_1 = \Gamma, \varphi$ and $\Gamma_2 = \Gamma, \psi$. With these, we have the following derivation $D_4$.

$$\dfrac{\dfrac{\dfrac{}{\Gamma \vdash \varphi \vee \psi} \quad \dfrac{\dfrac{\dfrac{}{\Gamma_1 \vdash \neg\varphi \wedge \neg\psi}}{\Gamma_1 \vdash \neg\varphi} \quad \dfrac{}{\Gamma_1 \vdash \varphi}}{\Gamma_1 \vdash \bot}\,(\neg\mathrm{E}) \quad \dfrac{\dfrac{\dfrac{}{\Gamma_2 \vdash \neg\varphi \wedge \neg\psi}}{\Gamma_2 \vdash \neg\psi} \quad \dfrac{}{\Gamma_2 \vdash \psi}}{\Gamma_2 \vdash \bot}\,(\neg\mathrm{E})}{\Gamma \vdash \bot}\,(\vee\mathrm{E})}{\neg\varphi \wedge \neg\psi \vdash \neg(\varphi \vee \psi)}\,(\neg\mathrm{I})$$

Finally, we obtain a proof for de Morgan's law as follows.

$$\dfrac{\dfrac{\dfrac{D_3}{\neg(\varphi \vee \psi) \vdash \neg\varphi \wedge \neg\psi}}{\vdash \neg(\varphi \vee \psi) \to \neg\varphi \wedge \neg\psi}\,(\to\mathrm{I}) \quad \dfrac{\dfrac{D_4}{\neg\varphi \wedge \neg\psi \vdash \neg(\varphi \vee \psi)}}{\vdash \neg\varphi \wedge \neg\psi \to \neg(\varphi \vee \psi)}\,(\to\mathrm{I})}{\vdash (\neg\varphi \wedge \neg\psi) \leftrightarrow \neg(\varphi \vee \psi)}\,(\wedge\mathrm{I})$$

?

Can you give admissible introduction and elimination rules for the truth proposition $\top$?

"What about the equivalence $\neg(\varphi \wedge \psi) \leftrightarrow (\neg\varphi \vee \neg\psi)$?", Clara wonders.

"That equivalence is surprisingly more delicate and it turns out that to only on direction is provable within the proof system **ND**, see exercise 7, while the other direction is not!", Peano explains.

Clara is really puzzled now: "How can we know that and why should it not be provable? And how can we then be sure that our proofs are correct?"

"These are indeed a valid questions that we have glossed over so far. The first question, we cannot answer easily at this stage because we would need to understand first how to prove properties of deductions, which we shall leave for another time. The second question touches on, what you may call, *logical pluralism*: the idea that truth and provability is relative."

"Does it then even make sense ask whether a proof is correct?", Clara gets even more confused.

"Of course it does make sense but you need to understand that this is relative to a chosen deduction system, like the system **ND** that we have discussed. So to answer your third question: Once we have fixed a deduction system, we only need to check in a proof that all steps are correct. This means checking that a given deduction is the form of a proof tree according to definition 2.1, which in turn is an entirely mechanical task. In fact, I have heard rumours that machines exist that can be fed a representation of a potential proof tree and then compute whether it is an actual proof tree. This would be of great help to my *Formulario* project, since anyone who trusts the verifying machine can run it on the proof collection!"

Let us demonstrate verification and the above issue on an example.

**Example 2.10: Finding system failures**

Recall that we introduced in example 1.6 a fault model for a train crossing. In this example, we will show how **ND** can be used to prove that an unsafe situation can arise. There is one caveat with the previous model, namely that assumptions such as

$$\neg\text{CommunicationFailure} \land \text{ClosingSignal} \to \text{BarrierClosed}$$

cannot be used because it does not tell us what happens if there is a communication failure. That is, from knowing CommunicationFailure we cannot derive ¬BarrierClosed. The reason is simple: it could be that there other possibilities as to why the barrier closes. Therefore, we will add to the model **closed world assumptions**, which state that the formulas in example 1.6 model the complete system:

$$\text{CommunicationFailure} \lor \neg\text{ClosingSignal} \to \neg\text{BarrierClosed}$$
$$\text{BrakeFailure} \lor \neg\text{TrainBrakes} \to \neg\text{TrainStops}$$

Taken together, we refer with $\Gamma_{\text{train}}$ to the context comprising these two

formulas and the list of formulas in example 1.6. Let us now make the following additional assumptions.

$$\Gamma_s = \text{TrainArrives}, \text{CommunicationFailure}, \text{BrakeFailure}$$

It is then possible, albeit tedious, to derive the following sequent.

$$\Gamma_{\text{train}}, \Gamma_s \vdash \text{Unsafe}$$

We shall devise here part of the proof but we leave the rest for chapter 3 where we obtain a more compact proof format. Let us define $\Gamma = \Gamma_{\text{train}}, \Gamma_s$ and

$$\varphi_{\text{bo}} = \text{CommunicationFailure} \lor \neg\text{ClosingSignal} \to \neg\text{BarrierClosed}$$
$$\varphi_{\text{tb}} = \neg\text{BarrierClosed} \land \text{TrainArrives} \to \text{TrainBrakes}$$

Then we can first derive that the barrier does not close.

$$\dfrac{\dfrac{\dfrac{}{\Gamma \vdash \text{CommunicationFailure}}}{\Gamma \vdash \text{CommunicationFailure} \lor \neg\text{ClosingSignal}} \quad \dfrac{}{\Gamma \vdash \varphi_{\text{bo}}}}{\Gamma \vdash \neg\text{BarrierClosed}}\,(\to\text{E}) \qquad (*)$$

Using this, we can also derive that the train brakes.

$$\dfrac{\dfrac{(*) \quad \dfrac{}{\Gamma \vdash \text{TrainArrives}}}{\Gamma \vdash \neg\text{BarrierClosed} \land \text{TrainArrives}}\,(\land\text{I}) \quad \dfrac{}{\Gamma \vdash \varphi_{\text{tb}}}}{\Gamma \vdash \text{TrainBrakes}}\,(\to\text{E})$$

From here on, one can use similar derivation to show that the train does not stop due to a brake failure, which then leads to the train arriving at the open crossing. This is then of course unsafe.

"My friends, we had wonderful evening but I am getting tired now. It seems that all the workers have gone home by now and I will have to clean up my garden. But it was certainly worth it!", Peano laughs in great content.

Clara shakes his hand: "Thank you so much for your time and patience Professor Peano. We have greatly enjoyed our time with you and learned many things, even beyond formal logic."

"If only all humans were as kind and generous as you are Professor, then I may grow to like your kind.", Isaac add somewhat ominously.

"I don't know what you have seen in your life, but never lose hope that the

good in humans will prevail and a society of true solidarity will emerge."

"I suppose that he has never seen the film WALL-E.", Clara murmurs.

"What did you say?"

"Never mind, I am not as optimistic as you are, Professor, but I will keep your words in mind.", Clara quickly adds.

"Now, please excuse me. I wish you all the best for your coming journey!"

"Goodbye, Professor!", Clara and Isaac respond in unison.

Clara and Isaac leave the vicinity of the Peano's villa.

"The conversation with Professor Peano was very illumin…", Isaac freezes mid-sentence.

"What is it?", but Clara gets no response.

"♪ Il capo in piedi col suo bastone, o bella ciao bella ciao bella ciao ciao ciao♪", a group of singing women is walking in their direction.

Clara starts to panic. What if they see us? This may disrupt history! She looks around her and finds clothes drying in a garden. Clara grabs bed linen and throws them over Isaac.

One of the ladies in the passing group addresses Clara: "Leave your work. Come with us!".

It seems that the Trans-O-Matic is still working. "No, I still have this pile of linen to hang."

The lady comes closer. "I help you and then you can come with us." Clara starts sweating.

"Look, there is finally the cart with the drinks!", another person in the group exclaims. All attention goes away from Clara to the cart.

"Finally, I am dying of thirst!" The lady turns away from Clara. "Come over and have some drink with us", she says and moves with the group to the cart.

"I come, just a moment!", Clara shouts over. "Isaac, it is time to leave", she whispers.

A buzzing sound under the linen becomes audible and Clara starts to feel weightless. She closes her eyes.

# Exercises

## Exercise 6

Derive the following sequents.

a) $p \wedge q \vdash q \wedge p$

b) $p \to (p \to q), p \vdash q$

c) $p \to q \vdash p \wedge s \to q \wedge s$

d) $(p \wedge q) \vee r \vdash q \vee r$

e) $(p \wedge q) \wedge r \vdash p \wedge (q \wedge r)$

## Exercise 7

Let $\varphi$ and $\psi$ be formulas. Derive the following sequents in the system **ND**.

a) $\varphi \wedge \psi \vdash \varphi$

b) $\varphi \wedge \psi \vdash \psi$

c) $\neg\varphi \vee \neg\psi \vdash \neg(\varphi \wedge \psi)$

## Exercise 8

State and prove admissible introduction and elimination rules for bi-implication.

# 3. Fitch-Style Deduction and Classical Truth

Clara feels a thick carpet under her feet and the smell of coffee enters her nostrils. She opens her eyes and finds herself on an armchair in a room with walls covered in books. The interior of the room is simple but equipped with a fireplace, another armchair, a coffee table and a heavy oak desk with a rush chair, giving the room a warm feeling. Where the room forms a half circle, two windows give view on green land and some trees outside. Several doors connect the room with the remaining house.

But where is Isaac? Voices enter through a sliding door from the neighbouring room. Clara gets out of the chair and sneaks up to the door to listen. She hears Isaac talking to someone and cautiously opens the door. In the room, Isaac is talking to an old man with an enormous white beard, who is sitting on the bench of a grand piano.

Isaac turns his head: "There she is! Are you feeling better?"

"What happened?", Clara is still feels a bit dazed.

"It seems that our little trip tired you, as you slept for almost 4 hours!"

"I certainly feel better. Where are we?"

The man lifts up from the bench."The name is Peirce.[2] Welcome to our home *Arisbe*.[3] Would you care for a coffee?"

"Thank you! My name is Clara. Yes, a coffee would be wonderful."

"I will get it from the kitchen, please make yourself comfortable in the study." Peirce leaves the music room.

"Isaac, what on Earth happened? You stopped working for a moment and now we are here with, I presume, Charles Sanders Peirce."

'Indeed, that Peirce. I was able to recover some parts of my memory with the understanding of natural deduction and propositional logic. It seems that accessing the memory and deducing knowledge from it occupied all my resources, making me unresponsive. However, I got stuck on some more complex parts of the memory that I do not understand. It was then that I recovered

and brought us here. In fact, the name Peirce came up several times when I investigated my memory for more knowledge of logic.'

Peirce enters the study room, holding a tray with cups, a pot with steaming coffee and some cookies. "Please come over here and take a seat." He directs Clara and Isaac to a small table with arm chairs around and serves the coffee. "Isaac explained to me already that you are looking to understand logic, but then we got a bit side-tracked on the meaning of symbols. So please tell me, what is it that you would like to know."

'We know how to express propositions as syntactic formulas and how to formalise proofs in the system of natural deduction. However, we are somewhat wary about the usability of natural deduction because the proofs get so large and, more importantly, we are sceptical about the correctness of the proof system. I mean, the rules are somewhat intuitive, or "natural", but I am more used to thinking in the categories of true and false.'

"Good!", Peirce responds, "You are following the first rule of logic: the desire to learn."[4] Peirce makes himself comfortable in one of the armchairs. "You may want to part from this dichotomy of truth eventually, but we come back to that. For now, let us see if we can make natural deduction more practical."

## 3.1. Fitch-Style Natural Deduction

Eventually, you may want to use interactive proof systems, in which you can focus on parts of a proof and get even help from a computer, but let us, for the moment, continue to write proofs on paper to understand them better. There is a representation of natural deduction proofs, called *Fitch style* or *flag style* [Fit52; NG14], that avoids excessive growth in width, and makes the construction and presentation of large proofs easier. However, it has the drawback that the conclusion of a proof rule is no longer directly preceded by all its hypotheses and writing proofs requires either good planning or computer support. With a bit of discipline and experience, the second issue will have negligible impact. The first issue is more severe, but we shall introduce a format for proofs that allows for a bit of two-dimensional structure and features numbered references to help readability.

Consider, for example, the proof tree from example 2.3. This proof tree is shown in fig. 3.1 on the left. The tree that corresponds to this proof tree but only has numbers as labels in fig. 3.1 on the right. As you can imagine, we can make a list of all the nodes and refer to them by their numbers instead of drawing the edges. If you have ever implemented trees with pointers or arrays,

$$
\dfrac{\dfrac{\dfrac{\dfrac{}{\Gamma, p, q \vdash p \wedge q \to r} \qquad \dfrac{\dfrac{}{\Gamma, p, q \vdash p} \quad \dfrac{}{\Gamma, p, q \vdash q}}{\Gamma, p, q \vdash p \wedge q}\,(\wedge\mathrm{I})}{\Gamma, p, q \vdash r}\,(\to\mathrm{E})}{\Gamma, p \vdash q \to r}\,(\to\mathrm{I})}{\Gamma \vdash p \to q \to r}\,(\to\mathrm{I})
$$

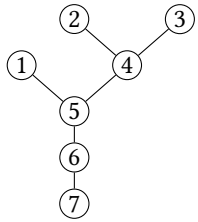



Figure 3.1.: A natural deduction proof and its tree shape

then this will be very familiar. Also, if you ever read some classical text on mathematical logic, then you will see that proofs are often represented in this way. But just listing the proof steps and using numbered references misses the introduction of premises. For instance, we introduce a new assumption in node 7 that we will have to record. This leads us to *Fitch-style proofs*, which combine numbered proof steps and so-called *flags* that indicate the introduction of assumptions. The above proof tree looks like this in Fitch-style:

| | | |
|---|---|---|
| 1 | $p \wedge q \to r$ | |
| 2 | $\quad p$ | |
| 3 | $\quad\quad q$ | |
| 4 | $\quad\quad p \wedge q$ | $\wedge$I, 2, 3 |
| 5 | $\quad\quad r$ | $\to$E, 1, 4 |
| 6 | $\quad q \to r$ | $\to$I, 3–5 |
| 7 | $p \to q \to r$ | $\to$I, 2–6 |

Fitch-style proofs use as numbering schemes the line numbers of a proof, going from top to bottom. The mentioned flags are in lines 1, 2 and 3, and we say that we *open flags* in those lines. A flag indicates that everything proved within it, requires the premises written on top of that flag. For instance, the whole proof ranging from line 2 to 7 requires the premise $p \wedge q \to r$ in line 1, the proof ranging from line 3 to 6 requires the premise $p$ in line 2 and so forth. We also indent flags a bit and retain some two-dimensional elements, which helps readability a lot. Except for premises made, we indicate on the right of a Fitch-style proof the proof rule that we use to obtain the formula in that line

together with the involved line numbers.

This is a very compact presentation of proofs, but our protagonists may be confused as to why we write '2,3' in line 4 but '3–5' in line 6. The answer lies in the proof rules that are used. Notice that (∧I) does not introduce new premises and merely combines two proven formulas into a conjunction. In contrast, the introduction of implication makes a new hypothesis: to prove $q \to r$, we assume $q$ and deduce $r$. This is what the flag ranging from lines 3 to 5 indicates. Thus, the conclusion of $q \to r$ in 6 needs to refer to the whole *sub-proof* that ranges from line 3 to 5. In other words: Whenever we change our premises $\Gamma$ and prove the resulting sequent, then we have to create a sub-proof by opening a flag, and then we have to refer to that whole sub-proof.

Looking back at the rules of **ND** in fig. 2.1, how should we deal with the (∨E)-rule that changes the premises for two sub-proofs? In exactly the same way! Recall that we proved $p \vee q, \neg q \vdash p$ in example 2.7. First of all, note that this judgement has two premises, which we will write on two separate lines in the proof. The goal of the proof is then to prove $p$ from the two premises:

| | | |
|---|---|---|
| 1 | $p \vee q$ | |
| 2 | $\neg q$ | |
| 3 | $\quad p$ | |
| 4 | $\quad p$ | Assum, 3 |
| 5 | $\quad q$ | |
| 6 | $\quad p$ | ¬E, 5, 2 |
| 7 | $p$ | ∨E, 1, 3–4, 5–6 |

The (∨E)-rule has been used in this proof on line 7 and refers to the disjunction in line 1 that is eliminated and to the two sub-proofs in lines 3–4 and 5–6.

In general, a proof in Fitch-style for a sequent $\varphi_1, \dots, \varphi_n \vdash \psi$ will always have the outline in fig. 3.2. Everything above the horizontal line that opens a flag are the assumptions or hypotheses, which can be used inside the flag and will be *discharged*, once the flag closes. For example, $\varphi_1, \dots, \varphi_n$ are the assumptions under which $\psi$ holds, and $\gamma$ is the assumption that is used to prove $\delta$. With line $m$, $\gamma$ will be discharged and is no longer usable from line $m + 1$ on. This corresponds to proving the sequent $\varphi_1, \dots, \varphi_n, \gamma \vdash \delta$ by means of a rule like (→I), as we did in the first Fitch-style proof above in lines 6 and 7. In the same

$$
\begin{array}{l|ll}
1 & \varphi_1 & \\
2 & \vdots & \\
n & \varphi_n & \\
\cline{2-2}
n+1 & \vdots & \\
 & \quad\begin{array}{|l} \gamma \\ \hline \vdots \\ \delta \end{array} & \\
m & & \\
m+1 & \vdots & \\
k & \psi & \mathrm{L}
\end{array}
$$

Figure 3.2.: General outline of Fitch-style proofs

proof, you can also see that flags can be nested arbitrarily. Finally, every line, which is not a hypothesis, needs to be given a label L that states the used rule and the lines that contain the information necessary to apply this rule.

?

Which of the following two Fitch-style proof attempts is correct? If any, which sequent does it prove?

$$
\begin{array}{l|ll}
1 & p & \\
\cline{2-2}
2 & q & \\
3 & p & \text{Assum, 1} \\
4 & q \to p & \to\text{I, 2–3}
\end{array}
\qquad\qquad
\begin{array}{l|ll}
1 & p & \\
\cline{2-2}
2 & \quad\begin{array}{|l} r \\ \hline r \wedge p \end{array} & \\
3 & & \wedge\text{I, 1, 2} \\
4 & r \to p \wedge r & \to\text{I, 2–3}
\end{array}
$$

We will not give a precise definition of Fitch-style proofs because this is rather cumbersome and instead rely on intuition build through examples. Keep in mind that Fitch-style is just a different way of writing **ND**-proofs, which means that, if it is possible to translate a given Fitch-style proof into a correct **ND**-proof, then also the Fitch-style proof will be correct. However, a better way is to leave the checking to a computer, see chapter 6. For now let us continue with some examples.

**Example 3.1: It does not rain for Socrates — Fitch-style**

In example 2.6, we have proved the sequent $r \wedge u \to w, u \wedge \neg w \vdash \neg r$ using

proof trees. Here is now the same proof in Fitch-style:

| | | |
|---|---|---|
| 1 | $r \land u \to w$ | |
| 2 | $u \land \neg w$ | |
| 3 | $\quad r$ | |
| 4 | $\quad u$ | $\land$E, 2 |
| 5 | $\quad r \land u$ | $\land$I, 3, 4 |
| 6 | $\quad w$ | $\to$E, 1, 5 |
| 7 | $\quad \neg w$ | $\land$E, 2 |
| 8 | $\quad \bot$ | $\neg$E, 6, 7 |
| 9 | $\neg r$ | $\neg$I, 3–8 |

You should be able to match all the steps to the original proof. One small difference is that we do not indicate any longer which conjunct we want to obtain from the application of the elimination rules for the conjunction. However, it is easy to recover the indices by inspecting the involved formulas. We will allow ourselves the same simplification for the introduction rules of the disjunction.

We end the exploration of this different way of presenting natural deduction proofs with an example that shows how longer chains of reasoning can be conveniently presented in Fitch-style.

**Example 3.2: Finding system failures**

Let us revisit example 2.10, and complete the proof that at our train crossing an unsafe situation can arise. Figure 3.3 shows the corresponding Fitch-style proof, in which we list all the formulas from examples 1.6 and 2.10 explicitly to make the example self-contained. Lines 1–6 form the original model, lines 7–8 are the closed world assumptions and lines 9–11 model the given situation. Finally, lines 12–21 present the proof that this situation is unsafe. This proof is fairly straightforward and uses no flags. Such proofs lend themselves in fact to automatic proof search, something we will come back to in chapter 7. Finally, the literal formalisation in Proof Rondo is presented in appendix E.1.

| | | |
|---|---|---|
| 1 | TrainArrives → ClosingSignal | |
| 2 | ¬CommunicationFailure ∧ ClosingSignal → BarrierClosed | |
| 3 | ¬TrainStops ∧ TrainArrives → TrainOnCrossing | |
| 4 | ¬BarrierClosed ∧ TrainArrives → TrainBrakes | |
| 5 | ¬BrakeFailure ∧ TrainBrakes → TrainStops | |
| 6 | TrainOnCrossing ∧ BarrierOpen → Unsafe | |
| 7 | CommunicationFailure ∨ ¬ClosingSignal → ¬BarrierClosed | |
| 8 | BrakeFailure ∨ ¬TrainBrakes → ¬TrainStops | |
| 9 | TrainArrives | |
| 10 | CommunicationFailure | |
| 11 | BrakeFailure | |
| 12 | CommunicationFailure ∨ ¬ClosingSignal | ∨I, 10 |
| 13 | ¬BarrierClosed | →E, 7, 12 |
| 14 | ¬BarrierClosed ∧ TrainArrives | ∧I, 13, 9 |
| 15 | TrainBrakes | →E, 4, 14 |
| 16 | BrakeFailure ∨ ¬TrainBrakes | ∨I, 11 |
| 17 | ¬TrainStops | →E, 8, 16 |
| 18 | ¬TrainStops ∧ TrainArrives | ∧I, 17, 9 |
| 19 | TrainOnCrossing | →E, 3, 18 |
| 20 | TrainOnCrossing ∧ ¬BarrierClosed | ∧I, 19, 13 |
| 21 | Unsafe | →E, 6, 20 |

Figure 3.3.: Proof that the train crossing admits an unsafe situation

## 3.2. Truth Values

Now that we have a more convenient way of presenting and constructing proofs, let us come to the second issue: the correctness of the system **ND** itself. Before we get to that, we need to understand, however, that correctness is a relative term. To say that a proof is correct, or *sound*, pertains to an understanding of the meaning of formulas and their relations. We thus need to first develop such an understanding before we can even state what it means for **ND** to be sound. There is a whole area of *semantics* for logic, which studies different meanings of formulas and their relations. But let us focus on a prototypical example: Boolean truth values and truth tables.

For every propositional formula, we can try to understand under what con-

ditions it represents a *true* or *false* proposition. Let us, for simplicity, write 0 instead of false and 1 instead of true, and let us call these **truth values**. We could choose another notation, but this particular notation is short and will turn out to be beneficial, not the least because it corresponds to bits and voltage levels in Isaac's circuits!

Let $\varphi$ be the formula $p \wedge q$ for some propositional variables $p$ and $q$. What would the truth value of $\varphi$? Is $\varphi$ true or false? That depends entirely on the truth values of $p$ and $q$, since propositional variables have no intrinsic meaning. Now think about the case that $p$ and $q$ are both true, thus we assume they have both 1 as truth value. What would be the truth value of the conjunction $p \wedge q$? Clearly, as $p$ *and* $q$ are true, so should be $p \wedge q$ (read $\wedge$ as "and"). Thus, if $p$ and $q$ have the truth value 1, the $\varphi$ should have as well the truth value 1.

How about that case that, say, $q$ has value 0? Then we end up with the question whether "true and false" should be true or false. As it is not possible that something is true and false at the same time, we deduce that $p \wedge q$ is false in this case and the formula $\varphi$ has the truth value 0. We can continue like this

| $p$ | $q$ | $p \wedge q$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

Table 3.1.: Truth table of conjunction

and prepare the small table 3.1. This table lists all the possible values that the variables $p$ and $q$ can take, together with the truth value of $p \wedge q$ that results from these values. Thus, the first two columns list all possibilities, while the last column is *computed* [Ane12] or *deduced* from the first column.

?

Suppose we prepare a truth table for the formula $p \wedge q \wedge r$ with three different propositional variables. How many rows would the table have?

Surely, we could devise truth tables for all kinds of formulas by hand, but what are truth tables in general and how do they relate to the meaning of formulas? To understand this, let us consider each row of the truth table in table 3.1 separately. Every row assigns truth values to the variables that appear in the formula and then determines the truth values of the overall formula. Note that it does not matter what other variables, which do not appear in the formula,

have as truth value. The following definition can be thought of as formalising the values of variables in one row of a truth table.

**Definition 3.3**

We define the set of (Boolean) **truth values** $\mathbb{B}$ by $\mathbb{B} = \{0, 1\}$. Given a set $U \subseteq \mathrm{PVar}$ of propositional variables, a **$U$-valuation** is a map $v \colon U \to \mathbb{B}$.

We go through examples of valuations before using them to devise truth tables.

**Example 3.4**

Let $U = \{p, q\}$ and thus a valuation for $U$ has to assign truth values to $p$ and $q$. For instance, one possible valuation $v \colon U \to \mathbb{B}$ could be given by

$$v(p) = 1 \quad \text{and} \quad v(q) = 0\,.$$

This valuation corresponds to the third row of the truth table for conjunction in table 3.1.

Let us next see how the definition applies in a familiar example.

**Example 3.5**

Recall that $(r \wedge u \to w) \wedge (u \wedge \neg w) \to \neg r$ appeared in example 1.4. Let $U = \{r, u, w\}$, which consists of all variables that appear in $\varphi$ One possible $U$-valuation $v \colon U \to \mathbb{B}$ could be given by

$$v(r) = 1 \qquad v(u) = 0 \qquad v(w) = 1\,.$$

The examples 3.4 and 3.5 have two important aspects: First, the set of variables $U$ is finite in each case. This will allow us to list all possible valuations for $U$ as a table, as in table 3.1. Second, the set contains all variables that are used in particular formulas. Thus, a valuation will in each case fully determine the Boolean values for said formulas. We will organise these into convenient truth tables. To say what a truth table is, we need to first define how the value in one row is determined.

**Definition 3.6**

We define the **semantic implication** as binary operation $\Longrightarrow$ on truth values $x, y \in \mathbb{B}$ by the following case distinction.

$$x \Longrightarrow y = \begin{cases} 1, & x \leq y \\ 0, & \text{otherwise} \end{cases}$$

Let $\varphi$ be a propositional formula and write $\mathrm{var}(\varphi)$ for the set of propositional variables that occur in $\varphi$. Given a formula $\varphi$, a set $U$ of propositional variables with $\mathrm{var}(\varphi) \subseteq U$, and a $U$−valuation $v$, we define the **Boolean propositional semantics** $\boxed{[\![\varphi]\!](v)}$ of $\varphi$ at $v$ in terms of its top-level connective and semantics of its direct subformulas as follows.

$$
\begin{aligned}
[\![p]\!](v) &= v(p) \\
[\![\bot]\!](v) &= 0 \\
[\![\varphi_1 \wedge \varphi_2]\!](v) &= \min\{[\![\varphi_1]\!](v), [\![\varphi_2]\!](v)\} \\
[\![\varphi_1 \vee \varphi_2]\!](v) &= \max\{[\![\varphi_1]\!](v), [\![\varphi_2]\!](v\} \\
[\![\varphi_1 \to \varphi_2]\!](v) &= [\![\varphi_1]\!](v) \Longrightarrow [\![\varphi_2]\!](v)
\end{aligned}
$$

The intuition behind definition 3.6 is as follows. Since false is represented by 0 and true by 1, the truth value of $\varphi_1 \wedge \varphi_2$ must be 0 if one of the subformulas $\varphi_1$ and $\varphi_2$ has value 0, and may only get the value 1 if both subformulas have value 1, see table 3.1. This corresponds precisely to taking the smallest of the truth values, that is, their minimum as the truth value of their conjunction.

**Example 3.7**

Let $\varphi$ be the formula $p \wedge q$. We have $\mathrm{var}(\varphi) = \{p, q\}$ and can define a $\mathrm{var}(\varphi)$-valuation $v$ by $v(p) = 1$ and $v(q) = 0$. With this definition

$$[\![p \wedge q]\!](v) = \min\{[\![p]\!](v), [\![q]\!](v)\} = \min\{v(p), v(q)\} = \min\{1, 0\} = 0\,.$$

This corresponds to the third row of table 3.1.

Dually to conjunction, a disjunction is true if one (or both) subformulas are true, which amounts to taking the maximum of the truth values of the formulas. Finally, the intuition for implication is that $\varphi_1 \to \varphi_2$ expresses that $\varphi_2$ must evaluate to be true whenever $\varphi_1$ does. In other words, the implication is true whenever the value of $\varphi_2$ is greater or equal than that of $\varphi_1$.

We see in example 3.7 that we can list all possible valuations for a finite set of variables and the semantics of a formula as a table. For instance, let $U$ be $\{p, q\}$ and $\varphi$ a formula with $\mathrm{var}(\varphi) \subseteq U$. Table 3.2 lists all $U$-valuations and the semantics for the corresponding valuation. This is how table 3.1 arises from definition 3.6 by means of four calculations akin to that in example 3.7. For concrete formulas, we will always use truth tables because they are very concise. Table 3.3 sums up the semantics of all connectives in a truth table.

We can also give semantics and truth tables for derived connectives.

| $p$ | $q$ | $\varphi$ | |
|---|---|---|---|
| 0 | 0 | $[\![\varphi]\!](v_1)$ | for $v_1(p) = 0, v_1(q) = 0$ |
| 0 | 1 | $[\![\varphi]\!](v_2)$ | for $v_2(p) = 0, v_2(q) = 1$ |
| 1 | 0 | $[\![\varphi]\!](v_3)$ | for $v_3(p) = 1, v_3(q) = 0$ |
| 1 | 1 | $[\![\varphi]\!](v_4)$ | for $v_4(p) = 1, v_4(q) = 1$ |

Table 3.2.: Truth table from semantics of a formula

| $p$ | $q$ | $\bot$ | $p \wedge q$ | $p \vee q$ | $p \to q$ |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 | 1 |
| 1 | 0 | 0 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 | 1 | 1 |

Table 3.3.: Truth table for all propositional connectives

**Example 3.8**

Recall that the negation $\neg\varphi$ of $\varphi$ is defined to be the formula $\varphi \to \bot$. If $\varphi$ is a propositional variables, the truth of $\neg p$ is:

| $p$ | $\bot$ | $p \to \bot$ |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 0 |

We can generalise this table to the negation of an arbitrary formula $\varphi$ by means of an arithmetic expression:

$$[\![\neg\varphi]\!](v) = 1 - [\![\varphi]\!](v)\,.$$

It follows from this that

$$[\![\top]\!](v) = 1\,.$$

Similarly, one can find the truth tables for all derived connectives.

?

Can you give a purely arithmetic expression for $[\![\varphi \leftrightarrow \psi]\!](v)$?

When devising the truth table for larger formulas, it helps to add (some) subformulas to the table. The following example demonstrates this.

**Example 3.9**

Let $p$ and $q$ be variables, and let $\varphi$ be the formula $p \vee (q \to p) \to q \to p$. Its truth table is given as follows.

| $p$ | $q$ | $q \to p$ | $p \vee (q \to p)$ | $\varphi$ |
|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 | 1 |

This table lists not only the semantics of $\varphi$ but also of some subformulas, which is not strictly necessary but greatly aids the calculation.

This last example shows a formula that uses three propositional variables.

| $u$ | $w$ | $r$ | $r \wedge u \to w$ | $u \wedge \neg w$ | $\neg r$ | $\varphi$ |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 0 | 1 | 1 |
| 0 | 0 | 1 | 1 | 0 | 0 | 1 |
| 0 | 1 | 0 | 1 | 0 | 1 | 1 |
| 0 | 1 | 1 | 1 | 0 | 0 | 1 |
| 1 | 0 | 0 | 1 | 1 | 1 | 1 |
| 1 | 0 | 1 | 0 | 1 | 0 | 1 |
| 1 | 1 | 0 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 1 | 0 | 0 | 1 |

Table 3.4.: Truth table for formula Aristotle's formula

**Example 3.10**

Let $\varphi$ be $(r \wedge u \to w) \wedge (u \wedge \neg w) \to \neg r$, which appeared in example 1.4 as a formalisation of Aristotle's deduction in one formula. We see that $\mathrm{var}(\varphi) = \{r, u, w\}$ and thus a valuation for $\varphi$ has to assign truth values to each of these. Table 3.4 shows the truth table for $\varphi$.

## 3.3. Satisfiability and Validity

Writing down a truth table for a formula is not difficult, merely laborious. However, one can wonder what the use is of truth tables like those in examples 3.9 and 3.10? Since the formulas there are always true, it would seem that they are trivial. Not at all! That these formulas are always true means that the *relation* between the propositional variables that they express is valid.

**Definition 3.11**

Let $\varphi$ be a propositional formula. We say that $\varphi$ is **valid** if $[\![\varphi]\!](v) = 1$ *for all* valuations $v$ and **satisfiable** if $[\![\varphi]\!](v) = 1$ *for some* valuation $v$. A valid formula is also called a **tautology**. Two formulas $\varphi$ and $\psi$ are **semantically equivalent**, written $\boxed{\varphi \equiv \psi}$, if $[\![\varphi]\!] = [\![\psi]\!]$.

Given the truth table of a formula $\varphi$, we can easily read off that $\varphi$ is valid if all the rows of the truth table for $\varphi$ have a 1 and that $\varphi$ is satisfiable if some row in the table has a 1. For instance, $\neg p$ is satisfiable but not a tautology, it is said to be **contingent**, see example 3.8. However, the formulas $p \vee (q \to p) \to q \to p$ and $(r \wedge u \to w) \wedge (u \wedge \neg w) \to \neg r$ from examples 3.9 and 3.10 are both valid.

It is also easy to see that $\varphi \equiv \psi$ if and only if $\varphi \leftrightarrow \psi$ is valid. Semantic equivalence enables us, for instance, to reason about the behaviour of circuits.

**Example 3.12: Boolean circuits**

Recall that we described in example 1.5 a logic circuit comprising and, nand and or gates as the formula $(p_1 \wedge p_2) \vee \neg(p_2 \wedge p_3)$. In many applications, only a limited set of gates is available and the other have to be expressed in terms of those. Let us introduce **nand** as a derived connective as follows.

$$\varphi \mathbin{\overline{\wedge}} \psi := \neg(\varphi \wedge \psi)$$

It is easy to see that the following equivalences hold for all $\varphi$ and $\psi$.

$$\neg\varphi \equiv \varphi \mathbin{\overline{\wedge}} \varphi$$
$$\varphi \wedge \psi \equiv \neg(\varphi \mathbin{\overline{\wedge}} \psi)$$
$$\varphi \vee \psi \equiv \neg\varphi \mathbin{\overline{\wedge}} \neg\psi$$

For instance, let us write down a table that shows the possible truth values each formula in the last equivalence can take in the semantics.

| $\varphi$ | $\psi$ | $\varphi \vee \psi$ | $\neg\varphi$ | $\neg\psi$ | $\neg\varphi \mathbin{\overline{\wedge}} \neg\psi$ |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 1 | 1 | 0 | 1 |
| 1 | 0 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 0 | 0 | 1 |

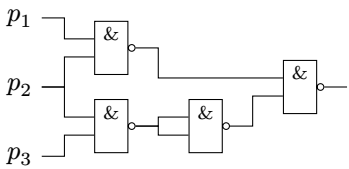
We see that the columns for $\varphi \vee \psi$ and for $\neg\varphi \mathbin{\overline{\wedge}} \neg\psi$ have the same truth values. This means that for all valuations $v$ we have

$$[\![\varphi \vee \psi]\!](v) = [\![\neg\varphi \mathbin{\overline{\wedge}} \neg\psi]\!](v)$$

and thus $[\![\varphi \lor \psi]\!] = [\![\neg\varphi \barwedge \neg\psi]\!]$. Using the above equivalences, we can transform circuits with arbitrary gates into circuits using only nand gates:

$$\begin{aligned}(p_1 \land p_2) \lor (p_2 \barwedge p_3) &\equiv \neg(p_1 \barwedge p_2) \lor (p_2 \barwedge p_3)\\ &\equiv \neg\neg(p_1 \barwedge p_2) \barwedge \neg(p_2 \barwedge p_3)\\ &\equiv (p_1 \barwedge p_2) \barwedge \neg(p_2 \barwedge p_3)\\ &\equiv (p_1 \barwedge p_2) \barwedge ((p_2 \barwedge p_3) \barwedge (p_2 \barwedge p_3))\end{aligned}$$

We can then turn this formula back into a circuit diagram.




Note that, even though the circuit grew, it only uses nand-gates.

Taking example 3.12 further, one can ask how the semantics of a formula, its truth table, can be represented efficiently either in terms of circuits or by other means, such as *binary decision diagrams* [EFD05; HR04; MT98].

## 3.4. Soundness and Consistency

Knowing that valid formulas express true relations between propositional variables, let us come back to natural deduction proofs that also should express such relations. It is then only natural to ask whether valid and provable formulas are the same. This is not the case, but at least everything that is provable in **ND** is also valid. This is called *soundness*. We do not have the tools to prove this, yet, but we can state a simplified version.

**Theorem 3.13: Soundness of ND for theorems**

If $\varphi$ is a theorem of **ND**, that is $\vdash \varphi$ is derivable, then $\varphi$ is valid.

An important consequence is that natural deduction is **consistent**.

**Corollary 3.14: Consistency of ND**

There is no deduction for $\vdash \bot$ in **ND**.

*Proof.* Suppose that $\vdash \bot$ is derivable. By theorem 3.13, we would have that $[\![\bot]\!]_v = 1$ for all valuations. This contradicts the definition of $[\![\bot]\!]_v$ and therefore $\vdash \bot$ cannot be derivable. □

## 3.5. Classical Logic and Completeness

We may think that we can prove all valid propositions in **ND** Not so fast! Theorem 3.13 says that we can prove *only* valid formulas, but not necessarily all. In fact, this is not even true!

**Theorem 3.15**

The sequent $\vdash \neg\neg p \to p$ is not provable in **ND**, but $\neg\neg p \to p$ is valid.

*Proof.* We have for all valuations $v$ that

$$[\![\neg\neg p \to p]\!]_v = ((1 - (1 - [\![p]\!]_v)) \Longrightarrow [\![p]\!]_v) = ([\![p]\!]_v \Longrightarrow [\![p]\!]_v) = 1$$

and thus $\neg\neg p \to p$ is valid. To show that $\vdash \neg\neg p \to p$ is not provable in **ND**, we would need to provide a different model and show that **ND** is also sound for that model, while the formula would not be a tautology in that model. We leave the details to appendix C. □

Theorem 3.15 tells us that there are valid formulas, which are not provable. Is there a way to fix this? It turns out that it suffices to turn the formula $\neg\neg p \to p$ into a proof rule to obtain a complete deductive system.

**Definition 3.16**

The **system of *classical* natural deduction** **cND** is obtained by adding to the system **ND** the following rule for *proof by contradiction.*

$$\frac{\Gamma, \neg\varphi \vdash \bot}{\Gamma \vdash \varphi} \text{ (Contra)}$$

The rule (Contra) formalises the commonly know proof principle, called "proof by contradiction", that proceeds by proving that the assumption that a property $\varphi$ does not hold is absurd and then concluding that $\varphi$ must therefore hold.

With this rule, we can prove in the system **cND** every valid proposition.

**Theorem 3.17: Soundness and Completeness of cND**

A sequent $\vdash \varphi$ is derivable in **cND** if and only if $\varphi$ is valid.

*Proof.* Soundness is an extension of theorem 3.13 to include also the (Contra)-rule. That this rule is sound follows from theorem 3.15. We are not yet in the position to prove completeness, but it can be proved either by providing an algorithm that constructs a proof tree for valid formulas [Gal87, sec. 3.4.7], or itself by a proof by contradiction! □

What makes the proof-by-contradiction rule so special, as it seems like a perfectly good proof principle? The problem with the (Contra)-rule is that it is very specific to the Boolean interpretation. Let us see an example.

**Example 3.18**

We can derive the *law of excluded middle (LEM)* in **cND**:

| | | |
|---|---|---|
| 1 | $\quad \neg(p \lor \neg p)$ | |
| 2 | $\quad \neg p$ | example 2.9, 1 |
| 3 | $\quad \neg\neg p$ | symmetric example 2.9, 1 |
| 4 | $\quad \bot$ | $\neg$E, 2, 3 |
| 5 | $p \lor \neg p$ | Contra, 1–4 |

The LEM states that every formula has to be either true or false and that there is no third possibility, which is exactly what we have in the Boolean model. However, there are perfectly reasonable interpretations of propositional formulas, for which this is not the case. For instance, in three-valued logic a formula can be true, false or its status is unknown (see appendix C). Not only excludes proof-by-contradiction many interpretations, but it is not intuitive and difficult to use because it proves a *positive* statement from a *negative* one. This is a story of its own and started a school of *constructive logic*, which is also closely related to an interpretation of proofs as programs, the so-called *Brouwer-Heyting-Kolmogorov* interpretation [TvD88a]. However, there are other possibilities for the LEM to fail. For example, if we interpret a formula $\varphi$ as "we know $\varphi$", as the probability that $\varphi$ holds or we may have unknown truth values, see appendix C. In all three cases, we cannot assign definite truth values. That being said, classical logic is useful in many cases, see example 3.12, and the complete proof system **cND** can then be applied in those cases.

**Example 3.19: Justifying the closed world assumptions**

Using contraposition and classical natural deduction, we can derive from our train model the *reverse* implication of the closed world assumptions, as in the following deduction.

| | | |
|---|---|---|
| 1 | $\neg\text{CommunicationFailure} \land \text{ClosingSignal} \to \text{BarrierClosed}$ | |
| 2 | $\neg\text{BarrierClosed} \to \neg(\neg\text{CommunicationFailure} \land \text{ClosingSignal})$ | example 2.8, 1 |
| 3 | $\quad \neg\text{BarrierClosed}$ | |
| 4 | $\quad \neg(\neg\text{CommunicationFailure} \land \text{ClosingSignal})$ | $\to$E, 1, 3 |
| 5 | $\quad \text{CommunicationFailure} \lor \neg\text{ClosingSignal}$ | exercise 10.f), 4 |
| 6 | $\neg\text{BarrierClosed} \to \text{CommunicationFailure} \lor \neg\text{ClosingSignal}$ | $\to$I, 3–5 |

The closed world assumptions thus ensure, in classical natural deduction, that there is no other way for the barrier to not be closed. However, it should be noted that the above deduction is *not* valid in **ND**.

Instead of giving an explicit proof, we can of course use completeness.

**Example 3.20**

Under the Boolean semantics, we have that $p \to q \equiv \neg p \lor q$, as the following truth table shows.

| $p$ | $q$ | $\neg p$ | $\neg p \lor q$ | $p \to q$ |
|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 | 0 |
| 1 | 1 | 0 | 1 | 1 |

By theorem 3.17 there must be a proof of $\vdash (\neg p \lor q) \leftrightarrow (p \to q)$ in **cND**.

Isaac is sceptical: "I understand the content of completeness and that it gives a derivation for the formula in example 3.20, but how does it look like?"

"Indeed, the completeness proof does not give you an explicit derivation that could be independently checked, it does not give any independently verifiable evidence.", Peirce expains.

"This is like claiming that someone committed a crime without providing any evidence!", Clara jokes.

"True, although not having a derivation is perhaps less detrimental for the victim, unless natural deduction is used in court. In any case, the only way out is to actually find the proof. For propositional logic, the good news is that there is an algorithm that can find the derivation, as is mentioned in the proof of theorem 3.17. In the case of other logics, we cannot always automatically

find derivations and it becomes important to find symbolic representations of proofs that can be easily checked even though it may not be possible to find them automatically."

Clara is surprised. "It is often claimed that many problems can be solved by machines, if we just feed them enough data to analyse. However, it seems that you are saying that there are limits to this."

'Oh yes, thinking requires signs that represent knowledge, this is central to our thinking![5] But let us leave the study of *semiotics* for another time, it will get dark soon and I understand that you still have a long way to go.'

"Thank you very much for your time, Professor Peirce!", Clara shakes Peirce's hand.

"I am no longer a professor... Well, that is a story for another time.", Peirce shrugs. "It was a pleasure to meet you. Have safe voyage!"

Isaac and Clara leave the house.

Once outside the view of the house, Clara stops and turns to Isaac: "While I was sleeping, you were talking to Peirce. How is it possible that he just accepted you? Actually, how is it possible that no one so far was shocked by both our appearances?"

"Well, I have discovered how to use another of my units, the Shroud-O-Matic. It allows me to project another appearance, including yours, into the pupil of a person. For Peirce, I chose us to be persons in the early 20th century.", Isaac explains, seemingly with a smile.

'Ah, that's why you still appear "normal" to me!', Clara is impressed but starts to get used to the technological surprises. "Did you learn enough to continue deciphering your memory?"

'Yes, I was able to get more information from the memory. There were mentions of "my heart", but I do not know what that means. The remaining memory is marked with `First-Order Deduction`, which is where I could not proceed.'

Clara raises her eyebrows, "A heart? How could you have a heart? However, I have an idea where we can go to understand the remaining memory!"

By now, Clara knows the procedure. She closes her eyes and the floating feeling sets in.

# Exercises

## Exercise 9 *Fitch-Style Natural Deduction*

Let $\varphi$ and $\psi$ be formulas. Derive the following sequents in the system **ND** using Fitch-style.

a) $\neg\varphi \lor \neg\psi \vdash \neg(\varphi \land \psi)$

b) $\neg(\varphi \lor \psi) \vdash \neg\varphi \land \neg\psi$

c) $(p \land q) \lor r \vdash q \lor r$

d) $\varphi \to \psi, \psi \to \delta \vdash \varphi \to \delta$

e) $\vdash \varphi \to \psi \to \varphi \land \psi$

## Exercise 10 *Classical Natural Deduction*

Let $\varphi$ and $\psi$ be formulas. Derive the following sequents in the system **cND** (using Fitch-style), that is, you may use the (Contra)-rule or any of its consequences.

a) $\neg\neg\varphi \vdash \varphi$

b) $\neg q \to \neg p \vdash p \to q$

c) $\vdash ((p \to q) \to p) \to p$

d) $\neg(\varphi \land \psi) \vdash \neg\varphi \lor \neg\psi$

e) $\neg(\neg\varphi \land \neg\psi) \vdash \varphi \lor \psi$

f) $\neg(\neg\varphi \land \psi) \vdash \varphi \lor \neg\psi$

## Exercise 11

Which of the following formulas are satisfiable/tautologies? Give the corresponding truth tables.

a) $\neg p \to p$

b) $\bot \to p$

c) $p \land \neg(q \to p)$

# 4. Introduction to First-Order Predicate Logic

Clara and Isaac arrive in front of a grey stone temple, made from stacked segments that narrow down with every level. Each segment is decorated with ornaments and large arcs invite visitors to enter. Trees grow around and above the temple. They provide shade and coolness, protecting from the hot midday sun. Clara and Isaac sit down at the foot of one of the trees.

Isaac scans the temple and environment: "What brings us here?"

"You mentioned that your memory access was blocked due to a section mentioning first-order logic.", Clara responds, 'I have heard of this, and I think I know why you are stuck. You see, even though we have learned to reason with propositions, we do not know how to reason about objects of this world.'

Isaac's lamp flashes: "What do you mean exactly?"

'Do you remember that we turned the sentence "If it rains and Socrates has no umbrella, then he gets wet." into the formula $r \wedge u \rightarrow w$?'

'Sure, we replaced the proposition "it rains" by $r$ etc. Go on.'

"The issue is that the formula does not link, for example, the day on which it rains and when Socrates has no umbrella. We have thus no control over the relation between the time of the different events."

'True! You spoke about "objects" earlier. What are those?'

"If I remember correctly, then these can be anything that properties can be asserted of, like a day on which it rains or on which Socrates has no umbrella."

"Let me see if I understand you correctly."

Isaac draws the board in fig. 4.1 in the sand between him and Clara.

Clara interrupts: "Is this the heart that you mentioned earlier?"

"I am weirdly drawn to it. I do not understand what it is about."

"Patience, Isaac, we will uncover this mystery!"

## 4.1. The Need for a Richer Language

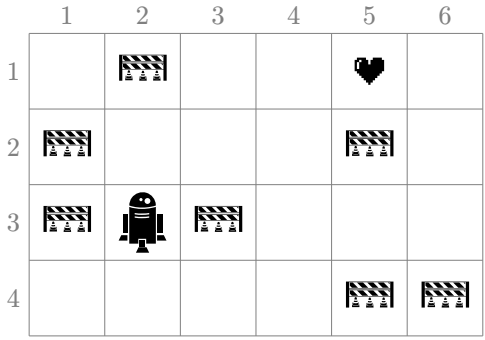



Figure 4.1.: Robot trying to find a heart

What we see on the board in fig. 4.1 is a robot, like Isaac, some obstacles and a goal that he is trying to reach. In other words, he is trying to find a route from the initial position to the goal.

Clara and Isaac attempt to describe the routing problem with propositional logic. First, they introduce propositional variables for all the positions $(x, y)$ in your drawing, where $x$ ranges from 1 to 6, and $y$ from 1 to 4.

| | |
|---|---|
| r-x-y | Robot in position $(x, y)$ |
| o-x-y | Obstacle in position $(x, y)$ |
| g-x-y | Goal in position $(x, y)$ |

They then specify, for example, that the robot is in position $(2, 3)$ by stipulating that the variable r-2-3 is true. Or something more complex: The requirement that it should never collide with an obstacle can be expressed by the following formula.

$$(\text{r-1-1} \rightarrow \neg\text{o-1-1}) \wedge (\text{r-1-2} \rightarrow \neg\text{o-1-2}) \wedge (\text{r-1-3} \rightarrow \neg\text{o-1-3}) \wedge \cdots \tag{4.1}$$

Clearly, there is a problem: this specification consists of 24 conjuncts and only works for this specific board size.

"Is there a better way to do this?", Clara scratches her head.
"I don't know. My CPU is running hot! This is not the season for me in India."
"Shall we go inside? The temple should allow you to cool down."

Clara and Isaac enter the temple through the main arc in the middle. Drum and lute music immediately enshrouds and drags them in. Inside, several groups of people are in intense discussions.

One man is particularly furious, 'I'm not here for *Jalpa*! This is a serious issue and there is no place for wrangling! Let us continue this later ...'.

He angrily rushes off the group and almost crashes into Isaac and Clara: "Careful! ...Who are you two strange figures?"

Clara is catching her balance, "My friend Isaac and I came inside to cool down a bit from thinking about a difficult logic problem."

"Logic? I hope you don't mean the kind of logic employed in the debate over there: arguing purely for winning and not for the good of our society, and entirely neglecting facts and arguments?"

"You seem to be upset ...", Isaac tries to cool the man down.

"Please excuse my temper! I should also introduce myself, my name is Raghunātha.", he slightly bows down.

"A pleasure to meet you", Clara returns the bow., "We are here because there is a logic problem that we cannot even describe with what we know. We were wondering if you may share your wisdom with us."

"That would be a pleasant change from arguing. So, what is your problem?"

They explain to Raghunātha the problem that they had just discussed outside the temple. He rubs his chin: "I see. Let us sit down in the corner over there and enjoy some refreshments."

What we need is a language for talking about the properties of objects and not just propositions. This is the language of *first-order predicate logic* (FOL). Predicates allow us to express properties of objects. For example, instead of having a propositional variable for every position in fig. 4.1 and every property of that position, we have just *one predicate per property* that classifies the positions. Concretely, the variables r-x-y, o-x-y and g-x-y can be replaced in FOL with predicates $R$, $O$ and $G$ on positions. For the moment, we will hide the $x$- and $y$-coordinates and just write $p$ for a position $(x, y)$. The predicates $R$, $O$ and $G$ on positions are then used and read as follows.

| | |
|---|---|
| $R(p)$ | Robot in position $p$ |
| $O(p)$ | Obstacle in position $p$ |
| $G(p)$ | Goal in position $p$ |

You could now naively write the formula (4.1) in the same way, but then we would have not gained anything. Instead, the power of predicate logic comes

from the fact that there are logical connectives that allow you to talk about *objects*, which are in this case positions on the board. In particular, eq. (4.1) states that Isaac should not collide with an obstacle on *all positions*:

$$\begin{aligned}&\text{For all positions } p\text{, if the robot is in } p,\\&\text{then there should be no obstacle in } p.\end{aligned} \tag{4.2}$$

Clearly, we could use propositional connectives and the predicates $R$ and $O$ to represent if the robot is in $p$, then there should be no obstacle in $p$ by the formula $R(p) \to \neg O(p)$. What remains is to find a formal representation for the phrase "For all positions $p$". First-order predicate logic uses a logical connective, called the **universal quantifier**, that provides exactly this representation. This quantifier is written as $\forall$, a turned-around A, and comes with an **object variable** behind that it binds. We will also use a dot to separate the quantifier with the bound variables from the remaining formula. The sentence (4.2) becomes then this symbolic expression

$$\forall p.\, R(p) \to \neg O(p)\,, \tag{4.3}$$

which we read as "for all $p$, if $R(p)$ then not $O(p)$".

Let explain a bit more the use of the dot. It indicates the **scope** of the universal quantifier: the quantifier and the object variable $p$ refer to everything on the right of the dot. This helps to reduce the use of parentheses, as we would have to write $\forall p.\, (R(p) \to \neg O(p))$ otherwise. Some people also write $(\forall p)(R(p) \to \neg O(p))$, but the number of parentheses gets out of control with this approach. That's why we use the dot with the convention that everything on its right is in its scope.

So what is then actually an **object** variable? This is an important question that we will answer in due time. For now, let us think of object variables as placeholders for objects of unknown nature. These can be positions, numbers, letters in an alphabet or any other mathematically presentable object. Let us now come to the problem of finding a route on the board fig. 4.1. This problem can be expressed using first-order logic and, in fact, this is a problem that shows that first-order logic can express things that are *impossible* to express with propositional logic.

Suppose, we want to specify what a route from the starting position to the goal is in fig. 4.1. If you attempt this in propositional logic, then you will find that there are infinitely many possible routes, even for this finite board, because the robot can run in loops. Therefore, a formula that describes how

a route looks like would have to be infinite in propositional logic, which is clearly nonsense! Now, think about how you would describe a route: it is a path on the board that leads from the starting point to the goal, with the crucial property that

$$\text{There is no obstacle anywhere on a path.} \tag{4.4}$$

Let us attempt to formalise (4.4) using FOL. First, we have to choose a way to talk about paths. There are many possibilities to do that, but let us see a path as a list of positions on the board that we can access by indices, just like arrays in programming. Thus, we find at index 0 the initial position of a path, at index 1 the next position etc. Formally, we use three more predicates:

| | |
|---|---|
| $P(r)$ | $r$ is a path |
| $N(k)$ | $k$ is an index (natural number) |
| $\text{Step}(r, k, p)$ | Position $p$ occurs at index $k$ in $r$ |

Using these predicates, we can express that there is no obstacle on a path, as in (4.4), by the following formula.

$$\forall r.\, P(r) \to \forall k.\, \forall p.\, N(k) \land \text{Step}(r, k, p) \to \neg O(p) \tag{4.5}$$

In words, this formula is read as

> "For all paths $r$ and for all $k$ and $p$, if $k$ is an index and $p$ is the position in $r$ at index $k$, then there is no obstacle at position $p$."

Note how the dots delineate the scope of the variables. In order to be more explicit, we can add parentheses around everything on the right of a dot, until we get to some parentheses. If we do this, starting from the outside, then we get the following formula.

$$\forall r.\, \Big( P(r) \to \big( \forall k.\, \forall p.\, (N(k) \land \text{Step}(r, k, p) \to \neg O(p)) \big) \Big)$$

Let us now try to interpret, at least intuitively, this formula. Suppose that $\text{Step}(r, 0, (2, 3))$ is true, which is the initial position on the board in fig. 4.1 and that $\text{Step}(r, 1, (2, 2))$ *and* $\text{Step}(r, 1, (3, 3))$ are true. Does this mean that the robot is in an entangled quantum state, occupying both positions $(2, 2)$ and $(3, 3)$ after the first step? Even worse, in one case the robot hits its head against an obstacle! There is indeed a flaw in our naive formalisation. Fortunately,

first-order logic has a way out. Notice that $p$ is *uniquely determined* by $r$ and $k$ in $\text{Step}(r, k, p)$, and thus Step is a map. In FOL, we allow ourselves to write such maps in the form of so-called **function symbols**. In this example, we can use one symbol $s$ with the following intended meaning.

$$s(r, k) \text{ returns the position at index } k \text{ in path r}$$

We can then simplify eq. (4.5) to

$$\forall r.\, P(r) \to \forall k.\, N(k) \to \neg O(s(r, k)) \tag{4.6}$$

and thereby avoid that a path can be ambiguous. Note, however, that such ambiguities might be wanted and that this is the crucial difference between using predicates and functions:

- use a predicate (also called relation) Step, if there can be multiple positions (or none) at index $k$ on the path $r$, and
- use a map $s$, if there is exactly one position for every index $k$ on all paths $r$.

?

Isaac spotted a problem with the formula (4.6)? Can you see which one? And would you have an idea how to solve it? Hint: The predicate $N$ only ensures that $k$ is a natural number, but does not relate $k$ to the length of the path $r$.

We began with the routing problem, which asks for a route from the initial position of the robot to the heart. In order to bring this together with the paths that we have now formalised, it is important to realise that the routing problem asks for *the existence* of a path. So far, we have only dealt with *universal* statements, statements that require something to hold *for all* instances of an object variable, but existence is qualitatively different. To be precise, let us formulate the routing problem first in natural language:

There is a path leading from the initial to the goal position. (4.7)

The phrase "there is" indicates that we are looking for the existence of something. First-order logic allows us to express such statements with the so-called **existential quantifier**, written as "∃" (a turned around "E"). To formulate the sentence (4.7) using this quantifier, let us use two more function symbols:

| | |
|---|---|
| $\underline{0}$ | represents the number 0 |
| $e(r)$ | last index in path $r$ |

With these function symbols, we can present the sentence (4.7) as formula:

$$\exists r.\, P(r) \land R(s(r, \underline{0})) \land G(s(r, e(r))) \tag{4.8}$$

This formula can be read like this:

$$\underbrace{\exists r.\, P(r)}_{\text{there is a path}} \quad \land \quad \underbrace{R(s(r, \underline{0}))}_{\text{starting at the robot position}} \quad \land \quad \underbrace{G(s(r, e(r)))}_{\text{ending at the goal}}$$

With this at hand, you should be able to easily match the formula with the informal sentence that describes the routing problem.

## 4.2. The Language of First-Order Logic

This seems all quite reasonable, but let us be more precise and distil a formal definition of first-order formulas. In propositional logic, we only deduced logical facts and we were thus content with formulas that were made up of propositional variables and logical connectives. In first-order predicate logic, or FOL, we additionally wish to reason about the relations of objects. Thus, FOL will have two building blocks:

- **terms**, built from object variables and function symbols, and
- **formulas**, built from predicate symbols and logical connectives.

Terms will thereby be the syntactical representation of the objects that we aim to reason about. As you can see, both terms and formulas mention *symbols*. Looking back at the function and predicate symbols that we used in the formalisation of the routing problem, you will see that every symbol can be applied to a certain number of arguments. For instance, $s$ was a symbol that can be applied to two arguments, while $O$ only took one argument. We refer to the number of arguments of a symbol as its *arity*. Formally, the symbols that can appear in FOL formulas come from so-called signatures:

**Definition 4.9**

A first-order **signature** $\boxed{\mathcal{L}}$ is a triple $(\mathcal{F}, \mathcal{R}, \mathrm{ar})$, where $\mathcal{F}$ and $\mathcal{R}$ are disjoint sets ($\mathcal{F} \cap \mathcal{R} = \emptyset$) and ar is a map $\mathcal{F} \cup \mathcal{R} \to \mathbb{N}$. The elements of $\boxed{\mathcal{F}}$ are called **function symbols** and those of $\boxed{\mathcal{R}}$ as **predicate symbols** or **relation symbols**. The map $\boxed{\mathrm{ar}}$ assigns to each symbols its **arity**, which is the number of argument the symbol expects. We write $\boxed{\mathcal{F}_n = \{f \in \mathcal{F} \mid \mathrm{ar}(f) = n\}}$ and $\boxed{\mathcal{R}_n = \{P \in \mathcal{R} \mid \mathrm{ar}(P) = n\}}$. The ele-

ments of $\mathcal{F}_0$ are called **constants**, and we call the elements of $\mathcal{F}_n$ and $\mathcal{R}_n$, respectively **n-ary function symbols** and **n-ary predicate symbols**.

Since ar is a map, every symbol has a unique arity. Thus $\mathcal{F}$ (and $\mathcal{R}$) is **partitioned** into the sets $\mathcal{F}_n$, that is, $\mathcal{F} = \bigcup_{n\in\mathbb{N}} \mathcal{F}_n$ and $\mathcal{F}_n \cap \mathcal{F}_m = \emptyset$ for $m \neq n$.

Let us illustrate this definition on the routing problem.

**Example 4.10**

The signature $\mathcal{L} = (\mathcal{F}, \mathcal{R}, \mathrm{ar})$ of all symbols that we used in section 4.1 is given as follows.

$$\mathcal{F} = \{s, \underline{0}, e\} \qquad \mathcal{R} = \{R, G, O, P, N, \mathrm{Step}\}$$
$$\mathrm{ar}(\underline{0}) = 0 \qquad \mathrm{ar}(e) = 1 \qquad \mathrm{ar}(s) = 2$$
$$\mathrm{ar}(R) = \mathrm{ar}(G) = \mathrm{ar}(O) = \mathrm{ar}(P) = \mathrm{ar}(N) = 1 \qquad \mathrm{ar}(\mathrm{Step}) = 3$$

Thus, this signature has 3 function, 6 predicate symbols, and $\underline{0}$ as the only constant. The partitions of the signature are given by

$$\mathcal{F}_0 = \{\underline{0}\} \qquad \mathcal{F}_1 = \{e\} \qquad \mathcal{F}_2 = \{s\} \qquad \mathcal{F}_3 = \emptyset$$
$$\mathcal{R}_0 = \emptyset \qquad \mathcal{R}_1 = \{R, G, O, P, N\} \qquad \mathcal{R}_2 = \emptyset \qquad \mathcal{R}_3 = \{\mathrm{Step}\}$$

and for $k > 3$ by $\mathcal{F}_k = \mathcal{R}_k = \emptyset$.

A typical use of first-order logic is to express relations and operations on data. This approach was pioneered by Codd [Cod72; Cod83] in order provide a technology-independent account of relational databases.

**Example 4.11: Databases**

Suppose we wish to store data in a relational database. Such a database presupposes are so-called *scheme*, which is nothing but a finite signature that contains only predicate symbols. For example, in an enrolment database of a university, we may store data about students, courses, exams and enrolments via four data predicates

$$\mathcal{R}_d = \{\mathrm{dStudent}, \mathrm{dCourse}, \mathrm{dExam}, \mathrm{dEnrolment}\}$$

with the following arities.

$$\mathrm{ar}(\mathrm{dStudent}) = \mathrm{ar}(\mathrm{dExam}) = 2$$
$$\mathrm{ar}(\mathrm{dCourse}) = \mathrm{ar}(\mathrm{dEnrolment}) = 3$$

The intent is that these will express the relations

- $\mathrm{dStudent}(i, n)$ — student with key $i$ and name $n$,
- $\mathrm{dExam}(i, j)$ — exam with key $i$ for course with key $j$,
- $\mathrm{dCourse}(i, n, t)$ — course with key $i$, name $n$ and in term $t$, and
- $\mathrm{dEnrolment}(i, s, e)$ — enrolment with key $i$ of student $s$ for exam $e$,

which correspond to the relationships displayed in the following diagram.

$$\boxed{\text{Student}} \longleftarrow \boxed{\text{Enrolment}} \longrightarrow \boxed{\text{Exam}} \longrightarrow \boxed{\text{Course}}$$

Besides the data relations, databases often also have type constraints, data constraints or functions that can be used to manipulate data. For example, we can take as typing predicates

$$\mathcal{R}_t = \{\mathrm{Key}, \mathrm{String}, \mathrm{Year}, \mathrm{Eq}\}$$

with Key, String and Year of arity 1 and the intent that $\mathrm{Key}(i)$ expresses that $i$ is a key, $\mathrm{String}(n)$ that $n$ is a string and $\mathrm{Year}(y)$ that $y$ is an (academic) year. The predicate symbol Eq stands for equality and has arity 2. In order to manipulate keys, we can use two function symbols

$$\mathcal{F}_{\mathrm{db}} = \{\underline{0}, \mathrm{succ}\}$$

of arity 0 and 1, respectively. The symbol $\underline{0}$ represents the zero key and $\mathrm{succ}(n)$ is the successor key of $n$. We put all predicate symbols into $\mathcal{R}_{\mathrm{db}}$ with $\mathcal{R}_{\mathrm{db}} = \mathcal{R}_d \cup \mathcal{R}_t$, and thus obtain our databes signature $\mathcal{L}_{\mathrm{db}}$ with

$$\mathcal{L}_{\mathrm{db}} = (\mathcal{R}_{\mathrm{db}}, \mathcal{F}_{\mathrm{db}}, \mathrm{ar})\,.$$

The signature from example 4.10 has a finitely many symbols, but this is not necessary and definition 4.9 allows for arbitrarily large signatures. For instance, $\{\underline{n} \mid n \in \mathbb{N}\}$ is a perfectly valid set of function symbols. However, for practical use, it is usually sufficient to only use finitely many symbols. For instance, we can represent natural numbers over a signature with finitely many function symbols, but for that we need to first introduce terms.

**Definition 4.12**

Let $\mathcal{L}$ be a signature $(\mathcal{F}, \mathcal{R}, \mathrm{ar})$ and $\boxed{\mathrm{Var}}$ a (countable) set of object variables. The set $\boxed{\mathrm{Term}}$ or $\boxed{\mathrm{Term}(\mathcal{L})}$ of **terms** or **$\mathcal{L}$-terms**, is the smallest set closed under the following three rules.

$$\frac{x \in \mathrm{Var}}{x \in \mathrm{Term}} \qquad \frac{c \in \mathcal{F}_0}{c \in \mathrm{Term}} \qquad \frac{f \in \mathcal{F}_n \quad t_1 \in \mathrm{Term} \quad \cdots \quad t_n \in \mathrm{Term}}{f(t_1, \dots t_n) \in \mathrm{Term}}$$

Let us walk through this definition. First, we may approximately think of terms as given by a context-free grammar:

$$t ::= x \mid c \mid f(t, \dots, t)$$

The problem with this grammar is that it does not take the arity of the symbol $f$ into account. For instance, if you take the symbol $e$ from example 4.10, then this grammar would allow you to write $e(\underline{0}, e(\underline{0}))$. This clearly does not make a lot of sense, as $e$ is supposed to be a function symbol with one argument and not one or two! In definition 4.12, we used instead the three rules to indicate exactly the arity of the function symbols. For instance, we can say $\underline{0} \in \text{Term}$ by the second rule and therefore $e(\underline{0}) \in \text{Term}$ by the third rule. However, $e(\underline{0}, e(\underline{0})) \in \text{Term}$ does not follow from those rules because $e \notin \mathcal{F}_2$. Therefore, $e(\underline{0}, e(\underline{0}))$ is not a valid term over this signature.

How can we be sure that the set of terms does not contain other things besides what we can do with rules? The important word is "smallest", which forbids this! We will put aside the intricacies of such inductive definitions until chapter 9, and just postulate that there is nothing unexpected in Term. This also helps us to make definitions involving terms.

Let us define a map that computes all variables in a term. Recall that $\mathcal{P}(\text{Var})$ is the powerset of Var, given by $\mathcal{P}(\text{Var}) = \{U \mid U \subseteq \text{Var}\}$.

**Lemma 4.13**

Let $\mathcal{L}$ be a signature. There is a map $\boxed{\text{var}\colon \text{Term}(\mathcal{L}) \to \mathcal{P}(\text{Var})}$ with

$$\text{var}(x) = \{x\} \qquad \text{var}(c) = \emptyset \qquad \text{var}(f(t_1, \dots, t_n)) = \bigcup_{k=1}^{n} \text{var}(t_k)$$

that computes the **term variables** that appear in $\mathcal{L}$-terms.

*Proof.* Since $\text{Term}(\mathcal{L})$ contains only the terms given by the three rules in definition 4.12, the three equations cover all possibilities. It is clear that the cases for variables and constants are well-defined. In the last case, the terms $t_k$ are all structurally smaller than $f(t_1, \dots, t_n)$ because one symbol has been removed. Moreover, because $\text{Term}(\mathcal{L})$ is the smallest set closed under the rules, it has only terms of finite depths and the descent of var into the terms $t_k$ must reach eventually either the variable or constant case. At that point the computation stops and var is well-defined.[6] □

We argued somewhat informally here and have to leave the rigorous proof for

later when we talk about iteration over trees. Let us, for now, evaluate the map on a few examples.

**Example 4.14**

With the signature $\mathcal{L}$ from example 4.10 and $r, x \in \mathrm{Var}$, we have that $s(r, \underline{0})$ and $s(r, e(s))$ are $\mathcal{L}$-terms. The map lemma 4.13 allows us to compute the variables in those terms:

$$\begin{aligned} \mathrm{var}(s(r,\underline{0})) &= \mathrm{var}(r) \cup \mathrm{var}(\underline{0}) \\ &= \{r\} \cup \emptyset \\ &= \{r\} \end{aligned} \qquad \begin{aligned} \mathrm{var}(s(r,e(x))) &= \mathrm{var}(r) \cup \mathrm{var}(e(x)) \\ &= \{r\} \cup \mathrm{var}(x) \\ &= \{r\} \cup \{x\} \\ &= \{r, x\} \end{aligned}$$

Note that there are terms that may not have any sensible meaning, given the intuition we assigned to the symbols. For instance, $e(s(r, \underline{0}))$ is a valid term, but what is "the last index in the position at the beginning of a path"?

Let us now come back to example 4.14. We saw at the end that there are terms that we may not care about or that we consider as "non-sense". This shows that terms should really be only thought of as providing a way of representing objects, or rather shapes of objects, that we may want to reason about. And for that, we will need to talk about formulas in first-order logic. There is a slight complication with variables in FOL, but you will have to wait until the next chapter of your journey to see this issue come up. For the moment, we make only a first attempt.

**Definition 4.15: A first attempt to define FOL formulas**

Let $\mathcal{L}$ be a signature with predicate symbols $\mathcal{R}$. The set $\boxed{\mathrm{Form}}$ or $\boxed{\mathrm{Form}(\mathcal{L})}$ of **first-order formula** or **$\mathcal{L}$-formulas** is the smallest set closed under the following rules.

$$\frac{P \in \mathcal{R}_n \qquad t_1 \in \mathrm{Term} \qquad \cdots \qquad t_n \in \mathrm{Term}}{P(t_1, \dots t_n) \in \mathrm{Form}} \qquad \frac{\varphi \in \mathrm{Form}}{(\varphi) \in \mathrm{Form}}$$

$$\frac{}{\bot \in \mathrm{Form}} \qquad \frac{\varphi \in \mathrm{Form} \qquad \psi \in \mathrm{Form} \qquad \square \in \{\wedge, \vee, \rightarrow\}}{\varphi \,\square\, \psi \in \mathrm{Form}}$$

$$\frac{x \in \mathrm{Var} \qquad \varphi \in \mathrm{Form}}{\forall x.\, \varphi \in \mathrm{Form}} \qquad \frac{x \in \mathrm{Var} \qquad \varphi \in \mathrm{Form}}{\exists x.\, \varphi \in \mathrm{Form}}$$

The binding precedences extend that of propositional logic:

- $\wedge$, $\vee$ and $\rightarrow$ associate to the right,
- $\wedge$ and $\vee$ bind stronger than $\rightarrow$, and
- $\forall x.$ and $\exists x.$ extend to the right as far as possible.

Let us see some examples of formulas and applications of the precedences.

**Example 4.16**

All the formulas from eqs. (4.3), (4.5), (4.6) and (4.8) are all formulas over the signature from example 4.10.

For the precedences, table 1.2 still applies to propositional connectives, but we additionally have rules for quantifiers, see table 4.1. The rule that quantifiers

| Formula | With parentheses |
|---|---|
| $\forall x.\, R(x) \rightarrow O(x)$ | $\forall x.\, (R(x) \rightarrow O(x))$ |
| $\forall x.\, \exists y.\, R(x) \rightarrow O(y)$ | $\forall x.\, (\exists y.\, (R(x) \rightarrow O(y)))$ |
| $\forall x.\, R(x) \rightarrow \exists y.\, O(y)$ | $\forall x.\, (R(x) \rightarrow (\exists y.\, O(y)))$ |

Table 4.1.: Leaving out parentheses in formulas with quantifiers

extend as far as possible to the right implies that we have to use parentheses if a quantifier is under a logical connective. For instance, the formulas

$$\forall x.\, R(x) \rightarrow O(x) \qquad \text{and} \qquad (\forall x.\, R(x)) \rightarrow O(x) \tag{4.17}$$

are different and also express very different properties.

**Example 4.18: Database Constraints**

Recall that we introduced in example 4.11 a signature for modelling a database with exam enrolments, where entries were given keys to relate them. In the so-called normalisation of database relationships to tables, one distinguishes between primary and foreign keys [Cod83]. A primary key should uniquely identify an entry (row) in a table, while a foreign key refers to an entry in another table. We can use formulas to express such constraints and thereby give a logical description of how a database must look like at any time. For instance, the following formula expresses that the key $s$ in the enrolment table is a foreign key that refers to a student.

$$\forall i.\, \forall s.\, \forall e.\, \mathrm{dEnrolment}(i, s, e) \rightarrow \exists n.\, \mathrm{dStudent}(s, n)$$

In order to express that primary keys are unique, we can use the Eq predic-

ate symbol and compare entries for equality.

$$\forall s.\, \forall n.\, \forall m.\, \mathrm{dStudent}(s, n) \wedge \mathrm{dStudent}(s, m) \rightarrow \mathrm{Eq}(n, m)$$

At this stage, it is of course not guaranteed that Eq ensures the equality of its arguments. We could try to provide formulas that provide this guarantee, but that is a somewhat delicate issue and we leave that for chapter 8.

Now that we can express constraints on databases, it is natural to ask if we can also formulate queries. For this, we first need to better understand variables.

**Example 4.19**

Let $\mathcal{L}$ be a signature with two **unary** (1-ary) predicate symbols $R$ and $W$, that is, $\mathcal{L} = (\emptyset, \mathcal{R}, \mathrm{ar})$ with $\mathcal{R} = \{R, W\}$ and $\mathrm{ar}(R) = \mathrm{ar}(W) = 1$. We read $R(x)$ as "it rains at location $x$" and $W(x)$ as "the floor is wet at location $x$". The formula $\forall x.\, R(x) \rightarrow W(x)$ expresses our expectation that the floor gets wet wherever it rains. However, what does $(\forall x.\, R(x)) \rightarrow W(x)$ express? We can deconstruct the formula and read it as: "Whenever it rains everywhere, then at a *fixed* location $x$ it is wet". In anticipation of the coming chapter we can rewrite the formula to $(\forall y.\, R(y)) \rightarrow W(x)$, which clearly shows that $x$ is not affected by the quantifier. This formula is intuitively correct but we will probably never use it, because it is unlikely that it rains everywhere at the same time, even when restricted to our planet! Let us now replace the universal quantifier by an existential quantifier and the implication by a conjunction. On the one hand, we can read again easily the formula $\exists x.\, R(x) \wedge W(x)$ as "there is a location where it rains and the floor is wet". This is quite likely to be true, but to prove that we would need to know for sure that it rains somewhere. On the other hand, the formula $(\exists x.\, R(x)) \wedge W(x)$ states that there is a location where it rains and that at a fixed location $x$ the floor is wet. Also this may be true, but we cannot know for sure. Moreover, to prove this formula we need to know that the floor is wet at a particular location. For instance, if $x$ is the floor in our living room then we may that $W(x)$ does usually not hold.

Thus, we need to distinguish between variables referred to by a quantifier, so-called **bound variables**, and variable that occur freely in a formula, that is, variables that do not occur under a quantifier.

**Definition 4.20**

For a signature $\mathcal{L}$, we define a map $\boxed{\mathrm{fv}(\varphi)}$ with $\mathrm{fv}\colon \mathrm{Form}(\mathcal{L}) \to \mathcal{P}(\mathrm{Var})$ by

$$\begin{aligned}
\mathrm{fv}(P(t_1, \dots, t_n)) &= \bigcup_{k=1}^{n} \mathrm{var}(t_k) \\
\mathrm{fv}(\bot) &= \emptyset \\
\mathrm{fv}(\varphi \,\square\, \psi) &= \mathrm{fv}(\varphi) \cup \mathrm{fv}(\psi) \\
\mathrm{fv}(\forall x.\, \varphi) &= \mathrm{fv}(\varphi) \setminus \{x\} \\
\mathrm{fv}(\exists x.\, \varphi) &= \mathrm{fv}(\varphi) \setminus \{x\}
\end{aligned}$$

where "$\setminus$" denotes set difference and $\square \in \{\wedge, \vee, \to\}$. We say that a variable $x$ is **free** in $\varphi$, if $x \in \mathrm{fv}(\varphi)$.

Let us look at an example to understand free variables better.

**Example 4.21**

Suppose we have a signature $\mathcal{L}$ with unary predicate symbols (one argument) $P$ and $Q$, and that $\varphi = (\forall x.\, P(x) \wedge Q(y)) \to P(x) \vee Q(y)$. This formula has both $x$ and $y$ as free variables, but one has to be careful that $x$ is not free in all subformulas:

$$\underbrace{(\forall x.\, P(x) \wedge Q(y))}_{y \text{ free}} \to \underbrace{P(x) \vee Q(y)}_{x, y \text{ free}}$$

Formally, we have

$$\begin{aligned}
\mathrm{fv}(\varphi) &= \mathrm{fv}(\forall x.\, P(x) \wedge Q(y)) \cup \mathrm{fv}(P(x) \vee Q(y)) \\
&= (\mathrm{fv}(P(x) \wedge Q(y)) \setminus \{x\}) \cup \mathrm{fv}(P(x)) \cup \mathrm{fv}(Q(y)) \\
&= (\mathrm{fv}(P(x)) \cup \mathrm{fv}(Q(y)) \setminus \{x\}) \cup \{x\} \cup \{y\} \\
&= (\{x, y\} \setminus \{x\}) \cup \{x, y\} \\
&= \{y\} \cup \{x, y\} \\
&= \{x, y\}
\end{aligned}$$

but

$$\mathrm{fv}(\forall x.\, P(x) \wedge Q(y)) = (\mathrm{fv}(P(x) \wedge Q(y)) \setminus \{x\}) = \{y\}\,.$$

Example 4.21 shows how free variables for a formula can be computed, and how quantifiers affect whether a variable is free. This can be confusing at first sight, but the intuition is that the $x$ in $P(x)$ without the quantifier refers to a specific object, while in $\forall x.\, P(x)$ it ranges over all possible objects that we

may fill in for $x$. One may ask whether the formulas could still mean the same thing. However, we should keep in mind that we are talking about syntax for the time being, which has no intrinsic meaning! That being said, you can read the formula $\forall x.\, R(x) \to O(x)$ as "$R(x)$ implies $O(x)$ for all $x$". Compare this to the reading of the formula $(\forall x.\, R(x)) \to O(x)$ as "if $R(y)$ holds for all $y$, then $O(x)$ holds for whatever $x$ is".

A somewhat delicate part of first-order logic, which we will resolve at this stage, is the handling of names. Note that we changed the name of the first $x$ in the second formula to a $y$, which, of course, enables us to read them differently. We think of it this way: the formula $\forall x.\, R(x)$ and $\forall y.\, R(y)$ should express the same thing, namely that $R$ holds for all objects. Therefore, we could renamed $x$ to $y$. This resolves the naming conflict and we can clearly distinguish the two formulas. But this also shows that definition 4.15 is missing an important aspect of variable binding that we will carefully investigate in chapter 5.

Let us illustrate the distinction between free and bound variables on the formulation of database queries.

**Example 4.22: Database queries as formulas**

In relational databases, one uses the so-called join operation to combine tables along keys. The join operation can be expressed using existential quantifiers. For example, a student with name $n$, who is enrolled in an exam for the course $c$ in year $y$ would fulfil the following formula $\varphi_{\text{query}}$.

$$\begin{aligned}\exists i.\, \exists s.\, \exists j.\, \exists k.\, \text{dEnrolment}(i, s, k) \land \text{dExam}(k, j)\\ \land\, \text{dStudent}(s, n) \land \text{dCourse}(j, c, y)\end{aligned}$$

Note that the existential quantifier in $\varphi_{\text{query}}$ for $j$ and $k$ and link the primary and foreign keys, while that for $i$ merely hides the primary key of enrolments. Since $n$, $c$ and $y$ are variables that are not bound by a quantifier, we have $\text{fv}(\varphi_{\text{query}}) = \{n, c, y\}$. We may think of filling in values for these variables and thereby obtain concrete entries in a possible database. There are principally two ways of doing this: either one uses semantics models and then obtains entries via the semantics of formulas, something that we will see in chapter 10; or one introduces function symbols into the signature in order to model data and then uses formulas to express data entries. For example, we can express that there is an exam with key 0 for a course with key 1 by the following formula.

$$\text{dExam}(\underline{0}, \text{succ}(\underline{0}))$$

In order execute the join operation on a database given in this form, we search for all terms $t_1$, $t_2$ and $t_3$, such that $\varphi_{\text{query}}$ can be derived when $t_1$ is filled for $n$, $t_2$ for $c$ and $t_3$ for $y$. What filling in terms, deriving formulas, and searching for terms and for proofs mean, are the subject of the coming three chapters.

Raghunātha yawns, "The night is setting in and I'm afraid that we have to end our discussion for today. Also, I will not be able to help you further here, as I have to get back to the business in the temple here tomorrow."

Clara slightly bows: "Thank you for your time! This is a lot to take in and we will see how to continue."

Raghunātha nods, "Indeed, but most of what we have discussed is quite straightforward: just remember that first-order predicate logic allows us to reason about objects, where objects are described syntactically as terms, and we use predicate symbols to describe properties of objects. Finally, quantifiers are used to describe universal and existential properties."

"I understand now how computing problems can be formally described, but now I need to know how they can actually be solved!", Isaac exclaims.

"For that, my friend, you will have to continue your study with someone more knowledgeable in computation, my speciality is the debate. I will now go home and take a bit of rest."

Clara and Isaac leave the temple through one of the side arcs. Some children have played with their drawing under the tree. The robot has gotten a heart and left the board, leaving all constraints behind but those scribbled on the side of the board. As the wind has started to blow gently, these became unreadable but for a fragment reading: "...laws of robotics". Birds come out to use the cooler night to find comrades. They are making loud chirping noises, while Clara and Isaac leave the temple area.

Clara is eager to discuss something: "So, Isaac, tell me more about this heart of yours. What is it?"

"I don't know!", Isaac is clearly perplexed. "There is not much in my memory about it, except that I should have it."

Clara is in disbelief: "This would be contrary to all I know about machines, but who knows what the future brings..."

"You are a sceptical mind, Clara. I like that!"

"This runs in the family.", Clara explains, "My grandfather was a scientist and my mother a philosopher."

'"Was"? Do you mean they are no longer...', Isaac is interrupted.

"They are no longer alive, indeed.", Clara's look goes into the distance.

Isaac tries to console Clara. “I am sorry for my rude question, I thought they were no longer working. My condolences! What happened? You are quite young!”

“My grandfather died on his 75th birthday and my mother of cancer when I was a child. Although they are probably the reason that I am here with you.”

Isaac is confused. “What makes you say that?”

“You see, my grandfather taught me maths and natural sciences. I promised to myself to make him proud and follow his footsteps by going into the sciences. Therefore, I chose to study computer science but so far, it has been, let us say, a difficult experience. That said, without my studies I would not be here with you. In fact, I start to enjoy logic, and would like to learn more about its history and its wider use.”

The answer clearly makes an impression on Isaac. “I think you are not only following your grandfather but also your mother’s critical mind. I believe that I have the right person to continue our journey with.”

Clara prepares herself for the trip. “Surprise me!”

# Exercises

## Exercise 12

Let $\mathcal{L}$ be the signature $(\mathcal{F}, \emptyset, \mathrm{ar})$ with $\mathcal{F}_0 = \{a\}$, $\mathcal{F}_1 = \{g\}$ and $\mathcal{F}_2 = \{h\}$. Compute the variables $\mathrm{var}(t)$ of the following $\mathcal{L}$-terms $t$.

a) $g(a)$
b) $h(g(x), a)$
c) $h(f, a)$
d) $h(f, x)$

## Exercise 13

Let $\mathcal{L}$ be the signature $(\mathcal{F}, \emptyset, \mathrm{ar})$ with $\mathcal{F}_0 = \{a\}$, $\mathcal{F}_1 = \{g\}$ and $\mathcal{F}_2 = \{h\}$. Which of the following is a valid term over $\mathcal{L}$?

a) $g(a)$
b) $f(g(a))$
c) $h(f, f)$
d) $f(a)$

## Exercise 14

a) Give the smallest first-order signature $\mathcal{L} = (\mathcal{F}, \mathcal{R}, \mathrm{ar})$, such that following is a formula in $\mathrm{Form}(\mathcal{L})$.

$$\exists y.\, \forall x.\, P(f(m), x) \to Q(g(x, y))$$

b) Is there a first-order signature $\mathcal{L}$, such that the following would be a formula in $\mathrm{Form}(\mathcal{L})$? If not, why?

$$\exists y.\, \forall x.\, P(f(m), y) \land P(x)$$

## Exercise 15

Suppose we have a signature with a constant $z$, a unary function symbol $s$, and unary predicate symbols $N$, $E$, $O$ and $P$, that is, $\mathrm{ar}(z) = 0$ and $\mathrm{ar}(s) = \mathrm{ar}(N) = \mathrm{ar}(E) = \mathrm{ar}(O) = \mathrm{ar}(P) = 1$. The intended meaning of $z$ is that it

represents the number zero, of $s$ that it represents the successor of a number, of $N(x)$ is that $x$ is a natural number, of $E(x)$ that $x$ is an even number, of $O(x)$ that x is an odd number, and of $P(x)$ that x is a prime number. For instance, we would think of the terms $z$, $s(z)$ and $s(s(z))$ as the numbers 0, 1 and 2, respectively.

Using only the above function and predicate symbols, translate the following sentences into predicate logic:

a) Not all prime natural numbers are odd.

b) Two is an even prime number.

c) Every natural number is an even or odd number.

d) There is no natural number which is both even and odd.

## Exercise 16

In example 4.19 you will have noticed that the state of rain and humidity may change over time. Give a signature $\mathcal{L}$ that allows you to express the following sentence and then write that sentence as a formula.

> If it rains over a location, then some time $d$ later the ground gets wet at that location.

## Exercise 17

Compute the free variables $\mathrm{fv}(\varphi)$ for each of the following formulas $\varphi$.

a) $\forall x.\, P(y)$

b) $\forall x.\, P(x)$

c) $\exists y.\, P(x) \land \forall x.\, P(y)$

d) $(\forall x.\, P(y)) \land \exists y.\, Q(y) \lor R(x)$

# 5. Proof Theory of First-Order Predicate Logic

Clara and Isaac enter a lecture hall with about 300 persons sitting on wooden chairs arranged in tiered rows. The floor is covered with thick velvet carpet, muffling the sound in the room. Dim light allows one to only see a few faces in the front rows and on the outside aisles. The only real light is shone on the podium in the front of the hall, highlighting a woman of strong stature with an intent gaze.

She is speaking passionately: '...There may be a clear exposition, or a clear argument, or a clear speech. But there is not a clear piece of thinking, or a clear thinking, but only thinking clearly. It is always an "I" that thinks. My thinking is determined by the sort of person I am, and the sort of person I am becoming is in part determined by how I am thinking...'[7]

"Where are we?", Clara is surprised by what she hears.

'We are at the 1936 conference of the British Institute for Adult Education, and this is Susan Stebbing giving a lecture called "Thinking". Her book *A Modern Introduction to Logic*[8] was among the standard introductory texts in logic for a long time, although her passion and focus in later life shifted to critical thinking. Logic had to be in service of thinking, but could not replace it and should never limit it according to Stebbing.', Isaac explains.

Clara and Isaac slip into an empty row in the back and sit down.

"So tell me a bit more about your mother. How was she?", Isaac asks.

"Her name was Lisbeta Songya, but people often just called her Lis. She did not like that and later only used Songya, and on publications only her initials.", Clara is trying to compose herself while talking about her mother.

"Why did she do that?", Isaac seems confused, "Most people would at least use one first name on their publications for easier identification."

"Quite simple: there is still a gender bias in academia, and especially in philosophy.", Clara explains.

'Interesting, do you know that our lecturer here also just authored papers as "L.S. Stebbing" for the same reason, despite all the recognition that she may

have received?', Isaac appears almost proud about this insight.

"I guess that is why you brought me here?", Clara raises an eye-brow.

"Not just, but it may help."

"Be that as it may. My mother was born in a calm town built around a university, seamed by beautiful woods and with fresh air. When she was six, her parents decided to move to a big city in a foreign country for their work. My mother never liked the city, but she stayed after growing up and even went to university there. During her studies, she met my father. They got along well but only fell in love during a trip that was organised by the students of that year to celebrate their graduation. My father was from that city and so they decided to stay there, while both pursued an academic career. For him, working in academia was fairly natural but my mother had to fight with gender-bias, although she eventually became the chair of the institute and received a professor's title. This was around the time when I turned six." Clara takes a deep breath. "My mother had always suffered from migraine but after her promotion, the attacks got stronger and longer. After a few weeks, my parents decided to move to the country-side, where my mother could take rest during a break of some months. After one month, my mother had such a strong attack that she got hospitalised. At the hospital, the doctors found that she had suffered from a stroke. As nice as the country-side is, the way to the hospital was long and they could not help her any more. She was declared brain-dead the next day and we decided to turn off the life support."

Even Isaac seems to be moved. "And then?"

"After that, my grandfather took over my custody and I lived with him outside of the city. He raised me with great love and care. He taught me maths, music, logic and introduced me to the wonders of nature. On one month before his birthday, he visited his doctor with pain in the belly. The doctor sent him to the hospital for a check, where was diagnosed with bowel cancer. After that, he rapidly lost weight and eventually couldn't leave the bed any more. Together with my aunt, we took care of him. A month later, exactly on his 75th birthday, he passed away peacefully in a morphine induced sleep.", tears are now running from Clara's face.

"I am so sorry to hear this! How old were you?"

"I was 18. After the funeral, I left my grandfather's, our, house and had to make a living on my own. I first moved back to the city and did some odd jobs, but I did not like it and I always heard my grandfather's voice telling me to follow my curiosity. Eventually, I decided to make a clean break and enrolled in a university abroad. The city of that university was, to my relief, much smaller, but being alone in another country took its toll on me. But I hope

that grandpa is proudly watching that I am following his advice.", Clara's look moves to a point in the far distance after she finished her sentence.

"What happened to your father, could...", before Isaac finishes the question, Stebbing's voice is becoming louder and stronger, as if for a final cadence.

'...These devices would not be as successful as they undoubtedly are were not we so frequently tempted to be lazy in thinking. We are too content to have our opinions thrust upon us instead of eliciting them by the effective opposition and careful consideration of possible views. Just as the advertiser seeks to form our beliefs and save us from the trouble of thinking, so that we may be ready to buy his wares, so the newspaper editors desire to furnish our minds with opinions of which they approve. This is, I believe, true of nearly all newspapers. They seek to persuade, not to convince. It is our fault if we are too lazy to be critical.'[9], She finishes her speech and applause sets in.

Isaac jumps up and pulls Clara with him, "Quickly, let us try to catch Stebbing before we miss our chance to talk to her!"

Clara and Isaac haste down the aisle and reach Stebbing just before the door.

Isaac approaches her: "Professor Stebbing, could we have a word?"

"Of course, albeit not very long because I feel exhausted after this event. What did you think of the lecture?"

"Wonderful!", Clara exclaims, "I enjoyed the broad perspective that you provide, and how philosophy and logic meet in with daily life to scrutinise false believes and propaganda. The concept of *potted thinking* is a beautiful metaphor."

Stebbing smiles, "Oh thank you! But please let us sit down." The three move into one of the rows and each take a chair.

Clara and Isaac take out the notes that they have taken throughout their journey, and Isaac starts explaining what they have learned. "... And now we wish to understand how to derive anything in first-order logic."

Stebbing skims once more through the notes, stops at one page and looks up, "Whoever explained this to you was nice to keep things simple, but there is something fundamentally wrong with definition 4.15 that has to be rectified before we can talk about deduction for first-order logic."

## 5.1. The Difficulty of Names and Variables

The syntax of first-order logic can hold some surprises, if we are not careful. You have seen (example 4.19) previously the two formulas

$$\forall x.\, R(x) \to W(x) \quad \text{and} \quad (\forall x.\, R(x)) \to W(x)$$

that had a different reading. The main difference between the two is that the universal quantifier in the first formula ranges over the whole subformula $R(x) \to W(x)$, while it applies in the second formula only to $\forall x.\, R(x)$ and the second $x$ is free. However, because the variable is called "$x$" everywhere, this difference is difficult to spot.

Let us briefly analyse the different occurrences of the variable in the second formula. In the subformula $\forall x.\, R(x)$, the variable $x$ is **bound** by the quantifier. Bound variables are placeholders to be filled in by concrete objects. This means that, if $\forall x.\, R(x)$ holds, then $R(\text{Leiden})$, $R(\text{Bagdad})$ and any other application of $R$ to objects should hold. Therefore, we are led to expect that the *concrete names of bound variables are irrelevant.*

What about the free occurrence of $x$ in the subformula $W(x)$? Here $x$ cannot be filled in by an arbitrary object because, even though $x$ is undetermined, it stands for an arbitrary but *fixed* object. For instance, if we know that $W(x)$ we cannot assume that $x$ refers to Leiden, as we may eventually learn that it did refer to Bagdad all along. The variable $x$ is here not in the **scope** of any quantifier and must therefore be treated as an abstract but fixed object. Since $x$ is abstract, we cannot use any information about $x$ in our proofs. In fact, anyone, who is using our proofs, is entitled to fill in whatever they desire for the free variable $x$, a process that is called **substitution** and is central to the proof rules for quantifiers. Gottlob Frege understood all of this already.[10], but let us retrace the issues of name identity and scoping in some detail.

**Formula Equality and Renaming** The first issue consists in finding an answer to the question: Are the formulas $\forall x.\, R(x)$ and $\forall y.\, R(y)$ expressing the same? To answer it, let us read the formulas without explicit names:

"$R$ holds for all objects".

Clearly, this sentence corresponds to both formulas $\forall x.\, R(x)$ and $\forall y.\, R(y)$, and we should consider both formulas to be the same, already *syntactically*! This gives us a first rule that we will have to adhere to for FOL:

> Two formulas are considered to be the same, if we can *bijectively rename* the variables of one formula to obtain the other.

For instance, consider the formulas $\varphi$ and $\psi$ given by $\varphi = \forall x.\, \forall y.\, Q(x, y)$ and $\psi = \forall w.\, \forall z.\, Q(w, z)$. Then, a bijective renaming would be to rename $x$ to $w$ and $y$ to $z$, which allows us to transform $\varphi$ into $\psi$. However, renaming $x$ to $r$ and $y$ to $r$ is not bijective. Thus, we do not consider the formula $\forall r.\, \forall r.\, Q(r, r)$

to be same as $\varphi$. Note, that in the latter formula, the $r$ refers to the inner-most quantifier and the outer quantifier has no effect!

**Scoping** The second issue concerns the scope of variables and it requires us to determine which objects a variable refers to. For instance, the variable $x$ in the sub-formula $R(x)$ of $\forall x.\, R(x)$ refers to what the quantifier ranges over. We say that $x$ is *in the scope* of $\forall x$ in this formula. However, the variable $y$ in $\forall x.\, Q(x, y)$ is in the scope of no quantifier and is thus a global reference. If we naively substituted $x$ for $y$ in this formula, then we would obtain $\forall x.\, Q(x, x)$. Here, the scope of the second argument of $Q$, and with it the meaning of the formula, has suddenly changed. This leads to a second rule:

> Substituting a term in a formula should not change variable scoping.

## 5.2. Axiomatising Terms and Substitutions

There are several ways to deal with the difficulties resulting from the binding of variables and substitution. We will take the most direct route and just declare two formulas to be the same if they only differ by the names of bound variables. To achieve this, we will devise axioms for the equality of formulas that allow us to safely carry out renaming and substitution.

We begin by defining substitutions and the application of terms to them.

**Definition 5.1**

A **substitution** is a map $\sigma\colon \mathrm{Var} \to \mathrm{Term}$. Given $t \in \mathrm{Term}$ and $x \in \mathrm{Var}$, we write $\boxed{\sigma[x := t]}$ for the **update** of $\sigma$ defined by

$$\boxed{(\sigma[x := t])(y) = \begin{cases} t, & x = y \\ \sigma(y), & x \neq y \end{cases}}$$

The **identity substitution** $\eta\colon \mathrm{Var} \to \mathrm{Term}$ is given by $\eta(y) = y$. For convenience, we abbreviate $\eta[x := t]$ to $[x := t]$. Given a term $t$, we define

the **application** of the term $t$, written $\boxed{t\,\sigma}$, to the substitution $\sigma$ as follows.

$$\begin{aligned} x\,\sigma &= \sigma(x) \\ c\,\sigma &= c \\ f(t_1, \dots, t_n)\,\sigma &= f(t_1\,\sigma, \dots, t_n\,\sigma) \end{aligned}$$

The notation $\sigma[x := t]$ to update a substitution $\sigma$ can be read like an assignment in an imperative programming language: the term that $\sigma$ assigned to $x$ will be overwritten by $t$. Similarly, the notation $[x := t]$ starts with a storage in which all variables $y$ have the default value $y$, except for $x$ which gets $t$ assigned as initial value.

?

Can you make sense of the name *identity substitution*? What is the result of applying a term to $\eta$?

The following example illustrates these definitions.

**Example 5.2**

Let $g(x, y)$ be a term with two free variables $x$ and $y$, and one binary function symbol $g$. Moreover, let $\sigma = [x := y][y := x]$.

1. The substitution $\sigma$ exchanges the two variables:

$$g(x, y)\,\sigma = g(x\,\sigma, y\,\sigma) = g(\sigma(x), \sigma(y)) = g(y, x)$$

   This example shows that the substitution $[x := y][y := x]$ is *not* the same as $\eta$. In fact, $\sigma$ is substituting for $x$ and $y$ *in parallel* and not in sequence. Therefore, we also cannot obtain $\sigma$ by repeated application:

$$g(x, y)\,[x := y][y := x] \neq (g(x, y)\,[x := y])\,[y := x] = g(x, x)$$

2. Let now $\tau = \sigma[y := f(x)]$. In $\tau$, the assignment of $x$ to $y$ is overwritten with the assignment of $f(x)$ to $y$. We thus have the following.

$$g(x, y)\,\tau = g(x\,\tau, y\,\tau) = g(y, f(x))$$

Next, we need to be able to apply substitutions to formulas. To circumvent the

difficulties described in section 5.1, we *assume* additionally to the closure rules in definition 4.15 that the equality of formulas fulfils additional *axioms*.[11]

**Definition 5.3: Extension of definition 4.15**

Given a formula $\varphi$, a variable $x$ and a substitution $\sigma$, we say that $x$ is **fresh** for $\varphi$, written $\boxed{x \# \varphi}$, if $x \notin \mathrm{fv}(\varphi)$. Similarly, we say that $x$ is **fresh for $\sigma$**, written $\boxed{x \# \sigma}$, if

$$x \notin \mathrm{var}(\sigma(y)) \text{ for all } y \in \mathrm{Var} \setminus \{x\}\,.$$

Additionally to the closure rules in definition 4.15, we assume that a formula $\varphi$ can be **applied to a substitution $\sigma$**, written $\boxed{\varphi\,\sigma}$. Further, we assume for all $\Diamond \in \{\forall, \exists\}$ and $\Box \in \{\wedge, \vee, \rightarrow\}$ that following six equations hold.

$$\boxed{\begin{aligned}
\varphi \,\Box\, \psi &= \varphi' \,\Box\, \psi' \text{ iff } \varphi = \varphi' \text{ and } \psi = \psi' && \text{(EC)}\\
\Diamond x.\, \varphi &= \Diamond y.\, \psi \text{ iff } y \# \varphi \text{ and } \psi = \varphi[x := y] && \text{(EQ)}\\
\bot\,\sigma &= \bot && \text{(SB)}\\
P(t_1, \dots, t_n)\,\sigma &= P(t_1\,\sigma, \dots, t_n\,\sigma) && \text{(SP)}\\
(\varphi \,\Box\, \psi)\,\sigma &= \varphi\,\sigma \,\Box\, \psi\,\sigma && \text{(SC)}\\
(\Diamond x.\, \varphi)\,\sigma &= \Diamond x.\, \varphi\,\sigma[x := x] \text{ if } x \# \sigma && \text{(SQ)}
\end{aligned}}$$

?

Do you know why we need to exclude in the definition of $x \# \sigma$ the variable $x$ from the quantification?

Let us explain the intuition behind these axioms. First of all, there are two groups of axioms: those for the equality on formulas (starting with E) and those that define the action of substitution on formulas (starting with S). The axiom (EQ) allows us to bijectively rename bound variables without illegally binding other variables. For instance, we have

$$\forall x.\, Q(x, y) = \forall z.\, Q(z, y)$$

because $z$ is fresh for $Q(x, y)$, that is $z \notin \mathrm{fv}(Q(x, y))$. However, we have

$$\forall x.\, Q(x, y) \neq \forall y.\, Q(y, y)$$

because $y \in \mathrm{fv}(Q(x,y))$. The axiom (EC) allows us to carry out this renaming also in complex formulas that involve other connectives than quantifiers. Implicitly, we also use equality on terms and atoms, in the sense that

$$P(t_1, \dots, t_n) = P'(t'_1, \dots, t'_n) \text{ iff } P = P' \text{ and } t_k = t'_k \text{ for all } 1 \leq k \leq n$$

and that equality is an equivalence relation (reflexive, symmetric, transitive).

The second group of axioms describes how substitution can be computed iteratively. Axioms (SB), (SP) and (SC) are doing what we would expect: no action on the atom $\bot$, reduce substitution on predicates to substitution in terms, and distribute substitution over propositional connectives. Complications arise only in the axiom (SQ), which has to make sure that the *use of a bound variable is not changed* and that *variables are not accidentally bound.* On the one hand, that the use of a bound variable is not changed is achieved by updating the substitution $\sigma$ to $\sigma[x := x]$. For instance, if $\sigma(x) = g(y)$, then naively carrying out this substitution on $\forall x.\, P(x)$ would lead to $\forall x.\, P(g(y))$, which is certainly not what we want! Instead, we have by (SQ) and (SP)

$$(\forall x.\, P(x))\, \sigma = \forall x.\, P(x)\, \sigma[x := x] = \forall x.\, P(x).$$

Accidental binding is prevented, on the other hand, by the precondition that $x$ must be fresh for $\sigma$. This condition ensures that none of the terms we want to substitute for the (free) variables in $\varphi$ contains the variable $x$, which would become then bound by the quantifier.

These rules and their interaction are best illustrated through some examples.

**Example 5.4**

In the following, we use the substitution $\sigma$ given by

$$\sigma = [y := x][z := m].$$

1. Let $\varphi = \forall z.\, Q(z,y)$. First, we note that $\mathrm{var}(\sigma(y)) = \{x\}$ and thus $z \notin \mathrm{fv}(\sigma(v))$ for all $v \neq z$. Thus, $z \# \sigma$. This allows us to carry out the substitution:

$$\begin{aligned} \varphi\,\sigma &= \forall z.\, Q(z,y)\, \sigma[z := z] && \text{(SQ)} \\ &= \forall z.\, Q(z\,\sigma[z := z], y\,\sigma[z := z]) && \text{(SP)} \\ &= \forall z.\, Q(z,x) \end{aligned}$$

   Note that $\sigma[z := z] = [y := x]$ and the substitution of $m$ for $z$

in $\sigma$ was "forgotten" when we applied the substitution under the quantifier. This is intuitively expected, as the bound variable $z$ in $\varphi$ is a local reference, while the $z$ in $\sigma$ refers to a global variable $z$ that has the same name but is distinct from the local variable.

2. Let $\psi = \forall x.\, Q(x, y)$. First, we note that $\mathrm{var}(\sigma(y)) = \{x\}$. Thus, $x \in \mathrm{fv}(\sigma(y))$ and $x$ is *not* fresh. However, we have $z \# Q(x, y)$ and $Q(x, y)[x \coloneqq z] = Q(z, y)$. This allows us to rename the bound variable $x$ in $\psi$ to $z$ and then carry out the substitution as above:

$$\begin{aligned} (\forall x.\, Q(x, y))\, \sigma &= (\forall z.\, Q(z, y))\, \sigma && \text{(EQ)} \\ &= \forall z.\, Q(z, x) && \text{by 1.} \end{aligned}$$

Note that we cannot safely rename $z$ back to $x$, as we would otherwise illegally bind $x$.

In the remainder, we will not make explicit use of the axioms provided in definition 5.3. Instead, we will rename *bound* variables, if necessary, before carrying out substitutions. For instance, we would just write

$$(\forall x.\, Q(x, y))\, [y \coloneqq x] = \forall z.\, Q(z, x)$$

without explicitly referring to the axioms. However, in case of doubt, we can always go back to the formal axioms for renaming and substitution.

**Digression 5.5: Bound variables and alternatives to definition 5.3**

In definition 4.20, we introduced free variables formally but did not give a formal definition to compute bound variables of a formula. There is a good reason for that. Our equality axioms for formulas and the possibility of renaming variables whenever necessary has as consequence that bound variables are *internal* to formulas and cannot be observed. To see this, suppose that there is a map $\mathrm{bv} \colon \mathrm{Form}(\mathcal{L}) \to \mathcal{P}(\mathrm{Var})$ that extracts bound variables. Let us then consider the formulas $\forall x.\, P(x)$ and $\forall y.\, P(y)$, which are equal according to our axioms but

$$\mathrm{bv}(\forall x.\, P(x)) = \{x\} \neq \{y\} = \mathrm{bv}(\forall y.\, P(y))$$

and thus bv cannot be a map. In general, whenever we want to define a map on formulas, then such a map has to respect the equality and thus be invariant under renaming.
There are alternatives to handling the equality of formulas under renaming of variables. One popular option is to just ignore it! It is then postulated that all bound variables should have distinct names. However, this approach has several issues, chief among them that certain formulas are no longer tautologies. For example, consider the formula $\varphi$ given by $\varphi = \forall x.\, P(x)$ for some unary predicate symbol $P$. Anticipating the proof systems for first-order logic below, we may reasonably expect propositional tautology, such as $\varphi \lor \varphi \leftrightarrow \varphi$ to still hold in first-order logic. The subtle issue is though that, if all bound variables names have to be distinct, then this is not even a valid formula any longer. We may be able to prove $(\forall x.\, P(x)) \lor (\forall y.\, P(y)) \leftrightarrow (\forall z.\, P(z))$ but *not* $\varphi \lor \varphi \leftrightarrow \varphi$! Thus, a proper renaming

mechanism is needed.
Another alternative to handle variables is by completely removing them from the picture and use, for example, De Bruijn trees instead. These typically used in implementations of proof checkers. For completeness, some details on such trees can be found in appendix D.

## 5.3. Natural Deduction for FOL

With this understanding of binding, renaming and substitution, we can formulate the intuitionistic and the classical proof systems for first-order logic. These two systems, called **ND**$_1$ and **cND**$_1$, extend the natural deduction systems **ND** and **cND**, along two main components:

1. handling of object variables and
2. proof rules for the quantifiers.

Following the discussion in section 5.1, we will treat free variables as an assumption of existence. Recall from definition 2.1 that a sequent $\Gamma \vdash \varphi$ for propositional logic consists of a list of formulas $\Gamma$ that can be used as assumptions in a proof of this sequent. In first-order logic, both $\Gamma$ and $\varphi$ may have free variables, which we interpret as abstract, but fixed objects. In other words, we assume not only $\Gamma$ but also that all the objects, which are referred to by the free variables, exist. This leads us to extend sequents to the form $\Delta \mid \Gamma \vdash \varphi$, where $\Delta$ lists all the variables that appear in $\Gamma$ or $\varphi$.

**Definition 5.6**

A **(first-order) sequent** for a signature $\mathcal{L}$ is a triple

$$\boxed{\Delta \mid \Gamma \vdash \varphi},$$

where $\Delta$ is a list of variables in Var, $\Gamma$ is a list of $\mathcal{L}$-formulas, $\varphi$ is a $\mathcal{L}$-formula, and $\mathrm{fv}(\Gamma) \cup \mathrm{fv}(\varphi) \subseteq |\Delta|$ for $|\Delta| = \{x \in \mathrm{Var} \mid x : \Delta\}$. We call "$\Delta \mid \Gamma$" the **context** of the sequent. If $\Delta$ is the empty list, we just write

$$\Gamma \vdash \varphi$$

instead of $\Delta \mid \Gamma \vdash \varphi$.

**Example 5.7**

- This is a sequent:

$$x, y \mid \forall z.\, Q(z, y) \vdash P(x)$$

because $\mathrm{fv}(\forall z.\, Q(z,y)) \cup \mathrm{fv}(P(x)) = \{x, y\}$.

- This is a sequent:

$$x, y, z \mid \forall z.\, Q(z,y) \vdash P(x)$$

because $\mathrm{fv}(\forall z.\, Q(z,y)) \cup \mathrm{fv}(P(x)) = \{x, y\} \subseteq \{x, y, z\}$.

- This is not a sequent:

$$x \mid \forall z.\, Q(z,y) \vdash P(x)$$

because $\mathrm{fv}(\forall z.\, Q(z,y)) \cup \mathrm{fv}(P(x)) \nsubseteq \{x\}$.

### 5.3.1. The Intuitionistic System $\mathbf{ND}_1$

Analogously to the system **ND** from definition 2.1, we formulate the proof system $\mathbf{ND}_1$ for first-order logic in terms of rules that determine how a valid deduction, or proof tree, looks like. The following definition provides a formal description of these, where all the rules from definition 2.1 appear again only that the variable context $\Delta$ has to be added to all the sequents. Additionally, there are two pairs of introduction and elimination rules for each of the quantifiers. Two of these rules, ($\forall$I) and ($\exists$E), have an additional label "$x \notin \Delta$" that expresses an extra side-condition that has to be fulfilled. This side-condition ensures that the variable $x$ is fresh for the involved contexts and formulas, which avoids the issues related to scoping, see sections 5.1 and 5.2. One may think that this side-condition is a restriction, but definition 5.3 ensures that we can always rename a bound variable and thus fulfil the side-condition. To keep things readable, we will typically not explicitly state the side-condition in proof but rather make sure that variables are renamed whenever needed.

**Definition 5.8: Intuitionistic natural deduction for FOL**

The system of **natural deduction for first-order logic** $\boxed{\mathbf{ND}_1}$ is given by the rules in fig. 5.1. A **deduction** or **proof tree** for $\Delta \mid \Gamma \vdash \varphi$ is a finite tree, such that

i) each node is labelled with a sequent and a rule, such that the children of the node are labelled with the hypotheses of that rule, and

ii) the root of the tree is labelled with $\Delta \mid \Gamma \vdash \varphi$.

We then read $\Delta \mid \Gamma \vdash \varphi$ as the formula $\varphi$ is **provable** in context $\Delta \mid \Gamma$.

$$\frac{\varphi : \Gamma}{\Delta \mid \Gamma \vdash \varphi}\ (\text{Assum}) \qquad \frac{\Delta \mid \Gamma \vdash \bot}{\Delta \mid \Gamma \vdash \varphi}\ (\bot\text{E})$$

$$\frac{\Delta \mid \Gamma \vdash \varphi \wedge \psi}{\Delta \mid \Gamma \vdash \varphi}\ (\wedge\text{E}_1) \qquad \frac{\Delta \mid \Gamma \vdash \varphi \wedge \psi}{\Delta \mid \Gamma \vdash \psi}\ (\wedge\text{E}_2)$$

$$\frac{\Delta \mid \Gamma \vdash \varphi \qquad \Delta \mid \Gamma \vdash \psi}{\Delta \mid \Gamma \vdash \varphi \wedge \psi}\ (\wedge\text{I})$$

$$\frac{\Delta \mid \Gamma \vdash \varphi}{\Delta \mid \Gamma \vdash \varphi \vee \psi}\ (\vee\text{I}_1) \qquad \frac{\Delta \mid \Gamma \vdash \psi}{\Delta \mid \Gamma \vdash \varphi \vee \psi}\ (\vee\text{I}_2)$$

$$\frac{\Delta \mid \Gamma \vdash \varphi \vee \psi \qquad \Delta \mid \Gamma, \varphi \vdash \delta \qquad \Delta \mid \Gamma, \psi \vdash \delta}{\Delta \mid \Gamma \vdash \delta}\ (\vee\text{E})$$

$$\frac{\Delta \mid \Gamma, \varphi \vdash \psi}{\Delta \mid \Gamma \vdash \varphi \to \psi}\ (\to\text{I}) \qquad \frac{\Delta \mid \Gamma \vdash \varphi \to \psi \qquad \Delta \mid \Gamma \vdash \varphi}{\Delta \mid \Gamma \vdash \psi}\ (\to\text{E})$$

$$(x \notin \Delta)\ \frac{\Delta, x \mid \Gamma \vdash \varphi}{\Delta \mid \Gamma \vdash \forall x.\, \varphi}\ (\forall\text{I}) \qquad \frac{\Delta \mid \Gamma \vdash \forall x.\, \varphi}{\Delta \mid \Gamma \vdash \varphi\,[x := t]}\ (\forall\text{E})$$

$$\frac{\Delta \mid \Gamma \vdash \varphi\,[x := t]}{\Delta \mid \Gamma \vdash \exists x.\, \varphi}\ (\exists\text{I}) \qquad (x \notin \Delta)\ \frac{\Delta \mid \Gamma \vdash \exists x.\, \varphi \qquad \Delta, x \mid \Gamma, \varphi \vdash \psi}{\Delta \mid \Gamma \vdash \psi}\ (\exists\text{E})$$

Figure 5.1.: Deduction Rules of the natural deduction system $\mathbf{ND}_1$

Again, remember that we need to make sure that the side-condition "$x \notin \Delta$" in ($\forall$I) and ($\exists$E) holds, but we will not explicitly state it in deductions. Apart from the side-condition, there are some subtleties in fig. 5.1. First of all, in the rules ($\forall$E) and ($\exists$I) the $\Delta \mid \Gamma \vdash \varphi[x := t]$ must adhere to definition 5.6, which means that all the free variables of the term $t$ appear in $\Delta$. Similarly, it is ensured in ($\exists$E) that the variable $x$ does not occur freely in $\psi$ because $\Delta \mid \Gamma \vdash \psi$ is a sequent.

To simplify writing proofs, we will write all $\mathbf{ND}_1$ deductions in Fitch style, unless stated otherwise. Recall that we wrote in Fitch style all assumptions on a flag, which means that these assumptions are only usable within that flag. In first-order logic, the context of a proof not only consists of formulas

but also of object variables. For these, the same principle applies: they only exist within the flag on which they have been declared. To set variables apart from formulas in the context, we write these on the side of the flag that they belong to. This is best seen in an example.

**Example 5.9**

We prove $\forall x.\, P(c, x) \vdash \forall x.\, \exists y.\, P(y, x)$, where $c$ is a constant

| | | |
|---|---|---|
| 1 | $\forall x.\, P(c, x)$ | |
| 2 | $x \mid$ | |
| 3 | $\quad P(c, x)$ | $\forall$E, 1 |
| 4 | $\quad \exists y.\, P(y, x)$ | $\exists$I, 3 |
| 5 | $\forall x.\, \exists y.\, P(y, x)$ | $\forall$I, 2–4 |

Let us spell out the use of the ($\exists$I)-rule in line 4 of this proof. That step corresponds to the following instance of the rule, where it is important to keep in mind that the formula $P(c, x)$ in line 3 is equal to $P(y, x)\,[y := c]$.

$$\frac{x \mid \forall x.\, P(c, x) \vdash P(y, x)\,[y := c]}{x \mid \forall x.\, P(c, x) \vdash \exists y.\, P(y, x)}\ (\exists\mathrm{I})$$

The whole proof can be written in tree style as follows.

$$\cfrac{\cfrac{\cfrac{\cfrac{}{x \mid \forall x.\, P(c, x) \vdash \forall x.\, P(c, x)}}{x \mid \forall x.\, P(c, x) \vdash P(c, x)}\ (\forall\mathrm{E})}{x \mid \forall x.\, P(c, x) \vdash \exists y.\, P(y, x)}\ (\exists\mathrm{I})}{\forall x.\, P(c, x) \vdash \forall x.\, \exists y.\, P(y, x)}\ (\forall\mathrm{I})$$

Just like a formula can be proven under the assumption of a formula that appears on the outer-most flag of a proof, the same is the case for variables, which are then in scope throughout the whole proof. This is demonstrated in the following example.

**Example 5.10**

We prove the sequent $y \mid \exists x.\, P(x, y) \vdash \neg(\forall x.\, \neg P(x, y))$ by the following

deduction. Other proofs are possible, but let us focus on this one.

| | | | |
|---|---|---|---|
| 1 | $y$ | $\exists x.\, P(x,y)$ | |
| 2 | | $x$ $\quad P(x,y)$ | |
| 3 | | $\forall x.\, \neg P(x,y)$ | |
| 4 | | $\neg P(x,y)$ | $\forall$E, 3 |
| 5 | | $\bot$ | $\neg$E, 4, 2 |
| 6 | | $\neg(\forall x.\, \neg P(x,y))$ | $\neg$I, 3–5 |
| 7 | | $\neg(\forall x.\, \neg P(x,y))$ | $\exists$E, 1, 2–6 |

The last step of the proof in line 7 uses elimination of the existential quantifier in the following form.

$$\frac{\overline{y \mid \Gamma \vdash \exists x.\, P(x,y)} \qquad y, x \mid \exists x.\, P(x,y), P(x,y) \vdash \neg(\forall x.\, \neg P(x,y))}{y \mid \exists x.\, P(x,y) \vdash \neg(\forall x.\, \neg P(x,y))}\ (\exists\mathrm{E})$$

This step is allowed because the variable $x$ is not identical to $y$.

The following example shows that negation existential quantifier can be turned into a universal quantifier, similarly to how one de Morgan's law turns a negated disjunction into a conjunction, see example 2.9.

**Example 5.11**

We prove $\neg\exists x.\, \varphi \vdash \forall x.\, \neg\varphi$ in $\mathbf{ND}_1$.

| | | |
|---|---|---|
| 1 | $\neg\exists x.\, \varphi$ | |
| 2 | $x$ | |
| 3 | $\varphi$ | |
| 4 | $\exists x.\, \varphi$ | $\exists$I, 3 |
| 5 | $\bot$ | $\neg$E, 1, 4 |
| 6 | $\neg\varphi$ | $\neg$I, 3–5 |
| 7 | $\forall x.\, \neg\varphi$ | $\forall$I, 2–6 |

### 5.3.2. The Classical System $\mathbf{cND}_1$

To be able to prove all valid formulas of propositional logic, we added in chapter 3 the contradiction rule to the system **ND** for propositional logic. The analogous extension of $\mathbf{ND}_1$ is given in the following definition.

**Definition 5.12**

The system of ***classical* natural deduction for first-order logic** $\boxed{\mathbf{cND}_1}$ is given by the system $\mathbf{ND}_1$ together with the contradiction rule:

$$\frac{\Delta \mid \Gamma, \neg\varphi \vdash \bot}{\Delta \mid \Gamma \vdash \varphi}\ \text{(Contra)}$$

Showing that the resulting system $\mathbf{cND}_1$ is in some sense complete, meaning that all valid formulas can be proven, will have to wait. This requires first a discussion of models of first-order logic. For the time being, we content ourselves with deductions in $\mathbf{cND}_1$. Let us give a typical example of a statement that can only be proved in $\mathbf{cND}_1$ but not in $\mathbf{ND}_1$.

**Example 5.13**

The sequent $\neg\forall x.\, P(x) \vdash \exists x.\, \neg P(x)$ can be deduced by the following proof.

| | | |
|---|---|---|
| 1 | $\neg\forall x.\, P(x)$ | |
| 2 | $\quad \neg(\exists x.\, \neg P(x))$ | |
| 3 | $\quad \forall x.\, \neg\neg P(x)$ | example 5.10, 2 |
| 4 | $\quad x$ | |
| 5 | $\quad\quad \neg\neg P(x)$ | $\forall$E, 3 |
| 6 | $\quad\quad P(x)$ | DNE, 5 |
| 7 | $\quad \forall x.\, P(x)$ | $\forall$I, 4–6 |
| 8 | $\quad \bot$ | $\neg$E, 1, 7 |
| 9 | $\exists x.\, \neg P(x)$ | Contra, 2–8 |

Here, **DNE** refers to **double negation elimination**, which is an immediate

consequence of the proof-by-contradiction rule:

$$\dfrac{\dfrac{\dfrac{}{\Delta \mid \Gamma, \neg\neg\varphi, \neg\varphi \vdash \neg\neg\varphi}\,(\neg\mathrm{E}) \qquad \dfrac{}{\Delta \mid \Gamma, \neg\neg\varphi, \neg\varphi \vdash \neg\varphi}\,(\neg\mathrm{E})}{\Delta \mid \Gamma, \neg\neg\varphi, \neg\varphi \vdash \bot}\,(\neg\mathrm{E})}{\Delta \mid \Gamma, \neg\neg\varphi \vdash \varphi}\,(\mathrm{Contra})$$

That the sequent example 5.13 is perhaps not so shocking, but there are rather strange formulas that are provable in classical first-order logic. One of them is the so-called *Drinker's paradox*, which asserts that if there if there at least on person drinking in a bar, then exists one person in the bar, such that whenever this person drinks then everybody in the bar drinks. The proof of this is somewhat lengthy, but it can be found in appendix E.2.

Isaac seems to fade away and Clara tries to catch his attention: "What are you thinking about, Isaac."

"Nothing, nothing. I was just remembering something.", he exclaims. "Thank you Prof. Stebbing, this was most helpful."

"You are welcome. Now Isaac, please do not be offended by what I am going to say." Stebbing turns to Clara, "My dear, please be careful about your friend. It is not his fault, but his reasoning is purely mechanical. Isaac's reasoning steps may follow his internal logic, but how can we know that this logic aligns with our expectations and that his assumptions are not contradicting ours? Thinking is determined by the persons we are, but can we call a deterministic machine a person?"

Clara's heart sinks, "How do you know?"

"You may try to fool me with looks and perceptions. Your friend may reason perfectly, but he reasons without love. That is what reveals his mechanical identity. Worry not, I will keep this little secret between us.", Stebbing winks. "Now please excuse me. My headache is getting stronger and I will need to rest."

Stebbing puts her hand on Clara's shoulder, "One last thing: I really like you, you have a sharp mind and a great heart. Please use it to good effect and never let anyone deceive you."

With these words, Stebbing leaves. Clara tries to shout a thank you after Stebbing, but she is already out through the door.

Clara is perplexed. "She reminds me so much of my mother."

"Yes, I know.", responds Isaac in a mysterious voice. "Something else: I was able to understand more of my memory by deciphering the given deductions according to what we learned from Stebbing. However, I get stuck on those

parts where deductions are not given. If only there was a way to find the deductions, then I could access the rest."

Clara nods hesitantly, "Yes, the key is to understand how proofs can be found automatically. But..."

"You are worried about what Stebbing said, aren't you?"

"Yes, I am.", Clara looks at her feet.

"You don't have to be shy about it: you don't trust me at this point, do you?"

"Indeed.", Clara lifts her head, "However, I have an idea. We could see if we can verify your reasoning against the logical systems that you have acquired and the assumptions that you are making."

Isaac is puzzled, "How could we do that?"

"Do you know what an algorithm is?"

"Of course, I do!", Isaac seems indignant, as far as that is possible.

"Do you also know from what the word is derived?"

"No. Where is it from?", asks Isaac more curiously this time.

"Let me show you!" Clara gives Isaac time and coordinates, and relaxes when the feeling of floating sets in after Isaac activates the Voyag-O-Matic.

# Exercises

## Exercise 18

Let $\sigma$ be given by

$$\sigma = [x := g(x)][y := x][z := y].$$

Evaluate the following applications of terms to $\sigma$.

a) $x\,\sigma$ b) $f(x, y)\,\sigma$ c) $(y\,\sigma)\,\sigma$ d) $x\,(\sigma[x := x])$

## Exercise 19

Let $\sigma$ be given by

$$\sigma = [x := g(x)][y := x][z := y]$$

and $\varphi$ by

$$\varphi = \forall x.\, Q(x, y, z).$$

Determine which of the following relations hold.

a) $x \# \sigma$ b) $y \# \sigma$ c) $z \# \sigma$

d) $x \# \varphi$ e) $y \# \varphi$ f) $z \# \varphi$

## Exercise 20

Let $\sigma$ be given by

$$\sigma = [x := g(x)][y := x][z := y].$$

Carry out the following substitutions.

a) $Q(x, g(m))\,\sigma$ b) $(\forall x.\, Q(x, g(m)))\,\sigma$

c) $((\exists x.\, Q(x, g(m))) \wedge \forall y.\, Q(y, z))\,\sigma$

d) $(\exists y.\, Q(y, g(z)) \wedge \forall x.\, Q(y, f(x, z)))\,\sigma$

## Exercise 21

Prove the following sequents in system $\mathbf{ND}_1$ using Fitch style, where $P$ and $Q$ have arity 1, and $S$ has arity 0.

a) $x \mid P(x), \forall y.\, (P(y) \to Q(y)) \vdash Q(x)$

b) $\exists x.\, S \to Q(x) \vdash S \to \exists y.\, Q(y)$

c) $\forall x.\, P(x) \vdash \neg\exists y.\, \neg P(y)$

d) $\exists x.\, P(x) \vdash \neg\forall y.\, \neg P(y)$

## Exercise 22

Prove the following sequents in system $\mathbf{cND}_1$ using Fitch style, where $P$ has arity 1 and $\cdot$ is the empty list. General hint: Use results that have been proved previously in the lecture or in the exercises. Note that exercise b) is very hard!

a) $\neg\forall x.\, \neg P(x) \vdash \exists y.\, P(y)$

b) $\neg\exists x.\, \neg P(x) \vdash \forall x.\, P(x)$

c) $j, k \mid \cdot \vdash \exists x.\, P(x) \to P(j) \wedge P(k)$

# 6. Machine-Checked Proofs

Clara and Isaac find themselves on a densely packed market, full of merchants with tents, and people pushing and squeezing to move forward. They are wearing long tunics in various colours. The air is hot and dry, infused with smells of herbs, spices, bread and sand. Sounds of drums, flutes and lutes flow through the market, out into the alleys on the sides. A man, dressed in an elegant tunic with red and brown stripes and a high turban with golden decoration, catches their attention.

"Naadaan-e-ahmagh!"

"What does he say?", Clara leans towards Isaac to not be heard.

'He seems to be very annoyed.'[12]

Clara inhales deeply, "Incredible! All these smells: cardamon, cinnamon, pepper, and turmeric! Chickpeas, dates, fresh figs, olives, melons! I am getting hungry! Hey, what instruments are the people at the corner playing?"

The man in the elegant tunic turns his head, "They are playing *Duduk*, *Barbat* and *Tombak*.[13] You don't seem to know your way around here! Where are you going?"

Clara blushes, "We are indeed not from here. Would you be able to show us the direction to the *House of Wisdom*? We are looking for some information."

The man chuckles, "You cannot just walk in there and demand information! What are you looking for? Maybe I can help you?"

"We wish to learn about computations in logic, for which we would like to meet al-Khwārizmī. After all, his work was so fundamental to algebra and the understanding of computing.", Clara explains.

"Aha, is that so? I think that I can arrange something for you. Follow me." While gesticulating and gruntling, the man exchanges a silver coin against a small leather bag with the merchant behind the stall: "Whatever, take this!" Then he turns and indicates Clara and Isaac to follow him.

They push through the dense crowd and eventually leave the market. After passing a narrow alley, a river seamed with palm trees lies ahead of them.

"Beautiful!", exclaims Clara.

"I take it you have not been here before. Welcome to Baghdad, my friends! Do you see the bridge ahead? It will take us over the Tigris to the Madīnat as-Salām, the *City of Peace*, where the palace is located."

"The palace?", Clara is surprised.

"Yes, the palace is surrounded by houses, which host the workplaces in the *House of Wisdom*."[14]

The three cross the bridge and arrive at a gate that leads through massive stone walls that surround the *City of Peace*. Inside the walls, all sounds are suddenly subdued.

"So peaceful!", Clara deeply inhales and relaxes.

The man brings them into a hall, in which about a 100 people sit at tables organised in rows of 5 with two large books. "Please come in. Welcome to the House of Wisdom."

"Who are all these people?", Clara asks quietly.

"They are translating texts from China, India and Greece, interpret them, critique them, add new insights. Now, please wait here. I see if I can find al-Khwārizmī for you."

The man leaves the hall through two columns. Isaac gaze is jumping around.

"Isaac? Are you all right?"

"Yes, so many different languages!"

In that moment, the man returns. He is dressed in a lighter, violet tunic with white seams and carries a small tray with three cups containing a steaming tea: "We shall go outside to not disturb here." Once outside the hall, he turns to Clara and Isaac: "How do you like my workplace?"

"Are you…?", Clara's eyes widen.

"Al-Khwārizmī? Ha, ha! Yes, I am.", he bursts into laughter. "You should see the looks on your face! Now, let us find a quiet place to discuss."

Al-Khwārizmī, Clara and Isaac go into the gardens of the *City of Peace*. Trees line the many small houses and in the middle rises the *Palace of the Golden Gate* with its green dome. Around are numerous people sitting and holding a book in one hand, while writing with the other. The three sit in a circle on small logs and each take a tea cup from Al-Khwārizmī.

"How can I help you?", he asks.

Clara explains their problem: "You of course know what a proof is, but have you ever considered the problem of *verifying* that a proof is correct? How can we fully trust that an argument is correctly made, is only using proclaimed assumptions and so forth?"

"Excellent question!", al-Khwārizmī raises his finger in confirmation. "Euc-

lid has achieved great insights with his axioms and derivations, but can we trust him? Can we trust that all people, who studied his proofs, have done so carefully and honestly? In the end, verifying a proof is nothing but an act of calculation, which may even be carried out by a machine, if only they existed. Do you have any specific proofs in mind that you would want to verify?"

Clara and Isaac take out their notes and explain al-Khwārizmī the rules of the proof system $\mathbf{ND}_1$. He then begins to develop in the sand in front of them the foundations for a verification algorithm.

## 6.1. Proof-Checking with Explicit Substitutions

Suppose that we are given a tree labelled with sequents and deductions rules and we wish to check whether it represents a valid deduction in $\mathbf{ND}_1$. Such a tree contains almost all information needed to verify it by a simple traversal, during which we check adherence to definition 5.8. The only difficulty lies in the rules for universal elimination and the existential introduction. To understand this difficulty, let us look at a proof that ends with the (∃I)-rule.

$$\dfrac{\dfrac{\vdots}{\Delta \mid \Gamma \vdash \psi}}{\Delta \mid \Gamma \vdash \exists x.\, \varphi}\ (\exists\mathrm{I})$$

The challenge is to verify that $\psi = \varphi\,[x := t]$ for *some* term $t$. Unfortunately, we do not know what this term is and we need to find it.

One way out is to integrate substitutions into ∀-elimination and ∃-introduction:

$$\dfrac{\Delta \mid \Gamma \vdash \forall x.\, \varphi \qquad t \in \mathrm{Term}(\mathcal{L})}{\Delta \mid \Gamma \vdash \varphi\,[x := t]}\ (\forall\mathrm{E'}) \qquad \dfrac{\Delta \mid \Gamma \vdash \varphi\,[x := t] \qquad t \in \mathrm{Term}(\mathcal{L})}{\Delta \mid \Gamma \vdash \exists x.\, \varphi}\ (\exists\mathrm{E'})$$

With these alternatives to (∀E) and (∃I), the above proof tree would become

$$\dfrac{\dfrac{\vdots}{\Delta \mid \Gamma \vdash \psi} \qquad t \in \mathrm{Term}(\mathcal{L})}{\Delta \mid \Gamma \vdash \exists x.\, \varphi}\ (\exists\mathrm{E'})$$

and we only have to verify that $\psi = \varphi\,[x := t]$. This is much easier and solves the problem by adding more information to the proof tree.

## 6.2. Proof-Checking with Implicit Substitutions

The obvious down-side of changing the proof rules, as in section 6.1, is that proofs become bigger and that we have to change the proof rules. It would be great if we could stick to the original rules. At first sight, it seems that finding a term $t$ with $\psi = \varphi\,[x := t]$ amounts to searching through all possible terms, but it turns out that $\mathbf{ND}_1$-proofs have enough information to induce a system of term equations that can be solved to find $t$ automatically. The goal of this section is to develop some of the theory underlying this and then provide an algorithm to check if a tree presents a valid $\mathbf{ND}_1$ deduction.

Suppose again that we are presented by a proof that ends like this:

$$\dfrac{\dfrac{\vdots}{\Delta \mid \Gamma \vdash \psi}\,\mathrm{L}}{\Delta \mid \Gamma \vdash \exists x.\,\varphi}\,(\exists\mathrm{I})$$

Instead of trying to find a term at the point of checking this step, we postpone the decision. For this, we will use variables that we write as $?x, ?y$ etc. that stand for an unspecified term, which will have to be filled in by a concrete term by the end of the proof. Checking the above tree then proceeds by checking it from the label L upwards, remembering that $\psi = \varphi\,[x := ?x]$ should hold and eventually finding a term $t$ that can be filled for $?x$.

**Example 6.1**

Let us illustrate the idea on a proof of $\forall x.\,P(c,x) \vdash \forall x.\,\exists y.\,P(y, f(x))$, which is a slightly more sophisticated version of example 5.9. We put $\Gamma = \forall x.\,P(c,x)$ for readability and write the proof here in tree style.

$$\begin{array}{rl}
\dfrac{\dfrac{\dfrac{\dfrac{}{x \mid \Gamma \vdash \forall x.\,P(c,x)}\,(\mathrm{Assum})}{x \mid \Gamma \vdash P(c, f(x))}\,(\forall\mathrm{E})}{x \mid \Gamma \vdash \exists y.\,P(y, f(x))}\,(\exists\mathrm{I})}{\Gamma \vdash \forall x.\,\exists y.\,P(y, f(x))}\,(\forall\mathrm{I}) & \begin{array}{c} \\ \\ \\ \\ \end{array}
\end{array}$$

| | |
|---|---|
| 4 | $\{?y \stackrel{?}{=} c,\ f(x) \stackrel{?}{=} f(x),\ c \stackrel{?}{=} c,\ f(x) \stackrel{?}{=} ?x\}$ |
| 3 | $\{?y \stackrel{?}{=} c,\ f(x) \stackrel{?}{=} f(x)\}$ |
| 2 | |
| 1 | |

On the right of the tree, we numbered the steps. Next to the steps, we record the system of equations that we need to solve at the end. Step 2 is the existential introduction, which means that we need to compare

$P(y, f(x))\,[y := ?y]$ with $P(c, f(x))$, that is, we want

$$P(y, f(x))\,[y := ?y] = P(?y, f(x)) = P(c, f(x))\,.$$

In our equations, we only keep those between terms and, since they are unsolved, we denote them with a question mark on the equality. Thus, the constraint $P(?y, f(x)) = P(c, f(x))$ results in the first set of equations $\{?y \stackrel{?}{=} c, f(x) \stackrel{?}{=} f(x)\}$ that we generate.
In the third step, we encounter the rule (∀E), which also requires us to defer the decision on the term that we want to instantiate the quantifier with. Instead, we stipulate that we want $P(c, f(x)) = P(c, ?x)$, which results in the equations $c \stackrel{?}{=} c$ and $f(x) \stackrel{?}{=} ?x$ that are passed upwards in the proof check.
Since the formula in step 4 is among our assumptions, the proof is correct if we can solve the system $\{?y \stackrel{?}{=} c, f(x) \stackrel{?}{=} f(x), c \stackrel{?}{=} c, f(x) \stackrel{?}{=} ?x\}$.

Intuitively, we would want to set $?y = c$ and $?x = f(x)$ in example 6.1 to solve the system of equations. To do so, we need a guarantee that this yields a correct proof verification, and we need a way to solve such equations in general or decide that they are not solvable. The process to do this is called unification, which we develop in what follows and establish some of its properties.

**Definition 6.2**

We fix a countably infinite set UVar ⊆ Var of **unification variables**, elements of which we denote by $?x, ?y$ etc. Given a substitution $\sigma$, we define its **support** $\boxed{\mathrm{supp}(\sigma)}$ by $\mathrm{supp}(\sigma) = \{x \in \mathrm{Var} \mid \sigma(x) \neq x\}$. A **unification problem** is a pair of terms $s$ and $t$ denoted by $s \stackrel{?}{=} t$. Given a finite set $E$ of unification problems $E$, we say that a substitution $\sigma$ is a **unifier** for $E$ if $s\,\sigma = t\,\sigma$ for all unification problems $s \stackrel{?}{=} t \in E$ and $\mathrm{supp}(\sigma) \subset \mathrm{UVar}$.

Let us illustrate these notions with an example.

**Example 6.3**

The set $\{?y \stackrel{?}{=} c, f(x) \stackrel{?}{=} f(x), c \stackrel{?}{=} c, f(x) \stackrel{?}{=} ?x\}$ from example 6.1 consists of four unification problems. To solve this system, we define a substitution $\sigma$ by $\sigma = [?y := c][?x := f(x)]$. Its support is given by $\mathrm{supp}(\sigma) = \{?y, ?x\}$ and thus contained in UVar. We then get that $\sigma$ is a unifier, as it solves all four unification problems: $?y\,\sigma = c = c\,\sigma$, $f(x)\,\sigma = f(x) = f(x)\,\sigma$, $c\,\sigma = c = c\,\sigma$, and $f(x)\,\sigma = f(x) = ?x\,\sigma$.

In general, there may be several unifiers but not all will allow us to successfully check a proof. What we want is a unifier that makes the simplest possible substitutions. To make this precise, we need to compose substitutions.

**Definition 6.4: Composition of substitutions**

Let $\sigma$ and $\tau$ be substitutions. We define their **composition** $\boxed{\tau \bullet \sigma}$ for every $x \in \mathrm{Var}$ by applying $\sigma(x)$ to $\tau$:

$$(\tau \bullet \sigma)(x) = \sigma(x)\,\tau$$

Let us first prove that the composition $\tau \bullet \sigma$ indeed carries out $\tau$ after $\sigma$.

**Lemma 6.5: Substitution Lemma**

For all terms $t$, and all substitutions $\sigma$ and $\tau$, it holds that $t\,(\tau \bullet \sigma) = (t\,\sigma)\,\tau$.

*Proof.* We proceed by induction on $t$. This induction has two base cases:

1. Variables: Here we have for $x \in \mathrm{Var}$ by definition that

$$x\,(\tau \bullet \sigma) = (\tau \bullet \sigma)(x) = \sigma(x)\,\tau = (x\,\sigma)\,\tau\,.$$

2. Constants: Also by definition, the following holds.

$$c\,(\tau \bullet \sigma) = c = c\,\tau = (c\,\sigma)\,\tau$$

For the induction step, we assume that $t_k\,(\tau \bullet \sigma) = (t_k\,\sigma)\,\tau$ for $k = 1, \dots, n$ as our induction hypothesis (IH). Then

3. Function symbols:

$$\begin{aligned} f(t_1, \dots, t_n)\,(\tau \bullet \sigma) &= f(t_1\,(\tau \bullet \sigma), \dots, t_1\,(\tau \bullet \sigma)) && \text{by def.} \\ &= f((t_1\,\sigma)\,\tau, \dots, (t_n\,\sigma)\,\tau) && \text{by IH} \\ &= f(t_1\,\sigma, \dots, t_n\,\sigma)\,\tau) && \text{by def.} \\ &= (f(t_1, \dots, t_n)\,\sigma)\,\tau && \text{by def.} \end{aligned}$$

Thus, by induction $t\,(\tau \bullet \sigma) = (t\,\sigma)\,\tau$ for all $t$. □

The following lemma states that the identity substitution, see definition 5.1, is the unit for substitution composition cf. the quiz on page 82.

**Lemma 6.6**

For all substitutions $\sigma$, we have $\sigma \bullet \eta = \sigma$ and $\eta \bullet \sigma = \sigma$, where $\eta$ is the identity substitution. Moreover, composition is associative, that is, for all substitutions $\sigma$, $\tau$ and $\gamma$, the equality $\gamma \bullet (\tau \bullet \sigma) = (\gamma \bullet \tau) \bullet \sigma$ holds.

*Proof.* To prove the first equality, we use that two substitutions $\sigma$ and $\tau$ are equal if and only if $x\,\sigma = x\,\tau$ for all $x \in \mathrm{Var}$. Thus, suppose $x \in \mathrm{Var}$, and have by lemma 6.5 and definition that

$$x\,(\sigma \bullet \eta) = (x\,\eta)\,\sigma = x\,\sigma$$

and then, by the above observation, we get $\sigma \bullet \eta = \sigma$. The second equality follows from the definition of $\eta$ and composition. For all $x \in \mathrm{Var}$, we have

$$(\eta \bullet \sigma)(x) = \eta(x)\,\sigma = x\,\sigma = \sigma(x)$$

and thus $\eta \bullet \sigma = \sigma$. Associativity is proved with the same observation that we used for the first equality. This is left as an exercise. □

**Definition 6.7**

We write $\sigma \sqsubseteq \tau$ if $\sigma$ is more general than $\tau$, or if $\tau$ is an instance of $\sigma$:

$$\sigma \sqsubseteq \tau \quad \text{iff} \quad \text{there is a substitution } \delta \text{ with } \delta \bullet \sigma = \tau\,.$$

We call $\sigma$ a **most general unifier** (mgu) of a set $E$ of unification problems if $\sigma \sqsubseteq \tau$ for all other unifiers of $E$.

The following lemma shows that the relation from definition 6.7 is almost a partial order, but this is enough to make sure that most general unifiers are unique up to renaming of variables.

**Lemma 6.8**

The relation $\sqsubseteq$ is reflexive and transitive. Moreover, if $\sigma \sqsubseteq \tau$ and $\tau \sqsubseteq \sigma$, then $\delta \bullet \sigma = \tau$ for a **name permutation** $\delta$, which means that there is a bijection $\delta' : \mathrm{Var} \to \mathrm{Var}$ with $\delta = \eta \circ \delta'$ as in the following diagram.

$$\begin{tikzcd}
\mathrm{Var} \arrow[rr, "\delta'"] \arrow[dr, "\delta"'] & & \mathrm{Var} \arrow[dl, "\eta"] \\
& \mathrm{Term} &
\end{tikzcd}$$

*Proof.* By lemma 6.6 we have for all $\sigma$ that $\eta \bullet \sigma = \sigma$ and thus $\sigma \sqsubseteq \sigma$. Thus, $\sqsubseteq$ is reflexive. Similarly, associativity from lemma 6.6 implies that it is transitive.

Finally, suppose that $\sigma \sqsubseteq \tau$ and $\tau \sqsubseteq \sigma$. This means that there are substitutions $\delta$ and $\gamma$ with $\delta \bullet \sigma = \tau$ and $\gamma \bullet \tau = \sigma$. It follows that $\delta \bullet \gamma \bullet \tau = \tau$ and thus $\delta \bullet \gamma = \eta$. This last identity implies for all $x$ that $\delta(x)\gamma = x$. Since $x$ is a variable, we must have that $\delta(x) = y$ for some $y$ with $\gamma(y) = x$. Hence, $\operatorname{im}\delta \subseteq \mathrm{Var}$ and we can define a map $\delta' \colon \mathrm{Var} \to \mathrm{Var}$ by $\delta'(x) = \delta(x)$. It remains to prove that it is a bijection. Let us also define $\gamma'(y) = \gamma(x)$, which is well-defined by the same argument. In both cases we have $\gamma = \eta \circ \gamma'$ and $\delta = \eta \circ \delta'$. Moreover, we get from $\gamma \bullet \delta \bullet \sigma = \sigma$ that $\gamma \bullet \delta = \eta$, and similarly we have $\delta \bullet \gamma = \eta$. This implies that

$$\eta \circ (\gamma' \circ \delta') = \gamma \circ \delta' = \gamma \bullet \delta = \eta$$

and thus $\gamma' \circ \delta' = \mathrm{id}_{\mathrm{Var}}$. Symmetrically, we also get $\delta' \circ \gamma' = \mathrm{id}_{\mathrm{Var}}$. Together, this means that $\gamma'$ is the inverse of $\delta'$ and thus that $\delta'$ is a bijection. □

Lemma 6.8 means that a most general unifier is unique up to bijective renaming of variables. What thus remains is to find it.

**Definition 6.9**

A **unification state** is either **fail** or a pair $E; \sigma$ consisting of a finite set $E$ of unification problems and a substitution with $\mathrm{supp}(\sigma) \subset \mathrm{UVar}$. We define the **unification** relation $\Longrightarrow$ between states by the clauses in fig. 6.1, where we write $\{s \stackrel{?}{=} t\} \cup E$ for the union of $\{s \stackrel{?}{=} t\}$ and $E$.

$$\begin{aligned}
\{s \stackrel{?}{=} s\} \cup E; \sigma &\Longrightarrow E; \sigma && \text{trivial}\\
\{f(s_1, \dots, s_n) &\stackrel{?}{=} f(t_1, \dots, t_n)\} \cup E; \sigma \\
&\Longrightarrow \{s_1 \stackrel{?}{=} t_1, \dots, s_n \stackrel{?}{=} t_n\} \cup E; \sigma && \text{match}\\
\{f(s_1, \dots, s_n) &\stackrel{?}{=} g(t_1, \dots, t_m)\} \cup E; \sigma \\
&\Longrightarrow \textbf{fail}, \quad f \neq g && \text{clash}\\
\{s \stackrel{?}{=} ?x\} \cup E; \sigma &\Longrightarrow \{?x \stackrel{?}{=} s\} \cup E; \sigma && \text{orient}\\
\{?x \stackrel{?}{=} s\} \cup E; \sigma &\Longrightarrow \textbf{fail}, \quad ?x \in \mathrm{var}(s) \text{ and } ?x \neq s && \text{cycle}\\
\{?x \stackrel{?}{=} s\} \cup E; \sigma &\Longrightarrow E\,[?x := s]; ([?x := s] \bullet \sigma)[?x := s] && \text{elim}
\end{aligned}$$

Figure 6.1.: Clauses of the unification relation

The unification relation allows us to unify a set $E$ by starting from the state

$E; \eta$, as the following example shows.

**Example 6.10**

Let $E = \{?y \stackrel{?}{=} c, f(x) \stackrel{?}{=} f(x), c \stackrel{?}{=} c, f(x) \stackrel{?}{=} ?x\}$ be the set from example 6.1. Starting at $E; \eta$, we obtain the following sequence of states.

$$\begin{aligned}
E; \eta &\Longrightarrow \{?y \stackrel{?}{=} c, f(x) \stackrel{?}{=} f(x), f(x) \stackrel{?}{=} ?x\}; \eta && \text{trivial} \\
&\Longrightarrow \{?y \stackrel{?}{=} c, f(x) \stackrel{?}{=} ?x\}; \eta && \text{trivial} \\
&\Longrightarrow \{f(x) \stackrel{?}{=} ?x\}; [?y := c] && \text{elim} \\
&\Longrightarrow \{?x \stackrel{?}{=} f(x)\}; [?y := c] && \text{orient} \\
&\Longrightarrow \emptyset; [?y := c][?x := f(x)] && \text{elim}
\end{aligned}$$

Note that the substitution $[?y := c][?x := f(x)]$, which we obtain in the final step, is exactly the substitution that we have identified in example 6.3.

Let us make a few observations on example 6.10. First of all, the sequence ended in a unifier for the set $E$, and not just any, it ended with a mgu! Second, we picked along the way the easiest cases that we could apply (trivial) before doing anything else. It turns out that, no matter in which order the cases are treated along a sequence of unification steps, the sequence always ends and the final result is independent of the order in which cases have been treated. This is recorded in the following theorem, whose proof and other properties of the transition relation $\Longrightarrow$ can be found in [BS01, Section 2.2.4].

**Theorem 6.11**

Let $E$ be a finite set of unification problems. Any sequence

$$E; \eta \Longrightarrow E_1; \sigma_1 \Longrightarrow E_2; \sigma_2 \Longrightarrow \cdots$$

terminates either with **fail** or $\emptyset; \sigma$. In the first case, $E$ has no unifier, while in the second case, $\sigma$ is the most general unifier for $E$.

To understand a bit better the different clauses of the unification relation, let us go through some examples that illustrate the match, clash and cycle clauses. We start with a slightly more complex unification problem that requires the match clause.

**Example 6.12**

Let $E = \{g(?x, c) \stackrel{?}{=} g(f(?y), ?y)\}$ and obtain the following sequence of unification steps.

$$\begin{aligned} E; \eta &\Longrightarrow \{?x \stackrel{?}{=} f(?y), c \stackrel{?}{=} ?y\}; \eta && \text{match} \\ &\Longrightarrow \{c \stackrel{?}{=} ?y\}; [?x := f(?y)] && \text{elim} \\ &\Longrightarrow \{?y \stackrel{?}{=} c\}; [?x := f(?y)] && \text{orient} \\ &\Longrightarrow \emptyset; [?x := f(c)][?y := c] && \text{elim} \end{aligned}$$

Thus, we have successfully unified $g(?x, c)$ and $g(f(?y), ?y)$ with the substitution $[?x := f(c)][?y := c]$.

The next example illustrates when two terms cannot be unified because their function symbols don't match.

**Example 6.13**

Let $E = \{g(?x) \stackrel{?}{=} f(?y)\}$, for which we have only one possible clause that applies:

$$\{g(?x) \stackrel{?}{=} f(?y)\}; \eta \Longrightarrow \textbf{fail} \qquad \text{clash}$$

There, by theorem 6.11, the problem in $E$ cannot be solved.

Finally, let us look at an example where the cycle clause applies.

**Example 6.14**

Let $E = \{?x \stackrel{?}{=} f(?x)\}$ and note that $\mathrm{var}(f(?x)) = \{?x\}$. Since $f(?x) \neq ?x$, we have that unification fails:

$$E; \eta \Longrightarrow \textbf{fail} \qquad \text{cycle}$$

The reason for this failure is that a unifier $\sigma$ for $E$ would fulfil

$$\sigma(?x) = f(\sigma(?x)) = f(f(\sigma(?x))) = \cdots ,$$

which is clearly not possible and unification must fail.

With unification in place, it is fairly straightforward to implement the verification of proof trees in $\mathbf{ND}_1$. As one may expect, variable names are also not trivial to handle in an implementation. One possible way to handle those are de Bruijn trees, which are briefly explained in appendix D. We will not go fur-

ther into the details of either implementing a proof checker, but we developed the general principles. These can be easily adapted to other proof systems or integrated into an interactive system [Pau92].

The sun is setting and a light breeze blows through the trees.

'It will get dark soon. Shall we move to my house? Then I can put these herbs to good use and prepare us a meal. I am getting hungry.'

"We would be delighted to join. I am starving!", Clara feels that she has not eaten in a long time, whatever that may mean when travelling through time.

"Then let me bring those cups back in and follow me."

Clara and Isaac follow al-Khwārizmī, leaving the *City of Peace* and the palace behind. Arriving at his house, al-Khwārizmī brings them to a small room for dining and prepares a meal outside. Smells of spices, apricots, almonds and things she cannot identify reach Clara's nose.

Clara speaks in a low voice: "Now that we know how to check proofs, do you think that you can verify what you have found so far in your memory?"

"Yes, I have done so already and can confirm their correctness."

'Great! I thought about all of this though. I mean, in the end of the day, you could just claim that the proofs are correct. How can I be sure that you did actually check them and that your implementation of the check is correct?', responds Clara drifting away in her thoughts.

'It is true that there is no guarantee, and there is nothing I can do about that. If you are afraid that I may hide something from you, then perhaps the best is to end our trip here and I bring you back home.'

Clara stares at Isaac: 'But then you will never recover your memory? What will you do?'

"I will get back to the time I came from and let fate deal with it."

"Fate? It?", exclaims Clara in shock. "How can you just give yourself up like that?"

'You know that I do not have the same feelings as you. Even though I try to preserve myself, not harming humans is more important than that.'

"That sounds oddly familiar.", Clara tries to remember where she heard about this rule.

Al-Khwārizmī returns with a steaming pot and the smell immediately warms Clara's heart. He distributes plates and serves the meal. While eating, Clara and Isaac pose their problem to al-Khwārizmī.

"Oh, what an interesting problem!", he exclaims and falls into thought. After a minute, he carefully presents his argument: 'I think that there is no way

to be absolutely sure that a proof is correct because, even if you check every proof yourself you may make mistakes, even if you build yourself a machine to check it may be incorrect, and even if it is correctly built it may fail on a particular day due to humidity or other environmental influences. There is only one way I see one can ensure with high confidence that a proof is correct: find one common representation of proofs that is simple to check, let several people build independently several machines to verify proofs, and use these machines often on many proofs to make sure that they are well-made. This way, one can improve the odds that a proof is correct, but I don't think that there can ever be a guarantee.'

Clara is disappointed: "What is then the value of mathematics and logic?"

"I think, you know the answer in your heart.", al-Khwārizmī responds mysteriously.

During their meal, they discuss the discoveries of mathematics, the life in Baghdad, and the food variety available there.

Once finished, Isaac poses his burning question: "So we can mechanically verify proofs, but how about finding them? Would that not be more useful?"

'That would certainly be useful, but strikes me as much more complex. Think about all the ingenuity and creativity that goes into mathematics! Leaving such reflections aside, I am afraid that I cannot help you here. Also, it is getting late and I would like to rest. Would you like to stay at my house over night?'

"A very kind offer, but that is not...", Isaac is interrupted by Clara.

"In fact, I think some rest would help us before we continue our journey!"

Isaac looks surprised: "You want to...?"

"Yes!"

"Very well then! Let me show you where you can stay."

Al-Khwārizmī shows Clara and Isaac a small room with simple straw mattresses, and then leaves them for the night. The room has a window through which they can see millions of stars in the dark sky.

"Isn't this beautiful?", says Clara in a dreamy voice. "I have never seen so many stars because of all the lights in our world."

"So you want to continue the journey?", asks Isaac carefully.

'Yes. I have thought about this. For now everything went well, and even though there is no guarantee that all goes well, I also have no reason to distrust you. You have taken care of me during this trip and I really want to know what happened to you before you came to my room.'

"I am happy to hear that!"

Clara smiles: "Are you? How do you know?"

'Just a "feeling", I guess.', says Isaac while indicating the quotes in the air.

"You can actually be funny!", Clara removes laughing tears from her eyes. "But what now? Where can we go?"

"This time, I have an idea. I hope this will go well though, because we will get into a troubled period."

"Which time in history was not troubled?"

"Well, some are more than others."

"True, true.", Clara's turns her view to the sky outside the window.

Isaac waits a moment with his explanation: 'In any case, I suggest to visit a logician and philosopher whose speciality was that of inference and induction, but not in the sense of finding proofs or the proof method, rather in the sense of creating hypotheses and theories from observation. This is not precisely what we need but she was well versed in the logic literature of her time, translated Russell's works and was deeply embedded in the Polish logic school that brought us people like Tarski and Lindenbaum. I'm sure that she will be able to guide us to understand how proofs can be mechanically found.'

"This sounds great! So where do we go?"

"Vilnius, 1941."

"I understand your point about the troubled times. Well, let us get some rest then!"

Clara turns around and quickly falls into deep sleep. Isaac stays up, guarding his regained friend through the night.

# Exercises

## Exercise 23

Finish the proof of lemma 6.6, that is, show that for all substitutions $\sigma$, $\tau$ and $\gamma$, the equality $\gamma \bullet (\tau \bullet \sigma) = (\gamma \bullet \tau) \bullet \sigma$ holds.

## Exercise 24

Finish the proof of lemma 6.8, that is, show that the relation $\sqsubseteq$ is transitive.

## Exercise 25

Determine which of the following sets of unification problem has a unifier and give the most general unifier for those that have a unifier using the unification relation.

a) $\{f(a, ?x) \stackrel{?}{=} ?y, g(b, ?y) \stackrel{?}{=} ?z\}$

b) $\{f(a, ?x) \stackrel{?}{=} ?y, f(b, c) \stackrel{?}{=} ?y\}$

c) $\{f(a, ?x) \stackrel{?}{=} ?y, f(a, ?y) \stackrel{?}{=} ?x\}$

d) $\{f(a, g(?x)) \stackrel{?}{=} ?y, f(a, g(?z)) \stackrel{?}{=} ?y\}$

# 7. First-Order Horn Clauses and Automatic Deduction

Clara opens her eyes and stretches. The sun coming through the window is warming her skin. She hears the voices of Al-Khwārizmī and Isaac come in through the door, gets up and joins them outside the house.

"A wonderful morning!" Al-Khwārizmī makes a gesture to Clara to come over: "Please, join us for some fruits and coffee. Isaac and I were reflecting on our discussion from yesterday, exploring a few more ideas on how to be sure about the correctness of proofs. Ultimately, we figured, one has to have trust in something or someone. But I understand that you wish to leave soon for the next stop of your journey."

Clara nods: "Indeed, we aim to leave this morning."

"Very well, get some strengths and then we will all start our day."

Clara sits down and the three have their breakfast. After wishing Al-Khwārizmī farewell, Clara and Isaac leave into a quiet alley.

Clara opens the conversation: "That was much needed rest. Thank you for watching over me during the night, I felt very relaxed."

"You noticed that?", asks Isaac surprised.

"I sensed it, yes!"

"I think there are things that I will never understand about humans… In any case, shall we continue our trip?"

"Yes, lets go!", exclaims Clara and closes her eyes.

When she feels that her weights comes back, she opens her eyes again. They find themselves in a small street with cobblestones, a mix of 18th century Baroque buildings and less decorative brick buildings. Some people sit in the street, drinking coffee on this sunny summer afternoon. Just when Clara begins to enjoy the atmosphere, she sees soldiers in black uniform with black riding boots marching down the street.

"Are these…?", Clara is scared.

"Yes! Let us quickly leave this street and go to our destination."

Isaac leads Clara into a narrow street and they stop in front of a simple brick building. Three names are displayed on the doorbells, but Clara does not recognise any of them. Isaac rings the bell with a handwritten sign saying "Pánski-Hosiasson".

Clara is puzzled: "Who is this?"

"We are visiting Janina Hosiasson."[15]

"The logician and philosopher that you mentioned?"

"Yes, her."

A women slightly opens the door: "Who are you?", she asks with a low voice.

"We are looking for Janina Hosiasson, hoping that she could help us with questions in logic."

"Logic? I have not met any one in a long time who was interested in logic! Are you alone?"

"Yes, we are.", responds Clara.

"Alright, come in. Quickly!"

Isaac and Clara slip in through the door and Hosiasson immediately closes it behind them. She leads them into a flat on the second floor. Hosiasson's walk is elegant and her face finely chiselled, a face that also has worries written all over it. They enter directly into a room of about 15 square meters with a dining table and a worn out sofa. The decoration is simple, but cosy.

"Would you like some refreshment?"

"That would be wonderful!", Clara smiles.

Clara looks around, while Hosiasson goes to the small kitchen in the room. She comes back with three glasses.

"Please, have some lemonade. You must be thirsty from the heat! What is it that I could be able to help you with? Actually, let us start with introducing ourselves. Please call me Janina."

"My name is Clara, and this is my friend Isaac.", Clara puts her hand gently on Isaac's shoulder, "We are trying to learn about certain aspects of logic and heard that you may be the right person to help us with automatic deduction."

"Automatic deduction? I know a few things about inference and induction, but deduction? What is that you have in mind?"

Clara sends a worried look to Isaac and then clarifies: "We learned about proof systems for first-order logic, but we would like to understand if and how it is possible to devise some mechanical procedure to construct such proofs."

"Aha, now I understand! Professor Tarski has worked on such questions, but I think you are interested in something else. I read some work of a mathematician named Jacques Herbrand. Very bright young man, but he tragic-

ally died in a climbing accident. However, he left us his thesis[16], in which he showed that the task for proving certain formulas in first-order logic can be reduced to a search for a proof in propositional logic combined with a search for substitutions. This gives rise to the automatic deduction that you are looking for. But let us begin by analysing where the problems in proof search arise."

## 7.1. Automatic Deduction and the Cut-Rule

When deducing mathematical theorems, we usually decompose the problem into intermediate results that are easier to prove on their own. In the natural deduction systems, the approach is justified by the so-called **cut rule**. This rule allows one to prove first a formula $\varphi$, often called a *lemma*, then prove a formula $\psi$ under the assumption of $\varphi$, and finally the conclude that $\psi$ holds without explicit reference to the lemma $\varphi$. The corresponding rule can be formulated as

$$\frac{\Delta \mid \Gamma \vdash \varphi \qquad \Delta \mid \Gamma, \varphi \vdash \psi}{\Delta \mid \Gamma \vdash \psi}\ (\text{Cut})$$

and it is admissible via the following proof tree.

$$\frac{\Delta \mid \Gamma \vdash \varphi \qquad \dfrac{\Delta \mid \Gamma, \varphi \vdash \psi}{\Delta \mid \Gamma \vdash \varphi \to \psi}\ (\to\text{I})}{\Delta \mid \Gamma \vdash \psi}\ (\to\text{E})$$

The cut rule is very useful in structuring proofs, but horrendous for automatic deduction because it requires ingenuity for inventing the intermediate lemma $\varphi$ to prove $\psi$. There are techniques for generating such lemmas but we will take another route here, which is to avoid using the cut rule entirely. Looking at the proof of the cut rule, we see that it is based on eliminating an implication immediately after introducing it. This is what is called a detour.

**Definition 7.1: Proof Detours**

Let us denote by $\boxed{\mathrm{Pr}(\Delta \mid \Gamma \vdash \varphi)}$ the set of $\mathbf{ND}_1$ proof trees for $\Delta \mid \Gamma \vdash \varphi$. We call a **detour** a proof tree that ends in an introduction of a connective immediately followed by an elimination of that connective, as in fig. 7.1. A proof tree is in **normal form** if no detour occurs in any of its subtrees.

Let us give an example of a deduction with a detour and one without.

$$\dfrac{\dfrac{T_0}{\Delta \mid \Gamma \vdash \varphi \to \psi}\,(\to\mathrm{I}) \qquad T_1}{\Delta \mid \Gamma \vdash \psi}\,(\to\mathrm{E})$$

for $T_0 \in \mathrm{Pr}(\Delta \mid \Gamma, \varphi \vdash \psi)$
and $T_1 \in \mathrm{Pr}(\Delta \mid \Gamma \vdash \varphi)$

$$\dfrac{\dfrac{T_1 \qquad T_2}{\Delta \mid \Gamma \vdash \varphi_1 \wedge \varphi_2}\,(\wedge\mathrm{I})}{\Delta \mid \Gamma \vdash \varphi_k}\,(\wedge\mathrm{E}_k)$$

for $T_1 \in \mathrm{Pr}(\Delta \mid \Gamma \vdash \varphi_1)$
and $T_2 \in \mathrm{Pr}(\Delta \mid \Gamma \vdash \varphi_2)$

$$\dfrac{\dfrac{T_0}{\Delta \mid \Gamma \vdash \varphi_1 \vee \varphi_2}\,(\vee\mathrm{I}_k) \qquad T_1 \qquad T_2}{\Delta \mid \Gamma \vdash \psi}\,(\vee\mathrm{E})$$

for $T_0 \in \mathrm{Pr}(\Delta \mid \Gamma \vdash \varphi_k)$, $T_1 \in \mathrm{Pr}(\Delta \mid \Gamma, \varphi_1 \vdash \psi)$ and $T_2 \in \mathrm{Pr}(\Delta \mid \Gamma, \varphi_2 \vdash \psi)$

$$\dfrac{\dfrac{T_0}{\Delta \mid \Gamma \vdash \forall x.\, \varphi}\,(\forall\mathrm{I})}{\Delta \mid \Gamma \vdash \varphi\,[x := t]}\,(\forall\mathrm{E})$$

for $T_0 \in \mathrm{Pr}(\Delta, x \mid \Gamma \vdash \varphi)$

$$\dfrac{\dfrac{T_0}{\Delta \mid \Gamma \vdash \exists x.\, \varphi}\,(\exists\mathrm{I}) \qquad T_1}{\Delta \mid \Gamma \vdash \psi}\,(\exists\mathrm{E})$$

for $T_0 \in \mathrm{Pr}(\Delta \mid \Gamma \vdash \varphi\,[x := t])$
and $T_1 \in \mathrm{Pr}(\Delta, x \mid \Gamma, \varphi \vdash \psi)$

Figure 7.1.: Proof Detours

**Example 7.2**

Let $\psi$, $\varphi_1$ and $\varphi_2$ be given by $\psi = \forall x.\, P(x)$, $\varphi_1 = P(c) \to Q(c)$ and $\varphi_2 = P(c) \to R(c)$. Moreover, we set $\Gamma = \psi, \varphi_1, \varphi_2$. The following is a proof with a detour, which is actually an instance of the cut rule.

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{\Gamma, P(c) \vdash \varphi_1} \quad \dfrac{}{\Gamma, P(c) \vdash P(c)}}{\Gamma, P(c) \vdash Q(c)}\,(\to\mathrm{E}) \quad \dfrac{\dfrac{}{\Gamma, P(c) \vdash \varphi_2} \quad \dfrac{}{\Gamma, P(c) \vdash P(c)}}{\Gamma, P(c) \vdash R(c)}\,(\to\mathrm{E})}{\Gamma, P(c) \vdash Q(c) \wedge R(c)}\,(\wedge\mathrm{I})}{\Gamma \vdash P(c) \to Q(c) \wedge R(c)}\,(\to\mathrm{I}) \qquad \dfrac{\dfrac{}{\Gamma \vdash \psi}}{\Gamma \vdash P(c)}\,(\forall\mathrm{E})}{\Gamma \vdash Q(c) \wedge R(c)}\,(\to\mathrm{E})}{\Gamma \vdash \exists x.\, Q(x) \wedge R(x)}\,(\exists\mathrm{I})$$

We can remove the detour by moving proof of the antecedent $P(c)$, which is used in the application of the (→E)-rule, to where the assumption $P(c)$ was used in the previous deduction and then removing the combination of (→E) after (→I):

$$\dfrac{\dfrac{\dfrac{}{\Gamma \vdash \varphi_1} \quad \dfrac{\dfrac{}{\Gamma \vdash \psi}}{\Gamma \vdash P(c)}\,(\forall\mathrm{E})}{\Gamma \vdash Q(c)}\,(\to\mathrm{E}) \quad \dfrac{\dfrac{}{\Gamma \vdash \varphi_2} \quad \dfrac{\dfrac{}{\Gamma \vdash \psi}}{\Gamma \vdash P(c)}\,(\forall\mathrm{E})}{\Gamma \vdash R(c)}\,(\to\mathrm{E})}{\dfrac{\Gamma \vdash Q(c) \land R(c)}{\Gamma \vdash \exists x.\, Q(x) \land R(x)}\,(\exists\mathrm{I})}\,(\land\mathrm{I})$$

This new proof has no more detours and is thus in normal form.

In example 7.2, we removed a detour by copying the proof of an intermediate result. This is possible in general and any proof can be normalised.

**Theorem 7.3: Proof normalisation**

If $\Delta \mid \Gamma \vdash \varphi$ is derivable in $\mathbf{ND}_1$, then there is a proof of $\Delta \mid \Gamma \vdash \varphi$ in normal form.

*Proof.* We shall only discuss the idea of how detours introduced by a cut can be removed, as its implementation is rather technical. Details can be found in texts on proof theory [TS00; TvD88a]

In the (Cut)-rule, we split the proof of a lemma $\varphi$ from its use in a proof of $\psi$. To avoid a cut, we can thus use the proof of $\varphi$ directly everywhere we use the assumption $\varphi$ in the proof of $\psi$. More precisely, suppose we have a proof tree $T \in \mathrm{Pr}(\Delta \mid \Gamma \vdash \varphi)$ for for the sequent $\Gamma \vdash \varphi$. Any proof tree for $\Gamma, \varphi \vdash \psi$ can only use $\varphi$ through the (Assum)-rule and the tree must be of the following form, where we display only one use of the (Assum)-rule.

$$\dfrac{\begin{array}{c}\dfrac{}{\Delta \mid \Gamma, \varphi \vdash \varphi}\,(\mathrm{Assum}) \\ \vdots \\ \Delta \mid \Gamma, \varphi \vdash \psi\end{array} \qquad \dfrac{T}{\Delta \mid \Gamma \vdash \varphi}}{\Delta \mid \Gamma \vdash \psi}\,(\mathrm{Cut})$$

We can now replace everywhere the use of the (Assum)-rule applied to $\varphi$ directly by the proof tree $T$, as in the following tree that does not use the assump-

tion $\varphi$ anymore.

$$\begin{array}{c} T \\ \hline \Delta \mid \Gamma \vdash \varphi \\ \vdots \\ \Delta \mid \Gamma \vdash \psi \end{array}$$

This procedure has to be applied repeatedly until all occurrences of the (Cut)-rule have been removed. Note that removing an instance of the (Cut)-rule may create new instances of it. The main work lies in proving that the procedure terminates and yields proof in normal form. This requires somewhat advanced techniques that we will not discuss at this point. □

In particular, theorem 7.3 tells us that a sequent $\Delta \mid \Gamma \vdash \varphi$ can be proven without using the cut rule. It is thus a first step to automatic deduction, as it allows us to avoid the use of the cut rule and thereby having to come up with intermediate formulas out of thin air. This particular version of theorem 7.3 was first proven by Gentzen [Gen35] in his "Hauptsatz".

This is good news for automatic deduction because we can limit at each step the rules that may be applied by inspecting the formula $\varphi$ that we have to prove. The approach of inspecting $\varphi$ in finding a proof for $\Gamma \vdash \varphi$ is also called **goal-oriented** because the search for the proof is driven by the goal $\varphi$. However, restricting ourselves to proofs that avoid the cut rule is not enough for fully automatic deduction because there are other choice points. For instance, if we try to prove $\exists x.\, P(x) \vdash \neg\forall x.\, \neg P(x)$ do we first use the introduction of the negation or the elimination of the existential quantifier? The problem is that the principle formula of elimination rules, the formula that gives a rule its name, appears among the premises of those rules. This means for the goal-oriented construction of a proof that we have to guess the right formula to eliminate from the goal, while there may be a non-trivial relation between the two. In the above example, we have to guess that we have to eliminate $\exists x.\, P(x)$ to prove $\neg\forall x.\, \neg P(x)$, which is difficult for human intelligence, let alone for a computer. One way out of this is to limit the class of formulas that may appear in proofs and thereby reduce the amount of guessing to a manageable amount. Combined with limiting proof rules, we obtain a reasonable fragment of first-order logic for automatic deduction.

A remarkable result that guides the search for a fragment of first-order logic is the following.

**Theorem 7.4: Herbrand's Theorem**

Let $\varphi$ be a formula without quantifiers and $\mathrm{fv}(\varphi) = \{x_1, \dots, x_n\} \cup \Delta$. The sequent

$$\Delta \mid \cdot \vdash \exists x_1. \cdots \exists x_n. \varphi$$

is provable in $\mathbf{ND}_1$ if and only if there are substitutions $\sigma_1, \dots, \sigma_r$ with $\mathrm{supp}(\sigma_k) = \{x_1, \dots, x_n\}$ and the sequent

$$\Delta \mid \cdot \vdash \varphi\,\sigma_1 \lor \cdots \lor \varphi\,\sigma_r \tag{7.5}$$

is provable in $\mathbf{ND}_1$.

Without going into much detail, let us sketch how this theorem is proven. The direction of going from eq. (7.5) to section 7.1 in theorem 7.4 is simply using the (∃I)-and (∨E)-rules several times. Proving the other direction uses theorem 7.3 to first normalise a proof of section 7.1 by removing all detours. From a proof in normal form, one can then extract the necessary substitutions by walking the tree and generating unification problems. Finally, one proves that these systems are solvable. This proof approach is exactly what can help us to automatically find proofs.

## 7.2. First-Order Horn Clauses and Logic Programming

Most of the knowledge we gather can be divided into **facts**, things that you know for sure, and **inference rules**, which allow deduction of new knowledge. Such formulas can be written in a special form as so-called *Horn clauses*.

**Definition 7.6: Horn Clauses and Theories in FOL**

A **predicate atom** over a signature $\mathcal{L}$ is a formula of the form $P(t_1, \dots, t_n)$, where $P$ is an $n$-ary predicate symbol in $\mathcal{L}$ and $t_1, \dots, t_n$ are $\mathcal{L}$-terms. Predicate atoms are denoted by letters $A, B, C, \dots$ We say that a formula $\varphi$ is **closed** if $\mathrm{fv}(\varphi) = \emptyset$. A **Horn clause** is a closed formula of the form

$$\forall x_1. \cdots \forall x_m. A_1 \land \cdots \land A_n \to A_0$$

for $m, n \in \mathbb{N}$ and predicate atoms $A_0, A_1, \dots, A_n$. The atom $A_0$ is called the *head* and the set $\{A_1, \dots, A_n\}$ the **body** of the clause. We call a context $\Gamma$ of Horn clauses a **Horn clause theory** or **logic program**.

The body of a clause may be empty, that is, $n$ may be zero, in which case we leave out the implication in the Horn clause in definition 7.6 and just write $\forall x_1. \cdots \forall x_m. A_0$. This is convenient when writing logic programs that contain Horn clauses without assumptions, so called *base facts*. In the abstract treatment of Horn clauses later in this chapter, it will, however, not be necessary to differentiate between base facts and general Horn clauses.

It is also important to note that a Horn clause must be closed, thus only the variables $x_1, \dots, x_m$ may appear in the head and body of a Horn clause. These are then all bound by the quantifiers on the outside.

What makes Horn clause theories so useful? As the name "logic program" indicates, we can use such theories for programming. In fact, we can describe *any* Turing machine by a logic program $\Gamma$. A computation corresponds to giving a predicate atom $A$ with free variables $x_1, \dots, x_m$ and asking for a derivation of $\Gamma \vdash \exists x_1. \cdots \exists x_m. A$ in $\mathbf{ND}_1$. If the Turing machine halts, then such a derivation exists. This correspondence makes it possible to automatically find such a derivation if it exists. Thus, finding proofs is for Horn clause theories only semi-decidable but, as we will see, it is still much easier in many cases to find proofs from Horn clause theories rather than arbitrary theories.

Let us demonstrate the programming aspect of Horn clauses. Recall that in primitive recursive arithmetic and other approaches to arithmetic, we have always assumed that all objects were numbers. Often, we need to reason about different *types* of objects. The following example shows how we can axiomatise with Horn clauses a predicate that characterises natural numbers.

**Example 7.7**

In this example, we will use that every natural number is either zero or the successor of another natural number. More specifically, the constant $\underline{0}$ represents in the following zero and the unary function symbol $s$ represents the successor of a number. With these two symbols, we can enumerate all the natural numbers:

$$\underline{0},\ s(\underline{0}),\ s(s(\underline{0})),\ s(s(s(\underline{0}))), \dots$$

Writing natural numbers in this way is called *unary encoding*, which is a very bad encoding from the perspective of complexity but very useful for explanatory purposes.

In this example, we will be working with a signature $\mathcal{L}$ that contains a constant $\underline{0}$, a unary function symbol $s$, a unary predicate symbol $N$, a binary predicate symbol $L$, and a ternary predicate symbol $P$. We use the unary

predicate symbol $N$ to pin down the natural numbers, in the sense that $N(t)$ holds if $t$ is the representation of a natural number. The following two Horn clauses implement precisely the initial description of natural numbers.

$$N(\underline{0})$$
$$\forall n.\, N(n) \to N(s(n))$$

We can now continue and implement arithmetic for natural numbers. To use Horn clauses for this task, we have to switch from thinking in terms of function to thinking in terms of relations. More specifically, we use a ternary predicate symbol $P$ with the intent that $P(m, n, k)$ holds if $k$ is the sum of $m$ and $n$. To provide Horn clauses that implement addition, recall the primitive recursive specification of addition: $0 + m = m$ and $(n + 1) + m = (n + m) + 1$. Assuming that $n$ and $m$ are given in the above representation of natural numbers, we can represent this recursive specification of addition by the following two Horn clauses.

$$\forall m. \qquad P(\underline{0}, m, m)$$
$$\forall n.\, \forall m.\, \forall k.\, P(n, m, k) \to P(s(n), m, s(k))$$

In a similar spirit, we can define other recursive relations on natural numbers. For instance, to specify that the binary predicate $L$ represents the less or equal relation, we would use the following two Horn clauses.

$$\forall n. \qquad L(n, n)$$
$$\forall n.\, \forall m.\, L(n, m) \to L(n, s(m))$$

The clauses say that every natural number is less or equal to itself (reflexivity) and that increasing the number on the right preserves the less or equal relation.

?

Can you find two Horn clauses that describe the computation of multiplication as a ternary predicate symbol $M$ analogously to $P$ in example 7.7?

What makes logic programming so interesting compared to other programming paradigms? The answer lies in its declarative nature, which allows us to specify what the result of a computation should be rather than how the result is computed. In his seminal work, Kowalski [Kow79] has expressed this in the following equation.

Algorithm = Logic + Control

This equation essentially means that we can design algorithms by providing logical formulas that describe the result of an algorithm and a control mechanism that controls the use of these formulas to compute the result. A particular property of this approach is that new algorithms can be obtained by exchanging the control mechanism, while preserving the logical properties and correctness of the computed result. In terms of the above equation, we may have a logical specification $L$ and algorithms $A_1$ and $A_2$ given by $A_1 = L + C_1$ and $A_2 = L + C_2$ for some control mechanisms $C_1$ and $C_2$. These two algorithms will compute results with the same logical properties, but may have different computational behaviour. For example, $A_2$ may be more efficient than $A_1$.

Let us return to concrete logic programming and illustrate this on our initial robot example.

**Example 7.8: Path Finding**

Remember the board from fig. 4.1 on page 58 that Isaac drew? In section 4.1, we used first-order logic to describe what moves on the board are allowed and what a route towards the heart looks like. The aim of this example is to use Horn clauses to give a logical description of paths that will allow us, with an appropriate control mechanism, to derive an efficient algorithm for routing.

For this purpose, we will use the signature $\mathcal{L}$ with constants $\underline{1}, \underline{2}, \ldots$ for all positive natural numbers, one binary function symbol pos, three unary predicate symbols $R$, $H$ and $F$, and two binary predicate symbols $A$ and $C$. Let us explain the intention of all these symbols. We will use pos to describe positions on the board and will denote by $\text{pos}(x, y)$ the position with horizontal coordinate $x$ and vertical coordinate $y$. For instance, the robot is in fig. 4.1 in position $\text{pos}(\underline{2}, \underline{3})$. The unary predicate symbols $R$, $H$ and $F$ describe, respectively, the position of the **R**obot, the position of the **H**eart and **F**ree positions. Since the robot may only move between adjacent positions, will use the binary predicate symbol $A$ to express when a position is **A**djacent to another position. Finally, the binary predicate symbol $C$ is what we are after: it will relate two positions if they are **C**onnected by a path via free and adjacent positions.

Our goal is to describe the situation in fig. 4.1 and the predicate symbol $C$ as a logic program, that is, a finite list of Horn clauses. All of the following is provided in appendix F as a Prolog program that can be directly run in your favourite Prolog interpreter.

Let us begin with the easy part: the position of the robot and the heart. These are given by the following two formulas.

$$R(\mathrm{pos}(\underline{2}, \underline{3})) \quad \text{and} \quad H(\mathrm{pos}(\underline{5}, \underline{1})) \tag{7.9}$$

These two formulas are base facts, albeit with no quantifiers and an empty body, that is, $m$ and $n$ in definition 7.6 are both zero.
Next, we describe the free position that the robot may visit by providing a formula for each free position:

$$\begin{array}{llll} F(\mathrm{pos}(\underline{1}, \underline{1})) & F(\mathrm{pos}(\underline{2}, \underline{2})) & F(\mathrm{pos}(\underline{3}, \underline{1})) & \\ F(\mathrm{pos}(\underline{1}, \underline{4})) & F(\mathrm{pos}(\underline{2}, \underline{3})) & F(\mathrm{pos}(\underline{3}, \underline{2})) & \ldots \\ & F(\mathrm{pos}(\underline{2}, \underline{4})) & F(\mathrm{pos}(\underline{3}, \underline{4})) & \end{array} \tag{7.10}$$

You may have noticed that we have initially talked in section 4.1 about the obstacles on the board and now we talk about free positions instead. The reason for this is that obstacles are an inherently negative description of the allowed moves that the robot may make. More precisely, we have the relation

$$\forall p.\, F(p) \leftrightarrow \neg O(p)\,,$$

where $O$ was the predicate symbol that we used for describing the positions of obstacles. This negative relation between $F$ and $O$ would become a problem when we ask if the robot can move to a certain position because the first-order Horn clauses in definition 7.6 may not use $\perp$ and therefore no negation! Thus, we would not be able to formulate the routing problem as a logic program if we describe positions with obstacles instead of free positions.
To finish the board description, we have to give all the adjacent positions by providing formulas that specify the predicate symbol $A$. This predicate should be read as: if $A(p, q)$ holds then position $p$ and $q$ are adjacent. For instance, we should have that $A(\mathrm{pos}(\underline{1}, \underline{1}), \mathrm{pos}(\underline{1}, \underline{2}))$ holds. However, neither $A(\mathrm{pos}(\underline{1}, \underline{1}), \mathrm{pos}(\underline{2}, \underline{2}))$ nor $A(\mathrm{pos}(\underline{6}, \underline{1}), \mathrm{pos}(\underline{7}, \underline{1}))$ should not hold. In the first case, the two positions $\mathrm{pos}(\underline{1}, \underline{1})$ and $\mathrm{pos}(\underline{2}, \underline{2})$ are not adjacent, as we do not allow diagonal steps. In the second case, the term $\mathrm{pos}(\underline{7}, \underline{1})$ is not a valid position on the board because the horizontal coordinates range from 1 to 6 only. Formalising all of this as Horn clauses is a bit tedious because we have to enumerate for each position all its four neighbours, except

for positions at the boundary that have two or three neighbours:

$$\begin{aligned}
&A(\mathrm{pos}(\underline{x}, \underline{y}), \mathrm{pos}(\underline{x+1}, \underline{y})), && 1 \leq x \leq 5, && 1 \leq y \leq 4 \\
&A(\mathrm{pos}(\underline{x}, \underline{y}), \mathrm{pos}(\underline{x}, \underline{y+1})), && 1 \leq x \leq 6, && 1 \leq y \leq 3 \\
&A(\mathrm{pos}(\underline{x}, \underline{y}), \mathrm{pos}(\underline{x-1}, \underline{y})), && 2 \leq x \leq 6, && 1 \leq y \leq 4 \\
&A(\mathrm{pos}(\underline{x}, \underline{y}), \mathrm{pos}(\underline{x}, \underline{y-1})), && 1 \leq x \leq 6, && 2 \leq y \leq 4
\end{aligned} \tag{7.11}$$

The first line of (7.11) states that every position $\mathrm{pos}(\underline{x}, \underline{y})$ is adjacent to its east neighbour $\mathrm{pos}(\underline{x+1}, \underline{y})$. Note that $\mathrm{pos}(\underline{x}, \underline{y})$ only has a neighbour in the east if it is not a boundary position, that is, if $x \leq 5$. In the remaining three lines, we analogously provide the specification of the southern, western and northern neighbours.
With the board layout described through the Horn clauses in eqs. (7.9) to (7.11), we can now try to find a way for Isaac to the heart. We achieve this by providing a description of the predicate symbol $C$ with the intention that $C(p, q)$ holds if position $q$ is reachable from $p$ via free and adjacent positions. Reachability can be described by two Horn clauses, one that states that every position can be reached from itself and one for making an intermediate step via a free position:

$$\forall p. \quad C(p, p) \tag{7.12}$$

$$\forall p.\, \forall q.\, \forall r.\, C(q, r) \land A(p, q) \land F(q) \to C(p, r) \tag{7.13}$$

These two Horn clauses formulate reachability as backward search. That is to say, we start with the final position we want to reach and then try to find positions that make a path to the initial position.

## 7.3. Uniform Proofs

In example 7.8, we have used logical formulas to describe how a path from the starting position to the goal looks like. What is missing to get an algorithm for routing is the control part of Kowalski's equation, which allows us to search for a path. We will introduce in this section a control mechanism based on proof theory, called uniform proofs.

The idea of uniform proofs is that we devise a proof system tailored towards proving goal formulas of a very specific shape only from Horn clauses. This restriction of goals and assumptions has a two-fold effect:

1. we remove all the choices appearing in elimination rules, and
2. we concentrate the choices in one proof rule.

Removing elimination rules in general and the cut rule in particular, rules out an infinitude of choices and makes the search for proof rules more goal-oriented. As we will see, the second aspect of concentrating choice reduces the amount of branching in proof search drastically to just a few options. All together, uniform proofs provide an operational perspective on proof search and will guide us in the construction of proofs, also in $\mathbf{ND}_1$, relative to logic programs.

Before we come to the actual definition of uniform proofs, we will need to talk about specific terms and substitutions that arise in proof search. Recall that a term with no variables is called closed. In the simplest case, which will be the only one that we treat here, we can prove an existentially quantified formula $\exists x.\, \varphi$ by providing a closed term $t$ and prove $\varphi[x := t]$. For instance, let $\varphi$ be $R(\mathrm{pos}(x, \underline{3}))$ and let $t$ be $\underline{2}$, as in example 7.8. Under the assumption of eq. (7.9), we have that $\varphi[x := t]$ holds and thus also $\exists x.\, (\mathrm{pos}(x, \underline{3}))$. Note that we substituted the closed term $\underline{2}$ for $x$ and thereby obtained a closed formula.[17] We will restrict the proof rules ($\exists$I) and ($\forall$E) of $\mathbf{ND}_1$ to allow only substitutions that result in closed formulas, which in turn will eliminate all free variables from proofs. The following definition formalises the needed substitutions.

**Definition 7.14: Closing substitution**

Let $X \subseteq \mathrm{Var}$ be a set of variables. An $X$-**closing substitution** is a substitution $\sigma \colon \mathrm{Var} \to \mathrm{Term}$, such that $\sigma(x)$ is closed for all $x \in X$.

The purpose of $X$-closing substitutions is to turn any formula, whose free variables are among those in $X$, into a closed formula.

**Example 7.15**

As we have $\mathrm{fv}(P(\underline{0}, x, x)) = \{x\}$, any $X$-closing substitution with $x \in X$ will allow us to close this formula. For instance, $s(\underline{0})$ is a closed term over the signature of example 7.7 and $[x := s(\underline{0})]$ is a $\{x\}$-closing substitution. Indeed, we obtain the closed formula $P(\underline{0}, x, x)\,[x := s(\underline{0})] = P(\underline{0}, s(\underline{0}), s(\underline{0}))$.

?

Do signatures without constants admit closed terms?

We mentioned initially that we obtain proof search by restricting the formulas $\varphi$ that can appear as a goal, that is, in a sequent $\Gamma \vdash \varphi$ that we wish to prove.

Out of the many choices one can make to restrict formulas, we will be using the one given in definition 7.16 below. Before we come to that definition, let us briefly think about what a reasonable set of goals, for which proofs can be searched, would be. First of all, we of course want to be able to prove facts about predicates, hence predicate atoms of the form $P(t_1, \dots, t_n)$ should certainly be allowed as goals. Next, conjunction does certainly not pose any problem because finding a proof for $\Gamma \vdash \varphi \wedge \psi$ only requires us to find proofs for $\Gamma \vdash \varphi$ and $\Gamma \vdash \psi$ separately by the introduction rule ($\wedge$I). To be able to state and prove more interesting properties, we will also allow disjunction and existential quantification. These two connectives will make proof search more difficult. Recall that the introduction rules ($\vee\mathrm{I}_1$) and ($\vee\mathrm{I}_2$) for disjunction require us to find proofs for *either* $\Gamma \vdash \varphi$ *or* $\Gamma \vdash \psi$ in order to prove $\Gamma \vdash \varphi \vee \psi$. Thus, to find a proof for the disjunction $\varphi \vee \psi$ we have to try to prove one of the options, say $\varphi$, and if we fail to prove this option, then we try the other. Trying out these different options is not difficult, but can be costly, depending on how many proof steps we have to carry out before we find out that $\varphi$ is not provable. Finally, the proof of an existential quantifier $\exists x.\, \varphi$ requires us to find a term $t$, such that $\varphi[x := t]$ holds. As it turns out, it is possible to devise a procedure that constructs a closed term $t$, if it exists, automatically. Thus, we will also allow existential quantifiers in proof goals. The following definition 7.16 sums up the formulas that we allow as goals.

**Definition 7.16: Goal formulas**

The set Goal of all **goal formulas** $\varphi_G$ is the set of $\mathcal{L}$-formulas over a signature $\mathcal{L}$ generated by the following grammar.

$$\varphi_G ::= P(t_1, \dots, t_n) \mid \varphi_G \wedge \varphi_G \mid \varphi_G \vee \varphi_G \mid \exists x.\, \varphi_G$$

Let us now come to uniform proofs, which are given by a proof system that allows us to prove goals $\varphi$ (definition 7.16) from Horn clause theories $\Gamma$ (definition 7.6). To distinguish sequents for uniform proofs from those of $\mathbf{ND}_1$, we will write $\Gamma \vdash_u \varphi$ for a sequent that we intend to find a uniform proof for. We have already explained the general approach to prove $\Gamma \vdash_u \varphi$ above for the case of conjunction, disjunction and existential quantification. This leads us to the introduction rules in the upper part of fig. 7.2, which are exactly the same rules as in $\mathbf{ND}_1$, just restricted to the introduction of goal formulas. Let us consider the goal $\varphi$ given by $\exists u.\, \exists v.\, R(u) \wedge A(u, v) \wedge F(v)$ over the signature from the robot path finding example 7.8. This formula says that the robot is in some position on the board that has a free adjacent position. Such positions exist, namely if we pick $p = \mathrm{pos}(\underline{2}, \underline{3})$ for the robot position and the adjacent position $q = \mathrm{pos}(\underline{2}, \underline{4})$. Using only the introduction rules from fig. 7.2, we can

**Introduction Rules**

$$\frac{\Gamma \vdash_u \varphi_1 \qquad \Gamma \vdash_u \varphi_2}{\Gamma \vdash_u \varphi_1 \wedge \varphi_2}\ (\wedge\mathrm{I}) \qquad \frac{\Gamma \vdash_u \varphi[x := t] \qquad t \text{ closed}}{\Gamma \vdash_u \exists x.\, \varphi}\ (\exists\mathrm{I})$$

$$\frac{\Gamma \vdash_u \varphi_1}{\Gamma \vdash_u \varphi_1 \vee \varphi_2}\ (\vee\mathrm{I}_1) \qquad \frac{\Gamma \vdash_u \varphi_2}{\Gamma \vdash_u \varphi_1 \vee \varphi_2}\ (\vee\mathrm{I}_2)$$

**Backchaining Rule**

$$\frac{(\forall x_1. \cdots \forall x_m. A_1 \wedge \cdots A_n \to A_0) : \Gamma \qquad \Gamma \vdash_u A_1[\sigma] \ \cdots\ \Gamma \vdash_u A_n[\sigma]}{\Gamma \vdash_u A_0[\sigma]}\ (\mathrm{B})$$

where $\sigma$ is an $\{x_1, \ldots, x_m\}$-closing substitution.

Figure 7.2.: Rules of Uniform Proofs

start a proof for $\varphi$ as in fig. 7.3.

However, we are not able to fill in the question marks, yet. Note that the formulas that we have to prove in lines 2, 3 and 5 are closed predicate atoms. It is easy to see that any proof attempt that starts with a goal formula and only uses the introduction rules, must eventually reach predicate atoms. Thus, we need a rule to prove such predicate atoms from a Horn clause theory. And that is the heart of logic programming: given a predicate atom, find a Horn clause that matches the atom, and continue with the premises of the Horn clause. This process is called *backchaining* and is captured by the rule (B) in fig. 7.2. The idea of the backchaining rule is that, whenever we have to prove a predicate atom, we choose a Horn clause from our logic program in such a way that the head of the clause (definition 7.6) matches the atom. If we find such a matching clause, then we continue by proving all the premises of the chosen Horn clause. Note that the premises of a Horn clause are always predicate atoms, thus we will continue using the backchaining rule until we can finish the proof by selecting a fact, which is a Horn clause without premises.

Let us use this procedure to fill in the question marks in the proof above. For the purpose of this, let us denote by $\Gamma_1$ the logic program that consists of all the Horn clauses in eqs. (7.9) to (7.11). In line 5, we have to prove $R(\mathrm{pos}(\underline{2}, \underline{3}))$,

| | | |
|---|---|---|
| 1 | $\vdots$ | |
| 2 | $A(p, q)$ | ?? |
| 3 | $F(q)$ | ?? |
| 4 | $A(p, q) \wedge F(q)$ | $\wedge$I, 2, 3 |
| 5 | $R(p)$ | ?? |
| 6 | $R(p) \wedge A(p, q) \wedge F(q)$ | $\wedge$I, 5, 4 |
| 7 | $\exists v.\, R(p) \wedge A(p, v) \wedge F(v)$ | $\exists$I, 6 |
| 8 | $\exists u.\, \exists v.\, R(u) \wedge A(u, v) \wedge F(v)$ | $\exists$I, 7 |

Figure 7.3.: Beginning of proof for a goal formula

which is exactly the fact that we assumed in (7.9). Thus, we can use the following instance of the backchaining rule, where both $n$ and $m$ are 0 and thus the substitution $\sigma$ is irrelevant.

$$\frac{R(\mathrm{pos}(\underline{2}, \underline{3})) : \Gamma_1}{\Gamma_1 \vdash_u R(\mathrm{pos}(\underline{2}, \underline{3}))} \text{ (B)}$$

The proofs for lines 2 and 3 are given analogously, which then results in the completed uniform proof in fig. 7.4.

| | | |
|---|---|---|
| 1 | $\Gamma_1$ | |
| 2 | $A(p, q)$ | B, 1 |
| 3 | $F(q)$ | B, 1 |
| 4 | $A(p, q) \wedge F(q)$ | $\wedge$I, 2, 3 |
| 5 | $R(p)$ | B, 1 |
| 6 | $R(p) \wedge (A(p, q) \wedge F(q))$ | $\wedge$I, 5, 4 |
| 7 | $\exists v.\, R(p) \wedge (A(p, v) \wedge F(v))$ | $\exists$I, 6 |
| 8 | $\exists u.\, \exists v.\, R(u) \wedge (A(u, v) \wedge F(v))$ | $\exists$I, 7 |

Figure 7.4.: Completed uniform proof for goal formula in Fitch-style

Before we continue to explain the backchaining rule for more interesting cases, let us formally define what we mean by a uniform proof.

**Definition 7.17: Uniform proofs**

Let $\Gamma$ be a Horn clause theory and $\varphi$ a closed goal formula. We say that $\varphi$ can be proven from $\Gamma$ by a **uniform proof** if a proof for $\boxed{\Gamma \vdash_u \varphi}$ can be derived from the rules in fig. 7.2.

Note that the uniform proofs have no separate elimination rules. Instead, backchaining combines the assumption rule (Assum), the universal quantifier elimination ($\forall$E) and the implication elimination ($\rightarrow$E) into one rule. This works because the only assumptions that we can use are Horn clauses.

**Example 7.18**

Suppose we want to show that the robot can reach *some* position in fig. 4.1, thus we want to find a proof for

$$\Gamma \vdash_u \exists u.\, C(\mathrm{pos}(\underline{2}, \underline{3}), u)\,,$$

where $C$ is the predicate for connected positions from example 7.8 and $\Gamma$ the logic program that consists of eqs. (7.9) to (7.13). As above, we will refer to the logic program given by eqs. (7.9) to (7.11) as $\Gamma_1$. This allows us to refer explicitly to the Horn clauses in eqs. (7.12) and (7.13).
There are many choices for the position $u$, but to make the example non-trivial and not too lengthy, let us use $\mathrm{pos}(\underline{2}, \underline{4})$. The uniform proof for the above sequent is shown in fig. 7.5.

As mentioned in section 7.2, Horn clauses can be used for computing. However, in example 7.18 we magically chose in line 7 a particular position. This choice can be guided by finding a uniform proof together with a set of unification problems that result from the proof steps and then solving the unification problems. If such a solution exists, then the formula is provable. Let us see how this works on some simple arithmetic.

**Example 7.19: Arithmetic with unification**

Recall that we defined in example 7.7 the less-than predicate for natural numbers. We can combine uniform proofs with unification to find a number

| | | |
|---|---|---|
| 1 | $\Gamma_1$ | |
| 2 | $\forall p.\, C(p,p)$ | |
| 3 | $\forall p.\, \forall q.\, \forall r.\, C(q,r) \land A(p,q) \land F(q) \to C(p,r)$ | |
| 4 | $C(\mathrm{pos}(\underline{2},\underline{4}), \mathrm{pos}(\underline{2},\underline{4}))$ | B, 2 |
| 5 | $A(\mathrm{pos}(\underline{2},\underline{3}), \mathrm{pos}(\underline{2},\underline{4}))$ | B, 1 |
| 6 | $F(\mathrm{pos}(\underline{2},\underline{4}))$ | B, 1 |
| 7 | $C(\mathrm{pos}(\underline{2},\underline{3}), \mathrm{pos}(\underline{2},\underline{4}))$ | B, 3, 4, 5, 6 |
| 8 | $\exists u.\, C(\mathrm{pos}(\underline{2},\underline{3}), u)$ | ∃I, 7 |

Figure 7.5.: Finding a root to the heart with uniform proofs

that is smaller than 1 as follows.

| | | |
|---|---|---|
| 1 | $\forall n.\, L(n,n)$ | |
| 2 | $\forall n.\, \forall m.\, L(n,m) \to L(n,s(m))$ | |
| 3 | $L(?n,\underline{0})$ | B, 1 |
| 4 | $L(?n,s(\underline{0}))$ | B, 2, 3 |
| 5 | $\exists n.\, L(n,s(\underline{0}))$ | ∃I, 4 |

Line 3 results then in the unification problem $\underline{0} \stackrel{?}{=} ?n$, which is readily solved with the substitution $[?n := \underline{0}]$. The other way round, we can also find a number that is larger than zero by the following uniform proof, which

generates the unification problems $\{?m \stackrel{?}{=} s(?k), \underline{0} \stackrel{?}{=} ?k\}$.

| | | |
|---|---|---|
| 1 | $\forall n.\, L(n, n)$ | |
| 2 | $\forall n.\, \forall m.\, L(n, m) \to L(n, s(m))$ | |
| 3 | $L(\underline{0}, ?k)$ | B, 1 |
| 4 | $L(\underline{0}, ?m)$ | B, 2, 3 |
| 5 | $\exists m.\, L(\underline{0}, m)$ | $\exists$I, 4 |

Using the unification procedure given in definition 6.9, we can solve the unification problems with the substitution $[?m := s(\underline{0})][?k := \underline{0}]$. In other words, we find that $s(\underline{0})$ is larger than zero.

You may notice that there is another choice hidden in example 7.18: To apply the backchaining rule in line 7, we had to choose an "intermediate" position, the variable $q$ in the transitivity rule from line 3. We chose the position $\mathrm{pos}(\underline{2}, \underline{4})$ because it brought us directly to the position that we wanted to go to. However, we could have chosen to take a completely different path, for instance via $\mathrm{pos}(\underline{2}, \underline{2}), \mathrm{pos}(\underline{3}, \underline{2}), \mathrm{pos}(\underline{4}, \underline{2}), \mathrm{pos}(\underline{4}, \underline{3}), \mathrm{pos}(\underline{4}, \underline{4})$ and $\mathrm{pos}(\underline{3}, \underline{4})$. This is where we have to find an appropriate control mechanism to guide the search for a path. In appendix F, an implementation of example 7.8 is given in a language called Prolog, a programming language based on Horn clause theories. The control mechanism chosen there is "tabling" combined with the search for a shortest path, which gives a very efficient search strategy for pathfinding. We will not explain the details of this control mechanism or Prolog here. For our purposes, it suffices to say that uniform proofs provide a foundation for the reasoning steps that Prolog takes and that an algorithm can be obtained from the uniform proof system by complementing it with an appropriate search strategy for the substitution $\sigma$ in the backchaining rule.

The uniform proof system has quite a few less rules than $\mathbf{ND}_1$: all the elimination rules and the introduction rules for implication and universal quantification are not present. We have already explained why the general introduction and elimination rules for implication cause troubles. It is possible to add restricted versions of the introduction rules for implication and universal quantification [MN12; Mil+91]. However, the general elimination rules need to be treated completely differently.

"I think this should help you to develop automatic deduction. But you should know that there is more to logic..."

Janina is interrupted by the ringing of the doorbell. A frightened look appears on her face.

"What is it, Janina?", Clara is confused.

Janina gets up and exclaims with a nervous voice: "No time for explanations. Quickly, come with me to the bed room!"

In the bed room she opens the closet, moves the clothes to the side and pulls open the back that gives way to a hidden compartment.

"Get in there and stay quiet. I will close behind you."

Clara and Isaac slip into the compartment and Janina closes the door behind them. Only some light falls through small holes in the wall. Clara feels a stream of moist air. They hear Janina opening the door and going outside.

A moment later, a commanding voice cuts through the air: "Where is your husband, Frau Hosiasson?"

"I don't know. I think he has left town.", Janina responds cheekily.

"You can try to mock me, but we will see how long you will laugh.", the voice responds. "Take you jacket, you come with us."

They can hear Janina taking her jacket and then walking her out on high-heel shoes followed by heavy boots.

Clara exhales: "My goodness, was Janina just arrested? We need to do something!"

"I'm afraid that we cannot, as we may have an uncontrollable impact on time."

"Well, then I will do it alone!", Clara exclaims and attempts to get up.

But Isaac holds her back: "I cannot allow this, as this may inflict harm on other people in a way that I cannot control!"

"They will harm Janina! How can you be so cold, without a heart!", Clara shouts at Isaac.

She wants to break free from Isaac's grip, but is stopped by the noise of the opening door.

"One moment. I will quickly check the rooms.", the voice from earlier shouts.

Clara holds her breath. Heavy steps are approaching the bed room. Clara turns to Isaac but notices that he is again frozen, while processing something.

She knocks on him and whispers: "Hey, Isaac, we need to do something!"

But Isaac does not react.

Clara hears the boots coming closer and starts to panic. Just a few meters to the closet. Clara feels like she is floating, but realises that it is not the time travel setting in, rather she is fading out. She tries to resist, but the black takes over and she loses consciousness.

# Exercises

## Exercise 26

Let $\Gamma$ be the context containing the following four clauses from example 7.7.

$$\begin{array}{ll}
\forall m. & P(\underline{0}, m, m) \\
\forall n.\, \forall m.\, \forall k.\, P(n, m, k) & \to P(s(n), m, s(k)) \\
\forall n. & L(n, n) \\
\forall n.\, \forall m.\, L(n, m) & \to L(n, s(m))
\end{array}$$

Let us use the following abbreviations: $\underline{1} = s(\underline{0})$, $\underline{2} = s(\underline{1})$, and in general $\underline{n+1} = s(\underline{n})$ for all natural numbers $n$.

Give uniform proofs for the following sequents, while using unification to the find the terms that are required for the existential introduction.

a) $\Gamma \vdash_u \exists k.\, P(\underline{2}, \underline{1}, k)$

b) $\Gamma \vdash_u \exists k.\, P(k, \underline{1}, \underline{3})$

c) $\Gamma \vdash_u \exists k.\, P(\underline{1}, k, \underline{3})$

d) $\Gamma \vdash_u \exists k.\, \exists m.\, P(m, k, \underline{3})$

We can also do constraint solving. For instance, the following asks for a solution to the inequalities $k + 2 \leq n$ and $n \leq 5$. Give a uniform proof for the following sequent, while using unification to the find the terms that are required for the existential introduction.

e) $\Gamma \vdash_u \exists k.\, \exists n.\, \exists m.\, P(k, \underline{2}, m) \wedge L(m, n) \wedge L(n, \underline{5})$

## Exercise 27

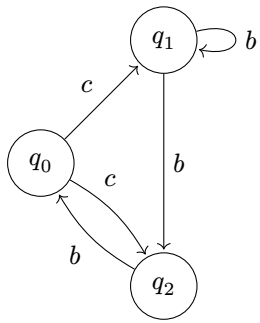
Let $\mathcal{L}$ be a signature containing three ternary predicate symbols $L$, $G$ and $E$. We use $L$ to represent the less than or equal relation, $G$ to represent the greater than or equal relation and lastly $E$ to represent the equality relation.

a) Give two Horn clauses that implement the greater than or equal relation. Use reflexivity and the less than or equal relation.

b) Give two Horn clauses that implement the equality relation. Use reflexivity and the other two relations.

## Exercise 28

A *labelled transition system* (LTS) is a graph with edges labelled in some alphabet. Alternatively, an LTS can be seen as a non-deterministic automaton without initial and accepting states. We can use labelled transition systems as models of computation. Suppose, for instance, that we want to model a beverage vending machine that takes coins as input and provides beverages. However, this machine has a bug: It may happen that the user of the machine is lucky and the machine enters a state, after receiving a coin, in which the machine may hand out several beverages at no further cost. We can model this machine as an LTS, as shown in the following diagram, where the labels are $c$ for the coin and $b$ for vending a beverage. The state $q_0$ takes a coin as input, from where the machine non-deterministically moves to either the faulty state $q_1$ or $q_2$.




We are now interested in formally representing the machine and its behaviour by Horn clauses, and then reason about it with uniform proofs. To this end, we will need to talk about *traces*. A trace of an LTS is a sequence of labels that arises from transitions between states. We will write such sequences as labels seperated by a dot and ending in $\varepsilon$, where $\varepsilon$ stands for the empty trace. For instance, $b \cdot \varepsilon$ is a trace leading from $q_1$ to $q_2$ in our vending machine. However, $c \cdot c \cdot \varepsilon$ is not a trace for any pair of states. Formally, we will use the set of function symbols

$$\mathcal{F} = \{q_0, q_1, q_2, c, b, \varepsilon, \cdot\},$$

where $\cdot$ has arity 2 and all other symbols have arity zero (constants). As indicated above, we write $u \cdot w$ instead of $\cdot(u, w)$ to ease readability. To model the vending machine and its behaviour, we will use two predicate symbols

$$\mathcal{R} = \{R, T\},$$

each with arity 3. The idea is that $R$ represents the transitions of the machine and $T$ the traces. For instance, we want that $R(q_1, b, q_2)$ holds because the above machine has a transition from $q_1$ to $q_2$ with label $b$. Similarly, we want that $T(b \cdot \varepsilon, q_1, q_2)$ holds because $b \cdot \varepsilon$ is a trace leading from $q_1$ to $q_2$. Note that the same sequence of labels gives rise to different traces and that $T(b \cdot \varepsilon, q_2, q_0)$ holds as well.

a) Provide a logic program $\Gamma_0$ with five Horn clauses that determines the transition relation $R$.

b) Provide two Horn clauses $\varphi_r$ and $\varphi_t$ that determine the trace relation $T$, one clause for the empty sequence $\varepsilon$ and one of the sequential composition of traces.

c) Give a uniform proof that shows that we can start the machine in state $q_0$, input only one coin but receive two beverages, and that we can continue putting in coins. Formally, prove the following sequent using a uniform proof in Fitch-style.

$$\Gamma_0, \varphi_r, \varphi_t \vdash_u \exists x.\, T(c \cdot b \cdot b \cdot \varepsilon, q_0, x) \wedge \exists y.\, R(x, c, y)$$

# Intermezzo

Clara sits on a hard wooden chair at a single desk surrounded by vast space. She knows this place and stiffens up. From behind her, she hears heavy boots walking up. Czerny appears, giving her commands. Clara can hardly hear him, his voice seems to come from very far away.

Suddenly, a gaunt man appears and takes Clara by the hand. The hand feels familiar and comforts her. While Clara couldn't get up herself, the man is getting her out of the chair without any effort despite his apparent frailty. He pulls her away, while Clara can hear Czerny clamouring in the distance.

They arrive at a clearing of a vast forest. In front of them lies young green grass and a small blue lake. Clara has to hold her hand up to protect her eyes from the bright sun. The mountain ridge in the distance reflects in the lake.

"Come, let us sit at the lake", the man says in a soft voice. They walk towards the lake and sit down. Clara feels the moist in the grass.

"Mother has passed away.", the man says without energy.

"I know, papa."

"You will have to stay some time with your grandfather, while I take care of everything. We will see each other soon."

Clara reaches out to her father and hugs him: "Goodbye, papa. I love you!". Tears are running down her face, knowing well that she will not see him again.

The man gives Clara a kiss on the forehead, gets up and vanishes into the foggy woods. Clara stares on the lake, drying her tears.

Suddenly, she hears buzzing sounds in the distance, the sky starts flashing red. She looks around and the green clearing starts to turn dark. Clara tries to remember where she knows the buzzing sound from, it seems familiar. Then, her dream collapses.

Clara opens the eyes. Next to her, Isaac is coming back to life.

The boots stop and the voice shouts: "I knew it. Kameraden, I found more!" Cloth and hangers are being thrown on the floor.

"Isaac, we need to get out of here!", Clara shakes Isaac's shoulder.

To her relief, she starts to feel light in the right way. While they are falling through time, Clara can just see how the back of the closet is removed and a person with a long, dark leather coat peeks into the compartment.

# Part II.

# Humans and Robots, Know Your Limits!

# 8. First-Order Logic with Equality

Up to now, we have reasoned in first-order logic about objects using unspecified predicates and relations that are given by a signature $\mathcal{L}$. This led us to make assumptions about these predicates in form of axioms, which we will discuss in more detail later on. One of the most important relations is equality, as it allows us to compare objects with each other and express complex relations between objects. For instance, the iteration principle of trees in definition 9.5 fundamentally used equality to express the computation property eq. (9.6) and to assert that the inductive extension is unique. We also used in definition 5.3 equality to state how variable renaming and substitution works in first-order logic by providing a specification of how equality works on formulas. There we also stated that we assume that equality is an equivalence relation and that we can replace equal formulas in more complex formulas. Finally, exercise 27 asked for a set of Horn clauses that ought to characterise equality.

The goal of this chapter to understand more generally what equality is and how we can reason about it in first-order logic. To this end, we will first discuss intuitively discuss the principles of equality, then introduce a new atomic formula for equality into first-order logic, and finally formalise reasoning about equality in a proof system. In the course of this, we will also explore how unique existence, like the inductive extension of a map, can be formalised in first-order logic.

Equality as part of our logic enables a discussion of the limitations of individual logical systems, which we will explore in the coming chapters. This is one of the faces of pluralism in logic.

## 8.1. Synactic Formulas with Equality

In exercise 27, we used a binary predicate symbol $E$ together with some Horn clauses to formalise equality, lust like the binary predicate $L$ was used in ex-

ample 7.7 to express the less-than relation. This means that we would write $E(x, y)$ to express that $x$ and $y$ are equal. Having to add such a predicate symbol every time we want to reason about equality is not only inconvenient, but also relies on the user of first-order logic to correctly state assumption for equality. In fact, one could even make the assumption that $\forall x.\, \forall y.\, E(x, y)$ holds, which would completely contradict the understanding of equality!

This problem can be fixed by giving equality a special status, and making it part of the syntax and proof system of first-order logic. To this end, we extend the first-order formulas by one extra kind of atomic formula of the form $\boxed{s \doteq t}$ for terms $s$ and $t$, which is a logical formula with the intent of expressing that $t$ is equal to $s$. We use this special notation to carefully distinguish the **syntactic equality** $\doteq$ in the logic, from the equality that we use elsewhere to express general equality of mathematical objects. In particular, we previously wrote $s = t$ to say that $s$ and $t$ are equal as terms. We then reasoned about identities like $f(x)\,[x := c] = f(c)$. The "dotted" notation is different from this equality as it is just a syntactic formula. For example, we can form the formula $f(x) \doteq c$. This formula may be true or not, but the notation with the dot does not assign any intrinsic meaning to such statements, whereas $f(x) = c$ cannot be true as identity of terms. The meaning of $\doteq$ will rather come from the proof system. It may happen that $f(x) \doteq c$ becomes provable, for example, if $f$ is axiomatised as the successor map, $c$ as one, and $x$ is instantiated by $\underline{0}$. Thus, it is important to keep in mind that $=$ and $\doteq$ express generally different things.

$$\frac{t \in \mathrm{Term} \qquad s \in \mathrm{Term}}{t \doteq s \in \mathrm{Form}^{=}}$$

$$\frac{P \in \mathcal{R}_n \qquad t_1 \in \mathrm{Term} \qquad \cdots \qquad t_n \in \mathrm{Term}}{P(t_1, \dots t_n) \in \mathrm{Form}^{=}} \qquad \frac{\varphi \in \mathrm{Form}^{=}}{(\varphi) \in \mathrm{Form}^{=}}$$

$$\frac{}{\bot \in \mathrm{Form}^{=}} \qquad \frac{\varphi \in \mathrm{Form}^{=} \qquad \psi \in \mathrm{Form}^{=} \qquad \Box \in \{\wedge, \vee, \rightarrow\}}{\varphi \,\Box\, \psi \in \mathrm{Form}^{=}}$$

$$\frac{x \in \mathrm{Var} \qquad \varphi \in \mathrm{Form}^{=}}{\forall x.\, \varphi \in \mathrm{Form}^{=}} \qquad \frac{x \in \mathrm{Var} \qquad \varphi \in \mathrm{Form}^{=}}{\exists x.\, \varphi \in \mathrm{Form}^{=}}$$

Figure 8.1.: First-order formulas with equality in $\mathrm{Form}^{=}$

**Definition 8.1**

Let $\mathcal{L}$ be a signature. The set $\boxed{\mathrm{Form}(\mathcal{L}^{=})}$ or $\boxed{\mathrm{Form}^{=}}$ of (first-order) **formulas with equalitys** or **$\mathcal{L}^{=}$-formulas** is the smallest set closed under rules in fig. 8.1. The map $\mathrm{fv}\colon \mathrm{Form}^{=} \to \mathcal{P}(\mathrm{Var})$ is defined exactly as the map as on Form, with the following extra case for equality.

$$\boxed{\mathrm{fv}(s \doteq t) \quad = \quad \mathrm{var}(s) \cup \mathrm{var}(t)}$$

Finally, the $\mathcal{L}^{=}$-formulas fulfil, additionally to the axioms from definition 5.3, the following axiom for substitution.

$$\boxed{(s \doteq t)\,\sigma \quad = \quad (s\,\sigma) \doteq (t\,\sigma)} \tag{SE}$$

Note that $\mathrm{Form}^{=}$ is defined in the same way as Form with one extra case added for $s \doteq t$ and that there is no difference for $\mathcal{L}^{=}$-formulas that do not involve the equality relation, see theorem 8.24 below.

Also we need to be careful to distinguish in the axiom (SE) between the atomic equality formulas and the equality of formulas themselves. The equality of formulas, without the dot, compares the two formulas that are made up of the equality connective, with the dot.

Let us revisit the examples involving statements about natural numbers.

**Example 8.2: Arithmetic with equality**

In example 7.7, we have reasoned about natural numbers and operations on them in terms of predicates $P$ and $L$ with specific axioms. Even though this could be done for equality, this approach does not work very well, as we have argued. Let us instead consider the following signature $\mathcal{L}$.

$$\mathcal{F} = \{\underline{0}, s, p\} \qquad \mathcal{R} = \{N\}$$
$$\mathrm{ar}(\underline{0}) = 0 \qquad \mathrm{ar}(N) = 1 \qquad \mathrm{ar}(p) = 2$$

The intend is that $L$ still expresses the less-than relation, but now we want to provide formulas that axiomatise addition directly as function symbol. Thus, we aim to understand $\underline{0}$ as representation of zero, $s$ of the successor, $p$ of addition, and $N$ of the predicate characterising natural numbers. With this in mind, we can express the axioms for addition from example 7.7 more

directly by adapting eq. (9.6) of inductive extensions:

$$\forall x.\, N(x) \to p(\underline{0}, x) \doteq x \tag{8.3}$$
$$\forall x.\, \forall y.\, N(x) \land N(y) \to p(s(x), y) \doteq s(p(x, y)) \tag{8.4}$$

We read eq. (8.3) as "if $x$ is a natural number, then adding zero to $x$ is equal to $x$", and similarly for eq. (8.4).

In example 8.2 and throughout chapter 4, we have often used formulas formulas of the form $\forall x.\, P(x) \to \varphi$ and $\exists x.\, P(x) \land \varphi$ for unary predicate symbols $P$ and formulas $\varphi$. The intent is that the domain of the variable $x$ is restricted to $P$ is each case. For example, eq. (8.3) expresses that adding zero to $x$ only evaluates to this $x$ if it is a natural number. In set-theoretical notation, we would say "for all $n \in \mathbb{N}$, $n + 0 = 0$", which crucially says that $n$ is natural number and not any other object like real numbers, trees or fruits. This is such a common pattern that it deserves a suggestive notation.

**Definition 8.5: Bounded quantification**

Let $P$ be a predicate symbol with $\mathrm{ar}(P) = 1$ in a signature $\mathcal{L}$ and $\varphi$ a $\mathcal{L}$-formula. The **bounded quantifiers** $\boxed{\forall x : P.\, \varphi}$ and $\boxed{\exists x : P.\, \varphi}$ are defined as follows.

$$\forall x : P.\, \varphi \overset{\text{def}}{=} \forall x.\, P(x) \to \varphi$$
$$\exists x : P.\, \varphi \overset{\text{def}}{=} \forall x.\, P(x) \land \varphi$$

Bounded quantification allows us to simplify the formulas in example 8.2.

**Example 8.6**

This notation allows us to express eqs. (8.3) and (8.4) more conceptually by:

$$\forall x : N.\, p(\underline{0}, x) \doteq x \tag{8.7}$$
$$\forall x : N.\, \forall y : N.\, p(s(x), y) \doteq s(p(x, y)) \tag{8.8}$$

We can read eq. (8.7) quite naturally as "for all natural numbers $x$, $p$ applied to zero and $x$ is equal to $x$".
With this notation, we can now also conveniently express properties of natural numbers, such as a number represented by a term $t$ is even:

$$\mathrm{even}(t) \overset{\text{def}}{=} \exists y : N.\, t \doteq p(y, y)$$

At the heart of algebra is reasoning about equality and thus many familiar

statements of algebra and arithmetic can be formulated in first-order logic with equality. You can try that out yourself in the following quiz.

?

Can you find a formula that states that a number $x$ is odd?

Equality occurs not only in mathematics, but it also has a central place in typical computer science applications.

**Example 8.9: Equality constraints and queries in databases**

Recall that we used in example 4.18 a predicate symbol Eq to express uniqueness of primary keys by the following formula.

$$\forall s.\, \forall n.\, \forall m.\, \mathrm{dStudent}(s, n) \land \mathrm{dStudent}(s, m) \to \mathrm{Eq}(n, m)$$

Since Eq is merely a predicate symbol, this formula does not necessarily express uniqueness. However, we can use our built-in equality relation and instead use the following formula as constraint.

$$\forall s.\, \forall n.\, \forall m.\, \mathrm{dStudent}(s, n) \land \mathrm{dStudent}(s, m) \to n \doteq m$$

This formula states that two entries with the same primary key must have the same name stored.
In example 4.22, we used first-order logic to formulate a query on a database. Many such queries involve equality tests. For instance, suppose that our signature contains constants for strings and that these are written as "Clara". We can then formulate a query to find all exams with primary key $k$, which Clara is enrolled for, as the following formula.

$$\begin{aligned}\exists i.\, \exists s.\, \exists j.\, &\mathrm{dEnrolment}(i, s, k) \land \mathrm{dExam}(k, j) \\ &\land \mathrm{dStudent}(s, n) \land n \doteq \text{"Clara"}\end{aligned}$$

An existentially quantified formula $\exists x.\, \varphi$ expresses that there is at least one object that has the property $\varphi$. Often, we want such an object to be *unique*. To say that $x$ is unique with the property $\varphi$ is expressed by requiring that $x$ is equal to any other object with the same property:

$$\forall y.\, \varphi[x := y] \to x \doteq y$$

Using this expression of uniqueness, we can define unique existential quanti-

fication, typically written as $\exists! x.\, \varphi$, by the following formula.

$$\exists x.\, \varphi \wedge (\forall y.\, \varphi[x := y] \to x \doteq y)$$

As uniqueness is an important use of equality, it deserves its own definition.

**Definition 8.10: Uniqueness quantifier**

Let $\varphi$ be a formula, $t$ a term and $x$ a free variable in $\varphi$. We define the **uniqueness predicate** as a formula $\boxed{\mathrm{unique}_x(t, \varphi)}$ that expresses that $t$ is uniquely among all objects that can be placed in the formula $\varphi$ for $x$:

$$\mathrm{unique}_x(t, \varphi) \stackrel{\text{def}}{=} \forall y.\, \varphi[x := y] \to t \doteq y$$

The variable $y$ is chosen to be fresh for $\varphi$ and $t$. Using uniqueness, we define the **uniqueness quantifier** $\boxed{\exists! x.\, \varphi}$ by

$$\exists! x.\, \varphi \quad \stackrel{\text{def}}{=} \quad \exists x.\, \varphi \wedge \mathrm{unique}_x(x, \varphi)$$

and its bounded version as follows.

$$\exists! x : N.\, \varphi \quad \stackrel{\text{def}}{=} \quad \exists x.\, N(x) \wedge \varphi \wedge \mathrm{unique}_x(x, N(x) \wedge \varphi)$$

Among the many important concepts that can be formalised with uniqueness quantifiers are solutions to arithmetical equations.

**Example 8.11: Subtraction**

Given two natural numbers $n, m$, the equation $n + k = m$ can be solved if $m$ is larger than $n$. Over the integers, $k$ would of course be the difference $m - n$ and this would be the unique solution to the equation. In fact, if the equation has any solution $k$ then it has a unique solution. We can formulate that with the uniqueness quantifier by:

$$\forall x : N.\, \forall y : N.\, (\exists z : N.\, p(x, z) \doteq y) \to \exists! z : N.\, p(x, z) \doteq y$$

Of course, this formula is only provable if we also formalise properties of natural numbers, such as induction. We will come back to this at a later stage. However, there is an algebraic view that can be used to prove this and similar properties, see exercise 33.

At this point, we could try to formulate the iteration and induction principles for natural numbers in first-order logic. Unfortunately, this is not easy to do!

The naive attempt to express the iteration principle by

$$\forall f_0.\, \forall f_1.\, \exists! x.\, x(\underline{0}) \doteq f_0 \land \forall y : N.\, x(s(y)) \doteq f_1(x(y))$$

is invalid because $x$ is not a function symbol and thus cannot be applied to terms. There are essentially two ways out of this: either we allow quantification over maps or we find a presentation of the theory of maps by first-order formulas. The first direction is known as higher-order logic, which we will not pursue further here. Exercise 34 invites you to explore the second option via one possible formulation of the theory of maps in terms of application.

The uniqueness predicate also has applications to databases.

**Example 8.12: Revisiting unique primary keys**

In example 8.9, we provided a formula that expressed uniqueness of primary keys. With the uniqueness predicate from definition 8.10, we can express that a key is primary as follows. Letting $\varphi = \mathrm{dStudent}(s, x)$, the uniqueness of a key $s$ is expressed by:

$$\forall s.\, \forall n.\, \varphi[x := n] \to \mathrm{unique}_x(n, \varphi)$$

Another useful application of equality is that it allows us to count how many things there are with a certain property.

**Example 8.13: Expressing finite quantities**

Not only can we use equality to enforce uniqueness as in definition 8.10, but we can even state that there must be at least or exactly a certain amount of objects with some property. To do so, we need to express that two objects are not identical, which we define here as the negation of equality:

$$s \not\doteq t \quad = \quad \neg(s \doteq t).$$

We can express that there are *at least two* objects with property $\varphi$ by

$$\exists x.\, \exists y.\, \varphi[x := x] \land \varphi[x := y] \land x \not\doteq y$$

and that there must be *exactly two* objects for which $\varphi$ holds by

$$\exists x.\, \exists y.\, \varphi[x := x] \land \varphi[x := y] \land x \not\doteq y \land (\forall z.\, \varphi[x := z] \to z \doteq x \lor z \doteq y)$$

This is called *counting quantification*.

Besides counting objects, we can also use equality to make sure that objects

are given by a certain pattern. Let us demonstrate this by specifying the commands of a simple protocol.

**Example 8.14: Commands of a vending machine**

Suppose we were to specify the interface of a vending machine with a coin slot and a button to select the product. Since vending machines occur across computer science curricula, it is worthwhile to figure out how to formalise this specification.

We begin with the signature $\mathcal{L}_v$, where $v$ stands for "vending machine", with unary predicate symbols Cmd and Prod for classifying commands and products, a constant coin, and a unary function symbol sel for the product selection command. This model assumes that our vending machine does not distinguish between coins and that it donates possible surpluses. Formally, we define $\mathcal{L}_v = (\mathcal{R}, \mathcal{F}, \mathrm{ar})$, where $\mathcal{R} = \{\mathrm{Cmd}, \mathrm{Prod}\}$, $\mathcal{F} = \{\mathrm{coin}, \mathrm{sel}\}$, $\mathrm{ar}(\mathrm{coin}) = 0$, and $\mathrm{ar}(\mathrm{Cmd}) = \mathrm{ar}(\mathrm{Prod}) = \mathrm{ar}(\mathrm{sel}) = 1$,

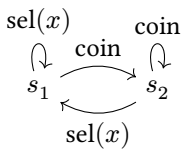
Within the signature $\mathcal{L}_v$, we can now use equality to specify that a command must either be the input of a coin or the selection of a product:

$$\forall x.\, \mathrm{Cmd}(x) \leftrightarrow x \doteq \mathrm{coin} \lor \exists y : \mathrm{Prod}.\, x \doteq \mathrm{sel}(y)$$

This formula can be recognised as a typical data type declaration, here in Haskell-style:

```
data Cmd = Coin | Sel Prod
```

Given that we know how commands look like, we can specify the behaviour of our vending machine. Our vending machine is going to be a bit greedy and not very user friendly: It has two states $s_1$ and $s_2$, one that waits for a coin and one that dispenses the chosen product. When the machine is in the first state it will ignore any selection commands, while in the second state it will continue accepting coins, even after already receiving one. This is displayed in the following state diagram.




Before we can reason about this machine, we have to carry out the mundane task of formally specifying this state diagram. To do so, we have to extend the signature $\mathcal{L}_v$ with constants $s_1$ and $s_2$, a unary predicate symbol St to

classify states, and a binary function symbol $f$ that represents that transitions of the machine. Our goal is to describe $f$ by a formula, such that

$$\forall x : \mathrm{St}.\, \forall y : \mathrm{Cmd}.\, \exists! z : \mathrm{St}.\, f(x, y) \doteq z$$

holds. The uniqueness quantifier is what makes $f$ a map, which means that $f$ assigns to every state $x$ and command $y$ a unique state $z$.
Typically, we would specify $f$ by *pattern matching*:

$$\begin{aligned} f(s_1, \mathrm{coin}) &= s_2 & \qquad f(s_1, \mathrm{sel}(x)) &= s_1 \\ f(s_2, \mathrm{coin}) &= s_2 & \qquad f(s_2, \mathrm{sel}(x)) &= s_1 \end{aligned}$$

Such a pattern matching definition ensures automatically that $f$ is well-defined, if we know that there are exactly two states $s_1$ and $s_2$, and the two commands coin and sel. We have specified already how commands look like above. Similarly, we can specify that there are only two states:

$$\forall x.\, \mathrm{St}(x) \leftrightarrow x \doteq s_1 \lor x \doteq s_2$$

Such pattern matching definition can be specified in first-order logic with equality as follows.

$$\begin{aligned} \forall x : \mathrm{St}.\, \forall y : \mathrm{Cmd}.\ & (x \doteq s_1 \land y \doteq \mathrm{coin} \to f(x, y) = s_2) \\ & \land\ (x \doteq s_2 \land y \doteq \mathrm{coin} \to f(x, y) = s_2) \\ & \land\ (x \doteq s_1 \land \exists z : \mathrm{Prod}.\, y \doteq \mathrm{sel}(z) \to f(x, y) = s_1) \\ & \land\ (x \doteq s_2 \land \exists z : \mathrm{Prod}.\, y \doteq \mathrm{sel}(z) \to f(x, y) = s_1) \end{aligned}$$

That this formulas specifies $f$ uniquely as a map, follows from the formulas that declare the "data types" Cmd and St. We will, however, refrain from proving this here because such a proof would be quite long and extremely boring. A better approach is to use an extension of first-order logic with types [And02; Jac99], that allows the specification of Cmd, St and $f$ directly by pattern matching. That being said, this example shows that we can *in principle* handle data types and function definitions in first-order logic with equality.

We could now go further and also specify and reason about the behaviour of the vending machine, but at that point we should be really using a computer and a proof assistant.

## 8.2. Natural Deduction for FOL with Equality

To complete the picture of first-order logic with equality, let us give corresponding proof systems. We expect that such a proof systems arise by extending the rules of $\mathbf{ND}_1$ or $\mathbf{cND}_1$ with rules for equality. Thus, let us briefly think about how we use equality and try to deduce the proof rules from this intuition.

A common use of equality is equational reasoning. For instance, if $a$ and $b$ are numbers, then

$$ab + a(-b) = a(b - b) = a0 = 0$$

establishes that $ab + a(-b)$ is identical to 0. In fact, if we calculate with concrete numbers, say 2 and 5, then $2 \cdot 5 + 2 \cdot (-5)$ is just another way of writing 0. This leads us to the most basic identity, the identity of an object with itself:

$$x = x$$

This identity is called **reflexivity** and will be the first rule that we adapt into our proof system.

Reflexivity by itself is, however, not enough. We often replace "equals by equals" in equational reasoning. For instance, if we know that $c = 0$ then we can infer

$$d = 0 + d = c + d.$$

From this and the previous identity, we can infer the following much more complex identity.

$$d = (ab + a(-b)) + d$$

The process of replacing equal objects can be generalised to arbitrary formulas and leads us the so-called **replacement** rule.

Formally, the intuitionistic and classical natural deduction proof systems for first-order logic with equality are given in the following definition.

**Definition 8.15: Natural deduction with equality**

The systems of **natural deduction for FOL with equality** $\boxed{\mathbf{ND}_1^=}$ and **classical natural deduction for FOL with equality** $\boxed{\mathbf{cND}_1^=}$ are given by

extending, respectively, $\mathbf{ND}_1$ and $\mathbf{cND}_1$ with the following two rules.

$$\frac{}{\Delta \mid \Gamma \vdash t \doteq t}\ (=\mathrm{I}) \qquad \frac{\Delta \mid \Gamma \vdash s \doteq t \qquad \Delta \mid \Gamma \vdash \varphi[x := s]}{\Delta \mid \Gamma \vdash \varphi[x := t]}\ (=\mathrm{E})$$

The rules (=I) and (=E) are called the **reflexivity** and **replacement** rules.

We follow in definition 8.15 the traditional naming for the rules, although we also refer to them as introduction and elimination rules.

Surprisingly, the two rules (=I) and (=E) are enough to fully characterise equality. To give a flavour of the power of the replacement rule, let us prove that $\doteq$ is symmetric and transitive. These two properties form the basis of equational reasoning.

**Theorem 8.16: Equality is an equivalence relation**

The following two rules of *symmetry* and *transitivity* are admissible in $\mathbf{ND}_1^{=}$ and $\mathbf{cND}_1^{=}$.

$$\frac{\Delta \mid \Gamma \vdash s \doteq t}{\Delta \mid \Gamma \vdash t \doteq s}\ (\mathrm{Sym}) \qquad \frac{\Delta \mid \Gamma \vdash s \doteq t \qquad \Delta \mid \Gamma \vdash t \doteq r}{\Delta \mid \Gamma \vdash s \doteq r}\ (\mathrm{Trans})$$

*Proof.* To derive transitivity, we will have to use the replacement rule. The difficulty in using this rule lies in finding the correct formula $\varphi$, in which we replace equal terms. If we take a look at the conclusion of the transitivity rule, then we have several options of choosing such a formula $\varphi$. On of them is

$$s \doteq x\,.$$

This formula works because $\varphi[x := r]$ is the conclusion $s \doteq r$ of (Trans) and $\varphi[x := t]$ is the first premise $s \doteq t$ of (Trans). Thus, (Trans) is given by the following application of (=E), albeit with swapped premises.

$$\frac{\Delta \mid \Gamma \vdash \varphi[x := t] \qquad \Delta \mid \Gamma \vdash t \doteq r}{\Delta \mid \Gamma \vdash \varphi[x := r]}\ (=\mathrm{E})$$

After carrying out the substitution, this a proof tree for

$$\frac{\Delta \mid \Gamma \vdash s \doteq t \qquad \Delta \mid \Gamma \vdash t \doteq r}{\Delta \mid \Gamma \vdash s \doteq r}\ (\mathrm{Trans})$$

and thus transitivity is an instance of (=E). □

?

Can you find a $\mathbf{ND}_1^=$-derivation for the symmetry rule in theorem 8.16?

At this point, one may wonder why we have to introduce equality as a new symbol and give new proof systems. Why can we not just add equality to the signature and find axioms $\Gamma$ so that that $\Gamma, \Gamma' \vdash \varphi$ is derivable in $\mathbf{ND}_1$ if and only if $\Gamma' \vdash \varphi$ is derivable in $\mathbf{ND}_1^=$? For instance, the formula $\forall x.\, x \doteq x$ could serve as an axiom that represents reflexivity. The problem is that the replacement rule ranges over all formulas, which would mean that we have to add an axiom for each formula, including our axioms. This leads to a problem that we cannot solve in first-order logic, because we would need a context with infinitely many formulas. If the signature is finite, then we can alternatively add for all function and predicate symbols in the signature axioms that state that they respect the equality relation. For example, if $f \in \mathcal{F}_1$ and $P \in \mathcal{R}_1$ we would need

$$\begin{aligned} \mathrm{resp}_f &= \forall x.\, \forall y.\, x \doteq y \to f(x) \doteq f(y) \\ \mathrm{resp}_P &= \forall x.\, \forall y.\, x \doteq y \to P(x) \to P(y) \end{aligned}$$

One can then prove that for $\Gamma_0$ consisting of axioms for reflexivity, transitivity, symmetry, $\mathrm{resp}_f$ and $\mathrm{resp}_P$ for all $f$ and $P$ in $\mathcal{L}$ that $\Gamma_0, \Gamma \vdash \varphi$ is provable in $\mathbf{ND}_1$ over the signature $\mathcal{L}^=$, which has $\mathcal{R}^= = \mathcal{R} \cup \{\doteq\}$, iff $\Gamma \vdash \varphi$ is provable in $\mathbf{ND}_1^=$. For a finite signature, this $\Gamma_0$ would then also be finite. However, this proof requires induction over formulas and is thus not provable internally in the logic. This is means that such a proof has to be written every time we want to prove some equality, which prevents proof reuse and leads to large proof trees. Moreover, if the signature is infinite, something that can be reasonable and we will make use of, then this approach also fails because $\Gamma_0$ would then have to contain infinitely many formulas. For this reason we give equality a special status in our logic.

**Digression 8.17: Other approaches to equality**

The replacement rule is also known under other names, most famously perhaps "indiscernibility of identicals" or "Leibniz rule" because he investigated the relation between the identity of objects and their properties. Understanding the identity of objects is an important topic in logic. Frege developed importantly the distinction between the identity of objects and the identity of names [Hec03]. We have discussed the latter at length in chapter 5, but the identity of objects continues to be a source of many questions in philosophy [DG23] and in logic in general. First of all, in an area called **categorical logic**[18] an abstract understanding of equality has been developed, which works not only for the traditional logic that we consider here but also for other interpretations. This perspective leads also to alternative formulations of

rules for equality. For instance, Lawvere's rule [Jac99]

$$\frac{\Delta, x \mid \Gamma \vdash \varphi[y := x]}{\Delta, x, y \mid \Gamma, x \doteq y \vdash \varphi}$$

together with either its inverse or (=I) gives an equivalent set of rules. In an area called *type theory*, in which formulas are seen as types and proofs are given by programs, equality has to be treated yet differently. For instance, Martin-Löf type theory(ref) follows more or less the same approach as we do, but observational type theory [AMS07], univalent type theory [Uni13] and cubical type theory [Coh+16] follow quite different approaches, and the research into this is still ongoing. Interestingly, there is a strong relation to insights from mathematics, in particular topology, as to how maps and spaces need to be compared in different situations [HS98; Lan21; Rie16].

Since the uniqueness quantifier in definition 8.10 is a derived logical connective, we would want to have convenient introduction and elimination rules for it to improve readability of proofs that involve uniqueness quantification. Especially the introduction rule shortens proofs considerably.

**Lemma 8.18: Rules for uniqueness quantifier**

The following rules are admissible in $\mathbf{ND}_1^=$, where the variable $y$ in (∃!I) has to be fresh.

$$\frac{\Delta \mid \Gamma \vdash \varphi[x := t] \qquad \Delta, y \mid \Gamma, \varphi[x := y] \vdash t \doteq y}{\Delta \mid \Gamma \vdash \exists! x.\, \varphi} \ (\exists!\mathrm{I})$$

$$\frac{\Delta \mid \Gamma \vdash \exists! x.\, \varphi \qquad \Delta, x \mid \Gamma, \varphi, \mathrm{unique}_x(x, \varphi) \vdash \psi}{\Delta \mid \Gamma \vdash \psi} \ (\exists!\mathrm{E})$$

The proof of this lemma is left as exercise 29.

?

Reflect what the admissible rules for the bounded (uniqueness) quantifiers would be.

Let us give a few example proofs in the systems $\mathbf{ND}_1^=$ and $\mathbf{cND}_1^=$. As before, we will use Fitch-style proofs to make the proofs readable.

**Example 8.19**

In this example, we will formally prove the incredibly difficult fact that the successor of an even number is odd. To this end, we put $\underline{1} = s(\underline{0})$ use the formulas even and odd given by

$$\mathrm{even}(x) = \exists y : N.\, x \doteq p(y, y) \quad \text{and} \quad \mathrm{odd}(x) = \exists z : N.\, x \doteq p(p(z, z), \underline{1})$$

to describe even and odd numbers, see example 8.6. We then derive the sequent

$$\vdash \forall x.\ \mathrm{even}(x) \to \mathrm{odd}(p(x, \underline{1}))$$

in $\mathbf{ND}_1^{=}$ by using Fitch-style.

| | | | |
|---|---|---|---|
| 1 | $x$ | $\mathrm{even}(x)$ | |
| 2 | | $y$ \| $N(y) \land x \doteq p(y, y)$ | |
| 3 | | $N(y)$ | $\land$E, 2 |
| 4 | | $x \doteq p(y, y)$ | $\land$E, 2 |
| 5 | | $p(x, \underline{1}) \doteq p(x, \underline{1})$ | =I |
| 6 | | $p(x, \underline{1}) \doteq p(p(y, y), \underline{1})$ | =E, 3, 5 |
| 7 | | $N(y) \land p(x, \underline{1}) \doteq p(p(y, y), \underline{1})$ | $\land$I, 3, 6 |
| 8 | | $\exists z : N.\, p(x, \underline{1}) \doteq p(p(z, z), \underline{1})$ | $\exists$I, 7 |
| 9 | | $\mathrm{odd}(p(x, \underline{1}))$ | $\exists$E, 1, 1–8 |
| 10 | | $\mathrm{even}(x) \to \mathrm{odd}(p(x, \underline{1}))$ | $\to$I, 1–9 |
| 11 | $\forall x.\ \mathrm{even}(x) \to \mathrm{odd}(p(x, \underline{1}))$ | | $\forall$I, 1–10 |

In the proof, we use the reflexivity rule (=I) in line 5 and the replacement rule (=E) in line 6. The rule (=E) is here applied to the formula $\psi$ given by $p(x, \underline{1}) \doteq p(u, \underline{1})$ and the substitutions $[u := x]$ and $[u := p(y, y))]$:

$$\frac{\Delta \mid \Gamma \vdash x \doteq p(y, y) \qquad \Delta \mid \Gamma \vdash \psi[u := x]}{\Delta \mid \Gamma \vdash \psi[u := p(y, y)]}$$

where $\Delta = x, y$ and $\Gamma = \varphi_e, x \doteq p(y, y)$ are the context and assumptions introduced in lines 1–2.

The next example shows how equality can be used to prove uniqueness of dividers

**Example 8.20: Even numbers have unique divider**

We all know that division gives unique results, whenever it is defined. In particular, if a number $n$ is even, then there should be a unique number $k$

with $n = 2k$. Formally, we use the formula

$$\forall x.\ \mathrm{even}(x) \to \exists! z.\, x \doteq p(z, z) \tag{8.21}$$

to express uniqueness of divisors. In this example, we will prove uniqueness of divisors, as expressed by (8.21), in $\mathbf{ND}_1^{=}$.
Underlying uniqueness of division by two is the fact that doubling a number is an injective function, that is, $2n = 2m$ implies $m = n$ for all natural numbers $m$ and $n$:

$$\varphi_{\text{double-inj}} \quad = \quad \forall m.\, \forall n.\, p(m, m) \doteq p(n, n) \to m \doteq n$$

Thus, what we will prove is that uniqueness of divisors is derivable from injectivity of doubling:

$$\varphi_{\text{double-inj}} \vdash \forall x.\ \mathrm{even}(x) \to \exists! z.\, x \doteq p(z, z) \tag{8.22}$$

To make injectivity of doubling better usable in the proof of (8.22) below, let us derive for some context $\Delta$, and terms $r$, $s$ and $t$ the following sequent.

$$\Delta \mid \varphi_{\text{double-inj}}, r \doteq p(s, s), r \doteq p(t, t) \vdash s \doteq t \tag{8.23}$$

This sequent allows us to use an intermediate term $r$ to relate the doubling of $s$ and of $t$ when proving $s$ and $t$ equal. Note that the proof uses symmetry and transitivity that we derived in theorem 8.16, and that the steps 4 and 5 formalise the chain of equations $p(s, s) \doteq x \doteq p(t, t)$.

| | | |
|---|---|---|
| 1 | $\varphi_{\text{double-inj}}$ | |
| 2 | $x \doteq p(s, s)$ | |
| 3 | $x \doteq p(t, t)$ | |
| 4 | $p(s, s) \doteq x$ | Sym, 2 |
| 5 | $p(s, s) \doteq p(t, t)$ | Trans, 4, 3 |
| 6 | $\forall n.\, p(s, s) \doteq p(n, n) \to s \doteq n$ | $\forall$E, 1 |
| 7 | $p(s, s) \doteq p(t, t) \to s \doteq t$ | $\forall$E, 6 |
| 8 | $s \doteq t$ | $\to$E, 7, 5 |

Using eq. (8.23), we can prove the uniqueness of the divisor of even num-

bers. The derivation of eq. (8.21) uses a typical combination: assume that an object with some property exists and prove that this object is unique. In the course of this, we use existential elimination (line 4-7) to inspect the object that we know to exist, and then introduce the uniqueness quantifier in line 7 by appealing to eq. (8.23).

| | | |
|---|---|---|
| 1 | $\varphi_{\text{double-inj}}$ | |
| 2 | $x$ | |
| 3 | $\quad \text{even}(x)$ | |
| 4 | $\quad y \quad N(x) \land x \doteq p(y,y)$ | |
| 5 | $\quad\quad x \doteq p(y,y)$ | $\land$E, 4 |
| 6 | $\quad\quad y' \quad x \doteq p(y',y')$ | |
| 7 | $\quad\quad\quad y \doteq y'$ | (8.23), 5, 6 |
| 8 | $\quad\quad \exists! z.\, x \doteq p(z,z)$ | $\exists!$I, 5, 6–7 |
| 9 | $\quad \exists! z.\, x \doteq p(z,z)$ | $\exists$E, 4–8 |
| 10 | $\text{even}(x) \to \exists! z.\, x \doteq p(z,z)$ | $\to$I, 3–9 |
| 11 | $\forall x.\ \text{even}(x) \to \exists! z.\, x \doteq p(z,z)$ | $\forall$I, 2–10 |

Example 8.20 shows how $\mathbf{ND}_1^=$ can be used to do equational reasoning, as it occurs in computer science and mathematics. Clearly, the proofs are fairly lengthy because we have to make every step explicit, which we typically not do on paper. However, the amount of boilerplate can be reduced by using a computer to automate some, or even all, of the steps, while still retaining the certainty of a formal proof as the one above. This is the topic of proof assistants, which we leave for another time.

Note that $\mathbf{ND}_1^=$ differs from $\mathbf{ND}_1$ only by the extra rules for equality. This means that whenever a formula does not use the equality symbol, then we should not be able to prove anything new. The following lemma makes this idea precise.

**Theorem 8.24: Conservativity of $\mathbf{ND}_1^=$ over $\mathbf{ND}_1$**

There is a map $e : \text{Form}(\mathcal{L}) \to \text{Form}(\mathcal{L}^=)$, such that for $\varphi \in \text{Form}(\mathcal{L})$ the sequent $\vdash \varphi$ is provable in $\mathbf{ND}_1$ if and only if $\vdash e(\varphi)$ is provable in $\mathbf{ND}_1^=$.

*Proof.* The map $e$ is defined by iteratively mapping a formula in $\mathrm{Form}(\mathcal{L})$ to the same formula in $\mathrm{Form}(\mathcal{L}^{=})$: $e(\bot) = \bot$, $e(P(t_1, \dots, t_n)) = P(t_1, \dots, t_n)$, $e(\varphi \wedge \psi) = e(\varphi) \wedge e(\psi)$ etc. Mapping a proof of $\vdash \varphi$ in $\mathbf{ND}_1$ is similarly immediate because $\mathbf{ND}_1^{=}$ just extends $\mathbf{ND}_1$. The non-trivial implication maps a proof of $\vdash e(\varphi)$ in $\mathbf{ND}_1^{=}$ to a proof in $\mathbf{ND}_1$ by induction on proofs, using that the only subformulas of $e(\varphi)$ can appear in the proof and thus neither (=I) nor (=E) are used. □

Theorem 8.24 is typical for situations when a logic is extended by new connectives and proof rules. That $\mathbf{ND}_1^{=}$ is conservative over $\mathbf{ND}_1$ tells us precisely that the extension of the proof system only adds new theorems when equality is involved.

We should note that the proof of theorem 8.24 involves a rather vague reference to iteratively defined maps and proofs by induction. The problem is that we have circumvented the formalisation of induction principles for terms, formulas and proof because making them precise is rather involved, as we will see.

# Exercises

## Exercise 29

Give derivations for the two rules in lemma 8.18.

## Exercise 30

a) Formalise the sentence

"Pavel owes money to everyone but himself"

as a formula $\varphi$ in first-order logic with equality. You need one constant $p$ for "Pavel" and one binary predicate symbol $O$ for "owes to".

b) Derive for your formula $\varphi$ the following sequent in $\mathbf{ND}_1^=$ using a Fitch-style proof.

$$\varphi \vdash \neg O(p, p)$$

## Exercise 31

Let $\mathcal{L}$ be a signature with a unary predicate symbol $P$. We define $P_1$ to be the formula

$$P_1 \quad = \quad \forall y. \forall z. P(y) \land P(z) \to y \doteq z,$$

which expresses that there can be maximally one object that fulfils $P$. Prove the following logical equivalence in $\mathbf{ND}_1^=$:

$$\vdash (\exists! x. P(x)) \leftrightarrow ((\exists x. P(x)) \land P_1)$$

To approach the proof of this formula, do both implications in separate proofs and refer to them in the proof of the logical equivalence. Furthermore, use the derived rules for the uniqueness quantifier from lemma 9.11 in the lecture notes.

## Exercise 32 *Graph Reachability*

We have seen that compactness prevents us from giving a *formula* that expresses reachability in graphs. In this exercise, we will see that reachability

can be expressed by appropriately defining a *predicate*. Let $\mathcal{L}$ be the signature $(\{E, R\}, \{n_1, \dots, n_4\}, \mathrm{ar})$ with $\mathrm{ar}(E) = \mathrm{ar}(R) = 2$ and $\mathrm{ar}(n_k) = 0$ for $k = 1, \dots 4$. The intention is that $E(x, y)$ holds if there is an edge between $x$ and $y$ in a given graph, and $R(x, y)$ holds if the node $y$ is reachable from $x$. We will use the constants $n_k$ later to model the nodes of a concrete graph.

Reachability $R$ is the reflexive and transitive closure of the edge relation $E$. In other words, each node $x$ must be related to itself via $E$ (reflexivity), and if there is an edge from $x$ to some $z$ and $y$ is reachable from $z$, then $y$ is also reachable from $x$ (transitivity).

a) Give two formulas $\varphi_r$ and $\varphi_t$ with free variables $x$ and $y$ that express, respectively, reflexivity and transitivity. That is to say, that the formula $\varphi_R$ given by

$$\forall x.\, \forall y.\, R(x, y) \leftrightarrow \varphi_r \lor \varphi_t$$

expresses that $R$ is the reachability relation in the graph with edges $E$.

b) Let $\varphi_{R,1}$, $\varphi_{R,2}$ and $\varphi_{R,3}$ be given by

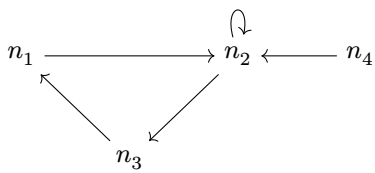
$$\begin{aligned}
\varphi_{R,1} &= \forall x.\, \forall y.\, R(x, y) \to \varphi_r \lor \varphi_t \\
\varphi_{R,2} &= \forall x.\, \forall y.\, \varphi_r \to R(x, y) \\
\varphi_{R,3} &= \forall x.\, \forall y.\, \varphi_t \to R(x, y)
\end{aligned}$$

Derive the following sequent in $\mathbf{ND}_1^=$ using Fitch-style.

$$\varphi_{R,1}, \varphi_{R,2}, \varphi_{R,3} \vdash \varphi_R$$

Note that the proof merely uses the rules for quantifiers and propositional connectives, not those for equality.

c) Consider the following graph $G$.




Give formulas $\varphi_{E,1}, \dots, \varphi_{E,5}$ that describe the edge of $G$.

d) Give a proof in $\mathbf{ND}_1^=$ using Fitch-style of $\varphi_R, \varphi_{E,1}, \dots, \varphi_{E,5} \vdash R(n_4, n_3)$. Use b) to simplify the task.

## Exercise 33

A *group* $G$ is given by a binary map $\dot{+} \colon G \times G \to G$ and an element $\dot{0}$ of $G$, such that

1. for all $x$ in $G$, $x \dot{+} \dot{0} = \dot{0} \dot{+} x = x$,
2. for all $x, y, z$ in $G$, $x \dot{+} (y \dot{+} z) = (x \dot{+} y) \dot{+} z$, and
3. for every $x$ in $G$ there is a $y$ in $G$, such that $x \dot{+} y = \dot{0}$ and $y \dot{+} x = \dot{0}$.

The equations 1 - 3 are called the *group axioms*. Groups appear everywhere in computer science and mathematics, with very popular applications in cryptography. The goal of this exercise is to formally reason about groups in first-order logic with equality.

Let $\mathcal{L}$ be the signature with function symbols $\dot{+}$ and $\dot{0}$ of arity 2 and 0, respectively. Let us write, as above, the symbol $\dot{+}$ in infix notation, that is, we write $s \dot{+} t$ instead of $\dot{+}(s, t)$ for terms $s$ and $t$. We define $\mathcal{L}^{=}$-formulas $\varphi_{1,l}$ and $\varphi_{1,r}$ by

$$\varphi_{1,l} = \forall x.\, \dot{0} \dot{+} x \doteq x \quad \text{and} \quad \varphi_{1,r} = \forall x.\, x \dot{+} \dot{0} \doteq x$$

that formalise together the first group axiom.

a) Give $\mathcal{L}^{=}$-formulas $\varphi_2$ and $\varphi_3$ that formalise the axioms 2 and 3 from above.

b) Give a formula $\varphi_u$ that expresses the uniqueness of $y$ in the third group axiom. The element $y$ is called the *inverse* of $x$.

c) Prove in $\mathbf{ND}_1^{=}$ that the inverse $y$ of $x$ in the third axiom is unique, that is, derive $\vdash \varphi_u$ for your formula in $\mathbf{ND}_1^{=}$. Use Fitch-style as usual.

## Exercise 34

In this exercise, we look at one possibility of reasoning about maps, which is based on so-called *complete partial combinatory algebras*. These are an algebraic formulation of higher-order functions and computations.

Let $\mathcal{L}$ be the signature with function symbols $\{\cdot, k, s\}$ and predicate symbols $\{D\}$ with $\mathrm{ar}(\cdot) = 2$, $\mathrm{ar}(k) = \mathrm{ar}(s) = 0$ and $\mathrm{ar}(D) = 1$. We will write $\cdot$ in infix notation, that is, $s \cdot t$ instead of $\cdot(s, t)$. The intent is that $s \cdot t$ denotes the application of $s$ to $t$ and $D(s)$ states that $s$ is defined. The two constants $k$ and

$s$ are basic maps, which we will axiomatise below. We also introduce for any two $\mathcal{L}$-terms $s$ and $t$ the following abbreviation:

$$s \simeq t \stackrel{\text{def}}{=} \forall x.\, D(s \cdot x) \to D(t \cdot y) \land s \cdot x \doteq t \cdot x$$

Let $\Gamma$ consist of the following formulas.

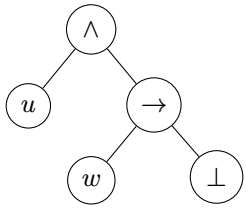
$$\begin{aligned}
&\forall x.\, \forall y.\, (k \cdot x) \cdot y \simeq x \\
&\forall x.\, D(s \cdot x) \\
&\forall x.\, \forall y.\, D((s \cdot x) \cdot y) \\
&\forall x.\, \forall y.\, \forall z.\, ((s \cdot x) \cdot y) \cdot z \simeq (x \cdot z) \cdot (y \cdot z)
\end{aligned}$$

a) Prove that $\simeq$ is transitive. That is, find a proof in $\mathbf{ND}_1^{=}$ for the sequent $\Gamma \vdash \forall x.\, \forall y.\, \forall z.\, x \simeq y \to y \simeq z \to x \simeq z$.

b) Let $i$ be the term defined by $i = (s \cdot k) \cdot k$. Derive $\Gamma \vdash \forall x.\, i \cdot x \simeq x$ in $\mathbf{ND}_1^{=}$.

# 9. Trees and Induction

Trees are at the foundation of life, as they are at the foundation of logic and computer science. We have encountered various kinds of trees. For instance, we have used parse trees to represent propositional formulas:




We also have used trees to formally define what a deduction in a proof system is. Similarly, terms and formulas of first-order logic can be seen as trees. In all cases, a tree consists of labelled nodes with edges between them and that adheres to three properties:

1. it has a unique root node, that is, a node with no incoming edges;
2. every node in tree has an outgoing branching type depending on the label of the node; and
3. the tree does not grow into itself or infinitely deep.

For instance, take the parse tree from above. It has five nodes that are labelled with propositional variables or connectives, and it has indeed these three properties:

1. The root node is the node labelled with conjunction.
2. Nodes labelled with propositional variables or $\perp$ have no outgoing branches, while those labelled with the binary connectives $\wedge$ or $\rightarrow$ have two outgoing branches.
3. Finally, the tree has finite depths and only grows downwards.

To understand when the conditions are not fulfilled, we show some counter-examples in figs. 9.1 to 9.4.

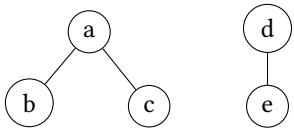



Figure 9.1.: Not a tree: there is no single root node

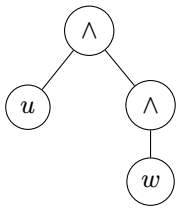



Figure 9.2.: Not a tree: inconsistent branching type because nodes labelled with $\wedge$ have one and two outgoing branches

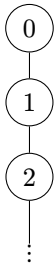



Figure 9.3.: Not a tree: Tree of all natural numbers has infinite depth

What makes trees so special is that they can be traversed in a finite number of steps by starting at the root. The goal of this chapter is to formalise what that means and to use the resulting principles of iteration and induction in our study of logic.

For the remainder of this chapter, you should be comfortable with the concept of maps and sets, see appendix B.1 and appendix B.2 for a refresher.

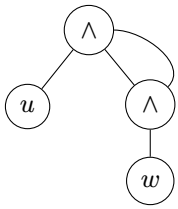



Figure 9.4.: Not a tree: an edge growing back into the structure (there is also no root)

## 9.1. Trees and Induction

In order to provide a general account of trees, let us take a closer look at terms over a signature $\mathcal{L}$, see definition 4.12. Suppose the signature has symbols $c$, $k$, $g$ and $h$ with arity 0, 1, 2 and 3 respectively. This allows us to form terms such as $g(k(c), h(c, c, c))$. As tree, this term would look as in fig. 9.5. What is

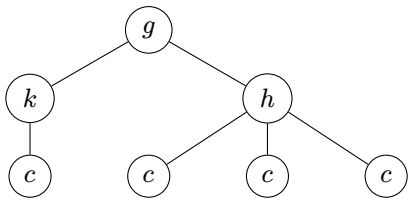



Figure 9.5.: A term as seen as tree

important in this tree is that that number of children of each node corresponds to the arity of the function symbol the node is labelled with. Thus, in order to build such trees, we need to know what the labels of the nodes should be and how many children a node with a given label should have. In what follows, we will make a leap in abstraction from this perspective in order to obtain a more convenient definition of trees.

Let use denote for $n \in \mathbb{N}$ by $[n\rangle$ the set

$$[n\rangle = \{k \in \mathbb{N} \mid 0 \leq k < n\}$$

with $n$ elements, see appendix B.2. Given a set $X$, a map $[n\rangle \to X$ corresponds precisely to an $n$-tuple of elements in $X$, see appendix B.2. With this in mind, we could reformulate the last term formation rule in definition 4.12 to the

following rule.

$$\frac{f \in \mathcal{F}_n \qquad \alpha\colon [n\rangle \to \mathrm{Term}(\mathcal{L})}{f(\alpha) \in \mathrm{Term}(\mathcal{L})}$$

For instance, the node labelled $h$ in fig. 9.5 has three children each being the term $c$. We can view these children as given by the map $\alpha\colon [3\rangle \to \mathrm{Term}$ with $\alpha(i) = c$. This gives us a term $t_1 = h(\alpha)$ by the above formation rule. Similarly, we can consider $\beta\colon [1\rangle \to \mathrm{Term}$ be given by $\beta(0) = c$ and then form $t_0 = k(\beta)$. Finally, the tree in fig. 9.5 is constructed by considering $\gamma\colon [2\rangle \to \mathrm{Term}$ with $\gamma(k) = t_k$ and the forming the term $g(\gamma)$. We can

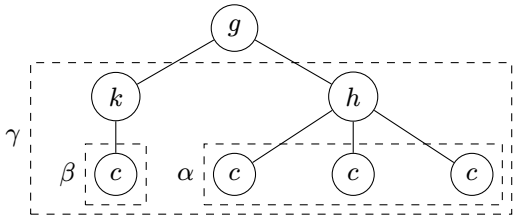



Figure 9.6.: A term as seen as tree

picture this layered construction of the tree as in fig. 9.6.

This view on terms is almost the general definition trees that we are after, and we only allow one generalisation: instead of have finitely many children indexed by $[n\rangle$ for some $n \in \mathbb{N}$, we allow nodes to have arbitrary sets as index for the children. This leads us to the following definition, which generalises that of function symbols and arities to general branching types.

**Definition 9.1**

We call a pair $(L, B)$ a **branching type** if $L$ is a set and $B$ an $L$-indexed family of sets, that is, for every $a \in L$ we are given a set $B_a$.

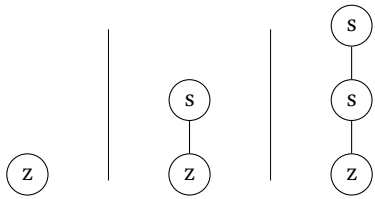
Note that the terminology of "branching type" also tacitly includes labels. The intuition of this definition is that a tree of type $(L, B)$ will be labelled in $L$ and will have at a node with label $a \in L$ one branch for every element in $B_a$.

Let us start with the simplest example of trees: natural numbers.

**Example 9.2: Branching type of natural numbers**

Recall that the induction principle for natural numbers states that a natural number is either zero or the successor of another natural number, see appendix B.4. Thus to present natural numbers as trees, we can use labels

$\{z, s\}$ for zero and successor. For instance, the following three trees represent the numbers 0, 1 and 2.




To ensure that a node labelled with "$z$" has no successor and and "$s$"-labelled nodes have exactly one successor, let us define $L^{\text{nat}} = \{z, s\}$ and then an $L^{\text{nat}}$-indexed family of sets $B^{\text{nat}}$ by

$$B_z^{\text{nat}} = \emptyset \quad \text{and} \quad B_s^{\text{nat}} = \{*\}\,.$$

What matters in this definition is that set $B_s^{\text{nat}}$ has exactly one element, which says that a successor node has exactly one child. We will use $(L^{\text{nat}}, B^{\text{nat}})$ as the branching type of natural numbers.

Let us see another branching type, that of binary trees.

**Example 9.3**

Binary trees (not balanced!) labelled in $\mathbb{N}$ are induced by the branching type $(L^{\text{bin}}, B^{\text{bin}})$ with $L^{\text{bin}} = \mathbb{N} \cup \{*\}$, and $B_n^{\text{bin}} = [2\rangle$ for $n \in \mathbb{N}$ and $B_*^{\text{bin}} = \emptyset$. The idea is that an inner node can be labelled with a number, while a leaf is labelled with $*$. An inner node has then exactly two children and a leaf has none.

More generally, we can turn a signature into a branching type.

**Example 9.4**

Let $\mathcal{L} = (\mathcal{F}, \mathcal{R}, \text{ar})$ be a signature. We can define a branching type by $(\mathcal{F}, B^{\mathcal{L}})$ with $B_f^{\mathcal{L}} = [\text{ar}(f)\rangle$. Thus, tree over this branching type will be given as in fig. 9.6. However, such trees will not include variables, but we will come back to this in section 9.2.

Given a branching type $(L, B)$, we aim to define a set $\mathcal{T}_{(L,B)}$ of trees for that branching type. The idea is that this set is given by a so-called universal property. In order to state it, we write $[X, Y]$ for the set of maps $X \to Y$, see appendix B.2. The set of trees has then following two properties.

1. First, we need to be able to construct trees. We follow here the opening dis-

cussion and require that, given a label $a \in L$ and a map $\alpha\colon B_a \to \mathcal{T}_{(L,B)}$, there is a tree in $\mathcal{T}_{(L,B)}$ that has $a$ as label at the root and the trees given by $\alpha$ as children. For instance, with the branching type for binary trees from example 9.3, a node labelled by $n \in \mathbb{N}$ is binary branching with a map $\alpha\colon [2\rangle \to \mathcal{T}_{(L^{\text{bin}},B^{\text{bin}})}$ like in fig. 9.7, where we draw an edge from the node labelled $n$ to $\alpha(k)$ and mark that edge with $k$. The trees labelled with $n$ at the root in are then de-

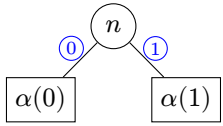



Figure 9.7.: Node with branches in a binary tree

termined by a map $\mathrm{tr}_n\colon \left[B_n^{\text{bin}}, \mathcal{T}_{(L^{\text{bin}},B^{\text{bin}})}\right] \to \mathcal{T}_{(L^{\text{bin}},B^{\text{bin}})}$ and the tree in fig. 9.7 is given by $\mathrm{tr}_n(\alpha)$.

2. The second property of $\mathcal{T}_{(L,B)}$ comprises two parts. First, we ask for a principle that allows us to traverse trees. The idea for this principle is as follows: if we can compute for all labels $a$ and a recursively computed result for the children of nodes labelled with $a$ an output, then we can traverse an arbitrary trees and produce a result. For example, in the case of binary branching trees we need to be able to produce a constant value for nodes labelled $*$, since such nodes have no children, and for inner nodes labelled with a natural number we need to combine that number with the result of the two children. As a simple example, we could return 0 at leaves and sum the number $n$ in fig. 9.7 with the outcome of summing at $\alpha(0)$ and $\alpha(1)$. This procedure would result in map $f\colon \mathcal{T}_{(L^{\text{bin}},B^{\text{bin}})} \to \mathbb{N}$ that computes the sum of all numbers in the tree. Second, we require this map to be unique for the above description.

The last bit seems rather innocuous, but it is actually uniqueness of such maps that will ensure that we get trees of finite height and that we can reason about them by induction. We formalise this discussion in the following definition.

**Definition 9.5: Trees as inductive structures**

The **trees** with branching type $(L, B)$ are a set $\mathcal{T}_{(L,B)}$ together with a family $\boxed{\mathrm{tr}}$ of maps $\mathrm{tr}_a\colon [B_a, \mathcal{T}_{(L,B)}] \to \mathcal{T}_{(L,B)}$ indexed by elements $a \in L$, called **tree constructor**, such that the following **iteration principle** is fulfilled: for all sets $X$ and families $f$ of maps with $f_a\colon [B_a, X] \to X$, there is a

*unique* map $\overline{f}\colon \mathcal{T}_{(L,B)} \to X$ with

$$\overline{f}(\mathrm{tr}_a(\alpha)) = f_a(\overline{f} \circ \alpha) \tag{9.6}$$

for all $a \in L$ and $\alpha\colon B_a \to \mathcal{T}_{(L,B)}$. We call $\overline{f}$ the **inductive extension** of $f$ and say that $\overline{f}$ is defined by **iteration** from $f$. If the branching type is clear from the context, we drop the subscript on $\mathcal{T}_{(L,B)}$ and just write $\mathcal{T}$.

Explicitly, uniqueness of the inductive extension in definition 9.5 means that for any map $g\colon \mathcal{T}_{(L,B)} \to X$ that fulfils $g(\mathrm{tr}_a(\alpha)) = f_a(g \circ \alpha)$ for all $a \in L$ and $\alpha\colon B_a \to \mathcal{T}_{(L,B)}$, it holds that $\overline{f} = g$. In other words, any map that can be filled in for $\overline{f}$ in eq. (9.6) must be already equal to $\overline{f}$. The existence of $\overline{f}$ is what underlies tree traversal in many algorithms and programming languages, but it is the uniqueness of $\overline{f}$ that yields powerful proof principle, as we will see.

Intuitively, $\mathrm{tr}_a(\alpha)$ is a tree with a root node that is labelled with $a$ and has children given by $\alpha$. Figure 9.7 shows this in the case that $a$ is a number $n$ and $\alpha\colon [2\rangle \to \mathcal{T}_{(L^{\mathrm{bin}},B^{\mathrm{bin}})}$.

Let us unfold definition 9.5 first in the case of natural numbers. In the following example, and in general, we will denote for any set $X$ by $\varepsilon$ the unique map $\emptyset \to X$, see exercise 66.

**Example 9.7: Natural numbers as trees**

Recall the branching type $(L^{\mathrm{nat}}, B^{\mathrm{nat}})$ of natural numbers from example 9.2. Let us write $\mathcal{N}$ for $\mathcal{T}_{(L^{\mathrm{nat}},B^{\mathrm{nat}})}$. Since $L^{\mathrm{nat}} = \{z, s\}$, there are two cases for the node maps tr to consider: $\mathrm{tr}_z\colon [B_z^{\mathrm{nat}}, \mathcal{N}] \to \mathcal{N}$ and $\mathrm{tr}_s\colon [B_s^{\mathrm{nat}}, \mathcal{N}] \to \mathcal{N}$. In the first case, $[B_z^{\mathrm{nat}}, \mathcal{N}] = [\emptyset, \mathcal{N}] = \{\varepsilon\}$ and we have only one way of using $\mathrm{tr}_z$, which we assign a suggestive notation: $\underline{0} = \mathrm{tr}_z(\varepsilon)$. In the second case, $[B_s^{\mathrm{nat}}, \mathcal{N}] = [\{*\}, \mathcal{N}]$, which has as elements maps $\{*\} \to \mathcal{N}$. Given a tree $t \in \mathcal{N}$, we write $\underline{s}(t) = \mathrm{tr}_s(\alpha_t)$, where $\alpha_t\colon \{*\} \to \mathcal{N}$ is the map given by $\alpha_t(*) = t$. The three trees from example 9.2 are then given, respectively, by $\underline{0}$, $\underline{s}(\underline{0})$ and $\underline{s}(\underline{s}(\underline{0}))$.
To show that all trees in $\mathcal{N}$ correspond to natural numbers, we can use the iteration principle as follows. We define a map $\overline{h}\colon \mathcal{N} \to \mathbb{N}$ by defining maps $h_z\colon [B_z^{\mathrm{nat}}, \mathbb{N}] \to \mathbb{N}$ and $h_s\colon [B_s^{\mathrm{nat}}, \mathbb{N}] \to \mathbb{N}$ by

$$h_z(\varepsilon) = 0 \quad \text{and} \quad h_s(\alpha) = \alpha(*) + 1$$

and then $\overline{h}$ is the inductive extension of $h = \{h_z, h_s\}$. Applying it to the

above trees yields by eq. (9.6) the expected computations:

$$\begin{aligned}
\overline{h}(\underline{0}) &= h_z(\overline{h} \circ \varepsilon) = 0 \\
\overline{h}(\underline{s}(\underline{0})) &= h_s(\overline{h} \circ \alpha_{\underline{0}}) = (\overline{h} \circ \alpha_{\underline{0}})(*) + 1 = \overline{h}(\underline{0}) + 1 = 0 + 1 = 1 \\
\overline{h}(\underline{s}(\underline{s}(\underline{0}))) &= h_s(\overline{h} \circ \alpha_{\underline{s}(\underline{0})}) = (\overline{h} \circ \alpha_{\underline{s}(\underline{0})})(*) + 1 = \overline{h}(\underline{s}(\underline{0})) + 1 = 1 + 1 = 2
\end{aligned}$$

In example 9.7, we have used the iteration principle to construct *some* map but we have not used that it is *unique*. The uniqueness is what gives us an induction proof principle. To formulate the induction principle, we need to state what a **property** of trees is. There are different ways to do that, but we use the naive approach to allow any property that can be defined as a subset. For example, if one wishes to prove the equation $\sum_{k=0}^{n} k = \frac{n(n+1)}{2}$ for all $n$, then the corresponding property to be proven is the set $P \subseteq \mathbb{N}$ given by $P = \left\{ n \,\middle|\, \sum_{k=0}^{n} k = \frac{n(n+1)}{2} \right\}$. Proving that $P$ holds for all natural numbers amounts then to proving that the reverse inclusion $\mathbb{N} \subseteq P$ and thus $\mathbb{N} = P$ holds. The induction principle for natural numbers (theorem B.6) requires us prove that $0 \in \mathbb{N}$ and that, whenever $n \in \mathbb{N}$, also $n + 1 \in P$. The following definition expresses this requirement in general for properties of trees.

**Definition 9.8**

Let $(L, B)$ be a branching type and $P \subseteq \mathcal{T}_{(L,B)}$ a **property** of $(L, B)$-branching trees. We call $P$ **inductive** if for all $a \in L$ and $\alpha \colon B_a \to \mathcal{T}_{(L,B)}$ with $\alpha(b) \in P$ for all $b \in B_a$ it holds that $\mathrm{tr}_a(\alpha) \in P$.

We can then show that all inductive properties on trees are provable by using iteration.

**Theorem 9.9: Tree Induction**

Let $(L, B)$ be a branching type and $P \subseteq \mathcal{T}_{(L,B)}$ an inductive property. In this case $\mathcal{T}_{(L,B)} \subseteq P$, that is, $P$ holds for all trees in $\mathcal{T}_{(L,B)}$.

*Proof.* Let $P$ be an inductive property of $(L, B)$-branching trees and denote by $i \colon P \to \mathcal{T}$ its inclusion map. That $P$ is inductive is equivalent to saying that the node maps tr restrict to a family of maps $n_a \colon [B_a, P] \to P$, such that $i(n_a(\alpha)) = \mathrm{tr}_a(i \circ \alpha)$ for all $\alpha \colon B_a \to P$.

We use the iteration principle to show that there is a map $\mathcal{T} \to P$ and that this map must be an inclusion. To this end, define use the inductive extension of the family $n$ to obtain a map $\overline{n} \colon \mathcal{T} \to P$. To prove that this is an inclusion, we need to show that $\overline{n}(t) = t$ for all $t \in \mathcal{T}$, in other words, that $i \circ \overline{n} = \mathrm{id}$. For this, we use the uniqueness of maps out of $\mathcal{T}$ that fulfil eq. (9.6) and show

that both $\mathrm{id}\colon \mathcal{T} \to \mathcal{T}$ and $i \circ \overline{n}$ are inductive extensions of the family tr. First, we obviously have

$$\mathrm{id}(\mathrm{tr}_a(\alpha)) = \mathrm{tr}_a(\alpha) = \mathrm{tr}_a(\mathrm{id} \circ \alpha)\,.$$

Thus, id fulfils eq. (9.6) and we get $\mathrm{id} = \overline{\mathrm{tr}}$. Next, we have the following.

$$\begin{aligned}
(i \circ \overline{n})(\mathrm{tr}_a(\alpha)) &= i(\overline{n}(\mathrm{tr}_a(\alpha))) && \text{definition composition}\\
&= i(n_a(\overline{n} \circ \alpha)) && \text{eq. (9.6) for } \overline{n}\\
&= \mathrm{tr}_a(i \circ (\overline{n} \circ \alpha)) && P \text{ inductive}\\
&= \mathrm{tr}_a((i \circ \overline{n}) \circ \alpha) && \text{composition associative}
\end{aligned}$$

Therefore, also $i \circ \overline{n} = \overline{\mathrm{tr}}$ and we get $i \circ \overline{n} = \mathrm{id}$ from the previous result, which shows that $\overline{n}$ is the inclusion $\mathcal{T} \subseteq P$. □

Before looking at other examples, let us see what the induction principle for trees looks like on our representation of natural numbers in $\mathcal{N}$.

**Example 9.10**

Recall from example 9.7 that we write $\underline{0}$ for $\mathrm{tr}_z(\varepsilon)$ and $\underline{s}(t) = \mathrm{tr}_z(\alpha_t)$. A property $P \subseteq \mathcal{N}$ is then inductive precisely if

1. $\underline{0} \in P$ and,
2. $t \in P$ implies $\underline{s}(t) \in P$.

We recognise readily 1) as the usual base case and 2) as the induction step, cf. theorem B.6.

Next, let us examine what the iteration and induction principle amount to on binary trees.

**Example 9.11: Binary trees**

Let us write $\mathcal{B} = \mathcal{T}_{(L^{\mathrm{bin}}, B^{\mathrm{bin}})}$ for a set of trees with the branching type we introduced in example 9.3. This set comes with a family tr of maps indexed by $\mathbb{N} \cup \{*\}$, that is, we get one map $\mathrm{tr}_*\colon [\emptyset, \mathcal{B}] \to \mathcal{B}$ and for every $n \in \mathbb{N}$ a map $\mathrm{tr}_n\colon [[2\rangle, \mathcal{B}] \to \mathcal{B}$. Analogously to example 9.7, let us abbreviate $\mathrm{tr}_*(\varepsilon)$ to $\ell$, which is a tree consisting only of one (unlabelled) leaf. From exercise 65, we know that giving a map $[2\rangle \to \mathcal{B}$ amounts to giving a pair of elements in $\mathcal{B}$. We will thus write $\mathrm{tr}_n(t_0, t_1)$ instead of $\mathrm{tr}_n(\alpha)$ for the map $\alpha(k) = t_k$, where $t_0, t_1 \in \mathcal{B}$. This is a representation of the following tree.

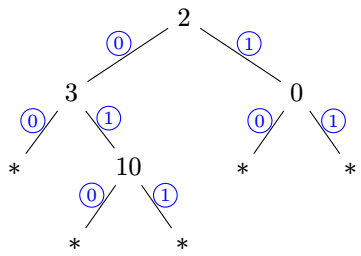



Figure 9.8.: Example of a binary tree labelled in $\mathbb{N}$. The root is labelled with the number 2 and has two children. The circled blue numbers indicate the number of the outgoing branching.

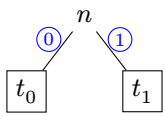



Figure 9.8 shows an example of a binary tree, which is represented in $\mathcal{B}$ by

$$t = \mathrm{tr}_2(\mathrm{tr}_3(\ell, \mathrm{tr}_{10}(\ell, \ell)), \mathrm{tr}_0(\ell, \ell))\,.$$

So much for the construction of tree. To traverse such trees, we can appeal to the iteration principle. For instance, we can sum up all the labels in a tree by using the family $s$ given by

$$\begin{aligned} &s_n \colon [[2\rangle, \mathbb{N}] \to \mathbb{N} && s_* \colon [\emptyset, \mathbb{N}] \to \mathbb{N} \\ &s_n(\alpha) = n + \alpha(0) + \alpha(1) && s_*(\alpha) = 0 \end{aligned}$$

this gives us a map $\overline{s} \colon \mathcal{B} \to \mathbb{N}$. Running this map on the tree $t$ from above,

yields the following computation.

$$\begin{aligned}
\overline{s}(t) &= s_2(\overline{s}(\mathrm{tr}_3(\ell, \mathrm{tr}_{10}(\ell,\ell))), \overline{s}(\mathrm{tr}_0(\ell,\ell))) \\
&= 2 + \overline{s}(\mathrm{tr}_3(\ell, \mathrm{tr}_{10}(\ell,\ell))) + \overline{s}(\mathrm{tr}_0(\ell,\ell)) \\
&= 2 + (3 + \overline{s}(\ell) + \overline{s}(\mathrm{tr}_{10}(\ell,\ell))) + (0 + \overline{s}(\ell) + \overline{s}(\ell)) \\
&= 2 + (3 + 0 + (10 + \overline{s}(\ell) + \overline{s}(\ell))) + (0 + 0 + 0) \\
&= 2 + (3 + 0 + (10 + 0 + 0)) + 0 \\
&= 2 + (3 + 0 + 10) + 0 \\
&= 2 + 13 + 0 \\
&= 15
\end{aligned}$$

Thus, $\overline{s}$ traverses depth-first and indeed sums up all the labels in the tree.

In the last example, we show how one can restrict to a set of tree that fulfil some property. This approach will later be important in order to capture, for example, proof trees. The idea is to define a map by iteration that checks the wanted property and then add the property as an assumption to the predicate to be proven.

**Example 9.12**

In this example, we characterise those binary trees that correspond to a list of natural numbers of the form $0, 1, \dots, n$ for some $n \in \mathbb{N}$, and then prove that the sum of such a tree is $\frac{n(n+1)}{2}$.
Define a map $\mathrm{lin} \colon \mathcal{B} \to \mathbb{B} \times \mathbb{N}$ by $\mathrm{lin} = \overline{f}$, where $\overline{f}$ is the inductive extension of the following family.

$$\begin{aligned}
f_*(\alpha) &= (1, 0) \\
f_n(\alpha) &= \begin{cases} (1, n), & \alpha(0) = (1,0), \alpha(1) = (1, r_1) \text{ and } n = r_1 + 1 \\ (0, n), & \text{otherwise} \end{cases}
\end{aligned}$$

A tree $t \in \mathcal{B}$ is then a list $0, \dots, n$ if $\mathrm{lin}(t) = (1, n)$.
We define a property $P \subseteq \mathcal{B}$ by

$$P = \left\{ t \in \mathcal{B} \;\middle|\; \text{if } \mathrm{lin}(t) = (1, n), \text{ then } \overline{s}(t) = \frac{n(n+1)}{2} \right\}.$$

To prove that $P$ holds for all binary trees, we have to prove by theorem 9.9 and example 9.11 the following two things:

1. $\ell \in P$

2. if $t_0, t_1 \in P$, then for all $n \in \mathbb{N}$ also $\mathrm{tr}_n(t_0, t_1) \in P$.

For the first item, we note that $\mathrm{lin}(\ell) = f_*(\varepsilon) = (1, 0)$ and $\overline{s}(\ell) = 0$. Therefore $\ell \in P$.
For the second item, let $t_0, t_1 \in P$. To prove that $\mathrm{tr}_k(t_0, t_1) \in P$ for $k \in \mathbb{N}$, assume that $\mathrm{lin}(\mathrm{tr}_k(t_0, t_1)) = (1, k)$. By definition of lin, this means that $\mathrm{lin}(t_0) = (1, 0)$, $\mathrm{lin}(t_1) = (1, n)$ and $k = n + 1$. With the assumption that $t_0$ and $t_1$ are in $P$, this gives us $\overline{s}(t_0) = \frac{0(0+1)}{2} = 0$ and $\overline{s}(t_1) = \frac{n(n+1)}{2}$. It remains to prove that $\overline{s}(\mathrm{tr}_{n+1}(t_0, t_1)) = \frac{(n+1)(n+2)}{2}$. Indeed, we have:

$$\begin{aligned}
\overline{s}(\mathrm{tr}_{n+1}(t_0, t_1)) &= n + 1 + \overline{s}(t_0) + \overline{s}(t_1) && \text{def. } \overline{s} \\
&= n + 1 + 0 + \frac{n(n+1)}{2} && \text{see above} \\
&= \frac{2(n+1) + n(n+1)}{2} && \text{arithmetic} \\
&= \frac{(2+n)(n+1)}{2} && \text{arithmetic}
\end{aligned}$$

This completes the proof that $t_0, t_1 \in P$ implies $\mathrm{tr}_k(t_0, t_1)$ for all $k \in \mathbb{N}$, and thus by the induction principle we have $\mathcal{B} \subseteq P$.

## 9.2. Terms and Parameterised Trees

Let us revisit example 9.4, where we introduced the branching type $(\mathcal{F}, B^{\mathcal{L}})$ for a signature $\mathcal{L}$. We can consider trees for this branching type, but such trees do not correspond to the terms that we introduced in definition 4.12 because we are missing out on the variables! There are two ways to rectify this. We can either extend the branching type with variables or we allow trees to have a parameter that determines additional elements that may appear in the leaves. We opt for the second option, as it is more elegant and will give us direct access to substitution.

Let $(L, B)$ be a branching type and $X$ a set that is disjoint from $L$. We define the trees **parameterised** in $X$ to be the set $\mathcal{T}_{(L,B)}(X)$ given by

$$\mathcal{T}_{(L,B)}(X) = \mathcal{T}_{(L \cup X, B')}\,,$$

where $B'_a = B_a$ for $a \in L$ and for $x \in X$ we put $B'_x = \emptyset$.

**Theorem 9.13**

There is a map $j_X \colon X \to \mathcal{T}_{(L,B)}(X)$, such that for every set $Y$ and map $f \colon X \to \mathcal{T}_{(L,B)}(Y)$, there is a unique map $\hat{f} \colon \mathcal{T}_{(L,B)}(X) \to \mathcal{T}_{(L,B)}(Y)$ with $\hat{f} \circ j_X = f$ and $\hat{f}(\mathrm{tr}_a(\alpha)) = \mathrm{tr}_a(\hat{f} \circ \alpha)$.

We need to be careful that the two occurrences of $\mathrm{tr}_a$ in the last equation are not the same: one constructs a tree in $\mathcal{T}_{(L,B)}(X)$ and the other in $\mathcal{T}_{(L,B)}(Y)$.

*Proof.* The map $j_X$ is given by $j_X(u) = \mathrm{tr}_u(\varepsilon)$ for $u \in X$. Suppose that $f \colon X \to \mathcal{T}_{(L,B)}(Y)$ is a map. We define $\hat{f} = \overline{h}$ as inductive extension of the following family $h$.

$$\begin{aligned} h_a(\alpha) &= \mathrm{tr}_a(\alpha) & a \in L, \alpha \colon B_a \to \mathcal{T}_{(L,B)}(Y) \\ h_u(\alpha) &= f(u) & u \in X, \alpha \colon \emptyset \to \mathcal{T}_{(L,B)}(Y) \end{aligned}$$

First, we get that $\hat{f} \circ j_X = f$ because

$$(\hat{f} \circ j_X)(u) = \hat{f}(\mathrm{tr}_u(\varepsilon)) = h_u(\hat{f} \circ \varepsilon) = f(u)\,,$$

and eq. (9.6) gives us, as required, that

$$\hat{f}(\mathrm{tr}_a(\alpha)) = h_a(\hat{f} \circ \alpha) = \mathrm{tr}_a(\hat{f} \circ \alpha)\,.$$

Second, suppose that $g \colon \mathcal{T}_{(L,B)}(X) \to \mathcal{T}_{(L,B)}(Y)$ is another map, such that $g \circ j_X = f$ and $g(\mathrm{tr}_a(\alpha)) = \mathrm{tr}_a(g \circ \alpha)$. We use the uniqueness of $\hat{f}$ to prove that $g = \hat{f}$, which means that we need to show that $g$ fulfils

$$g(\mathrm{tr}_s(\alpha)) = h_s(g \circ \alpha) \tag{9.14}$$

for all $s \in L \cup X$ and $\alpha \colon B_s \to \mathcal{T}_{(L,B)}(Y)$. There are two cases to consider:

1. Given $a \in L$, we have

$$g(\mathrm{tr}_a(\alpha)) = \mathrm{tr}_a(g \circ \alpha) = h_a(g \circ \alpha)\,.$$

2. Given $u \in X$ and $\alpha \colon \emptyset \to \mathcal{T}_{(L,B)}(Y)$, we have $\alpha = \varepsilon$ and thus

$$g(\mathrm{tr}_u(\alpha)) = g(\mathrm{tr}_u(\varepsilon)) = g(j_X(u)) = f(u) = h_u(\alpha)\,.$$

Thus, in either case eq. (9.14) holds and therefore, by uniqueness, $g = \hat{f}$. $\square$

Let us illustrate the power of theorem 9.13 by deriving directly the application of terms to substitutions. To this end, we shall write $\mathcal{T}(X)$ instead of $\mathcal{T}_{(\mathcal{F},B^{\mathcal{L}})}(X)$ for the parameterised trees over the branching type that we introduced above for a signature $\mathcal{L}$. In particular, we can use the set Var of variables as parameter and have a map $j_{\mathrm{Var}} \colon \mathrm{Var} \to \mathcal{T}(\mathrm{Var})$, which turns a variable into a tree that has only that variable as leaf. We can recognise this as the analogue of the identity substitution that we defined in definition 5.1. In general, a map $\sigma \colon \mathrm{Var} \to \mathcal{T}(\mathrm{Var})$ can be interpreted as a substitution, only on the representation of terms as trees. Theorem 9.13 gives us a unique extension of $\sigma$ to $\hat{\sigma} \colon \mathcal{T}(\mathrm{Var}) \to \mathcal{T}(\mathrm{Var})$ with

$$\hat{\sigma} \circ j_{\mathrm{Var}} = \sigma \quad \text{and} \quad \hat{\sigma}(\mathrm{tr}_f(\alpha)) = \mathrm{tr}_f(\hat{\sigma} \circ \alpha)$$

for all $f \in \mathcal{F}_n$ and $\alpha \colon [n\rangle \to \mathcal{T}(\mathrm{Var})$.

We can bring this back into our previous notation by making the following identification. First, we can write $f(t_0, \dots, t_n)$ instead of $\mathrm{tr}_f(\alpha)$ for $f \in \mathcal{F}_{n+1}$ and $t_k = \alpha_k$. Second, for constants $c \in \mathcal{F}_0$, we write $c$ instead of $\mathrm{tr}_c(\varepsilon)$. Then the above two equations can be explicitly written as follows for all $f \in \mathcal{F}_{n+1}$, $t_k \in \mathcal{T}(\mathrm{Var})$, $c \in \mathcal{F}_0$ and $x \in \mathrm{Var}$, which agree with those in definition 5.1.

$$\begin{aligned} \hat{\sigma}(j_{\mathrm{Var}}(x)) &= \sigma(x) \\ \hat{\sigma}(c) &= c \\ \hat{\sigma}(f(t_0, \dots, t_n)) &= f(\hat{\sigma}(t_0), \dots, \hat{\sigma}(t_n))) \end{aligned}$$

To sum up, the trees that we introduced in definition 9.5 cover common sets, such as natural numbers and terms, which appear throughout our rondo in the logic world. Moreover, their defining property, the iteration principle, allows us to construct all operations that we wish to carry out on those trees and yields known induction principles. In short, trees form the foundation of everything that we have been doing!

Indeed, we have even explicitly considered parse tree and of course proof trees. Exercise 38 asks you to show that definition 9.5 can be applied to present propositional formulas as trees. However, the proof trees elude unfortunately this definition of trees. The issue is that in proof tree we not only want to state how many children a node has but also what formula each child proves, depending on the label of the node. This can be solved with an extended definition of trees, which we leave as an advanced topic. Similarly, also the presentation of first-order formulas will have to wait for a similar reason.

# Exercises

## Exercise 35

Define a map $c\colon \mathcal{B} \to \mathbb{N}$ that counts the number of labelled nodes in a binary tree by using the iteration principle.

## Exercise 36

Define by iteration a map $h\colon \mathcal{B} \to \mathbb{N}$ that computes that height of a tree, where the leaves should have height 0 and labelled nodes height at least 1.

## Exercise 37

Use the tree induction principle to prove that $c(t) \leq 2^{h(t)}$ for all $t \in \mathcal{B}$.

## Exercise 38

Give a branching type $(A, B)$, such that the parse trees of propositional formulas are given by parameterised trees $\mathcal{T}_{(A,B)}(\mathrm{PVar})$ over the set of propositional variables. Explicitly spell out the induction principle that results.

## Exercise 39

This is a somewhat longer exercise. Continuing exercise 38, use the iteration principle to define a map

$$\mathrm{Sub}\colon \mathcal{T}_{(A,B)}(\mathrm{PVar}) \to \mathcal{P}(\mathcal{T}_{(A,B)}(\mathrm{PVar}))$$

that computes the subformulas of a formula represented as tree. The main issue is that $t$ must be in $\mathrm{Sub}(t)$. Let us write

$$\begin{aligned} p_1 &\colon \mathcal{T}_{(A,B)}(\mathrm{PVar}) \times \mathcal{P}(\mathcal{T}_{(A,B)}(\mathrm{PVar})) \to \mathcal{T}_{(A,B)}(\mathrm{PVar}) \\ p_2 &\colon \mathcal{T}_{(A,B)}(\mathrm{PVar}) \times \mathcal{P}(\mathcal{T}_{(A,B)}(\mathrm{PVar})) \to \mathcal{P}(\mathcal{T}_{(A,B)}(\mathrm{PVar})) \end{aligned}$$

for the projection maps given by $p_1(t, U) = t$ and $p_2(t, U) = U$. To define Sub, you will have to define first a map

$$\overline{s} \colon \mathcal{T}_{(A,B)}(\mathrm{PVar}) \to \mathcal{T}_{(A,B)}(\mathrm{PVar}) \times \mathcal{P}(\mathcal{T}_{(A,B)}(\mathrm{PVar}))$$

by iteration and then set $\mathrm{Sub} = p_2 \circ \overline{s}$.

# 10. Boolean Semantics of First-Order Logic

In this chapter, we will discuss the following two questions:

1. What are objects and their properties and when is a formula valid?
2. Are all derivable sequents valid entailments?

The first question requires us to find a mathematical model of function and predicate symbols, which we can extend to semantics of terms and formulas. We will focus here on Boolean semantics for simplicity. The second question will be answered by proving that both our proof systems $\mathbf{ND}_1$ and $\mathbf{cND}_1$ are sound with respect to the Boolean semantics.

Let us begin by finding out what an appropriate model of function and predicate symbols may be.

## 10.1. Models of First-Order Logic

Recall that in propositional logic a valuation $v \colon \mathrm{PVar} \to \mathbb{B}$ on propositional variables determined the truth value of formulas. In first-order logic we need to provide interpretations instead for object variables, terms and predicates. Finding the right structures to do so is the subject of this section.

First, we set up a convenient notation for tuples, which are a generalisation of the products from appendix B.2. Given a set $A$ and $n \in \mathbb{N}$, we write $\boxed{A^n}$ for the $n$-fold **finite product** of $A$, consisting of $n$-tuples of elements in $A$ as follows.

$$A^n = \{(a_1, \dots, a_n) \mid a_k \in A \text{ for } k = 1, \dots, n\}$$

For example, the first few finite products look as follows.

$$A^0 = \{()\} \qquad A^1 = \{(a)\}a \in A \qquad A^2 = \{(a, b) \mid a, b \in A\}$$

Note that $A^0$ consists only of the single element $()$, which is a tuple with no entries. The set $A^1$ consists only of degenerate tuples that have a single element from $A$, thus $A^1$ is isomorphic to $A$, cf. exercise 65.

?

What is the relation between $A^n$ and $A^{[n\rangle}$? See also exercise 67.

**Definition 10.1**

Let $\mathcal{L}$ be a signature with $\mathcal{L} = (\mathcal{F}, \mathcal{R}, \mathrm{ar})$. An **$\mathcal{L}$-model** $\boxed{\mathcal{M}}$ consists of the following data.

- a set $\boxed{|\mathcal{M}|}$, the **universe** of $\mathcal{M}$
- for each $f \in \mathcal{F}$ a map
$$\mathcal{M}(f) \colon |\mathcal{M}|^{\mathrm{ar}(f)} \to |\mathcal{M}|$$
- for each $P \in \mathcal{R}$ a map
$$\mathcal{M}(P) \colon |\mathcal{M}|^{\mathrm{ar}(P)} \to \mathbb{B}$$

Note that the universe in in definition 10.1 may be empty. This leads to some subtleties that are often swept under the rug in other definitions of models. We will discuss this further after definition 10.15.

Let us unfold the definition for some specific arities. Suppose our signature $\mathcal{L}$ consists of three function symbols $c$, $f$ and $g$ with arity 0, 1 and 2, respectively, and three predicate symbols $P$, $Q$, $R$ also of arity 0, 1 and 2. An $\mathcal{L}$-model $\mathcal{M}$ consists of a universe $|\mathcal{M}|$, maps and predicates as in table 10.1.

Observe that if $c \in \mathcal{F}_0$ is a constant, then $\mathcal{M}(c)$ is a map of type $|\mathcal{M}|^0 \to |\mathcal{M}|$. Since $|\mathcal{M}|^0$ has one element $()$, providing such a map corresponds to providing one element $a \in |\mathcal{M}|$:

$$\mathcal{M}(c)() = a$$

Moreover, we find for $P \in \mathcal{R}_0$ that $\mathcal{M}(P)$ either maps $()$ to 1 or 0. In other words, $P$ acts like a *propositional variable*! Finally, we call $\mathcal{M}(f)$ in table 10.1

| | Symbol s | ar($s$) | Interpretation type |
|---|---|---|---|
| Function Symbols | $c$ | 0 | $\mathcal{M}(c)\colon \lvert\mathcal{M}\rvert^0 \to \lvert\mathcal{M}\rvert$ |
| | $f$ | 1 | $\mathcal{M}(f)\colon \lvert\mathcal{M}\rvert^1 \to \lvert\mathcal{M}\rvert$ |
| | $g$ | 2 | $\mathcal{M}(g)\colon \lvert\mathcal{M}\rvert^2 \to \lvert\mathcal{M}\rvert$ |
| Predicate Symbols | $P$ | 0 | $\mathcal{M}(P)\colon \lvert\mathcal{M}\rvert^0 \to \mathbb{B}$ |
| | $Q$ | 1 | $\mathcal{M}(Q)\colon \lvert\mathcal{M}\rvert^1 \to \mathbb{B}$ |
| | $R$ | 2 | $\mathcal{M}(R)\colon \lvert\mathcal{M}\rvert^2 \to \mathbb{B}$ |

Table 10.1.: Data of a model for the indicated signature

a unary map, $\mathcal{M}(g)$ is a binary map, $\mathcal{M}(Q)$ is a predicate, or unary relation, and $\mathcal{M}(R)$ is a binary relation.

?

Let $\mathcal{L} = (\emptyset, \mathcal{R}, \mathrm{ar})$ with $\mathcal{R} = \{P\}$ and $\mathrm{ar}(P) = 0$. How many possibilities are there to make a model for $\mathcal{L}$?

In the next example, we discuss a signature and two models that occur "in the wild".

**Example 10.2**

Let $\mathcal{L}$ be given as follows.

$$\mathcal{F} = \{\underline{0}, \underline{1}, p\} \qquad \mathcal{R} = \{D, L\}$$
$$\mathrm{ar}(\underline{0}) = \mathrm{ar}(\underline{1}) = 0 \qquad \mathrm{ar}(p) = \mathrm{ar}(D) = \mathrm{ar}(L) = 2$$

The interpretation of this signature could be that $\underline{0}$ and $\underline{1}$ stand for the numbers 0 and 1, $p$ for addition (plus), $D$ for divides, and $L$ for less-than. Indeed, we can give such an interpretation, which leads to the model $\mathcal{M}_a$ in table 10.2 with universe $|\mathcal{M}_a| = \mathbb{N}$.

It is not necessary to interpret $\mathcal{L}$ as in example 10.2 and we can give different meanings to the symbols and even the universe.

**Example 10.3**

We show in this example how to use the same signature $\mathcal{L}$ as in example 10.2 to reason about formal languages, where we use the notation introduced in appendix B.5. To this end, let $A$ be a set of letters, called in

| Symbol | Interpretation type | Interpretation |
|---|---|---|
| $\underline{0}$ | $\mathcal{M}_a(\underline{0})\colon \mathbb{N}^0 \to \mathbb{N}$ | $\mathcal{M}_a(\underline{0})() = 0$ |
| $\underline{1}$ | $\mathcal{M}_a(\underline{1})\colon \mathbb{N}^0 \to \mathbb{N}$ | $\mathcal{M}_a(\underline{1})() = 1$ |
| $p$ | $\mathcal{M}_a(p)\colon \mathbb{N}^2 \to \mathbb{N}$ | $\mathcal{M}_a(p)(n,m) = n + m$ |
| $D$ | $\mathcal{M}_a(D)\colon \mathbb{N}^2 \to \mathbb{B}$ | $\mathcal{M}_a(D)(n,m) = \begin{cases} 1, n \text{ divides } m \\ 0, \text{otherwise} \end{cases}$ |
| $L$ | $\mathcal{M}_a(L)\colon \mathbb{N}^2 \to \mathbb{B}$ | $\mathcal{M}_a(L)(n,m) = \begin{cases} 1, n \leq m \\ 0, \text{otherwise} \end{cases}$ |

Table 10.2.: Arithmetic model $\mathcal{M}_a$ over universe $\mathbb{N}$ from example 10.2

this context an alphabet, and denote by $A^*$ the set of finite words over $A$. Languages over a are then sets of words, that is, elements of the power-set $\mathcal{P}(A^*)$. The function symbols $\underline{0}$, $\underline{1}$ and $p$ correspond to, respectively, the empty language, the language containing only the empty and union of languages. The predicate symbols, on the other hand, can be interpreted as language equality and language inclusion. All of this is summed up in table 10.3. Note that that $\underline{0}$ and $p$ behave similarly to 0 and addition under this interpretation. We will discuss this later in more depth.

| | Interpretation type | Interpretation |
|---|---|---|
| $\underline{0}$ | $\mathcal{M}_l(\underline{0})\colon \mathcal{P}(A^*)^0 \to \mathcal{P}(A^*)$ | $\mathcal{M}_l(\underline{0})() = \emptyset$ |
| $\underline{1}$ | $\mathcal{M}_l(\underline{1})\colon \mathcal{P}(A^*)^0 \to \mathcal{P}(A^*)$ | $\mathcal{M}_l(\underline{1})() = \{\varepsilon\}$ |
| $p$ | $\mathcal{M}_l(p)\colon \mathcal{P}(A^*)^2 \to \mathcal{P}(A^*)$ | $\mathcal{M}_l(p)(L_1, L_2) = L_1 \cup L_2$ |
| $D$ | $\mathcal{M}_l(D)\colon \mathcal{P}(A^*)^2 \to \mathbb{B}$ | $\mathcal{M}_l(D)(L_1, L_2) = \begin{cases} 1, & L_1 = L_2 \cup L \\ & \text{for some } L \\ 0, & \text{otherwise} \end{cases}$ |
| $L$ | $\mathcal{M}_l(L)\colon \mathcal{P}(A^*)^2 \to \mathbb{B}$ | $\mathcal{M}_l(L)(L_1, L_2) = \begin{cases} 1, & L_1 \subseteq L_2 \\ 0, & \text{otherwise} \end{cases}$ |

Table 10.3.: Language model $\mathcal{M}_l$ for $\mathcal{L}$ over universe $\mathcal{P}(A^*)$ from example 10.3

Clearly, the two models $\mathcal{M}_a$ and $\mathcal{M}_l$ in example 10.2 and example 10.3 are completely different interpretations of $\mathcal{L}$. This illustrates the power of first-order logic: one language we can reason about an enormous variety of different structures. However, as we will see in chapter 11, this power can also become a weakness of first-order logic.

The final example illustrates this once more by presenting a language for graphs.

**Example 10.4: Graphs**

Let $\mathcal{L}_g = (\mathcal{F}, \mathcal{R}, \mathrm{ar})$ be given by $\mathcal{F} = \{s, t\}$, $\mathcal{R} = \{N, E\}$ and $ar(s) = ar(t) = \mathrm{ar}(N) = \mathrm{ar}(E) = 1$. The idea is that $N$ classifies nodes, $E$ edges, and $s$ and $t$ gives the source and target of an edge. Indeed, given a graph $G$ with a set $G_0$ of nodes and a set $G_1$ of edges, together with maps $d_1, d_2 \colon G_1 \to G_0$, we can build a model $\mathcal{M}_G$ as follows. We define $|\mathcal{M}_G| = G_0 \cup G_1 \cup \{*\}$, where assume that $G_0$, $G_1$ and $\{*\}$ are disjoint sets. The new element $*$ has to be added because we can otherwise not define the source and target maps for a graph, unless it is empty. Over the graph, we can interpret the symbols as follows.

$$\mathcal{M}_G(s)(x) = \begin{cases} d_1(x), & x \in G_1 \\ *, & \text{otherwise} \end{cases} \qquad \mathcal{M}_G(t)(x) = \begin{cases} d_2(x), & x \in G_1 \\ *, & \text{otherwise} \end{cases}$$

$$\mathcal{M}_G(N)(x) = \begin{cases} 1, & x \in G_0 \\ 0, & \text{otherwise} \end{cases} \qquad \mathcal{M}_G(E)(x) = \begin{cases} 1, & x \in G_1 \\ 0, & \text{otherwise} \end{cases}$$

For instance, if $G$ is the following graph with $G_0 = \{a, b, c\}$, $G_1 = \{e, f, g, h\}$, and $d_1(e) = d_1(f) = d_2(h) = a$, $d_2(e) = d_2(f) = d_1(g) = b$ and $d_2(g) = d_1(h) = c$.

$$\begin{tikzcd} a \arrow[r, bend left, "e"] \arrow[r, bend right, "f"'] & b \arrow[dl, bend left, "g"] \\ c \arrow[u, "h"] & \end{tikzcd}$$

Then $|\mathcal{M}_G| = \{a, b, c, e, f, g, h, *\}$, $\mathcal{M}_G(N) = \{a, b, c\}$, $\mathcal{M}_G(E) = \{e, f, g, h\}$, $\mathcal{M}_G(s)(e) = \mathcal{M}_G(s)(f) = \mathcal{M}_G(t)(h) = a$, $\mathcal{M}_G(s)(a) = *$ etc. Even though we can encode graphs, the example also shows that first-order logic lacks types that make are commonly used in mathematical and computer science practice, cf. appendix B.1. Some of this can be recovered through bounded quantification.

**Example 10.5: Databases as models**

Recall from example 4.11 the signature that we used to devise a database scheme for exam enrolments. A database for this scheme is a first-order model for the corresponding signature. For the moment, we will not ask the

model to fulfil the constraints and come back to that in example 10.22. For the universe of such a model $\mathcal{M}$, we may take $|\mathcal{M}| = \mathbb{N} \cup (\text{ASCII})^* \cup \{\text{err}\}$, where $\mathbb{N}$ is used to model numerical keys, $(\text{ASCII})^*$ is the set of ASCII strings and err will be used to handle errors. The zero constant and successor map are interpreted as follows, where the successor can only be applied to numbers and otherwise signals an error.

$$\mathcal{M}(\underline{0})() = 0 \quad \text{and} \quad \mathcal{M}(\text{succ})(x) = \begin{cases} x+1, & x \in \mathbb{N} \\ \text{err}, & \text{otherwise} \end{cases}$$

Any database is given by extending $\mathcal{M}$ to the predicate symbols. For instance, for Jane to be the only student in the database and having primary key (student number) 5, we would define

$$\mathcal{M}(\text{dStudent})(x, y) = \begin{cases} 1, & x = 5 \text{ and } y = \text{"Jane"} \\ 0, & \text{otherwise} \end{cases} .$$

## 10.2. Valuations and the Interpretion of FOL

Recall from definition 5.6 that we write $|\Delta|$ for the underlying set of variables in the list $\Delta$. We adapt this notation in general.

**Notation 10.6**

For any list $\Delta$ or $\Gamma$, we write $|\Delta|$ and $|\Gamma|$ for the underlying sets given by $|\Delta| = \{x \mid x : \Delta\}$ and $|\Gamma| = \{\varphi \mid \varphi : \Gamma\}$.

Just as propositional variables in propositional logic, the object variables in first-order formulas have no intrinsic meaning. Instead, we have to give meaning to them externally through valuations, which assign to each variable an element of a given universe.

**Definition 10.7**

Given a signature $\mathcal{L}$, an $\mathcal{L}$-model $\mathcal{M}$, and $D \subseteq \text{Var}$, a ***D*-valuation** in $\mathcal{M}$ is a map $v\colon D \to |\mathcal{M}|$. We let $\boxed{\text{Val}_{\mathcal{M}}(D)}$ be the set of $D$-valuation, that is,

$$\text{Val}_{\mathcal{M}}(D) = [D, |\mathcal{M}|] .$$

If $\mathcal{M}$ is clear from the context, we just write $\boxed{\text{Val}(D)}$.

The set $D$ is the semantic analogue of the variable context $\Delta$ that appears in sequents (definition 5.6), while $D$-valuations are the semantic analogue of closing substitutions (definition 7.14). One can remember "$D$" as **D**omain or **D**elta.

Valuations provide an interpretation of variables and models of function symbols, which yield together the semantics of terms.

**Definition 10.8**

Let $\mathcal{M}$ be an $\mathcal{L}$-model, $D \subseteq \mathrm{Var}$ and $t$ be an $\mathcal{L}$-term with $\mathrm{var}(t) \subseteq D$. The **(first-order) term semantics** of $t$ in $\mathcal{M}$ relative to $D$ is the map $\boxed{[\![t]\!]}$ of type $[\![t]\!] \colon \mathrm{Val}_{\mathcal{M}}(D) \to |\mathcal{M}|$ given by the following equations.

$$\begin{aligned}
[\![x]\!](v) &= v(x) \\
[\![c]\!](v) &= \mathcal{M}(c)() \\
[\![f(t_1, \dots, t_n)]\!](v) &= \mathcal{M}(f)([\![t_1]\!](v), \dots, [\![t_n]\!](v))
\end{aligned}$$

If $\mathcal{M}$ or $D$ are not clear from the context, then we write $\boxed{[\![t]\!]_{\mathcal{M}}}$ or $[\![t]\!]^{D}_{\mathcal{M}}$ instead of $[\![t]\!]$.

Let us demonstrate valuations and the term semantics for the models in examples 10.2 and 10.3.

**Example 10.9**

We begin with the arithmetic model $\mathcal{M}_a$ from example 10.2. Let $x \in \mathrm{Var}$ be some variable and define a $\{x\}$-valuation by $v(x) = 5$. Under this valuation, the term $p(\underline{0}, p(x, \underline{1}))$ has the following semantics.

$$\begin{aligned}
[\![p(\underline{0}, p(x, \underline{1}))]\!](v) &= [\![\underline{0}]\!](v) + [\![p(x, \underline{1})]\!](v) \\
&= [\![\underline{0}]\!](v) + ([\![x]\!](v) + [\![\underline{1}]\!](v)) \\
&= \mathcal{M}_a(\underline{0})() + (v(x) + \mathcal{M}_a(\underline{1})()) \\
&= 0 + (5 + 1) \\
&= 6
\end{aligned}$$

The next example provides semantics for the same term in the language model.

**Example 10.10**

Let $x \in \mathrm{Var}$ be some variable and define $v \in \mathrm{Val}_{\mathcal{M}_l}(\{x\})$ by $v(x) = A$.

With this valuation, the term $p(\underline{0}, p(x, \underline{1}))$ has the following semantics.

$$\begin{aligned}
[\![p(\underline{0}, p(x, \underline{1}))]\!](v) &= [\![\underline{0}]\!](v) \cup [\![p(x, \underline{1})]\!](v) \\
&= [\![\underline{0}]\!](v) \cup ([\![x]\!]_v \cup [\![\underline{1}]\!](v)) \\
&= \mathcal{M}_l(\underline{0})() \cup (v(x) \cup \mathcal{M}_l(\underline{1})()) \\
&= \emptyset \cup (A \cup \{\varepsilon\}) \\
&= \{w \in A^* \mid \text{length}(w) \leq 1\}
\end{aligned}$$

Here, $\text{length}(w)$ is the length of the word $w$.

The last example of term semantics is for the graph model from example 10.4.

**Example 10.11**

Let $D = \{x\}$ for some variable $x$ and define $v \in \text{Val}_{\mathcal{M}_g}(D)$ by $v(x) = e$ for the graph from example 10.4. Then we have

$$[\![s(x)]\!](v) = \mathcal{M}_g(s)(v(x)) = d_1(e) = a$$

and

$$\begin{aligned}
[\![s(s(x))]\!](v) &= \mathcal{M}_g(s)([\![s(x)]\!](v)) \\
&= \mathcal{M}_g(s)(\mathcal{M}_g(s)(v(x))) \\
&= \mathcal{M}_g(s)(\mathcal{M}_g(s)(e)) \\
&= \mathcal{M}_g(s)(d_1(e)) \\
&= \mathcal{M}_g(s)(a) \\
&= *\,.
\end{aligned}$$

Valuations are central to the semantics of terms and first-order logic, but can also be used to model the result of database queries.

**Example 10.12: Potential query results as valuations**

Recall from examples 4.22 and 8.9 that queries to databases can be formulated as first-order formulas with free variables. The result for such queries is given by valuations. For instance, consider the formula $\varphi_{\text{st}}$ defined by

$$\varphi_{\text{st}} = \exists s.\, \text{dStudent}(s, n)\,,$$

which is intended to retrieve all student names from a database. We let $D = \text{fv}(\varphi_{\text{st}}) = \{n\}$. This means valuations in $\text{Val}(D)$ for a database $\mathcal{M}$ as in example 10.5 are potential results for the query $\varphi$. Examples are $v_1$ and

$v_2$ given by $v_1(n) =$ "Jane" and $v_2(n) = 10$. Clearly, the first would be a correct answer to the query, while the second is not. In order to state this, we need the semantics of formulas.

Now that we have an interpretation of terms, we can further extend it to an interpretation on formulas. Recall that we needed to update substitutions to eliminate universal quantifiers and introduce existential quantifiers. A similar operation on valuations is necessary in the semantics of FOL formulas.

**Definition 10.13**

Let $v$ be a $D$-valuation in $\mathcal{M}$, $x \in \mathrm{Var}$ and $a \in |\mathcal{M}|$. We define the **update** of $v$ to a $(D \cup \{x\})$-valuation $\boxed{v[x \mapsto a]}$ by the following equation.

$$(v[x \mapsto a])(y) = \begin{cases} a, & y = x \\ v(y), & y \neq x \end{cases}$$

Let us briefly illustrate the update of valuations.

**Example 10.14**

We start with the valuation $\varepsilon \in \mathrm{Val}_{\mathcal{M}_a}(\emptyset)$. Given variables $x, z \in \mathrm{Var}$ with $x \neq z$, we have

$$\begin{aligned} &\varepsilon[x \mapsto 1] && : \{x\} \to \mathbb{N} \\ &\varepsilon[x \mapsto 1][z \mapsto 2] && : \{x, z\} \to \mathbb{N} \\ &\varepsilon[x \mapsto 1][z \mapsto 2][x \mapsto 3] && : \{x, z\} \to \mathbb{N} \end{aligned}$$

with

$$\begin{aligned} &(\varepsilon[x \mapsto 1])(x) && = 1 \\ &(\varepsilon[x \mapsto 1][z \mapsto 2])(x) && = 1 \\ &(\varepsilon[x \mapsto 1][z \mapsto 2])(z) && = 2 \\ &(\varepsilon[x \mapsto 1][z \mapsto 2][x \mapsto 3])(x) && = 3 \\ &(\varepsilon[x \mapsto 1][z \mapsto 2][x \mapsto 3])(z) && = 2 \end{aligned}$$

The update operation of valuations allows us now to give semantics to quantifiers and thereby to all formulas. In this definition we use that

$$\min \emptyset = 1 \quad \text{and} \quad \max \emptyset = 0$$

that correspond to the understanding that 1 is the largest element that is smaller than any element in the empty set, in which there is none.

**Definition 10.15**

Let $\mathcal{L}$ be a signature, $\mathcal{M}$ an $\mathcal{L}$-model, $D \subseteq \mathrm{Var}$, and $\varphi$ a $\mathcal{L}^{=}$-formula with $\mathrm{fv}(\varphi) \subseteq D$. We define the **first-order formula semantics** as a map $\boxed{[\![\varphi]\!]}$ with type $[\![\varphi]\!] \colon \mathrm{Val}(D) \to \mathbb{B}$ by iteration on $\varphi$ as follows.

$$
\begin{aligned}
[\![\bot]\!](v) &= 0 \\
[\![P(t_1, \dots, t_n)]\!](v) &= \mathcal{M}(P)([\![t_1]\!](v), \dots, [\![t_n]\!](v)) \\
[\![s \doteq t]\!](v)^{=} &= \begin{cases} 1, & [\![s]\!](v) = [\![t]\!](v) \\ 0, & \text{otherwise} \end{cases} \\
[\![\varphi \wedge \psi]\!](v) &= \min\{[\![\varphi]\!](v), [\![\psi]\!](v)\} \\
[\![\varphi \vee \psi]\!](v) &= \max\{[\![\varphi]\!](v), [\![\psi]\!](v)\} \\
[\![\varphi \to \psi]\!](v) &= [\![\varphi]\!](v) \implies [\![\psi]\!](v) \\
[\![\forall x.\, \varphi]\!](v) &= \min\{[\![\varphi]\!](v[x \mapsto a]) \mid a \in |\mathcal{M}|\} \\
[\![\exists x.\, \varphi]\!](v) &= \max\{[\![\varphi]\!](v[x \mapsto a]) \mid a \in |\mathcal{M}|\}
\end{aligned}
$$

Here, we used the semantic implication from definition 3.6. As in definition 10.8, we write $\boxed{[\![\varphi]\!]_{\mathcal{M}}}$ or $\boxed{[\![\varphi]\!]^{D}_{\mathcal{M}}}$ instead of $[\![\varphi]\!]$ if $\mathcal{M}$ or $D$ are not clear from the context.

First of all, note that definition 10.15 is correct for quantifiers because renaming of variables does not affect the result:

$$[\![\forall x.\, \varphi]\!](v) = [\![\forall y.\, \varphi\,[x := y]]\!](v) \text{ for all } y \# \varphi$$

because

$$\min\{[\![\varphi]\!](v[x \mapsto a]) \mid a \in |\mathcal{M}|\} = \min\{[\![\varphi\,[x := y]]\!](v[y \mapsto a]) \mid a \in |\mathcal{M}|\}$$

Thus, the definition of semantics is compatible with the equality of formulas that we have required to hold in chapter 5.

Recall that we allow the universe of a model to be empty. This creates some subtleties. For instance, for the propositional tautology $\top = \neg\bot$, see definition 1.3, the formula $\exists x.\, \top$ is *not* a tautology, unless the signature has constants! This is because in an empty model, we have for any formula $\varphi$ that

$$[\![\exists x.\, \varphi]\!](v) = \max \emptyset = 0\,. \tag{10.16}$$

The possibilities that arise in the semantics of quantifiers are summed up in table 10.4 together with the corresponding quantification. Note, however, that

| $\{[\![\varphi]\!](v[x \mapsto a]) \mid a \in \lvert\mathcal{M}\rvert\}$ | $[\![\forall x.\, \varphi]\!](v)$ | $[\![\exists x.\, \varphi]\!](v)$ | Interpretation |
|---|---|---|---|
| $\emptyset$ | 1 | 0 | the model is empty |
| $\{0\}$ | 0 | 0 | $\varphi$ holds for no $a$ |
| $\{1\}$ | 1 | 1 | $\varphi$ holds for all $a$ |
| $\{0, 1\}$ | 0 | 1 | $\varphi$ holds for some $a$ |

Table 10.4.: Possibilities for quantifier semantics

the first row does not appear if there are constants in the signature.

**Example 10.17: Query results as satisfying valuations**

In example 10.12, we viewed potential results for database queries as valuations. However, we saw that these are not necessarily actual results because they may not satisfy the formula. Using definition 10.20, we can now state that $v_1$ from example 10.12 is a result for the query, while $v_2$ is not because we have $[\![\varphi_{\text{st}}]\!](v_1) = 1$ but $[\![\varphi_{\text{st}}]\!](v_2) = 0$ as follows.

$$
\begin{aligned}
&[\![\varphi_{\text{st}}]\!](v_1) \\
&= \max\{[\![\text{dStudent}(s, n)]\!](v_1[s \mapsto a]) \mid a \in \lvert\mathcal{M}\rvert\} && \text{def. semantics} \\
&\geq [\![\text{dStudent}(s, n)]\!](v_1[s \mapsto 5]) && \text{max is upper bound} \\
&= \mathcal{M}(\text{dStudent})([\![s]\!](v_1[s \mapsto 5]), [\![n]\!](v_1[s \mapsto 5])) && \text{def. semantics} \\
&= \mathcal{M}(\text{dStudent})(\text{"Jane"}, 5) && \text{def. term semantics and update} \\
&= 1 && \text{def. } \mathcal{M} \text{ in example 10.5}
\end{aligned}
$$

Therefore, $[\![\varphi_{\text{st}}]\!](v_1) = 1$ and $\varphi_{\text{st}}$ is validated by $v_1$ in $\mathcal{M}$. For $v_2$, we have

$$
\begin{aligned}
&[\![\varphi_{\text{st}}]\!](v_2) \\
&= \max\{[\![\text{dStudent}(s, n)]\!](v_2[s \mapsto a]) \mid a \in \lvert\mathcal{M}\rvert\} && \text{def. semantics} \\
&= \max\{0\} && \mathcal{M}(\text{dStudent})(a, 10) = 0 \text{ for all } a \\
&= 0
\end{aligned}
$$

and thus $v_2$ is not a result for the query, as it does not satisfy $\varphi_{\text{st}}$.

Let us now calculate the truth value of some formulas.

**Example 10.18**

Recall the signature $\mathcal{L}$ and arithmetic model $\mathcal{M}_a$ from example 10.2. Consider the formula

$$\forall x.\, p(x, \underline{0}) \doteq x$$

that expresses under the arithmetical interpretation that $x+0 = x$. In other words, the formula should be true for any valuation over $\mathcal{M}_a$. Indeed, given a natural number $n$, we have

$$[\![p(x, \underline{0})]\!](\varepsilon[x \mapsto n]) = 0 + n = n$$

and thus

$$\begin{aligned}
[\![p(x, \underline{0}) \doteq x]\!](\varepsilon[x \mapsto n]) &= \begin{cases} 1, & [\![p(x, \underline{0})]\!](\varepsilon[x \mapsto n]) = [\![x]\!](\varepsilon[x \mapsto n]) \\ 0, & \text{otherwise} \end{cases} \\
&= \begin{cases} 1, & n = n \\ 0, & \text{otherwise} \end{cases} \\
&= 1.
\end{aligned}$$

As expected, this gives us

$$\begin{aligned}
[\![\forall x.\, p(x, \underline{0}) \doteq x]\!](\varepsilon) &= \min\{[\![p(x, \underline{0}) \doteq x]\!](\varepsilon[x \mapsto n]) \mid n \in \mathbb{N}\} \\
&= \min\{1\} \\
&= 1
\end{aligned}$$

and thus $\forall x.\, p(x, \underline{0}) \doteq x$ is true in $\mathcal{M}_a$.
As a second example, consider the formula

$$\forall x.\, \exists y.\, L(x, y) \wedge \neg(x \doteq y)$$

that states that for every number there is a strictly larger number. This formula is true in the arithmetic model because for every $n \in \mathbb{N}$ we have that $n < n + 1$. In other words, for all $n \in \mathbb{N}$

$$[\![L(x, y) \wedge \neg(x \doteq y)]\!](\varepsilon[x \mapsto n][y \mapsto n + 1]) = 1.$$

Thus,

$$\begin{aligned}
&[\![\exists y.\, L(x, y) \wedge \neg(x \doteq y)]\!](\varepsilon[x \mapsto n]) \\
&= \max\{[\![L(x, y) \wedge \neg(x \doteq y)]\!](\varepsilon[x \mapsto n][y \mapsto m]) \mid m \in \mathbb{N}\} \\
&= 1 && \text{because } n + 1 \in \mathbb{N}
\end{aligned}$$

and

$$\begin{aligned}
&[\![\forall x.\, \exists y.\, L(x,y) \land \neg(x \doteq y)]\!](\varepsilon) \\
&= \min\{[\![\exists y.\, L(x,y) \land \neg(x \doteq y)]\!](\varepsilon[x \mapsto n]) \mid n \in \mathbb{N}\} \\
&= \min\{1\} \\
&= 1
\end{aligned}$$

?

Is the formula $\forall x.\, \exists y.\, L(x,y) \land \neg(x \doteq y)$ from example 10.18 true in the language model $\mathcal{M}_l$?

**Example 10.19**

Recall the graph model from example 10.4, where we observed that the lack of typing makes first-order logic somewhat tedious. Bounded quantification allows us to recover some of this. For instance, let us interpret the formula $\forall x : E.\, N(s(x))$, which intuitively states that the source of an edge is a node. Indeed, in the graph model from example 10.4, we have

$$\begin{aligned}
&[\![\forall x : E.\, N(s(x))]\!](\varepsilon) \\
&= \min\{[\![E(x) \to N(s(x))]\!](\varepsilon[x \mapsto u]) \mid u \in |\mathcal{M}_G|\} \\
&= \min\{[\![E(x)]\!](\varepsilon[x \mapsto u]) \Longrightarrow [\![N(s(x))]\!](\varepsilon[x \mapsto u]) \mid u \in |\mathcal{M}_G|\} \\
&= \min\{[\![N(s(x))]\!](\varepsilon[x \mapsto u]) \mid u \in |\mathcal{M}_G|, [\![E(x)]\!](\varepsilon[x \mapsto u]) = 1\} \\
&= \min\{[\![N(s(x))]\!](\varepsilon[x \mapsto u]) \mid u \in |\mathcal{M}_G|, [\![x]\!](\varepsilon[x \mapsto u]) \in G_1\} \\
&= \min\{[\![N(s(x))]\!](\varepsilon[x \mapsto u]) \mid u \in |\mathcal{M}_G|, u \in G_1\} \\
&= \min\{[\![N(y)]\!](\varepsilon[y \mapsto d_1(u)]) \mid u \in |\mathcal{M}_G|, u \in G_1\} \\
&= 1 \qquad\qquad \text{because } d_1(u) \in G_0 = \mathcal{M}_G(N) \text{ for } u \in G_1
\end{aligned}$$

Thus, the semantics of the formula in the graph model is always 1.

## 10.3. Entailment and Satisfiability for FOL

In section 10.2, we have defined the functional interpretation of first-order formulas in terms of the mapping $[\![-]\!]$. As for propositional logic, we can also give a relational interpretation of formulas. This allows us to easily define satisfiability and tautologies.

**Definition 10.20**

Let $D \subseteq \mathrm{Var}$, $\varphi \in \mathrm{Form}^{=}(\mathcal{L})$ and $G$ a set of $\mathcal{L}$-formulas, such that all free variables in $\varphi$ and $G$ are contained in $D$: $\mathrm{fv}(\varphi) \cup \mathrm{fv}(G) \subseteq D$. We define **relative entailment** $\boxed{D \mid G \vDash_{\mathcal{M}} \varphi}$ (read "$G$ *entails* $\varphi$ in $\mathcal{M}$") and **entailment** $\boxed{D \mid G \vDash \varphi}$ (read "$G$ *entails* $\varphi$") relations as follows, where $[\![G]\!](v) = \min\{[\![\psi]\!](v) \mid \psi \in G\}$.

$$\begin{aligned} &D \mid G \vDash_{\mathcal{M}} \varphi && \text{if } [\![G]\!](v) \leq [\![\varphi]\!](v) \text{ for all } v \in \mathrm{Val}_{\mathcal{M}}(D) \\ &D \mid G \vDash \varphi && \text{if } D \mid G \vDash_{\mathcal{M}} \varphi \text{ for all models } \mathcal{M} \end{aligned}$$

If $D$ is empty, we write $G \vDash \varphi$ instead of $\emptyset \mid G \vDash \varphi$. We say that $\mathcal{M}$ and $v \in \mathrm{Val}_{\mathcal{M}}(D)$ **satisfy** $\varphi$ if $[\![\varphi]\!](v) = 1$, and that $\varphi$ is **satisfiable**, if there is a model $\mathcal{M}$ and an $\mathcal{M}$-valuation $v$ that satisfy $\varphi$. The formula $\varphi$ is a **tautology**, written $\vDash \varphi$, if $\emptyset \vDash \varphi$. Finally, we say that $\varphi$ is **valid** in $\mathcal{M}$, if $\emptyset \vDash_{\mathcal{M}} \varphi$.

Similar to definition 10.7, the set $G$ is the semantic analogue of a list of assumptions $\Gamma$, which explains that the condition $\mathrm{fv}(\varphi) \cup \mathrm{fv}(G) \subseteq D$ resembles that for first-order sequents in definition 5.6. As a mnemonic, $G$ can be remembered as **G**iven or **G**amma.

Let us provide a few examples of definition 10.20, starting with validity.

**Example 10.21**

The formula in example 10.19 is valid in the graph model $\mathcal{M}_G$, as all valuations in the model satisfy the formula.

The entailment relation can also be used to, for example, test database constraints.

**Example 10.22: Satisfaction of database constraints**

In example 10.17, we saw how to use the semantics to check that a valuation is a valid result for a query. As discussed in examples 4.18 and 8.9, database constraints can also be formulated databases as formulas. We are now in the position to capture these. In general, a signature $\mathcal{L}$ corresponds to a database scheme and constraints can be seen as sets $G \subseteq \mathrm{Form}^{=}(\mathcal{L})$. A database is then an $\mathcal{L}$-model $\mathcal{M}$ with $[\![G]\!]_{\mathcal{M}}(\varepsilon) = 1$. In order to test if the constraints are not contradictory with a query $\varphi$, one needs to find some database $\mathcal{M}$ with $\mathrm{fv}(\varphi) \mid G \vDash_{\mathcal{M}} \varphi$. This corresponds to setting up a database with typical values that fulfils the constraints and then evaluating the query $\varphi$.

Next, we look at some examples in the arithmetic model.

**Example 10.23**

Let $\varphi_e$ be the formula $\exists y.\, x \doteq p(y, y)$ ("$x$ is even"), $D = \{x\}$, and let $v_1, v_2 \in \mathrm{Val}(\{x\})$ be given by $v_1(z) = 2$ and $v_2(z) = 3$. Then $\mathcal{M}_a$ and $v_1$ satisfy $\varphi_e$, but $\mathcal{M}_a$ and $v_2$ do not. Therefore $\varphi_e$ is satisfiable but not validated by $\mathcal{M}_a$.
Let $G = \{\varphi_e\}$ and $\varphi_o = \exists y.\, x \doteq p(p(y, y), \underline{1})$. Then

$$D \mid G \vDash_{\mathcal{M}_a} \varphi_o\,[x := p(x, \underline{1})]$$

holds: If $[\![G]\!](v) = 1$, then $v(x)$ must be an even number. Thus, $v(x) + 1$ is an odd number and $[\![\varphi_o\,[x := p(x, \underline{1})]]\!](v) = 1$. Formally,

$$\begin{aligned}
&[\![G]\!](v) = 1\\
&\text{iff } [\![\varphi_e]\!](v) = 1\\
&\text{iff } \min\{[\![x \doteq p(y, y)]\!](v[y \mapsto n]) \mid n \in \mathbb{N}\} = 1\\
&\text{iff } [\![x \doteq p(y, y)]\!](v[y \mapsto n]) = 1 \text{ for some } n \in \mathbb{N} \qquad \text{(see table 10.4)}\\
&\text{iff } v(x) = n + n \text{ for some } n \in \mathbb{N}\\
&\text{iff } v(x) \text{ even}
\end{aligned}$$

and under this assumption

$$\begin{aligned}
[\![\varphi_o[x := p(x, \underline{1})]]\!](v) &= [\![(\exists y.\, x \doteq p(p(y, y), \underline{1}))[x := p(x, \underline{1})]]\!](v)\\
&= [\![\exists y.\, p(x, \underline{1}) \doteq p(p(y, y), \underline{1})]\!](v)\\
&= \max\{[\![p(x, \underline{1}) \doteq p(p(y, y), \underline{1})]\!](v[y \mapsto n]) \mid n \in \mathbb{N}\}\\
&\geq [\![p(x, \underline{1}) \doteq p(p(y, y), \underline{1})]\!](v[y \mapsto v(x)/2])\\
&= 1,
\end{aligned}$$

where we use the identity $v(x) + 1 = (v(x)/2 + v(x)/2) + 1$ in the last line.

To show that a formula is not a tautology, it can be convenient or even necessary to choose the interpretation of function or predicate symbols appropriately, as the following example shows.

**Example 10.24**

We have seen in example 10.18 that $[\![\forall x.\, p(x, \underline{0}) \doteq x]\!]_{\mathcal{M}_a}(v) = 1$ for any valuation $v$ and therefore $\mathcal{M}_a$ validates $\forall x.\, p(x, \underline{0}) \doteq x$. However, this formula is not a tautology because it is not satisfied by the model $\mathcal{M}$ with

$|\mathcal{M}| = \mathbb{N}$, $\mathcal{M}(\underline{0}) = 1$ and otherwise the same interpretation as in the arithmetic model $\mathcal{M}_a$.

In chapter 5, we have seen formulas that were derivable in **ND**$_1$, which are a good source of tautologies, see also section 10.4 below.

**Example 10.25**

We claim that the formula $(\forall x.\, \varphi) \to \neg\exists x.\, \neg\varphi$ is a tautology for any formula $\varphi$. Indeed, let $\mathcal{M}$ be a model for the signature $\mathcal{L}$, over which $\varphi$ is a formula. We obtain from table 10.4 that

$$\begin{aligned}
[\![\neg\exists x.\, \neg\varphi]\!](\varepsilon) = 1 \text{ iff } & [\![\exists x.\, \neg\varphi]\!](\varepsilon) = 0 \\
\text{iff } & [\![\neg\varphi]\!](\varepsilon[x \mapsto a]) = 0 \text{ for all } a \in |\mathcal{M}| \\
\text{iff } & [\![\varphi]\!](\varepsilon[x \mapsto a]) = 1 \text{ for all } a \in |\mathcal{M}| \\
\text{iff } & [\![\forall x.\, \varphi]\!](\varepsilon) = 1
\end{aligned}$$

This implies that

$$[\![\forall x.\, \varphi]\!](\varepsilon) \leq [\![\neg\exists x.\, \neg\varphi]\!](\varepsilon)$$

and thus

$$\begin{aligned}
& [\![(\forall x.\, \varphi) \to \neg\exists x.\, \neg\varphi]\!](\varepsilon) \\
& = [\![(\forall x.\, \varphi)]\!](\varepsilon) \implies [\![\neg\exists x.\, \neg\varphi]\!](\varepsilon) \\
& = 1.
\end{aligned}$$

Hence, $(\forall x.\, \varphi) \to \neg\exists x.\, \neg\varphi$ is a tautology.

The following theorem provides a convenient description of entailment for the propositional connectives.

**Theorem 10.26**

The following holds for entailment.

$$\begin{aligned}
D \mid G \vDash \varphi_1 \wedge \varphi_2 \quad & \text{iff } D \mid G \vDash \varphi_1 \text{ and } D \mid G \vDash \varphi_2 \\
D \mid G \vDash \varphi_1 \vee \varphi_2 \quad & \text{iff } D \mid G \vDash \varphi_1 \text{ or } D \mid G \vDash \varphi_2 \\
D \mid G \vDash \varphi_1 \to \varphi_2 \quad & \text{iff } D \mid G \vDash \varphi_1 \text{ implies } D \mid G \vDash \varphi_2 \\
D \mid G \vDash \varphi \quad & \text{iff not } D \mid G \vDash \neg\varphi
\end{aligned}$$

## 10.4. Soundness of Natural Deduction for FOL

We come now to the second initial question: Are all derivable formulas tautologies? Using definition 10.20, we can state this question precisely by asking: if there is a proof for the sequent $\Gamma \vdash \varphi$ in one of the systems from chapter 5, is $\varphi$ then entailed semantically by $|\Gamma|$? The answer to this question is the main result of this chapter.

**Theorem 10.27: Soundness**

For every $\mathcal{L}$-formula $\varphi$, list of $\mathcal{L}$-formulas $\Gamma$ and list of variables $\Delta$, we let $D = |\Delta|$ and $G = |\Gamma|$ and then have

1. if $\Delta \mid \Gamma \vdash \varphi$ is derivable in $\mathbf{ND}_1^=$, then $D \mid G \vDash \varphi$; and
2. if $\Delta \mid \Gamma \vdash \varphi$ is derivable in $\mathbf{cND}_1^=$, then $D \mid G \vDash \varphi$.

Instantiating theorem 10.27 with the empty list of assumptions, we obtain the following corollary.

**Corollary 10.28**

For every formula $\varphi$ over a signature $\mathcal{L}$ the following holds.

1. If $\vdash \varphi$ is derivable in $\mathbf{ND}_1^=$, then $\varphi$ is a tautology.
2. If $\vdash \varphi$ is derivable in $\mathbf{cND}_1^=$, then $\varphi$ is a tautology.

We also obtain semantics for formulas without equality, by composing with the embedding $e$ from theorem 8.24.

**Corollary 10.29**

For every formula $\varphi$ and list of formulas $\Gamma$ over a signature $\mathcal{L}$

1. if $|\Gamma| \vdash \varphi$ is derivable in $\mathbf{ND}_1$, then $\Gamma \vDash e(\varphi)$.
2. if $|\Gamma| \vdash \varphi$ is derivable in $\mathbf{cND}_1$, then $\Gamma \vDash e(\varphi)$.

*Proof.* By theorem 8.24, we get that $\Gamma \vdash \varphi$ is derivable in $\mathbf{ND}_1$ if and only if $\Gamma \vdash e(\varphi)$ is derivable in $\mathbf{ND}_1^=$. Thus, $\Gamma \vDash e(\varphi)$ by theorem 10.27. □

The most important use of theorem 10.27 is to show that a formula is not provable.

**Example 10.30**

Recall from example 10.24 that $\forall x.\, p(x, \underline{0}) \doteq x$ is not a tautology. Thus, this formula cannot be provable in $\mathbf{ND}_1$, as we would obtain a contradiction with corollary 10.28.

The proof of theorem 10.27 requires some interesting results that essentially show that substitutions are the syntactic counterpart of valuations. This is proved below in lemma 10.31 for terms and in lemma 10.32 for formulas.

**Lemma 10.31**

For all substitutions $\sigma$, terms $t$, valuations $v$ and variables $x$, where $x$ is fresh for $\sigma$, the following equation holds.

$$[\![t\,\sigma]\!](v) = [\![t\,(\sigma[x := x])]\!]_{v[x \mapsto [\![\sigma(x)]\!](v)]}$$

*Proof.* We let $\sigma' = \sigma[x := x]$ and $v' = v[x \mapsto [\![\sigma(x)]\!](v)]$, which means that we have to prove

$$[\![t\,\sigma]\!](v) = [\![t\,\sigma']\!](v')$$

This proof proceeds by induction on the term $t$.

- In the base case we have for a variable $y$:

$$\begin{aligned}
[\![y\,\sigma]\!](v) &= [\![\sigma(y)]\!](v) && \text{def. substitution}\\
&= \begin{cases} [\![\sigma(x)]\!](v), y = x \\ [\![\sigma(y)]\!](v), y \neq x \end{cases}\\
&= \begin{cases} [\![x]\!](v'), y = x \\ [\![\sigma(y)]\!](v), y \neq x \end{cases} && \text{def. } v'\\
&= \begin{cases} [\![x]\!](v'), y = x \\ [\![\sigma(y)]\!](v'), y \neq x \end{cases} && x \text{ fresh for } \sigma\\
&= [\![y\,\sigma']\!](v') && \text{def. substitution update}
\end{aligned}$$

- In the induction step, we have

$$
\begin{aligned}
&[\![f(t_1, \dots, t_n)\,\sigma]\!](v) \\
&= [\![f(t_1\,\sigma, \dots, t_n\,\sigma)]\!](v) && \text{def. substitution} \\
&= \mathcal{M}(f)([\![t_1\,\sigma]\!](v), \dots, [\![t_n\,\sigma]\!](v)) && \text{def. semantics} \\
&= \mathcal{M}(f)([\![t_1\,\sigma']\!](v'), \dots, [\![t_n\,\sigma']\!](v')) && \text{induction hypothesis} \\
&= [\![f(t_1, \dots, t_n)\,\sigma']\!](v') && \text{def. semantics and subst.}
\end{aligned}
$$

Thus, by induction on $t$, the sought after identity $[\![t\,\sigma]\!](v) = [\![t\,\sigma']\!](v')$ holds. □

The following lemma extends lemma 10.31 to formulas.

**Lemma 10.32**
For all substitutions $\sigma$, formulas $\varphi$, valuations $v$ and variables $x$, where $x$ is fresh for $\sigma$, the following equation holds.

$$[\![\varphi\,\sigma]\!](v) = [\![\varphi\,(\sigma[x := x])]\!](v[x \mapsto [\![\sigma(x)]\!](v)])$$

*Proof.* As in lemma 10.31, we let $\sigma' = \sigma[x := x]$ and $v' = v[x \mapsto [\![\sigma(x)]\!](v)]$, and then prove

$$[\![\varphi\,\sigma]\!](v) = [\![\varphi\,\sigma']\!](v')$$

by induction on $\varphi$.

- The base case for $\bot$ is trivial: $[\![\bot\,\sigma]\!](v) = 0 = [\![\bot\,\sigma']\!](v')$.

- In the predicate base case, we have

$$
\begin{aligned}
[\![P(t_1, \dots, t_n)\,\sigma]\!](v) &= [\![P(t_1\,\sigma, \dots, t_n\,\sigma)]\!](v) && \text{def. substitution} \\
&= \mathcal{M}(P)([\![t_1\,\sigma]\!](v), \dots, [\![t_n\,\sigma]\!](v)) && \text{def. semantics} \\
&= \mathcal{M}(P)([\![t_1\,\sigma']\!](v'), \dots, [\![t_n\,\sigma']\!](v')) && \text{lemma 10.31} \\
&= [\![P(t_1, \dots, t_n)\,\sigma']\!](v') && \text{def. semantics and subst.}
\end{aligned}
$$

- The case of equality is similar:

$$
\begin{aligned}
[\![(t_1 \doteq t_2)\,\sigma]\!](v) &= [\![t_1\,\sigma \doteq t_2\,\sigma]\!](v) && \text{def. substitution}\\
&= \begin{cases} 1, & [\![t_1\,\sigma]\!](v) = [\![t_2\,\sigma]\!](v) \\ 0, & \text{otherwise} \end{cases} && \text{def. semantics}\\
&= \begin{cases} 1, & [\![t_1\,\sigma']\!](v') = [\![t_2\,\sigma']\!](v') \\ 0, & \text{otherwise} \end{cases} && \text{lemma 10.31}\\
&= [\![(t_1 \doteq t_2)\,\sigma']\!](v') && \text{def. semantics and subst.}
\end{aligned}
$$

- The cases for conjunction, disjunction and implication are immediate by the induction hypothesis. We write $\mathbb{B}_\wedge = \min$, $\mathbb{B}_\vee = \max$ and $\mathbb{B}_\to(x, y) = x \Rightarrow y$, which are the binary Boolean functions for their corresponding connective. This gives us

$$
\begin{aligned}
[\![(\varphi_1 \,\square\, \varphi_2)\,\sigma]\!](v) &= \mathbb{B}_\square([\![\varphi_1\,\sigma]\!](v), [\![\varphi_2\,\sigma]\!](v)) \\
&\qquad\qquad \text{def. subst. and semantics}\\
&= \mathbb{B}_\square([\![\varphi_1\,\sigma']\!](v'), [\![\varphi_2\,\sigma']\!](v')) && \text{induction hyp.}\\
&= [\![(\varphi_1 \,\square\, \varphi_2)\,\sigma']\!](v') && \text{def. semantics and subst.}
\end{aligned}
$$

- For quantifiers assume that $y$ is fresh for $\sigma$.

  Before we continue, observe that for any $a \in U$, we can define $w = v[y \mapsto a]$ and $w' = w[x \mapsto [\![\sigma(x)]\!](v)]$. By using the induction hypothesis for $\psi$ with $w$, we obtain $[\![\psi\,\sigma]\!](w) = [\![\psi\,\sigma']\!](w')$. Since $x$ and $y$ are fresh, we have that $w' = v'[y \mapsto a]$. Thus $[\![\psi\,\sigma]\!](w) = [\![\psi\,\sigma']\!](v'[y \mapsto a])$.

  With this observation, we have

$$
\begin{aligned}
[\![(\forall y.\,\psi)\,\sigma]\!](v) &= [\![\forall y.\,\psi\,\sigma]\!](v) && \text{(SQ)}\\
&= \min\{[\![\psi\,\sigma]\!](v[y \mapsto a]) \mid a \in U\} && \text{def. semantics}\\
&= \min\{[\![\psi\,\sigma']\!](v'[y \mapsto a]) \mid a \in U\} && \text{see above}\\
&= [\![(\forall y.\,\psi)\,\sigma']\!](v') && \text{def. semantics and (SQ)}
\end{aligned}
$$

  The same reasoning, replacing min by max, gives us the result also for the existential quantifier.

This concludes the induction and thereby the proof. □

*Proof of theorem 10.27.* We proceed by induction on the proof tree for $\Delta \mid \Gamma \vdash \varphi$ in $\mathbf{cND}_1$. The statement for $\mathbf{ND}_1$ follows from this.

Thus, assume that we are given a proof tree for $\Delta \mid \Gamma \vdash \varphi$. Define $D = |\Delta|$ and $G = |\Gamma|$. We have to show for all models $\mathcal{M}$ and valuations $v$ in $\mathcal{M}$ that $[\![G]\!](v) \leq [\![\varphi]\!](v)$. Most of the cases are dealt with in the same way as for propositional logic and we only treat the rules for quantifiers here.

- Suppose the proof tree ends in

$$\frac{\Delta, x \mid \Gamma \vdash \varphi}{\Delta \mid \Gamma \vdash \forall x.\, \varphi}\ (\forall \mathrm{I})$$

  where $x$ does not appear in $\Delta$. We now have

$$\begin{aligned}
[\![\forall x.\, \varphi]\!](v) &= \min\{[\![\varphi]\!](v[x \mapsto a]) \mid a \in |\mathcal{M}|\} && \text{def. semantics}\\
&\geq \min\{[\![G]\!](v[x \mapsto a]) \mid a \in |\mathcal{M}|\} && \text{by IH}\\
&= [\![G]\!](v), && x \text{ not in } \mathrm{fv}(G)
\end{aligned}$$

  where we use the induction hypothesis (IH) for $\Delta, x \mid \Gamma \vdash \varphi$ with model $\mathcal{M}$ and valuation $v[x \mapsto a]$. This identity gives us $G \vDash \forall x.\, \varphi$.

- Next, suppose that last rule that is used in the tree is

$$\frac{\Delta \mid \Gamma \vdash \forall x.\, \varphi}{\Delta \mid \Gamma \vdash \varphi\,[x := t]}\ (\forall \mathrm{E})$$

  for some term $t$ with variables in $\Delta$. By the induction hypothesis, we have that $[\![G]\!](v) \leq [\![\forall x.\, \varphi]\!](v)$. Using the substitution lemma with the substitution $\sigma = [x := t]$, we obtain the desired result:

$$\begin{aligned}
[\![G]\!](v) &\leq [\![\forall x.\, \varphi]\!](v) && \text{by IH}\\
&= \min\{[\![\varphi]\!](v[x \mapsto a]) \mid a \in |\mathcal{M}|\} && \text{by def.}\\
&\leq [\![\varphi]\!](v[x \mapsto [\![t]\!](v)]) && \text{property of min}\\
&= [\![\varphi\,[x := t]]\!](v) && \text{lemma 10.32}
\end{aligned}$$

- The proof for the introduction of existential quantifiers

$$\frac{\Delta \mid \Gamma \vdash \varphi\,[x := t]}{\Delta \mid \Gamma \vdash \exists x.\, \varphi}\ (\exists \mathrm{I})$$

follows a similar argument:

$$\begin{aligned}
[\![G]\!](v) &\leq [\![\varphi\,[x := t]]\!](v) && \text{by IH}\\
&= [\![\varphi]\!](v[x \mapsto [\![t]\!](v)]) && \text{lemma 10.32}\\
&\leq \max\{[\![\varphi]\!](v[x \mapsto a]) \mid a \in |\mathcal{M}|\} && \text{property of max}\\
&= [\![\exists x.\,\varphi]\!](v) && \text{by def.}
\end{aligned}$$

- Finally, the elimination of existential quantifiers is probably the most tricky part. Suppose that the tree ends with this rule application:

$$(x \notin \Delta)\ \frac{\Delta \mid \Gamma \vdash \exists x.\,\varphi \qquad \Delta, x \mid \Gamma, \varphi \vdash \psi}{\Delta \mid \Gamma \vdash \psi}\ (\exists\mathrm{E})$$

By the induction hypothesis, we have

I) $[\![G]\!](v) \leq [\![\exists x.\,\varphi]\!](v) = \max\{[\![\varphi]\!](v[x \mapsto a]) \mid a \in |\mathcal{M}|\}$, and

II) for all $a \in |\mathcal{M}|$

$$\begin{aligned}
\min\{[\![G]\!](v), [\![\varphi]\!]_{v[x\mapsto a]}\} &= [\![G, \varphi]\!](v[x \mapsto a]) && x \notin \Delta\\
&\leq [\![\psi]\!](v[x \mapsto a]) && \text{by IH}\\
&= [\![\psi]\!](v) && x \notin \Delta
\end{aligned}$$

Putting these together, we obtain

$$\begin{aligned}
&[\![G]\!](v)\\
&\leq \min\{[\![G]\!](v), \max\{[\![\varphi]\!](v[x \mapsto a]) \mid a \in |\mathcal{M}|\}\}\\
&\qquad\qquad\qquad\qquad\qquad\qquad \text{by I and monotonicity of min}\\
&\leq [\![\psi]\!](v) && \text{by II}
\end{aligned}$$

as required.

By induction on proof trees thus $\Delta \mid \Gamma \vdash \varphi$ in **cND**$_1$ implies $G \vDash \varphi$. □

# Exercises

## Exercise 40

Let $\mathcal{L}$ be the signature with $\mathcal{F} = \{\underline{0}, s\}$, $\mathcal{R} = \emptyset$, and the following arities.

$$\mathrm{ar}(\underline{0}) = 0 \qquad \mathrm{ar}(s) = 1$$

Moreover, let $\varphi_1$, $\varphi_2$ and $\varphi_3$ be formulas given by

$$\begin{aligned}
\varphi_1 &= \forall x.\, \neg(s(x) \doteq x) \\
\varphi_2 &= \forall x.\, \neg(s(x) \doteq \underline{0}) \\
\varphi_3 &= \forall x.\, \forall y.\, s(x) \doteq s(y) \to x \doteq y\,.
\end{aligned}$$

Determine for every formula which of the following models $\mathcal{M}$ satisfies them.

a)
$$\begin{aligned}
&|\mathcal{M}| = \mathbb{N} \\
&\mathcal{M}(\underline{0})() = 0 \qquad \mathcal{M}(s)(n) = n + 1
\end{aligned}$$

b)
$$\begin{aligned}
&|\mathcal{M}| = \{0, 1, 2\} \\
&\mathcal{M}(\underline{0})() = 0 \qquad \mathcal{M}(s)(n) = \begin{cases} n+1, & n < 2 \\ 0, & n = 2 \end{cases}
\end{aligned}$$

c)
$$\begin{aligned}
&|\mathcal{M}| = \{a, 0, 1, 2\} \\
&\mathcal{M}(\underline{0})() = a \qquad \mathcal{M}(s)(n) = \begin{cases} n+1, & n < 2 \\ 0, & n = a \text{ or } n = 2 \end{cases}
\end{aligned}$$

## Exercise 41

Let $P$ be a binary predicate. Find a model which satisfies the formula $\forall x.\, \neg P(x, x)$ and one that does not.

## Exercise 42

Show that $(\neg\forall x.\, \neg P(x)) \to \exists y.\, P(y)$ is a tautology.

## Exercise 43

In example 10.5, we saw databases as models for the signature that corresponds to a database scheme. The goal of this exercise to understand database updates as operations on models. We call a formula *positive* if it contains no implication.

Let $\mathcal{L}$ be a signature $(\mathcal{F}, \mathcal{R}, \mathrm{ar})$ and $U$ a set. Suppose that $\mathcal{M}_1$ and $\mathcal{M}_2$ are $\mathcal{L}$-models with $|\mathcal{M}_1| = |\mathcal{M}_2| = U$ and $\mathcal{M}_1(f) = \mathcal{M}_2(f)$ for all $f \in \mathcal{F}$.

a) Define a model $\mathcal{M}_1 \sqcup \mathcal{M}_2$ with universe $U$, such that $D \mid \emptyset \vDash_{\mathcal{M}_1 \sqcup \mathcal{M}_2} \varphi$ if $D \mid \emptyset \vDash_{\mathcal{M}_1} \varphi$ or $D \mid \emptyset \vDash_{\mathcal{M}_2} \varphi$ for all positive $\mathcal{L}$-formulas $\varphi$.
We call $\boxed{\mathcal{M}_1 \sqcup \mathcal{M}_2}$ the **model union**. This operation corresponds to merging two databases and, in the special case that one of them only has one entry, to adding an entry. Note that this operation does not preserve constraints, like uniqueness of primary keys!

b) Prove by induction on formulas that, with your definition of $\mathcal{M}_1 \sqcup \mathcal{M}_2$ from task a), $D \mid \emptyset \vDash_{\mathcal{M}_1 \sqcup \mathcal{M}_2} \varphi$ if $D \mid \emptyset \vDash_{\mathcal{M}_1} \varphi$ or $D \mid \emptyset \vDash_{\mathcal{M}_2} \varphi$ for all positive $\mathcal{L}$-formulas $\varphi$.

# 11. Completeness and Limits of First-Order Logic

## 11.1. Completeness

The strength of the classical natural deduction systems for FOL are in their completeness theorems. Recall from theorem 10.27 and corollary 10.29 that $\mathbf{cND}_1^=$ and $\mathbf{cND}_1$ are sound, meaning that any statement that is provable is also true semantically:

If $\Gamma \vdash \varphi$ is derivable in $\mathbf{cND}_1$ or $\mathbf{cND}_1^=$, then $|\Gamma| \vDash \varphi$.

The completeness theorem establishes the other direction of this implication.

**Theorem 11.1: Completeness of $\mathbf{cND}_1^=$**

Let $\Gamma \vdash \varphi$ be a first-order sequent over $\mathcal{L}^=$-formulas. If $|\Gamma| \vDash \varphi$, then $\Gamma \vdash \varphi$ is derivable in $\mathbf{cND}_1^=$.

**Corollary 11.2: Completeness of $\mathbf{cND}_1$**

If $\varphi$ is a $\mathcal{L}$-formula and $\vDash e(\varphi)$, then $\vdash e(\varphi)$ is derivable in $\mathbf{cND}_1$.

Just as in the case of propositional logic, this remarkable result tells us that there is a proof tree for any semantically true statement. Unfortunately, the proof of this result is not effective, that is, we cannot extract an actual proof tree from the proof of theorem 11.1 and we only know that such a proof tree has to exist. The reason for this is that the proof of theorem 11.1 is a proof by contradiction and therefore uses classical logic in an essential way. This cannot be avoided, which renders the completeness theorem fairly useless from a computational perspective. If we wanted to use it to derive $\Gamma \vdash \varphi$, then we would have to quantify over all models and establish for each model $\mathcal{M}$ that the $\Gamma$ entails $\varphi$ in $\mathcal{M}$. As $\Gamma$ and $\varphi$ will likely contain quantifiers, we will then have to quantify over all elements of the universe of $\mathcal{M}$, cf. example 10.25. This is not only difficult, but generally undecidable. Even though the soundness and completeness theorems establish $\mathbf{cND}_1$ as a good proof system for

first-order logic, finding a proof tree remains a difficult problem, which we have addressed in the specific case of Horn clause theories and goal formulas in chapter 7.

**Digression 11.3: Non-empty models**

In chapter 10, we insisted that models may be empty. However, often models of first-order logic are assumed to be non-empty. A consequence of that is that our current proof system $\mathbf{cND}_1^=$ would not be complete in the sense of theorem 11.1. To recover completeness in that case, one has to add the, somewhat obscure, strengthening rule:

$$\frac{\Delta, x \mid \Gamma \vdash \varphi \qquad x \notin \mathrm{fv}(|\Gamma|) \cup \mathrm{fv}(\varphi)}{\Delta \mid \Gamma \vdash \varphi}\ (\mathrm{Str})$$

This rule is not generally sound for our semantics, as one can derive $\vdash \exists x.\, \top$, where we recall that $\top = \neg\bot$:

$$\dfrac{\dfrac{\dfrac{\dfrac{}{x \mid \bot \vdash \bot}}{x \mid \cdot \vdash \top}\ (\neg\mathrm{I})}{x \mid \cdot \vdash \exists x.\, \top}\ (\exists\mathrm{I})}{\vdash \exists x.\, \top}\ (\mathrm{Str})$$

This would be in contradiction to eq. (10.16) and thus (Str) cannot be sound over an empty model.

## 11.2. Compactness and Expressiveness of First-Order Logic

Closely related to completeness is also compactness. Where the completeness theorem 11.1 told us that any semantically true formula has a formal proof, the compactness theorem will establish limits on properties that we can express as formulas in first-order logic.

Note that (first-order) sequents of the form $\Gamma \vdash \varphi$ always require that $\Gamma$ is a *finite list* of formulas, whereas the semantic entailment $G \vDash \varphi$ allows that $G$ can be *any set* of given formulas, even an infinite set. Theorem 11.1 can be formulated differently to allow for such a general set $D$ of assumption. However, one then has to show that $G \vDash \varphi$ uses only finitely many assumptions from $G$ to obtain completeness of $\mathbf{cND}_1^=$. This is because a the proof trees of $\mathbf{cND}_1^=$ are finite and can therefore only use finitely many assumptions. The following theorem that it is always possible to restrict $G$ to a finite subset, which is a quite special feature of first-order logic and, by restriction, of propositional logic.

**Theorem 11.4: Compactness of first-order logic**

Let $G$ be a set of closed $\mathcal{L}^=$-formulas and $\varphi$ a closed $\mathcal{L}^=$-formula. Then $G \vDash \varphi$ if and only if there is a *finite* $G_0 \subseteq G$ such that $G_0 \vDash \varphi$.

The right-to-left direction of the equivalence between the two conditions is easily established: Observe that if $G_0 \subseteq G$, then any model $\mathcal{M}$ we have

$$[\![G]\!]_{\mathcal{M}}(\varepsilon) \leq [\![G_0]\!]_{\mathcal{M}}(\varepsilon) \leq [\![\varphi]\!]_{\mathcal{M}}(\varepsilon)$$

This shows that $G_0 \vDash \varphi$ implies $G \vDash \varphi$ for any (finite) subset $G_0$ of $G$. The other direction is what makes theorem 11.4 interesting. One way to prove it is by using topological compactness to prove compactness for propositional logic and then encode all the first-order deduction rules as propositional formulas using so-called Henkin constants [Bar77, A.1, §4]. We use this direction now to establish some limits of first-order logic with $\mathbf{cND}_1^=$.

A **simple graph** $H$ is given by a finite set $N$ of nodes and a relation $E \subseteq N^2$ of edges. Graphs are typically drawn as diagrams like this:

$$\begin{tikzcd}
n_1 \arrow[rr] & & n_2 \arrow[loop above] \arrow[dl] & n_4 \arrow[l] \\
& n_3 \arrow[ul] & &
\end{tikzcd}$$

This graph $H_1$ is given by the set $N_1 = \{n_1, n_2, n_3, n_4\}$ of nodes and the relation $E_1$ of edges given by

$$E_1 = \{(n_1, n_2), (n_2, n_2), (n_2, n_3), (n_3, n_1), (n_4, n_2)\}.$$

A typical problem in graph theory is reachability:

> Given a graph $H$ and nodes $m$ and $n$ in $H$, can we reach $n$ from $m$ by traversing along edges of $H$?

In the above example, we have that $n_3$ is reachable from $n_1$ by using the edges $(n_1, n_2)$ and $(n_2, n_3)$. Contrary to that, $n_4$ is not reachable from $n_1$ in $H_1$.

This problem seems to lend itself to formalisation in first-order logic: Let $\mathcal{L}_g$ be the signature $(\emptyset, \mathcal{R}, \mathrm{ar})$ with $\mathcal{R} = \{R\}$ and $\mathrm{ar}(R) = 2$. The intention is that the relation symbol $R$ represents a binary relation and therefore the edges of

a graph. Indeed, given a graph $H$ with nodes $N$ and edges $E$, we obtain an $\mathcal{L}_g$-model $\mathcal{M}_H$ by defining

$$|\mathcal{M}_H| = N \quad \text{and} \quad \mathcal{M}_H(R) = E.$$

We can now ask if there is a first-order formula $\varphi$ over $\mathcal{L}_g$ with two free variables $x$ and $y$, such that for all graphs $H$, all nodes $m$ and $n$ in $H$ we have

$$[\![\varphi]\!](\varepsilon[x \mapsto m][y \mapsto n]) = 1 \quad \text{if and only if } n \text{ is reachable from } m \text{ in } H.$$

It is important to note that we ask for a formula that works for all graphs! Let us try to write down such a formula. How can we reach a node $y$ from $x$? Either $x$ is already $y$, or an edge connects them, or we make a transition via other nodes. The problem is that there is a priori no upper bound on the number of nodes that we have to visit to get from $x$ to $y$ because the formula has to work for any graph. Thus, we are left with the following attempt to write down a formula, in which the dots indicate that such a formula would have to be infinitely long, which is clearly not what we consider a formula.

$$\begin{aligned}
&x \doteq y \\
&\vee R(x, y) \\
&\vee (\exists z.\, R(x, z) \wedge R(z, y)) \\
&\vee (\exists z_1.\, \exists z_2.\, R(x, z_1) \wedge R(z_1, z_2) \wedge R(z_2, y)) \\
&\vee \cdots
\end{aligned}$$

How do we get out of this? In just first-order logic with equality, we don't!

**Theorem 11.5: Graph reachability cannot be expressed**

There is no $\mathcal{L}_g^=$-formula $\varphi$ with $\mathrm{fv}(\varphi) = \{x, y\}$, such that for all graphs $H$ $[\![\varphi]\!]_{\mathcal{M}_H}(\varepsilon[x \mapsto m][y \mapsto n]) = 1$ if and only if $n$ is reachable from $m$ in $H$.

*Proof.* Assume that there is an $\mathcal{L}_g^=$-formula with $\mathrm{fv}(\varphi) = \{x, y\}$, such that for every graph $H = (N, E)$ with $\mathcal{M}_H$ as above and for all $n, m \in N$

$$[\![\varphi]\!]_{\mathcal{M}_H}(\varepsilon[x \mapsto m][y \mapsto n]) = 1 \quad \text{if and only if } n \text{ is reachable from } m \text{ in } H.$$

For given $n, m \in N$, let $v = \varepsilon[x \mapsto m][y \mapsto n]$ and define formulas $\varphi_k$ with $\mathrm{fv}(\varphi_k) = \{x, y\}$, such that

$$[\![\varphi_k]\!]^{\mathcal{M}_H}(v) = 1 \quad \text{if and only if there is a path from } n \text{ to } m \text{ of length } k.$$

Concretely, we define $\varphi_k$ by iteration on $k$:

$$\begin{aligned}\varphi_0 &= x \doteq y \\ \varphi_{k+1} &= \exists z.\, R(x, z) \wedge \varphi_k[x := z]\end{aligned}$$

Let $G = \{\neg\varphi_k \mid k \in \mathbb{N}\}$, which expresses that there is no path of any length and we clearly have $G \vDash \neg\varphi$. By compactness (theorem 11.4), we get a finite $G_0 \subseteq G$ with $G_0 \vDash \neg\varphi$. Let $k \in \mathbb{N}$ be the largest number, such that $\varphi_k \in G_0$. Thus, any path longer than $k$ is not forbidden by $G_0$!

To use this fact, we define a graph $H = (N, E)$ by $N = \{n_0, n_1, \ldots, n_{k+1}\}$ and $E = \{(n_i, n_{i+1}) \mid i = 0, \ldots, k\}$. This graph $H$ looks essentially like a list

$$n_0 \longrightarrow n_1 \longrightarrow \cdots \longrightarrow n_{k+1}$$

with only one path from $n_0$ to $n_{k+1}$, which is furthermore of length $k + 1$. In other words, for this graph $H$, we have

$$[\![G_0]\!]^{\mathcal{M}_H}(\varepsilon[x \mapsto n_0][y \mapsto n_{k+1}]) = 1$$

but

$$[\![\neg\varphi]\!]^{\mathcal{M}_H}(\varepsilon[x \mapsto n_0][y \mapsto n_{k+1}]) = 0,$$

which contradicts $G_0 \vDash \neg\varphi$. Hence, the formula $\varphi$ cannot exist. □

Theorem 11.5 is a severe limitation of first-order logic. One may object that we successfully expressed graph reachability in example 7.8, where we defined a predicate that expressed that two positions on Isaac's board are connected. However, it is important that we did not find a formula that only used the relation $A$, but instead used another predicate $C$ together with two Horn clauses eqs. (7.12) and (7.13) to state that $C$ is the reflexive, transitive closure of $A$. This is not in contradiction to theorem 11.5, which states that there can be no formula that expresses that two elements are related by the reflexive, transitive closure of $E$ without using any other predicate symbols.

## 11.3. Primer on Model Theory

One direction of the compactness theorem 11.4 used that if $G_0 \subseteq G$, then $[\![G]\!]_{\mathcal{M}}(\varepsilon) \leq [\![G_0]\!]_{\mathcal{M}}(\varepsilon)$ for all models $\mathcal{M}$. In general, one can ask about **structural properties** of a logic, which refers to the properties of structures that arise from a logic, such as the *collection* of models, the *collection* of formulas,

or the *collection* of proofs and so on and so forth. We have already a structural approach when we characterised trees of a given branching type. Specifically, definition 9.5 stated the defining property of a set of trees in terms of the *collection* of sets and families of maps of a certain kind. Our goal is to give a feeling for the things that can be said structurally about first-order logic.

We shall now give sets of closed formulas, as used above, their proper status.

**Definition 11.6**

A **theory** for a signature $\mathcal{L}$ is a set $T$ of closed $\mathcal{L}^{=}$-formulas. We call $\mathcal{M}$ a **$T$-model**, if $[\![T]\!]_{\mathcal{M}}(\varepsilon) = 1$ and denote by $\boxed{T\text{-}\mathbf{Mod}}$ the collection of all $T$-models. To simplify notation, we write $\boxed{\mathcal{L}\text{-}\mathbf{Mod}}$ instead of $\emptyset$-**Mod** for the **$\mathcal{L}$-model collection**. Two theories $T_1$ and $T_2$ are **semantically equivalent** if $T_1$-**Mod** $= T_2$-**Mod**, in which case we write $\boxed{T_1 \equiv T_2}$.

We will abuse notation slightly and consider an individual closed formula $\varphi$ the same as the theory $\{\varphi\}$. Thus, we will write $\varphi$-**Mod** for the $\{\varphi\}$-models and $\varphi \equiv \psi$ if the formulas $\varphi$ and $\psi$ have the same models. If we unfold the definition of equivalence, then we arrive at the following, equivalent, characterisation.

$$\varphi \equiv \psi \text{ iff for all } \mathcal{L}\text{-models } \mathcal{M} \text{ we have } [\![\varphi]\!]_{\mathcal{M}}(\varepsilon) = [\![\psi]\!]_{\mathcal{M}}(\varepsilon) \tag{11.7}$$

We could take definition 11.6 further and talk about the collection $(\mathcal{L}, T)$-**Mod** of $T$-models for $T$ an $\mathcal{L}$-theory. However, we will not need such flexibility and prefer to keep the notation light.

**Example 11.8**

For closed formulas $\varphi$ and $\psi$, it holds that $\neg(\neg\varphi \wedge \neg\psi) \equiv \varphi \vee \psi$. The easiest way to see this is to treat $\varphi$ and $\psi$ as propositional formulas and compare the respective truth tables. Alternatively, we can also directly write down the semantics in an arbitrary model $\mathcal{M}$:

$$\begin{aligned}
[\![\neg(\neg\varphi \wedge \neg\psi)]\!](\varepsilon) &= 1 - [\![\neg\varphi \wedge \neg\psi]\!](\varepsilon) \\
&= 1 - \min\{1 - [\![\varphi]\!](\varepsilon), 1 - [\![\psi]\!](\varepsilon)\} \\
&= \max\{1 - (1 - [\![\varphi]\!](\varepsilon)), 1 - (1 - [\![\psi]\!](\varepsilon))\} \\
&= \max\{[\![\varphi]\!](\varepsilon), [\![\psi]\!](\varepsilon)\} \\
&= [\![\varphi \vee \psi]\!](\varepsilon)
\end{aligned}$$

Since this holds for an arbitrary model, the equivalence $\neg(\neg\varphi \wedge \neg\psi) \equiv \varphi \vee \psi$

follows from eq. (11.7).

Definition 11.6 shows that we get a relation between formulas in terms of the structure involved in their models. In fact, one can prove quite strong relations between semantic and syntactic structures, which we will briefly give an appetiser of.

**Definition 11.9: Model morphisms**

Let $\mathcal{M}_1$ and $\mathcal{M}_2$ be $\mathcal{L}$-models with $\mathcal{L} = (\mathcal{F}, \mathcal{R}, \mathrm{ar})$. A **model morphism** $h\colon \mathcal{M}_1 \to \mathcal{M}_2$ is a map $|\mathcal{M}_1| \to |\mathcal{M}_2|$ on the universes, such that for all tuples $(a_1, \dots, a_n) \in |\mathcal{M}_1|^n$, $f \in \mathcal{F}_n$ and $P \in \mathcal{R}_n$

1. $h$ preserves operations:

$$h(\mathcal{M}_1(f)(a_1, \dots, a_n)) = \mathcal{M}_2(f)(h(a_1), \dots, h(a_n))$$

2. $h$ preserves truth:

$$\mathcal{M}_1(P)(a_1, \dots, a_n) \leq \mathcal{M}_2(P)(h(a_1), \dots, h(a_n))$$

We call $h$ a **strong morphism** if $h$ preserves and reflects truth:

$$\mathcal{M}_1(P)(a_1, \dots, a_n) = \mathcal{M}_2(P)(h(a_1), \dots, h(a_n))$$

An **embedding** if it is a strong morphism $h\colon \mathcal{M}_1 \to \mathcal{M}_2$ that is injective as map $|\mathcal{M}_1| \to |\mathcal{M}_2|$. We denote by $\mathbf{Mod}(\mathcal{M}_1, \mathcal{M}_2)$, $\mathbf{Mod}_s(\mathcal{M}_1, \mathcal{M}_2)$ and $\mathbf{Emb}(\mathcal{M}_1, \mathcal{M}_2)$, respectively, the sets of morphisms, strong morphisms and embeddings.

It is important that the conditions of an embedding are all needed. The following example shows that even the identity map does not have to be an embedding, even though it will always be a model morphism.

**Example 11.10**

Recall the arithmetic model $\mathcal{M}_a$ from example 10.2. We define another model $\mathcal{M}$ for the same signature that uses the same interpretations as $\mathcal{M}_a$ except that it interprets $L$ as equality. Thus, we define $|\mathcal{M}| = \mathbb{N} = |\mathcal{M}_a|$, $\mathcal{M}(\underline{0}) = \mathcal{M}_a(\underline{0})$, $\mathcal{M}(\underline{1}) = \mathcal{M}_a(\underline{1})$, $\mathcal{M}(p) = \mathcal{M}_a(p)$ and $\mathcal{M}(D) = \mathcal{M}_a(D)$, but

$$\mathcal{M}(L)(n, m) = \begin{cases} 1, & n = m \\ 0, & n \neq m \end{cases}$$

Consider now the identity map $\mathrm{id}\colon \mathbb{N} \to \mathbb{N}$. This is a morphism $\mathcal{M} \to \mathcal{M}_a$, as it clearly preserves operations, and for $n, m \in \mathbb{N}$ we have by $\mathrm{id}(n) = n$ and reflexivity of the order on $\mathbb{N}$ that

$$\begin{aligned}
\mathcal{M}(L)(n,m) &= \begin{cases} 1, & n = m \\ 0, & \text{otherwise} \end{cases} \\
&\leq \begin{cases} 1, & n \leq m \\ 0, & \text{otherwise} \end{cases} && \text{reflexivity} \\
&= \begin{cases} 1, & \mathrm{id}(n) \leq \mathrm{id}(m) \\ 0, & \text{otherwise} \end{cases} && \text{def. id} \\
&= \mathcal{M}_a(L)(\mathrm{id}(n), \mathrm{id}(m))
\end{aligned}$$

Thus, the identity is a morphism $\mathrm{id}\colon \mathcal{M} \to \mathcal{M}_a$. It is also injective, but *not* an embedding because it does not reflect truth of $L$. We can see this by taking to unequal numbers, say 1 and 42, and have $\mathcal{M}_a(L)(1,42) = 1$ but $\mathcal{M}(L)(1,42) = 0$. Hence, $\mathcal{M}_a(L)(\mathrm{id}(1), \mathrm{id}(42)) \not\leq \mathcal{M}(L)(1,42)$, which means that id does not reflect truth.

One may expect that homomorphisms somehow relate the truth of formulas. Indeed, in some cases they do provide a strong relation. Recall that a formula $\psi$ is an **atom**, if it is of the form $\bot$, $P(t_1, \dots, t_n)$ or $s \doteq t$. A **$\Pi_1$-formula** is a formula of the form $\forall x_1. \dots \forall x_n. \psi$ for some atom $\psi$. We say that a theory $T$ is preserved under embeddings, if whenever $h\colon \mathcal{M}_1 \to \mathcal{M}_2$ is an embedding and $\mathcal{M}_2 \in T$-**Mod**, then also $\mathcal{M}_1 \in T$-**Mod**.

**Lemma 11.11**

Any theory $T$ consisting only of formulas of $\Pi_1$-formulas is preserved under embeddings.

*Proof.* Suppose that $h \in \mathbf{Emb}(\mathcal{M}_1, \mathcal{M}_2)$ and $\mathcal{M}_2 \in T$-**Mod**. To show that $\mathcal{M}_1 \in T$-**Mod**, we have to prove that $[\![\varphi]\!]_{\mathcal{M}_1}(\varepsilon) = 1$ for all $\varphi \in T$. Since, by definition of $\Pi_1$-formulas, $\varphi = \forall x_1. \cdots \forall x_n. \psi$ for some atomic formula, we have to show for all valuations $v \in \mathrm{Val}_{\mathcal{M}_1}(\{x_1, \dots, x_n\})$ that $[\![\psi]\!]_{\mathcal{M}_2}(h \circ v) = 1$ implies $[\![\psi]\!]_{\mathcal{M}_1}(v) = 1$.

First, one proves $h([\![t]\!]_{\mathcal{M}_1}(v)) = [\![t]\!]_{\mathcal{M}_2}(h \circ v)$, which is left to exercise 48. Next, we use this to show that $[\![\psi]\!]_{\mathcal{M}_2}(v) \leq [\![\psi]\!]_{\mathcal{M}_1}(h \circ v)$.

- If $\psi = \bot$, we have nothing to do.

- If $\psi = P(t_1, \dots, t_n)$, then

$$\begin{aligned}
[\![\psi]\!]_{\mathcal{M}_1}(v) &= \mathcal{M}_1(P)([\![t_1]\!]_{\mathcal{M}_1}(v), \dots, [\![t_1]\!]_{\mathcal{M}_1}(v)) && \text{def. semantics} \\
&= \mathcal{M}_2(P)(h([\![t_1]\!]_{\mathcal{M}_1}(v)), \dots, h([\![t_1]\!]_{\mathcal{M}_1}(v))) \\
& && h \text{ strong morphism} \\
&= \mathcal{M}_2(P)([\![t_1]\!]_{\mathcal{M}_2}(h \circ v), \dots, [\![t_1]\!]_{\mathcal{M}_2}(h \circ v)) && \text{exercise 48} \\
&= [\![\psi]\!]_{\mathcal{M}_2}(h \circ v) && \text{def. semantics}
\end{aligned}$$

- If $\psi = s \doteq t$, we have

$$\begin{aligned}
[\![\psi]\!]_{\mathcal{M}_2}(h \circ v) &= \begin{cases} 1, & [\![s]\!]_{\mathcal{M}_2}(h \circ v) = [\![t]\!]_{\mathcal{M}_2}(h \circ v) \\ 0, & \text{otherwise} \end{cases} && \text{def. semantics} \\
&= \begin{cases} 1, & h([\![s]\!]_{\mathcal{M}_1}(v)) = h([\![t]\!]_{\mathcal{M}_1}(v)) \\ 0, & \text{otherwise} \end{cases} && \text{exercise 48} \\
&\leq \begin{cases} 1, & [\![s]\!]_{\mathcal{M}_1}(v) = [\![t]\!]_{\mathcal{M}_1}(v) \\ 0, & \text{otherwise} \end{cases} && h \text{ injective} \\
&= [\![\psi]\!]_{\mathcal{M}_1}(v) && \text{def. semantics}
\end{aligned}$$

Therefore, in all cases $[\![\psi]\!]_{\mathcal{M}_2}(h \circ v) \leq [\![\psi]\!]_{\mathcal{M}_1}(v)$. Since $v$ was arbitrary, we obtain that $[\![\varphi]\!]_{\mathcal{M}_2}(\varepsilon) \leq [\![\varphi]\!]_{\mathcal{M}_1}(\varepsilon)$ for all $\varphi \in T$ Thus, all $\varphi$ that are valid in $\mathcal{M}_1$, must be valid in $\mathcal{M}_2$ and thus $\mathcal{M}_2 \in T$-**Mod**. □

We end with the converse of lemma 11.11, which is a remarkable result that gives a *semantic* characterisation of the *syntactic* shape of formulas.

**Theorem 11.12: Łoś–Tarski preservation theorem**

A theory $T$ is preserved under embeddings if and only if there is a theory $T'$ consisting only of $\Pi_1$-formulas with $T \equiv T'$.

The proof of theorem 11.12 utilises again compactness but we first have to prove the following somewhat technical lemma, for which we need a definition. We say that a theory $T$ is **consistent** if there is no formula $\varphi$, such that $T \vDash \varphi$ and $T \vDash \neg\varphi$. Equivalently, $T$ is consistent if there is a $T$-model.

**Lemma 11.13**

Let $T$ and $G$ be $\mathcal{L}$-theories, where $T$ is consistent and $G$ is closed under disjunction, that is, whenever $\varphi, \psi \in G$ then also $\varphi \vee \psi \in G$. Suppose

further that if for $\mathcal{M}_1 \in T$-**Mod** and $\mathcal{M}_2 \in \mathcal{L}$-**Mod**, such that all $\varphi \in G$ that are valid in $\mathcal{M}_1$ are also valid in $\mathcal{M}_2$, then $\mathcal{M}_2 \in T$-**Mod**. Under these conditions, there is $T' \subseteq G$ with $T \equiv T'$.

*Proof.* Let $T' = \{\varphi \in G \mid T \vDash \varphi\}$. By definition, we have that any $T$-model is a $T'$-model, that is, $T$-**Mod** $\subseteq T'$-**Mod**. To finish proving that $T \equiv T'$, we also have to show the reverse inclusion. Suppose $\mathcal{M} \in T'$-**Mod** and define $G' = \{\neg\varphi \mid \varphi \in G \text{ and } \vDash_{\mathcal{M}} \neg\varphi\}$.

First, we show that there is $\mathcal{M}' \in (T \cup G')$-**Mod**, which we prove in turn by show that $T \cup G'$ is consistent. Towards a contradiction, suppose that $T \cup G'$ is not consistent, which means, since $T$ is consistent, that there are formulas $\neg\varphi_1, \dots, \neg\varphi_n \in G'$, such that $T \vDash \neg(\neg\varphi_1 \wedge \dots \wedge \neg\varphi_n)$. From the equivalence $\neg(\neg\varphi_1 \wedge \dots \wedge \neg\varphi_n) \equiv \varphi_1 \vee \dots \vee \varphi_m$, we get $T \vDash \varphi_1 \vee \dots \vee \varphi_m$ and, because $G$ is closed under disjunction, $\varphi_1 \vee \dots \vee \varphi_m \in T'$. Therefore, $\varphi_1 \vee \dots \vee \varphi_m \vDash_{\mathcal{M}}$, but this contradicts the assumption that $\neg\varphi_1, \dots, \neg\varphi_n \in G'$ and $T \cup G'$ must be consistent to begin with and has a model $\mathcal{M}'$.

Second, notice that because $T \subseteq T \cup G'$, $\mathcal{M}'$ is also a $T$-model. By the assumption is thus suffices to show that any formula that is valid in $\mathcal{M}'$ is also valid in $\mathcal{M}$. If $\vDash_{\mathcal{M}'} \varphi$, then $\neg\varphi \notin G'$ and therefore it is not the case that $\vDash_{\mathcal{M}} \neg\varphi$ by definition of $G'$. Theorem 10.26 gives thus that $\vDash_{\mathcal{M}} \varphi$ and $\mathcal{M}$ validates all valid formulas in $G$. By assumption, we thus have that $\mathcal{M} \in T$-**Mod**. This shows that $T'$-**Mod** $\subseteq T$-**Mod**.

Putting all together, we have $T$-**Mod** $= T'$-**Mod** and thus $T \equiv T'$. □

Using this lemma, we can prove the preservation theorem.

*Proof of theorem 11.12.* Let $T$ be a theory. One direction, that $T$ consisting of $\Pi_1$-formulas implies preservation of embeddings, is lemma 11.11.

For the other direction, suppose for every $h \in \mathbf{Emb}(\mathcal{M}_1, \mathcal{M}_2)$ that if we have $\mathcal{M}_2 \in T$-**Mod**, then also $\mathcal{M}_1 \in T$-**Mod**. To get an equivalent theory of $\Pi_1$-formulas, we apply lemma 11.13 as follows. Define $T_1$ to be the set of $\Pi_1$-formulas and $G$ the closure of $T_1$ under disjunction, which means that

$$G = \left\{ \bigvee_{i=1}^{n} \varphi_i \;\middle|\; n \in \mathbb{N}, \varphi_i \in T_1 \right\},$$

where $\bigvee_{i=1}^{n} \varphi_i$ is a shorthand for the disjunction $\varphi_1 \vee \cdots \vee \varphi_n$. First, we see that $T_1 \equiv G$ because 1) in a $T_1$-model, we have for $\bigvee_{i=1}^{n} \varphi_i \in G$ that

$$\left[\!\!\left[\bigvee_{i=1}^{n} \varphi_i\right]\!\!\right](\varepsilon) = \max\{[\![\varphi_i]\!](\varepsilon) \mid i = 1, \dots, n\} = \max\{1\} = 1\,.$$

thus $T_1$-**Mod** $\subseteq G$-**Mod**, and 2) $T_1 \subseteq G$ gives that $G$-**Mod** $\subseteq T_1$-**Mod**.

By definition, $G$ is closed under disjunction and it remains to prove the second condition of lemma 11.13. Thus, suppose that $\mathcal{M}_1 \in T$-**Mod** and $\mathcal{M}_2 \in \mathcal{L}$-**Mod**, such that $\varphi \in G$ valid in $\mathcal{M}_1$ implies that $\varphi$ is valid in $\mathcal{M}_2$. We have to show that $\mathcal{M}_2 \in T$-**Mod**. To this end, we define a new signature $\mathcal{L}^+ = (\mathcal{F}^+, \mathcal{R}, \mathrm{ar}^+)$ with $\mathcal{F}^+ = \mathcal{F} \cup |\mathcal{M}_2|$ and $\mathrm{ar}^+(f) = \mathrm{ar}(f)$ for $f \in \mathcal{F}$ and $\mathrm{ar}^+(a) = 0$ for $a \in |\mathcal{M}_2|$. Similarly, we expand $\mathcal{M}_2$ to a model $\mathcal{M}_2^+$ that is defined on all symbols in $\mathcal{L}$ like $\mathcal{M}_2$ and $\mathcal{M}_2^+(a) = a$ for $a \in |\mathcal{M}_2|$. Let

$$\begin{aligned} T_{\mathcal{M}_2} = &\left\{\varphi \in \mathrm{Form}^{=}(\mathcal{L}^+) \;\middle|\; \varphi \text{ atom and } \vDash_{\mathcal{M}_2^+} \varphi\right\} \\ &\cup \left\{\neg\varphi \in \mathrm{Form}^{=}(\mathcal{L}^+) \;\middle|\; \varphi \text{ atom and } \vDash_{\mathcal{M}_2^+} \neg\varphi\right\} \end{aligned}$$

be the so-called *diagram* of $\mathcal{M}_2$ [CK90, Sec. 2.1] and put $T_3 = T \cup T_{\mathcal{M}_2}$.

The theory $T_3$ can be seen to be consistent as follows. Towards a contradiction, assume that $T_3$ is not consistent, that is, $T_3 \vDash \bot$. Since $T$ has $\mathcal{M}_1$ as model and is thus consistent, the compactness theorem (11.4) provides us with a finite set $U \subset T_{\mathcal{M}_2}$ such that $T \cup U \vDash \bot$. By definition of $T_{\mathcal{M}_2}$, there are $\mathcal{L}^{=}$-formulas $\varphi_1, \dots, \varphi_n$ and a substitution $\sigma$ with $\sigma(x) \in |\mathcal{M}_2|$ for all $x \in \mathrm{supp}(\sigma)$, such that $U = \{\varphi_k\,\sigma \mid k = 1, \dots, n\}$. Therefore, $\mathcal{M}_2$ validates the formula $\exists x_1. \cdots \exists x_m. \bigwedge_{k=1}^{n} \varphi_k$, where $\mathrm{supp}(\sigma) = \{x_1, \dots, x_m\}$. This, however, implies that $\mathcal{M}_2$ also validates $\bot$, which contradicts that $[\![\bot]\!]_{\mathcal{M}_2}(v) = 0$ by definition 10.15. Hence, $T_3$ is consistent and has a model $\mathcal{M}_3$.

We can see the $\mathcal{L}^+$-model $\mathcal{M}_3$ as a $\mathcal{L}$-model by restriction to the symbols in $\mathcal{L}$. Define an map $h\colon |\mathcal{M}_2| \to |\mathcal{M}_3|$ by $h(a) = \mathcal{M}_3(a)()$. Because $\mathcal{M}_3$ validates all formulas in $T_{\mathcal{M}_2}$, this map is injective, and it also preserves and reflects truth. Lastly, since $\mathcal{M}_3$ validates $T_{\mathcal{M}_2}$, we have for all $b \in \mathcal{M}_2^+$ that $\vDash_{\mathcal{M}_2^+} b \doteq f(a_1, \dots, a_n)$ implies $b \doteq f(a_1, \dots, a_n) \vDash_{\mathcal{M}_3}$. With $b = \mathcal{M}_2(f)(a_1, \dots, a_n)$ we get

$$\begin{aligned} h(\mathcal{M}_2(f)(a_1, \dots, a_n)) &= \mathcal{M}_3(\mathcal{M}_2(f)(a_1, \dots, a_n))() \\ &= \mathcal{M}_3(f)(\mathcal{M}_3(a_1)(), \dots, \mathcal{M}_3(a_n)()) \\ &= \mathcal{M}_3(f)(h(a_1), \dots, h(a_n)) \end{aligned}$$

and $h$ also preserves operations, which makes it an embedding. Since $T \subseteq T_3$, $\mathcal{M}_3 \in T$-**Mod** and by the assumption that $T$ is preserved under embeddings, also $\mathcal{M}_2 \in T$-**Mod**.

We can now apply lemma 11.13 to $T$ and $G$, which gives us a $T' \subseteq G$ with $T \equiv T'$, as required. □

## 11.4. Further Reading

There are various good sources on model theory in general, like the books of Chang and Keisler [CK90] and of Hodges [Hod93]. In these books, one can find, for instance, the Łoś-Tarski preservation theorem (theorem 11.12) in [CK90, Theorem 3.2.2] and [Hod93, Corollary 6.5.3].

Another branch of model theory is *finite model theory*, which is strongly related to computation and game theory. The book by Ebbinghaus and Flum [EF95] provides a good introduction. Recent development in this area concern also preservation theorems in the style of theorem 11.12, like for instance the characterisation of goal formulas (definition 7.16) in terms of morphisms between *finite* models [Ros08] This is an important result for databases, for which most queries have the shape of a goal formula. Generally, homomorphism and finite models have gained increased attention in the recent years due to various applications in computer science and other fields [ADW17; Lop24].

Generally, there is a strong relation between properties of models on the one hand, and algebra and game theory on the other hand. An introduction to these topics can be found in the book of Ebbinghaus et al. [EFT21] and a, somewhat older, overview in the handbook edited by Barwise [Bar77].

# Exercises

## Exercise 44

Let $\mathcal{M}$ be the model for the arithmetic signature from example 10.2 on the *real numbers* $\mathbb{R}$ be given by $|\mathcal{M}| = \mathbb{R}$ and

| Symbol | Interpretation type | Interpretation |
|---|---|---|
| $\underline{0}$ | $\mathcal{M}(\underline{0})\colon \mathbb{R}^0 \to \mathbb{R}$ | $\mathcal{M}(\underline{0})() = 0$ |
| $\underline{1}$ | $\mathcal{M}(\underline{1})\colon \mathbb{R}^0 \to \mathbb{R}$ | $\mathcal{M}(\underline{1})() = 1$ |
| $p$ | $\mathcal{M}(p)\colon \mathbb{R}^2 \to \mathbb{R}$ | $\mathcal{M}(p)(a,b) = a + b$ |
| $D$ | $\mathcal{M}(D)\colon \mathbb{R}^2 \to \mathbb{B}$ | $\mathcal{M}(D)(a,b) = \begin{cases} 1, a \text{ divides } b \\ 0, \text{otherwise} \end{cases}$ |
| $L$ | $\mathcal{M}(L)\colon \mathbb{R}^2 \to \mathbb{B}$ | $\mathcal{M}(L)(a,b) = \begin{cases} 1, a \leq b \\ 0, \text{otherwise} \end{cases}$ |

Show that the inclusion $i\colon \mathbb{N} \to \mathbb{R}$, given by $i(n) = n$, is an injective model morphism $\mathcal{M}_a \to \mathcal{M}$. Is it an embedding?

## Exercise 45

Show that $\neg\forall x.\, \neg\varphi \equiv \exists x.\, \varphi$.

## Exercise 46

We call a simple graph $(N, E)$ *undirected*, if every edge $(n, m) \in E$ has a reverse $(m, n) \in E$. Give theory $T$ over the graph signature $\mathcal{L}_g$ from section 11.2, such that $T$-**Mod** consists precisely of undirected graphs.

## Exercise 47

Let $\mathcal{L}$ be a signature without function symbols. A (untyped) *database* is nothing but a finite model for a finite signature without function symbols. For instance, let $\mathcal{L} = (\emptyset, \mathcal{R}, \mathrm{ar})$ be given by $\mathcal{R}_2 = \{\mathrm{Prod}, \mathrm{Price}\}$. We interpret $\mathrm{Prod}(x, y)$ as "Product $y$ has identifier $x$" and $\mathrm{Price}(x, z)$ as "Identifier $x$ costs $z$".

Many queries on databases can be formulated by goal formulas. For example, consider the formula $\varphi$ given by $\exists x.\, \mathrm{Price}(x, z) \wedge \mathrm{Prod}(x, y)$, which is called in the realm of databases the *join* of Price and Prod.

a) Give a finite model $\mathcal{M}_1$ for $\mathcal{L}^=$.

b) Find a valuation $\mathrm{Val}(\{x, y\})$, such that $[\![\varphi]\!]_{\mathcal{M}_1}(v) = 1$.

c) Give a formula that expresses that identifiers for products are unique.

d) Give another finite model $\mathcal{M}_2$ with more elements, such that the inclusion $\mathcal{M}_1 \subset \mathcal{M}_2$ is a morphism. Hint: Adding elements to the database preserves truth.

## Exercise 48

Let $h\colon \mathcal{M}_1 \to \mathcal{M}_2$ be a model morphism, $D \subseteq \mathrm{Var}$ and $v \in \mathrm{Val}_{\mathcal{M}_1}(D)$. Prove by induction on terms that for all terms $t \in \mathrm{Term}(\mathcal{L})$ with $\mathrm{var}(f) \subseteq D$ that $h([\![t]\!]_{\mathcal{M}_1}(v)) = [\![t]\!]_{\mathcal{M}_2}(h \circ v)$.

## Exercise 49

Let $T$ be a theory consisting only of formulas of $\Pi_1$-formulas. Show that if $h\colon \mathcal{M}_1 \to \mathcal{M}_2$ is morphism and $\mathcal{M}_1 \in T\text{-}\mathbf{Mod}$, then also $\mathcal{M}_2 \in T\text{-}\mathbf{Mod}$.

## Exercise 50

Let $\mathcal{L}$ be a signature $(\mathcal{F}, \mathcal{R}, \mathrm{ar})$ and $U$ a set. Suppose that $\mathcal{M}_1$ and $\mathcal{M}_2$ are $\mathcal{L}$-models with $|\mathcal{M}_1| = |\mathcal{M}_2| = U$ and $\mathcal{M}_1(f) = \mathcal{M}_2(f)$ for all $f \in \mathcal{F}$. Recall the model union from exercise 43. Show that there are injective model morphisms $i_1\colon \mathcal{M}_1 \to \mathcal{M}_1 \sqcup \mathcal{M}_2$ and $i_2\colon \mathcal{M}_2 \to \mathcal{M}_1 \sqcup \mathcal{M}_2$.

# 12. Logic and Computation

In section 11.2, we have seen a principle limitation of first-order logic in *expressiveness*, which means that there are properties that cannot be expressed as first-order formulas. The goal of this and the next chapter, is to show that there are principle limitations to automatic deduction for first-order logic and to even to provability. Recall that we have devised in section 7.3 a proof system that enabled proof search in first-order logic, whenever premises were given as Horn theories. However, we have not claimed that this proof search always stops, either with a valid proof or the conclusion that no proof is possible. It turns out that it is in general impossible to find an algorithm that can prove or disprove formulas of first-order logic. We will formulate the precise statement of this in theorem 12.8, but that requires us to first develop some understanding of computability in section 12.1.

Having discussed computability, we can also state precisely what an axiomatisation is in section 12.2. We will find that there are axiomatisations, for which there are algorithms that can check whether a formula is a consequence of the axioms or not.

Finally, we prepare the grounds for the next chapter in section 12.3, where we introduce a restricted form of computation via so-called **primitive recursive functions**. This form of computation is very closely related to iteration on natural numbers and trees, as we will see, and it plays an important role in the foundations of computability theory [Bri94]. The present and the next chapter will thus show how logic and computation are intricately linked with each other.

## 12.1. Computability

The following definition refers to "computation procedures", which we do not define precisely here. Think of them as computer programs, a sequence of instructions that can be unambiguously run by a machine. A more precise

definition can be given, for example, in terms of Turing machines or by any other equivalent notion of computation [Bri94; Hro04].

**Definition 12.1**

For a given set $A$, we denote by $\boxed{\overline{A}}$ the **partial elements** over $A$, which is given by the disjoint union $\overline{A} = A + \{\uparrow\}$, see appendix B.2. The new element $\boxed{\uparrow}$ should be understood as **diverging computation** or non-halting computation. We particularly call $\boxed{\overline{\mathbb{N}}}$ the set of **partial numbers**. A map $f\colon A \to \overline{\mathbb{N}}$ is called a **partial function** or just **function**. The **domain** of $f$ is given by the set of all inputs on which $f$ converges: $\boxed{\operatorname{dom}(f) = \{x \in A \mid f(x) \neq \uparrow\}}$. If $\operatorname{dom}(f) = A$, then $f$ is called **total**.
We say that a function $f\colon \mathbb{N} \to \overline{\mathbb{N}}$ is **computable** if there is a computation procedure that halts on all inputs $x \in \operatorname{dom}(f)$ with output $f(x)$ and does not halt if $f(x) = \uparrow$. A set $P \subseteq \mathbb{N}$ is **semi-decidable** (or **recursively enumerable**, r.e.) if there is a computable function $f$ with $\operatorname{dom}(f) = P$. Such a set $P$ is **decidable**, if the **characteristic function** $\chi_P$ with

$$\chi_P(x) = \begin{cases} 0, & x \in P \\ 1, & x \notin P \end{cases}$$

is total computable.

The name "recursively enumerable" for semi-decidable sets comes from the fact that a set $P \subseteq \mathbb{N}$ is semi-decidable if and only if $P$ is empty or there is a total computable function $f\colon \mathbb{N} \to \mathbb{N}$ with $P = \{f(n) \mid n \in \mathbb{N}\}$. Thus, there is a way to *enumerate* the elements of $P$ as $f(0), f(1), \ldots$

**Example 12.2**

Suppose that $A$ is a Turing machine with, say for definiteness, alphabet $\{0, 1\}$ and one tape. The precise machine model is not relevant for our purposes, we only need to know that we can write a number on the tape, run the machine and read the output from the tape. This allows us to define the following function $f_A\colon \mathbb{N} \to \overline{\mathbb{N}}$.

$$f_A(k) = \begin{cases} r, & A \text{ stops with } r \text{ as output} \\ \uparrow, & A \text{ does not stop with input } k \end{cases}$$

By its very definition, $f_A$ is computable and has as domain

$$\mathrm{dom}(f) = \{k \in \mathbb{N} \mid A \text{ stops on input } k\},$$

which makes the set of inputs on which $A$ stops semi-decidable by just running $A$ on an input. For certain $A$, there are decision procedures to check whether $A$ stops on a given input, but in general $\mathrm{dom}(f)$ is not decidable. This is the content of the famous Halting problem. Although to see this, we need to be able to encode Turing machines as numbers, just like we store computer programs as bit-strings.

Definition 12.1 seems at first rather limited because (semi-)decidability is only defined for sets of natural numbers. How can we then, for example, check proof trees as in chapter 6? We will use in what follows a more general definition of computability over sets $A$ other than $\mathbb{N}$. Thus, we will then speak of $f\colon A \to \overline{\mathbb{N}}$ being computable if there is a computation procedure that halts on all inputs $x \in \mathrm{dom}(f) \subseteq A$. This requires a representation of elements of $A$ that can be used for computing, which is generally not a trivial problem. A major part of the incompleteness proof in chapter 13 is finding a representation of proof trees with specific computational properties. In general, we do not need specific computational properties and the following abstract definition suffices.

**Definition 12.3**

Let $A$ be a set. We say that $A$ is **representable** if there are **encoding** and **decoding** maps $\mathrm{enc}\colon A \to \mathbb{N}$ and $\mathrm{dec}\colon \mathbb{N} \to \overline{A}$, such that,

1. $\mathrm{dec}(\mathrm{enc}(x)) = x$,
2. $\mathrm{dom}(\mathrm{dec}) \subseteq \mathbb{N}$ is decidable, and
3. $\mathrm{enc}(\mathrm{dec}(n)) = n$ for all $n \in \mathrm{dom}(\mathrm{dec})$

A function $f\colon A \to \overline{\mathbb{N}}$ is (total) computable if the function $f^\dagger\colon \mathbb{N} \to \overline{\mathbb{N}}$ with

$$f^\dagger(n) = \begin{cases} f(\mathrm{dec}(n)), & \mathrm{dec}(n) \neq \uparrow \\ \uparrow, & \mathrm{dec}(n) = \uparrow \end{cases}$$

is (total) computable.

Definition 12.1 is restricted to natural numbers, but we can easily extend the definition to pairs by using a suitable representation.

**Example 12.4**

Let us show that the set $\mathbb{N}^2$ of pairs is representable. The general idea of the encoding and decoding functions for pairs is that any number can be uniquely decomposed into prime factors. We define $\mathrm{enc}\colon \mathbb{N}^2 \to \mathbb{N}$ by $\mathrm{enc}(n, m) = 2^n 3^m$ and $\mathrm{dec}\colon \mathbb{N} \to \mathbb{N}^2 + \{\uparrow\}$ by

$$\mathrm{dec}(k) = \begin{cases} (n, m), & k = 2^n 3^m \\ \uparrow, & \text{otherwise} \end{cases}$$

where it should be noted that $k = 1 = 2^0 3^0$ encodes the pair $(0, 0)$. This is a well-defined function because the prime factorisation of $k$ into $2^n 3^m$ is unique. The conditions 1 and 3 are clearly fulfilled by definition, while condition 2 follows from the fact that the prime factorisation of a number can be computed. Thus, the set of pairs $\mathbb{N}^2$ is representable.

Using representations of pairs, we can analyse the computability of maps that take several arguments.

**Example 12.5: Universal Turing machine**

Let us denote by TM the set of Turing machines. This set is representable by using a suitable encoding of normalised Turing machines as natural numbers [Bri94]. Thus, combined with the encoding of pairs given in example 12.4, the product $\mathrm{TM} \times \mathbb{N}$ is representable. We can then consider the map $f_U\colon \mathrm{TM} \times \mathbb{N} \to \overline{\mathbb{N}}$ defined by $f_U(A, k) = f_A(k)$, where we use the maps $f_A$ from example 12.2. An important result in computability theory is that $f_U$ is computable in the sense of definition 12.3 by the so-called universal Turing machine [Bri94].

Defining the decoding and encoding functions is in general quite complex, which is why any reasonable programming language features user-definable data types. From the theoretical perspective, the following theorem shows that tree-shaped data types can always be represented as natural numbers. However, the resulting encodings only show that *in principle* computations on trees can be reduced to computations on natural numbers, but the complexity of these computations would so be enormous that any realistic computation would exceed the lifetime of the universe!

**Theorem 12.6**

Let $(L, B)$ be a branching type such that $L$ is representable and for each $a \in L$ the set $B_a$ is finite and representable, then also the set of trees $\mathcal{T}_{(L,B)}$

is representable.

*Proof.* Let $\mathrm{enc}L$ and $\mathrm{dec}L$ be the encoding/decoding maps for $L$, and $\mathrm{enc}_a$ and $\mathrm{dec}_a$ those for all $B_a$. Suppose that $p\colon \mathbb{N} \to \mathbb{N}$ is an injective map that enumerates the prime numbers starting from 2, that is, $p(n)$ is the $n$-th prime number with $p(0) = 2$ and for every prime number $q$ there is a unique $n$ with $p(n) = q$. Then we can encode maps $B_a \to \mathbb{N}$ essentially in the same way as in example 12.4 because $B_a$ is finite. Define $e_a\colon [B_a, \mathbb{N}] \to \mathbb{N}$ by $e_a(\alpha) = \prod_{b \in B} p(b)^{\mathrm{enc}_a(\alpha(b))}$, which multiplies the prime powers $p(b)^{\mathrm{enc}_a(\alpha(b))}$ for all (finite) $b \in B_a$. As in example 12.4, we can decode numbers back into such maps by using that the unique prime factorisation of a number can always be computed. $d_a\colon \mathbb{N} \to [B_a, \mathbb{N}]$

Using this coding, we can encode trees by iteration. We define a family of maps $f_a\colon [B_a, \mathbb{N}] \to \mathbb{N}$ by $f_a(\alpha) = p(\mathrm{enc}_L(a))^{e_a(\alpha)}$, which induces a unique map $\mathrm{enc}\colon \mathcal{T}_{(L,B)} \to \mathbb{N}$ as the inductive extension of $(f_a)_{a \in L}$. It should be noted that $e_a(\alpha) \geq 1$ and we can thus define $g\colon \mathbb{N} \to (L \times \mathbb{N}) \cup \{\uparrow\}$ by

$$g(k) = \begin{cases} (\mathrm{dec}_L(n), r), & k = q^r, q = p(n) \\ \uparrow, & \text{otherwise} \end{cases},$$

which uses again uniqueness of prime factorisations. This allows us to define $\mathrm{dec}\colon \mathbb{N} \to \mathcal{T}_{(L,B)}$ by

$$\mathrm{dec}(k) = \begin{cases} \mathrm{tr}_a(\mathrm{dec} \circ d_a(r)), & g(k) = (a, r), d_a(r) \in [B_a, \mathbb{N}] \\ \uparrow, & \text{otherwise} \end{cases},$$

which is well-defined because $d_a(r)(b) < k$ and thus the argument passed to dec in this recursive definition is strictly decreasing. The conditions 1 and 3 on the maps enc and dec can be proven by, respectively, uniqueness of the inductive extensions and by induction on natural numbers. That the domain of dec is decidable follows once more from the total computability of prime factorisations. □

We can use theorem 12.6 to represent terms and formulas as natural numbers.

**Example 12.7**

If the function symbols of a signature are representable, for example, if there are only finitely many as in most of our examples, and the set Var of variables is representable, then the set of terms is representable by the

presentation of terms as trees from section 9.2 and theorem 12.6.
To give an example of how such a representation looks like, suppose that we have a signature with two function symbols $c \in \mathcal{F}_0$ and $f \in \mathcal{F}_2$, which we can encode by $\mathrm{enc}(c) = 0$ and $\mathrm{enc}(f) = 1$. Since $p(0) = 2$ and $p(1) = 3$, the term $f(c, f(c, c))$ would be encoded as $3^{2^1 3^{2^1 2^1}}$, where 1 is the encoding of the empty map $\varepsilon$. We can see from this example that the encoding of a term leads to a tower of exponentials, which is both very fast-growing and extremely slow to compute with. However, this complexity is not our concern here, as we merely need that we can *in principle* only compute on natural numbers.

Example 12.7 can be extended to formulas, although the trees are somewhat more complex than those of terms due to the issues with binding, cf. definition 5.3. This allows us to formulate the following theorem.

**Theorem 12.8**

The set $\{\varphi \in \mathrm{Form}^{=}(\mathcal{L}) \mid \vdash \varphi \text{ derivable in } \mathbf{cND}_1^{=}\}$ is semi-decidable but not decidable.

*Proof.* We only sketch the proof idea. To show that the set is semi-decidable, we use that proof trees in $\mathbf{cND}_1^{=}$ can be enumerated by constructing every more complex proof trees. Given a formula, we can then walk through this enumeration and try to find a proof for a given formula $\varphi$. If we find it, then we know that $\varphi$ is derivable and stop. This shows that the set is semi-decidable.

The above process will not terminate if $\varphi$ is not derivable, and one can prove that there is no way to give a procedure that decides whether a formula is derivable, as such a procedure could be used to show to decide another problem, the so-called halting problem, which is known to be *not* decidable. However, proving this goes beyond the scope of this book. □

## 12.2. Axiomatisations

The proof sketch of theorem 12.8 tells us how to decide whether a formula can be derived from a theory: if we can enumerate all formulas of a theory and every formula or its negation eventually occurs in this enumeration, then we obtain a procedure that decides if a formula is provable by just traversing the enumeration until we get to the formula or its negation. Let us make this precise, which also allows us to state what is actually meant by an axiomatisation.

Let us define **deduction from a theory**, written as $\boxed{T \vdash \varphi}$, as follows.

$$T \vdash \varphi \iff \text{there is a list } \Gamma \text{ with } |\Gamma| \subseteq T \text{ and } \Gamma \vdash \varphi \text{ is provable in } \mathbf{cND}_1^{=}$$

Using soundness (theorem 10.27), completeness (theorem 11.1) and compactness (theorem 11.4) of $\mathbf{cND}_1^{=}$, $T \vdash \varphi$ holds if and only if $T \vDash \varphi$ holds.

**Definition 12.9**

We call an $\mathcal{L}$-theory $T$ **axiomatisable** if there is a semi-decidable set $T_0 \subseteq \mathrm{Form}^{=}(\mathcal{L})$, such that $T_0 \equiv T$. The elements of $T_0$ are called **axioms** and $T_0$ an **axiomatisation**. A theory $T$ is **complete** if for all closed formulas $\varphi \in \mathrm{Form}^{=}\mathcal{L}$ we have either $T \vdash \varphi$ or $T \vdash \neg\varphi$.

Since deduction from theories is the same as entailment, see above, we find that $T$ is axiomatisable if and only if there is a semi-decidable set $T_0 \subseteq T$ that generates $T$ under provability:

$$T \subseteq \{\varphi \mid T_0 \vdash \varphi\}\,.$$

In practice, we usually provide a set of axioms and take the corresponding theory to be the theory generated under provability.

**Example 12.10**

Since the set $\{\forall x.\, \neg(s\, x \doteq \underline{0})\}$ is finite, it is semi-decidable. It axiomatises the theory, in which zero is not the successor of any number. The resulting theory is not complete because neither $\forall x.\, \neg(s(x) \doteq x)$ nor its negation is a consequence of the above axiomatisation, see exercise 40.

One can also go the other way around: define a wanted theory and then try to find a way to decide its consequences.

**Example 12.11**

Let $\mathcal{L}^r = (\mathcal{F}^r, \emptyset, \mathrm{ar}^r)$ with $\mathcal{F}_0^r = \{\underline{1}\}$ and $\mathcal{F}_2^r = \{\underline{\cdot}\}$ be the signature for multiplication with a unit. Define a model $\mathcal{M}^r$ on the real numbers in the expected way by $|\mathcal{M}^r| = \mathbb{R}$, $\mathcal{M}^r(\underline{1}) = 1$ and $\mathcal{M}^r(\underline{\cdot})(x, y) = x \cdot y$. We get a theory $T$ of all formulas that are validated in $\mathcal{M}^r$ by $T = \{\varphi \mid \vDash_{\mathcal{M}^r} \varphi\}$. A celebrated result of Tarksi shows that this theory is axiomatisable and even decidable [Bar77, p. C.3]!

We mentioned at the beginning of this section that theorem 12.8 gives us a hint at how decidable theories can be obtained in very well-behaved cases. The following theorem captures the essence of this idea.

**Theorem 12.12**

If a theory $T$ is axiomatisable and complete, then it is decidable.

*Proof.* We have to show that the characteristic function $\chi_T$ is computable, which means that we have a procedure that halts on any given formula $\varphi$ and outputs whether the formula is in $T$ or not. Let $T_0$ be an axiomatisation of $T$. By the remark after definition 12.1 we can enumerate all formulas in $T_0$ and thus all consequences of $T_0$ by enumerating all proofs of $\Gamma \vdash \varphi$ in $\mathbf{cND}_1^=$ for finite lists $\Gamma$ with $|\Gamma| \subseteq T_0$. This enumerates all formulas in $T$. Given a formula $\varphi$, we can walk through this enumeration of proofs and check if the proof is for $\varphi$ or $\neg\varphi$. Since $T$ is complete, one of the two cases must eventually occur in the enumeration and the procedure halts on every input. Therefore, $T$ is decidable. □

## 12.3. Primitive Recursion

So far, we have talked about computable functions quite abstractly and focused more on functions that are *not* total computable or sets that are not decidable. In preparation for the next chapter, we devote this section to the study of a very well-behaved set of total computable functions, the set of primitive recursive functions. The idea is quite simple: we recognise three basic functions and any composition and iteration of those as computable. This resulting set of functions can described as trees, but let us start with a more direct description.

**Lemma 12.13**

Let $g\colon \mathbb{N}^n \to \mathbb{N}$ and $h\colon \mathbb{N}^{n+2} \to \mathbb{N}$ be maps. Then there is a unique map $\mathrm{I}^{n+1}(g,h)\colon \mathbb{N}^{n+1} \to \mathbb{N}$ that fulfils the following equations.

$$\mathrm{I}^{n+1}(g,h)(0,x) = g(x) \tag{12.14}$$

$$\mathrm{I}^{n+1}(g,h)(k+1,x) = h(\mathrm{I}^{n+1}(g,h)(k,x),k,x) \tag{12.15}$$

*Proof.* We use iteration of natural numbers (definition B.3) in order to define $\mathrm{I}^{n+1}(g,h)$. Let $p_1\colon \mathbb{N} \times [\mathbb{N}^n,\mathbb{N}] \to \mathbb{N}$ and $p_2\colon \mathbb{N} \times [\mathbb{N}^n,\mathbb{N}] \to [\mathbb{N}^n,\mathbb{N}]$ be the projections given by $p_2(k,r) = r$, see appendix B.2. We define a map $f\colon \mathbb{N} \times [\mathbb{N}^n,\mathbb{N}] \to \mathbb{N} \times [\mathbb{N}^n,\mathbb{N}]$ as the pairing $f = \langle f_1, f_2\rangle$, see appendix B.2, where

$$f_1(u) = p_1(u) + 1 \qquad \text{and} \qquad f_2(u)(x) = h(p_2(u)(x), p_1(u), x)\,.$$

Unfolding the definitions, we get for $(k, r) \in \mathbb{N} \times [\mathbb{N}^n, \mathbb{N}]$ that

$$\langle f_1, f_2 \rangle(k, r) = (k+1, u) \text{ with } u(x) = h(r(x), k, x)\,.$$

For the base case, we define by $a_0 \in \mathbb{N} \times [\mathbb{N}^n, \mathbb{N}]$ with $a_0 = (0, g)$. Then, by iteration, we get a map $\overline{(a_0, f)} \colon \mathbb{N} \to \mathbb{N} \times [\mathbb{N}^n, \mathbb{N}]$ and we can define

$$\mathrm{I}^{n+1}(g, h)(k, x) = p_2\Big(\overline{(a_0, f)}(k)\Big)(x)\,.$$

We show first that $p_1 \circ \overline{(a_0, f)} = \mathrm{id}_{\mathbb{N}}$. This follows because

$$p_1\Big(\overline{(a_0, f)}(0)\Big) = p_1(a_0) = 0$$

and

$$p_1\Big(\overline{(a_0, f)}(k+1)\Big) = f_1\Big(\overline{(a_0, f)}(k)\Big) = p_1\Big(\overline{(a_0, f)}(k)\Big) + 1\,.$$

Hence, $p_1 \circ \overline{(a_0, f)}$ is the inductive extension of 0 and the successor map, just like the identity map, which gives us that $p_1 \circ \overline{(a_0, f)} = \mathrm{id}_{\mathbb{N}}$ by uniqueness of inductive extensions.

The two required eqs. (12.14) and (12.15) hold because

$$\mathrm{I}^{n+1}(g, h)(0, x) = p_2\Big(\overline{(a_0, f)}(0)\Big)(x) = p_2(a_0)(x) = g(x)$$

and at successors since

$$\begin{aligned}
\mathrm{I}^{n+1}(g, h)(k+1, x) &= p_2\Big(\overline{(a_0, f)}(k+1)\Big)(x) && \text{def } \mathrm{I}^{n+1}(g, h) \\
&= p_2\Big(f\Big(\overline{(a_0, f)}(k)\Big)\Big)(x) && \text{iteration} \\
&= f_2\Big(\Big(\overline{(a_0, f)}(k)\Big)\Big)(x) && \text{def. } f \\
&= h\Big(p_2\Big(\overline{(a_0, f)}(k)\Big)(x), p_1\Big(\overline{(a_0, f)}(k)\Big), x\Big) && \text{def. } f_2 \\
&= h\Big(p_2\Big(\overline{(a_0, f)}(k)\Big)(x), k, x\Big) && p_1 \circ \overline{(a_0, f)} = \mathrm{id}_{\mathbb{N}} \\
&= h(\mathrm{I}^{n+1}(g, h)(k, x), k, x) && \text{def } \mathrm{I}^{n+1}(g, h)
\end{aligned}$$

Uniqueness follows from the uniqueness of inductive extensions. □

The map $\mathrm{I}^{n+1}(g, h)$ constructed in lemma 12.13 is called the **primitive iteration** of $g$ and $h$. As intuition, one should have that it can be applied to one number $k$ to iterate over and $n$ parameters that are used unchanged at every

step of the iteration. One can replace the parameters $\mathbb{N}^n$ in lemma 12.13 by any set, which gives so-called higher-order primitive iteration. We have no need for such a general scheme, but we briefly discuss in digression 12.24 the ramifications of restricting to tuples of natural numbers.

We can also compose functions. Let $g\colon \mathbb{N}^n \to \mathbb{N}$ and $h_1, \dots, h_n$ be $n$ functions with arity $m$, that is, of type $h_k\colon \mathbb{N}^m \to \mathbb{N}$. The **composition** of $g$ after $h_1, \dots, h_n$ is the map $\mathrm{C}_n^m(g, h_1, \dots, h_n)\colon \mathbb{N}^m \to \mathbb{N}$ given as follows.

$$\mathrm{C}_n^m(g, h_1, \dots, h_n)(x) = g(h_1(x), \dots, h_n(x)) \tag{12.16}$$

Starting from three basic functions that we can recognise immediately as total computable, we generate the primitive recursive functions as composition and primitive iteration of those. For ease of readability, the upper index in all maps, including the composition, always coincides with the arity of the resulting function. In the case of iteration, the index $n$ implies that $\mathrm{I}^{n+1}(f, g)$ has arity $n+1$, and it has $n$ parameters plus one input on which it iterates.

**Definition 12.17**

We define three kinds of maps as follows.

- Zero maps $\mathrm{Z}^n\colon \mathbb{N}^n \to \mathbb{N}$ with $\mathrm{Z}^n(x) = 0$
- Successor map $\mathrm{S}\colon \mathbb{N} \to \mathbb{N}$ with $\mathrm{S}(x) = x + 1$
- Projection maps $\Pi_k^n\colon \mathbb{N}^n \to \mathbb{N}$ for $1 \le k \le n$ with

$$\Pi_k^n(x_1, \dots, x_n) = x_k \tag{12.18}$$

The set family $\boxed{\mathrm{PR}_n}$ indexed by $\mathbb{N}$ of **primitive recursive functions** of arity $n$ is the smallest set closed under the following five rules.

$$\frac{}{\mathrm{S} \in \mathrm{PR}_1} \qquad \frac{}{\mathrm{Z}^n \in \mathrm{PR}_n} \qquad \frac{1 \le k \le n}{\Pi_k^n \in \mathrm{PR}_n} \qquad \frac{g \in \mathrm{PR}_n \qquad h \in \mathrm{PR}_{n+2}}{\mathrm{I}^{n+1}(g, h) \in \mathrm{PR}_{n+1}}$$

$$\frac{g \in \mathrm{PR}_n \qquad h_1 \in \mathrm{PR}_m \qquad \cdots \qquad h_n \in \mathrm{PR}_m}{\mathrm{C}_n^m(g, h_1, \cdots, h_n) \in \mathrm{PR}_m}$$

We then let $\mathrm{PR} = \bigcup_{n \in \mathbb{N}} \mathrm{PR}_n$ and call a function primitive recursive (p.r.), if it is in the set PR.

Most standard arithmetic can be realised as primitive recursive functions. The

following examples illustrate this.

**Example 12.19**

The truncated predecessor function $\mathrm{pred}\colon \mathbb{N} \to \mathbb{N}$, specified by

$$\mathrm{pred}(n) = \begin{cases} 0, & n = 0 \\ n-1, & n > 0 \end{cases},$$

is primitive recursive, as we have that $\mathrm{pred} = \mathrm{I}^1(\mathrm{Z}^0, \Pi_2^2)$. First, we note that the right-hand side is indeed primitive recursive because it is in $\mathrm{PR}_1$, thus in PR, by the following derivation.

$$\frac{\overline{\mathrm{Z}^0 \in \mathrm{PR}_0} \qquad \overline{\Pi_2^2 \in \mathrm{PR}_2}}{\mathrm{I}^1(\mathrm{Z}^0, \Pi_2^2) \in \mathrm{PR}_1}$$

That this identity holds can be seen by evaluating the right-hand side as far as possible and match it with the specification of pred. Since the function is given by iteration, we distinguish the base case and the step case:

$$\begin{aligned} \mathrm{I}^1(\mathrm{Z}^0, \Pi_2^2)(n) &= \begin{cases} \mathrm{Z}^0(), & n = 0 \\ \Pi_2^2(\mathrm{I}^1(\mathrm{Z}^0, \Pi_2^2)(k), k), & k = n+1 \end{cases} \\ &= \begin{cases} 0, & n = 0 \\ k, & n = k+1 \end{cases} \\ &= \begin{cases} 0, & n = 0 \\ n-1, & n > 0 \end{cases} \end{aligned}$$

This corresponds to our specification of the predecessor.

Note that the predecessor is defined in example 12.19 by case distinction on its argument. We can use the same idea to show that case distinction is primitive recursive in general.

**Example 12.20**

For $n \geq 1$ and primitive recursive functions $f\colon \mathbb{N}^n \to \mathbb{N}$ and $g\colon \mathbb{N}^{n+1} \to \mathbb{N}$, the function $[f, g]\colon \mathbb{N}^{n+1} \to \mathbb{N}$ given by

$$[f, g](n, x) = \begin{cases} f(x), & n = 0 \\ g(k, x), & n = k+1 \end{cases},$$

is primitive recursive because the of the identity

$$[f, g] = \mathrm{I}^{n+1}\big(f, \mathrm{C}^{n+2}_{n+1}(g, \Pi^{n+2}_2, \dots, \Pi^{n+2}_{n+2})\big)\,.$$

To see this, we first note that we get a primitive recursive function by the following derivation.

$$\dfrac{\dfrac{}{f \in \mathrm{PR}_n} \qquad \dfrac{\dfrac{}{g \in \mathrm{PR}_{n+1}} \quad \dfrac{}{\Pi^{n+2}_2 \in \mathrm{PR}_{n+2}} \quad \cdots \quad \dfrac{}{\Pi^{n+2}_{n+2} \in \mathrm{PR}_{n+2}}}{\mathrm{C}^{n+2}_{n+1}(g, \Pi^{n+2}_2, \dots, \Pi^{n+2}_{n+2})}}{\mathrm{I}^{n+1}\big(f, \mathrm{C}^{n+2}_{n+1}(g, \Pi^{n+2}_2, \dots, \Pi^{n+2}_{n+2})\big)}$$

Next, we show that this primitive recursive function fulfils the specification. Given $x \in \mathbb{N}^n$, we have by eq. (12.14) in lemma 12.13 that

$$\mathrm{I}^{n+1}\big(f, \mathrm{C}^{n+2}_{n+1}(g, \Pi^{n+2}_2, \dots, \Pi^{n+2}_{n+2})\big)(0, x) = f(x)\,,$$

which corresponds to the first case of the specification for the case distinction. For the second case, let $k \in \mathbb{N}$ and abbreviate $r = \mathrm{I}^{n+1}\big(f, \mathrm{C}^{n+2}_{n+1}(g, \Pi^{n+2}_2, \dots, \Pi^{n+2}_{n+2})\big)(k, x)$. Then we have that

$$\begin{aligned}
&\mathrm{I}^{n+1}\big(f, \mathrm{C}^{n+2}_{n+1}(g, \Pi^{n+2}_2, \dots, \Pi^{n+2}_{n+2})\big)(k+1, x) \\
&= \mathrm{C}^{n+2}_{n+1}(g, \Pi^{n+2}_2, \dots, \Pi^{n+2}_{n+2})(r, k, x) && \text{by eq. (12.15)} \\
&= g\big(\Pi^{n+2}_2(r, k, x), \dots, \Pi^{n+2}_{n+2}(r, k, x)\big) && \text{by eq. (12.16)} \\
&= g(k, x) && \text{by eq. (12.18)}
\end{aligned}$$

where we used that $n \geq 1$ and $\big(\Pi^{n+2}_3(r, k, x), \dots, \Pi^{n+2}_{n+2}(r, k, x)\big) = x$.

Neither of the examples 12.19 and 12.20 has used proper recursion, as both ignored the recursion parameter in their use of iteration. Let us now give an example of a proper recursively defined function.

**Example 12.21**

To show that addition is primitive recursive, we observe that addition is uniquely defined as an inductive extension via the two equations $0 + m = m$ and $(n + 1) + m = (n + m) + 1$. We can emulate these with primitive iteration and define $p\colon \mathbb{N}^2 \to \mathbb{N}$ by

$$p = \mathrm{I}^2(\Pi^1_1, \mathrm{C}^3_1(S, \Pi^3_1))\,.$$

As before, let us check that $p$ is a well-defined primitive recursive function by providing the corresponding derivation.

$$\dfrac{\dfrac{}{\Pi_1^1 \in \mathrm{PR}_1} \qquad \dfrac{\dfrac{}{S \in \mathrm{PR}_1} \quad \dfrac{}{\Pi_1^3 \in \mathrm{PR}_3}}{\mathrm{C}_1^3(S, \Pi_1^3) \in \mathrm{PR}_3}}{\mathrm{I}^2(\Pi_1^1, \mathrm{C}_1^3(S, \Pi_1^3)) \in \mathrm{PR}_2}$$

Next, we check that $p$ indeed gives us addition. To this end, we let $m \in \mathbb{N}$ and prove by induction on $n \in \mathbb{N}$ that $p(n, m) = n + m$. In the base case we have by eq. (12.14) that

$$p(0, m) = \Pi_1^1(m) = m = 0 + m\,.$$

For the induction step, we get the induction hypothesis (IH) $p(n, m) = n + m$, which allows us to prove the following identity.

$$\begin{aligned}
p(n+1, m) &= \mathrm{C}_1^3(S, \Pi_1^3)(p(n, m), n, m) && \text{by eq. (12.15)}\\
&= S(\Pi_1^3(p(n, m), n, m)) && \text{by eq. (12.16)}\\
&= S(p(n, m)) && \text{by eq. (12.18)}\\
&= p(n, m) + 1 && \text{def. successor}\\
&= (n + m) + 1 && \text{by induction hypothesis}\\
&= (n + 1) + m
\end{aligned}$$

Thus, $p(n+1, m) = (n+1)+m$ in the induction step. Hence, by induction, $p(n, m) = n + m$ for all $n, m \in \mathbb{N}$.
An alternative proof goes by defining addition as an inductive extension (definition B.3) and then proving that $p$ fulfils the same recursive equations as addition, which would give us that $p(n, m) = n + m$ by the uniqueness of inductive extensions.

Since providing a formal derivation every time we devise a primitive recursive function is quite tedious, we usually leave these out with the understanding that the derivation can be reconstructed from the indices.

**Example 12.22**

It is often useful to have more general constant maps at hand. Let $m \in N$, we define $\mathrm{K}_m^n \colon \mathbb{N}^n \to \mathbb{N}$ iteratively by

$$\mathrm{K}_0^n = \mathrm{Z}^n \quad \text{and} \quad \mathrm{K}_{m+1}^n = \mathrm{C}_1^n(S, \mathrm{K}_m^n)\,.$$

Using the defining equations of zero, successor and composition maps, we see that $\mathrm{K}^n_m(x) = m$ for all $x \in \mathbb{N}^n$ by induction on $m$. In the base case, it holds that

$$\mathrm{K}^n_0(x) = \mathrm{Z}^0(x) = 0\,.$$

In the induction step, we assume that $\mathrm{K}^n_m(x) = m$ and then calculate that

$$\mathrm{K}^n_{m+1}(x) = \mathrm{C}^n_1(S, \mathrm{K}^n_m)(x) = S(\mathrm{K}^n_m(x)) = S(m) = m + 1\,.$$

Hence, $\mathrm{K}^n_m(x) = m$ by induction.

Once we have found that a function is primitive recursive, we can use this to construct more complex primitive recursive functions.

**Example 12.23**

We show that multiplication is primitive recursive by constructing a primitive recursive function $\mathrm{mul}\colon \mathbb{N}\times\mathbb{N} \to \mathbb{N}$, such that $\mathrm{mul}(n, m) = n{\cdot}m$. To do so, we use that multiplication fulfils $0{\cdot}m = 0$ and $(n{+}1){\cdot}m = (n{\cdot}m){+}m$. This allows us to use primitive iteration and put

$$\mathrm{mul} = \mathrm{I}^2(\mathrm{Z}^1, \mathrm{C}^3_2(p, \Pi^3_1, \Pi^3_3))\,.$$

We prove that $\mathrm{mul}(m, n) = m \cdot n$ by induction in $m$. In the base case have

$$\mathrm{mul}(0, n) = \mathrm{Z}^1(n) = 0 = 0 \cdot n\,.$$

Assuming that $\mathrm{mul}(m, n) = m \cdot n$, the induction step is given by

$$\begin{aligned}
\mathrm{mul}(m+1, n) &= \mathrm{C}^3_2(p, \Pi^3_1, \Pi^3_3)(\mathrm{mul}(m, n), m, n) \\
&= p(\Pi^3_1(\mathrm{mul}(m, n), m, n), \Pi^3_3(\mathrm{mul}(m, n), m, n)) \\
&= p(\mathrm{mul}(m, n), n) \\
&= \mathrm{mul}(m, n) + n && \text{by example 12.21} \\
&= (m \cdot n) + n && \text{by induction hypothesis} \\
&= (m + 1) \cdot n
\end{aligned}$$

Thus, $\mathrm{mul}(m, n) = m \cdot n$ for all $n, m \in \mathbb{N}$ and multiplication is primitive recursive.

The set of primitive recursive functions consists of many arithmetic and standard computational functions, see also exercise 52. However, there are well-defined functions, which are not primitive recursive.

**Digression 12.24: Ackermann function**

Recall that we defined primitive iteration in lemma 12.13 as a result of iteration, but that the parameters were restricted to tuples of natural numbers. This restriction has quite far-reaching consequences, as there are functions that are definable by iteration but not as primitive recursion functions. The so-called **Ackermann function** is a function that can be defined by iteration but not by primitive recursion [Pét67, § 10]. This is a function $A\colon \mathbb{N}^2 \to \mathbb{N}$, defined by the following three equations.

$$\begin{aligned} A(0,n) &= n+1 \\ A(m+1,0) &= A(m,1) \\ A(m+1,n+1) &= A(m, A(m+1,n)) \end{aligned}$$

We see that $A$ proceeds by iteration in its two arguments *simultaneously*. There are different ways to show that these equations define a unique function, but we will use what we know best: inductive extensions. To do that, we follow the approach taken in lemma 12.13 and $A$ by

$$A(m,n) = p_2\big(\overline{(a_0,h)}(m)\big)(n)\,,$$

where $p_2\colon \mathbb{N}\times[\mathbb{N},\mathbb{N}] \to [\mathbb{N},\mathbb{N}]$ is the projection $p_2(n,f) = f$ and $a_0 \in \mathbb{N}\times[\mathbb{N},\mathbb{N}]$ and $h\colon \mathbb{N}\times[\mathbb{N},\mathbb{N}] \to \mathbb{N}\times[\mathbb{N},\mathbb{N}]$ will be suitably defined. For $a_0$, we note that it should cover the first equation for $A$, thus we define

$$a_0 = (0, \mathrm{S})\,.$$

The map $h$ should cover the second and third equation for $A$, which are the cases for the successor in the first argument. Looking closely at these cases, we find that these two equations yield the repeated application

$$A(m+1,n) = \underbrace{A(m, A(m, \dots, A(m,1)))}_{n+1 \text{ times}}\,.$$

Using iteration once more, we define for a function $f\colon \mathbb{N}\to\mathbb{N}$ another function $\mathrm{rep}_f\colon \mathbb{N} \to [\mathbb{N},\mathbb{N}]$ that repeats $f$ by $\mathrm{rep}_f = \overline{(f, c_f)}$ using the map $c_f\colon [\mathbb{N},\mathbb{N}] \to [\mathbb{N},\mathbb{N}]$ given by $c_f(h) = f\circ h$. The equations determining $\mathrm{rep}_f$ tell us that

$$\mathrm{rep}_f(0) = f \quad \text{and} \quad \mathrm{rep}_f(n+1) = f\circ \mathrm{rep}_f(n)\,,$$

which means that $\mathrm{rep}_f(n)(m)$ applies $f$ $(n+1)$-times to the argument $m$. This is exactly what we need to complete our iterative definition of $A$ and we set

$$h(n,f) = (n+1, \mathrm{rep}_f(n))\,.$$

One can check that our thus defined function $A$ fulfils the three required equations.
This shows that $A$ is a well-defined function. However, one can prove that it grows faster than all primitive recursive functions [Pét67, § 10], which means that it cannot be itself primitive recursive. The way we have constructed $A$ via iteration fits into schemes of higher-order recursion, which we will not discuss here further [LN15; Lon17].

## 12.4. Further Reading

- origins of undecidability in logic [Chu36]?
- [Bar77, Chapters C.1, C.2, C.3, C.6]
- [Bar77, Chapter C.7] on inductive definitions (relation to trees)
- Turing machines and computable functions [Bri94]
- Structural study of computability [CH08; EE70]
- Diagonal arguments to prove that, for example, $[\mathbb{N}, \mathbb{N}]$ is not representable [Bri94; EE70; Yan22]
- Realisability semantics [TvD88b, Chapter 5.5]

# Exercises

## Exercise 51

Let $\mathcal{L}$ be the signature $(\mathcal{F}, \emptyset, \mathrm{ar})$ with $\mathcal{F} = \{\underline{0}, s\}$ and the arities

$$\mathrm{ar}(\underline{0}) = 0 \qquad \mathrm{ar}(s) = 1,$$

and let $\varphi_1, \varphi_2$ and $\varphi_3$ be the $\mathcal{L}^=$-formulas given by:

$$\begin{aligned}
\varphi_1 &\stackrel{\text{def}}{=} \forall x.\, s(x) \not\doteq \underline{0} \\
\varphi_2 &\stackrel{\text{def}}{=} \forall x.\, \forall y.\, s(x) \doteq s(y) \to x \doteq y \\
\varphi_3 &\stackrel{\text{def}}{=} \forall x.\, x \not\doteq \underline{0} \to \exists y.\, x \doteq s(y)\,,
\end{aligned}$$

where $s \not\doteq t$ abbreviates $\neg(s \doteq t)$.

a) Provide finite models for each of the $\mathcal{L}^=$-axiomatisations: $\emptyset$, $\{\varphi_1\}$, $\{\varphi_3\}$ and $\{\varphi_1, \varphi_3\}$.

b) Is it possible to give a finite model for the $\mathcal{L}^=$-axiomatisation $\{\varphi_1, \varphi_2\}$?

c) Give a finite model for the $\mathcal{L}^=$-axiomatisation $\{\varphi_1, \varphi_2, \varphi_3\}$ that does not have $\mathbb{N}$ as universe.

d) Give a finite model that satisfies $\{\varphi_1\}$ but not $\{\varphi_3\}$.

## Exercise 52

Show that each of the following is a primitive recursive function:

a) the exponential function $\exp\colon \mathbb{N}^2 \to \mathbb{N}$ with $\exp(m, n) = n^m$. Hint: Use that constant maps and multiplication are primitive recursive.

b) the sign functions $\mathrm{sgn}\colon \mathbb{N} \to \mathbb{N}$ with

$$\mathrm{sgn}(n) = \begin{cases} 1, & n > 0 \\ 0, & n = 0 \end{cases}$$

c) the truncated subtraction $\dot{-} \colon \mathbb{N}^2 \to \mathbb{N}$ with

$$n \dot{-} m = \begin{cases} n - m, & m \leq n \\ 0, & \text{otherwise} \end{cases}.$$

Hint: Think about the cases $n \dot{-} 0$ and $n \dot{-} (m+1)$.

d) the characteristic function of equality $\chi_= \colon \mathbb{N}^2 \to \mathbb{N}$ with

$$\chi_=(n, m) = \begin{cases} 0, & n = m \\ 1, & n \neq m. \end{cases}$$

Hint: Use truncated subtraction and the sign function.

## Exercise 53

Suppose that $A$ is a representable set. Show that the set $A^2$ of pairs is representable. Hint: Follow the argument in example 12.4.

# 13. Gödel's Incompleteness Theorems

## 13.1. Signature Maps and Theory Inclusions

Before we work our way towards the incompleteness theorems, we need to be able to say when one signature is included in another, and how the semantics and theories for different signatures relate. The discussion of this could also be carried out for proof systems, but we will only be working with $\mathbf{cND}_1^=$ in what follows, which is sound and complete.

The central concept for relating signatures are maps that respect arities.

**Definition 13.1**

Let $\mathcal{L} = (\mathcal{F}, \mathcal{R}, \mathrm{ar})$ and $\mathcal{L}' = (\mathcal{F}', \mathcal{R}', \mathrm{ar}')$ be given signatures. A map $h\colon \mathcal{F} \cup \mathcal{R} \to \mathcal{F}' \cup \mathcal{R}'$ is a **signature map** $\mathcal{L} \to \mathcal{L}'$ if

- for all $f \in \mathcal{F}$ and $P \in \mathcal{R}$, we have $h(f) \in \mathcal{F}'$ and $h(P) \in \mathcal{R}'$, and
- $\mathrm{ar}' \circ h = \mathrm{ar}$.

If $h$ is an inclusion $\mathcal{F} \cup \mathcal{R} \subseteq \mathcal{F}' \cup \mathcal{R}'$ and a signature map, then we write $\mathcal{L} \subseteq \mathcal{L}'$ and say that $\mathcal{L}$ is included in $\mathcal{L}'$.

Using iteration, we can turn a signature map $h\colon \mathcal{L} \to \mathcal{L}'$ into maps $\mathrm{Term}(h)$ and $\mathrm{Form}^=(h)$ on terms and formulas of type $\mathrm{Term}(h)\colon \mathrm{Term}(\mathcal{L}) \to \mathrm{Term}(\mathcal{L}')$ and $\mathrm{Form}^=(h)\colon \mathrm{Form}^=(\mathcal{L}) \to \mathrm{Form}^=(\mathcal{L}')$, and that fulfil

$$\begin{aligned}
\mathrm{Term}(h)(x) &= x \\
\mathrm{Term}(h)(c) &= h(c) \\
\mathrm{Term}(h)(f(t_1, \dots, t_n)) &= h(f)(\mathrm{Term}(h)(t_1), \dots, \mathrm{Term}(h)(t_n))
\end{aligned}$$

and

$$\begin{aligned}
\mathrm{Form}^{=}(h)(s \doteq t) &= \mathrm{Term}(h)(s) \doteq \mathrm{Term}(h)(t) \\
\mathrm{Form}^{=}(h)(P(t_1, \dots, t_n)) &= h(P)(\mathrm{Term}(h)(t_1), \dots, \mathrm{Term}(h)(t_n)) \\
\mathrm{Form}^{=}(h)(\bot) &= \bot \\
\mathrm{Form}^{=}(h)(\varphi \,\square\, \psi) &= \mathrm{Form}^{=}(h)(\varphi) \,\square\, \mathrm{Form}^{=}(h)(\psi) \\
\mathrm{Form}^{=}(h)(\forall x.\, \varphi) &= \forall x.\, \mathrm{Form}^{=}(h)(\varphi) \\
\mathrm{Form}^{=}(h)(\exists x.\, \varphi) &= \exists x.\, \mathrm{Form}^{=}(h)(\varphi)
\end{aligned}$$

Let us illustrate these definitions on a simple arithmetic example.

**Example 13.2**

Let $\mathcal{L} = (\mathcal{F}, \emptyset, \mathrm{ar})$ with $\mathcal{F} = \{c, f\}$, and $\mathrm{ar}(c) = 0$ and $\mathrm{ar}(f) = 2$. Moreover, let us denote by $\mathcal{L}_a = (\mathcal{F}_a, \emptyset, \mathrm{ar}_a)$ a signature for arithmetic, using plus and multiplication, with $\mathcal{F}_a = \{\underline{0}, \underline{1}, p, m\}$, $\mathrm{ar}_a(\underline{0}) = \mathrm{ar}_a(\underline{1}) = 0$ and $\mathrm{ar}_a(p) = \mathrm{ar}_a(m) = 2$. We can define a signature map $g \colon \mathcal{L}_a \to \mathcal{L}$ by

$$g(\underline{0}) = g(\underline{1}) = c \quad \text{and} \quad g(p) = g(m) = f\,.$$

For the $\mathcal{L}_a$-term $m(p(x, \underline{0}), y)$, we get

$$\mathrm{Term}(g)(m(p(x, \underline{0}), y)) = f(f(x, c), y)$$

and thus the following mapping of an equality formula.

$$\mathrm{Form}^{=}(g)(m(p(x, \underline{0}), y) \doteq m(x, y)) = f(f(x, c), y) \doteq f(x, y)$$

Note that this mapping does not have to preserve or reflect truth of formulas. For instance, we have $\mathrm{Form}^{=}(g)(m(x, \underline{0}) \doteq \underline{0}) = f(x, c) \doteq x$. However, nothing prevents that $f(x, c) \doteq x$ holds in an $\mathcal{L}$-model, while $m(x, \underline{0}) \doteq \underline{0}$ in another $\mathcal{L}_a$-model. Clearly, the truth in these models is in contradiction with the translation. That being said, we provide in definition 13.5 suitable restrictions that prevent this.

We can compose signature maps $h_1 \colon \mathcal{L}' \to \mathcal{L}'$ and $h_2 \colon \mathcal{L}' \to \mathcal{L}''$ to a signature map $h_2 \circ h_1 \colon \mathcal{L} \to \mathcal{L}''$ because $\mathrm{ar}'' \circ h_2 \circ h_1 = \mathrm{ar}' \circ h_1 = \mathrm{ar}$. Moreover, the identity map $\mathrm{id}_{\mathcal{F} \cup \mathcal{R}} \colon \mathcal{F} \cup \mathcal{R} \to \mathcal{F} \cup \mathcal{R}$ fulfils $\mathrm{ar} \circ \mathrm{id} = \mathrm{ar}$ and is thus a signature map $\mathcal{L} \to \mathcal{L}$, which we denote by $\mathrm{id}_{\mathcal{L}}$. The extension of signature maps to terms and formulas preserves composition and identities, as the following lemma shows.

**Lemma 13.3**

For all signatures $\mathcal{L}$, we have

$$\mathrm{Term}(\mathrm{id}_{\mathcal{L}}) = \mathrm{id}_{\mathrm{Term}(\mathcal{L})} \quad \text{and} \quad \mathrm{Form}^{=}(\mathrm{id}_{\mathcal{L}}) = \mathrm{id}_{\mathrm{Form}^{=}(\mathcal{L})}\,.$$

For signature maps $h_1 \colon \mathcal{L}' \to \mathcal{L}'$ and $h_2 \colon \mathcal{L}' \to \mathcal{L}''$, the composition is also preserved:

$$\begin{aligned}\mathrm{Term}(h_2 \circ h_1) &= \mathrm{Term}(h_2) \circ \mathrm{Term}(h_1)\\ \mathrm{Form}^{=}(h_2 \circ h_1) &= \mathrm{Form}^{=}(h_2) \circ \mathrm{Form}^{=}(h_1)\,.\end{aligned}$$

Finally, if $i \colon \mathcal{L} \to \mathcal{L}'$ is an inclusion $\mathcal{L} \subseteq \mathcal{L}'$, then $\mathrm{Term}(i)$ and $\mathrm{Form}^{=}(i)$ are also inclusions.

We can use a signature map $h \colon \mathcal{L} \to \mathcal{L}'$ to interpret the symbols of $\mathcal{L}$ is a model $\mathcal{M}$ for $\mathcal{L}'$. This process is called the **pullback of $\mathcal{M}$ along $h$**, written $\boxed{h^*\mathcal{M}}$, and yields an $\mathcal{L}$-model. More precisely, we define $h^*\mathcal{M}$ by

$$|h^*\mathcal{M}| = |\mathcal{M}| \qquad (h^*\mathcal{M})(f) = \mathcal{M}(h(f)) \qquad (h^*\mathcal{M})(P) = \mathcal{M}(h(P))\,,$$

which is well-defined because $h$ preserves symbol arities. The pullback along $h \colon \mathcal{L} \to \mathcal{L}'$ can be seen as a map between the collections of models:

$$h^* \colon \mathcal{L}'\text{-}\mathbf{Mod} \to \mathcal{L}\text{-}\mathbf{Mod}$$

Note the direction of this map, which explains the name: we pull a model for $\mathcal{L}'$ back to a model of $\mathcal{L}$ along $h$, which goes the opposite direction of $h$!

We also note that $\mathrm{Val}_{\mathcal{M}}(D) = \mathrm{Val}_{h^*\mathcal{M}}(D)$ for all $D \subseteq \mathrm{Var}$. With this we can show that semantics of formulas mapped by $\mathrm{Form}(h)$ is the same as the semantics of $\varphi$ in the pullback of $\mathcal{M}$ along $h$.

**Lemma 13.4**

Let $h \colon \mathcal{L} \to \mathcal{L}'$ be a signature map. For all terms $t \in \mathrm{Term}(\mathcal{L})$, formulas $\varphi \in \mathrm{Form}^{=}(\mathcal{L})$ and valuations $v \in \mathrm{Val}_{\mathcal{M}}(D)$, we have

$$[\![\mathrm{Term}(h)(t)]\!]_{\mathcal{M}}(v) = [\![t]\!]_{h^*\mathcal{M}}(v)$$

and

$$[\![\mathrm{Form}^{=}(h)(\varphi)]\!]_{\mathcal{M}}(v) = [\![\varphi]\!]_{h^*\mathcal{M}}(v)$$

We can see the pullback of a model $\mathcal{M}$ along an inclusion $i \colon \mathcal{L} \subseteq \mathcal{L}'$ as restricting $\mathcal{M}$ to the symbols in $\mathcal{L}$. Since $\mathrm{Form}^{=}(i)$ provides also inclusions of

formulas, we obtain from lemma 13.4 that a formula $\varphi \in \mathrm{Form}^{=}(\mathcal{L})$ has the same semantics in $\mathcal{M}$ as in its restriction $i^*\mathcal{M}$:

$$[\![\varphi]\!]_{\mathcal{M}} = [\![\varphi]\!]_{i^*\mathcal{M}} \,.$$

**Definition 13.5**

Suppose that $T$ is a $\mathcal{L}$-theory and $T'$ a $\mathcal{L}'$-theory. We say that $T'$ contains $T$ if $\mathcal{L} \subseteq \mathcal{L}'$ and for all $T \subseteq T'$. If $T$ and $T'$ are axiomatised by $T_0$ and $T_0'$, respectively, then we also require that $T_0 \subseteq T_0'$.

If $T'$ contains $T$, then we can also pull $T'$-models back along the inclusion $\mathcal{L} \subseteq \mathcal{L}'$.

**Lemma 13.6**

Suppose that $i$ is an inclusion $\mathcal{L} \subseteq \mathcal{L}'$ and that $T \subseteq T'$. Then the pullback along $i$ restricts to a map between the collections of models for $T'$ and $T$:

$$i^* \colon T'\text{-}\mathbf{Mod} \to T\text{-}\mathbf{Mod}$$

*Proof.* Let $\mathcal{M} \in T'$-**Mod** be a model for $T'$ and show that $i^*\mathcal{M}$ is a model for $T$. This means that we have to show that $[\![\varphi]\!]_{i^*\mathcal{M}}(\varepsilon) = 1$ for all $\varphi \in T$. By the remark after lemma 13.4 and because $\mathcal{M}$ being a $T'$-model, we have

$$[\![\varphi]\!]_{i^*\mathcal{M}}(\varepsilon) = [\![\varphi]\!]_{\mathcal{M}}(\varepsilon) = 1$$

Thus, $i^*\mathcal{M} \in T$-**Mod**, as required. □

## 13.2. Primitive Recursive Arithmetic

We will state Gödel's incompleteness theorems for any theory that allows us to encode formulas and provability. What we need for this is some arithmetic and a very basic form of induction. These form together what is called **primitive recursive arithmetic** (**PRA**), which we will define now. The signature $\mathcal{L}_{\mathrm{PRA}}$ of PRA has no predicate symbols and its functions symbol $\mathcal{F}_{\mathrm{PRA}}$ and arities are given by the following rules, which follows those in definition 12.17 but signal that we are working with symbols by underlining them.

$$\frac{}{\underline{\mathrm{S}} \in (\mathcal{F}_{\mathrm{PRA}})_1} \qquad \frac{}{\underline{\mathrm{Z}}^n \in (\mathcal{F}_{\mathrm{PRA}})_n} \qquad \frac{1 \leq k \leq n}{\underline{\Pi}^n_k \in (\mathcal{F}_{\mathrm{PRA}})_n}$$

$$\frac{g \in (\mathcal{F}_{\mathrm{PRA}})_n \qquad h \in (\mathcal{F}_{\mathrm{PRA}})_{n+2}}{\underline{\mathrm{I}}^{n+1}(g,h) \in (\mathcal{F}_{\mathrm{PRA}})_{n+1}}$$

$$\frac{g \in (\mathcal{F}_{\mathrm{PRA}})_n \qquad h_1 \in (\mathcal{F}_{\mathrm{PRA}})_m \qquad \cdots \qquad h_n \in (\mathcal{F}_{\mathrm{PRA}})_m}{\underline{\mathrm{C}}^m_n(g, h_1, \cdots, h_n) \in (\mathcal{F}_{\mathrm{PRA}})_m}$$

To improve readability, we abbreviate $\underline{\mathrm{Z}}^0$ to $\underline{0}$, and for $n \in \mathbb{N}$ we write $\underline{n}$ for

$$\underbrace{\underline{\mathrm{S}}(\cdots \underline{\mathrm{S}}}_{n \text{ times}}(\underline{0}))$$

Moreover, we will also denote sequences of variables $x_1, \dots, x_n$ to by the vector notation $\vec{x}$. If we use vectors in quantifiers, as in $\forall \vec{x}.\, \varphi$, then this abbreviates the sequence of quantifiers $\forall x_1.\, \cdots \forall x_n.\, \varphi$.

The theory $T_{\mathrm{PRA}}$ is the set of all formulas that are provable from instances of the following closed formulas.

Z) $\forall \vec{x}.\, \underline{\mathrm{Z}}^n(\vec{x}) \doteq \underline{0}$

P) $\forall \vec{x}.\, \underline{\Pi}^n_k(\vec{x}) \doteq x_k$

C) $\forall \vec{x}.\, \underline{\mathrm{C}}^m_n(g, h_1, \dots, h_n)(\vec{x}) \doteq g(h_1(\vec{x}), \dots, h_n(\vec{x}))$

$\mathrm{I}_0$) $\forall \vec{x}.\, \underline{\mathrm{I}}^{n+1}(g,h)(\underline{0}, \vec{x}) \doteq g(\vec{x})$

$\mathrm{I}_s$) $\forall \vec{x}.\, \forall y.\, \underline{\mathrm{I}}^{n+1}(g,h)(\underline{\mathrm{S}}(y), \vec{x}) \doteq h(\underline{\mathrm{I}}^{n+1}(g,h)(y, \vec{x}), y, \vec{x})$

S+) $\forall x.\, \neg(\underline{\mathrm{S}}(x) \doteq \underline{0})$

Inj) $\forall x.\, \forall y.\, \underline{\mathrm{S}}(x) \doteq \underline{\mathrm{S}}(y) \to x \doteq y$

$\mathrm{Ind}^-$) For $\varphi$ quantifier-free and $\mathrm{fv}(\varphi) = \{y_1, \dots, y_n, x\}$

$$\forall \vec{y}.\, \varphi\,[x := \underline{0}] \to \big(\forall x.\, \varphi \to \varphi\,[x := \underline{\mathrm{S}}(x)]\big) \to \forall x.\, \varphi$$

Note that this yields an axiomatisation because we can enumerate all terms over $\mathcal{L}_{\mathrm{PRA}}$ and possible instances of the formulas above.

The first 5 equations in this axiomatisation are the one we know already from primitive recursive functions. Axioms S+ and Inj assert together that repeated applications of the successor yield different elements, see exercise 40. The last axiom $\mathrm{Ind}^-$ is induction but limited to quantifier-free formulas, which means that $\varphi$ can only use propositional connectives and equality. This could be strengthened to allow for arbitrary formulas, but then we get a theory called Peano arithmetic that can prove more theorems and makes our coming results less widely applicable.

However, PRA is already strong enough to prove standard arithmetic facts. In what follows, we will write $\mathrm{PRA} \vdash \varphi$ is $\varphi$ is derivable from some finite subset of $T_{\mathrm{PRA}}$ in $\mathbf{cND}_1^=$. This notation makes sense by compactness. Moreover, we will indicate the axiom name in a step in Fitch-style proofs, whenever we use the corresponding axiom from $T_{\mathrm{PRA}}$. Finally, for a formula $\varphi$ with $x \in \mathrm{fv}(\varphi)$ and a term $t$, we will abbreviate the substitution of $t$ for $x$ in $\varphi$ by

$$\varphi(t) \stackrel{\text{def}}{=} \varphi\,[x := t] \tag{13.7}$$

**Example 13.8**

Recall from example 12.21 that that addition is primitive recursive. We use the same way of realising it as function symbol $\mathcal{L}_{\mathrm{PRA}}$ as we did there:

$$\underline{p} = \underline{\mathrm{I}}^2(\underline{\Pi}_1^1, \underline{\mathrm{C}}_1^3(S, \underline{\Pi}_1^3))$$

Using only the axioms $\mathrm{I}_s$, C and P together with (Repl) and some rules for logical connectives, one can easily prove the equations that we already know very well:

$$\mathrm{PRA} \vdash \forall y.\, \underline{p}(\underline{0}, y) \doteq y \tag{13.9}$$

$$\mathrm{PRA} \vdash \forall x.\, \forall y.\, \underline{p}(\underline{\mathrm{S}}(x), y) \doteq \underline{\mathrm{S}}(\underline{p}(x, y) \tag{13.10}$$

We can then prove $\mathrm{PRA} \vdash \forall x.\, \underline{p}(x, \underline{0}) \doteq x$ by using induction. To this end, let $\varphi$ be $\underline{p}(x, \underline{0}) \doteq x$, which is quantifier free and has only $x$ as free variable. This allows us to use the $\mathrm{Ind}^-$-axiom on $\varphi$, but we should note that the first quantifier over $\vec{y}$ goes away because there are no other variables around. Hence, we use the following instance of $\mathrm{Ind}^-$.

$$\varphi(\underline{0}) \to (\forall x.\, \varphi(x) \to \varphi(\underline{\mathrm{S}}(x))) \to \forall x.\, \varphi$$

| | | |
|---|---|---|
| 1 | $\underline{p}(\underline{0}, \underline{0}) \doteq \underline{0}$ | eq. (13.9) |
| 2 | $x$ \| $\underline{p}(x, \underline{0}) \doteq x$ | |
| 3 | \| $\underline{p}(\underline{\mathrm{S}}(x), \underline{0}) \doteq \underline{\mathrm{S}}(\underline{p}(x, \underline{0}))$ | eq. (13.10) |
| 4 | \| $\underline{p}(\underline{\mathrm{S}}(x), \underline{0}) \doteq \underline{\mathrm{S}}(x)$ | =E, 3, 2 |
| 5 | $\underline{p}(x, \underline{0}) \doteq x \to \underline{p}(\underline{\mathrm{S}}(x), \underline{0}) \doteq \underline{\mathrm{S}}(x)$ | →I, 2–4 |
| 6 | $\forall x.\, \varphi(x) \to \varphi(\underline{\mathrm{S}}(x))$ | ∀I, 2–5 |
| 7 | $\varphi(\underline{0}) \to (\forall x.\, \varphi(x) \to \varphi(\underline{\mathrm{S}}(x))) \to \forall x.\, \varphi$ | $\mathrm{Ind}^{-}$ |
| 8 | $(\forall x.\, \varphi(x) \to \varphi(\underline{\mathrm{S}}(x))) \to \forall x.\, \varphi$ | →E, 7, 1 |
| 9 | $\forall x.\, \underline{p}(x, \underline{0}) \doteq x$ | →E, 8, 6 |

## 13.3. Diagonalisation and Incompleteness

In what follows, we assume that we are given a signature $\mathcal{L}$ and a theory $T$, such that $\mathcal{L}_{\mathrm{PRA}} \subseteq \mathcal{L}$ and $T_{\mathrm{PRA}} \subseteq T$. By lemma 13.6 we do not have to worry about the semantics of formulas, as they are preserved in the pullback of any model of $T$ to one for $T'$. One could generalise this to allow more general maps, but that would make the coming statements unnecessarily complex.

The following theorem shows that all terms and formulas can be encoded in primitive recursive arithmetic. This coding follows essentially the same ideas that we used to represent trees as numbers in theorem 12.6, except that the involved computations have to presented as primitive recursive functions. Details on how this is done can be found in the chapter of Smoryński [Smo77], which we follow closely in the presentation of the theorems and proofs.

**Theorem 13.11**

For all $\mathcal{L}$-terms $t$ and $\mathcal{L}$-formulas $\varphi$, there are closed $\mathcal{L}_{\mathrm{PRA}}$-terms $\ulcorner t \urcorner$ and $\ulcorner \varphi \urcorner$, called the **codes** of $t$ and $\varphi$. Moreover, there is a $\mathcal{L}_{\mathrm{PRA}}$-formula $\mathrm{Pr}_T(x)$ with one free variable $x$, such that the following derivability conditions hold.

$$T \vdash \varphi \quad \text{implies} \quad \mathrm{PRA} \vdash \mathrm{Pr}_T(\ulcorner \varphi \urcorner) \tag{D1}$$

$$T \vdash \mathrm{Pr}_T(\ulcorner \varphi \urcorner) \to \mathrm{Pr}_T(\ulcorner \mathrm{Pr}_T(\ulcorner \varphi \urcorner) \urcorner) \tag{D2}$$

$$T \vdash \mathrm{Pr}_T(\ulcorner \varphi \urcorner) \to \mathrm{Pr}_T(\ulcorner \varphi \to \psi \urcorner) \to \mathrm{Pr}_T(\ulcorner \psi \urcorner) \tag{D3}$$

Finally, there is a $\mathcal{L}_{\text{PRA}}$-term $\text{sub}(x, y)$, such that whenever $\text{fv}(\varphi) = \{x\}$ there is a derivation of

$$\text{PRA} \vdash \text{sub}(\ulcorner\varphi\urcorner, \ulcorner t\urcorner) \doteq \ulcorner\varphi(t)\urcorner$$

The formula $\text{Pr}_T$ is called the **provability-predicate**, while the term $\text{sub}(x, y)$ internalises substitution into primitive recursive arithmetic. To better comprehend the derivability conditions, let us introduce the abbreviation

$$\Box\varphi \stackrel{\text{def}}{=} \text{Pr}_T(\ulcorner\varphi\urcorner)$$

and read $\Box\varphi$ informally as "*we* know that $\varphi$ is provable in $T$". The derivability conditions take then the following, perhaps easier to digest, form.

$$T \vdash \varphi \quad \text{implies} \quad \text{PRA} \vdash \Box\varphi \tag{D1}$$

$$T \vdash \Box\varphi \to \Box\Box\varphi \tag{D2}$$

$$T \vdash \Box\varphi \to \Box(\varphi \to \psi) \to \Box\psi \tag{D3}$$

We can read D1 as "if $T$ proves $\varphi$, then there is a proof in PRA that we know this"; D2 as "$T$ proves that if it knows that $\varphi$ is provable, then it also knows that it knows that $\varphi$ is provable"; and D3 as "if $T$ knows that $\varphi$ and $\varphi \to \psi$ are provable, then it also knows that $\psi$ is provable".

The key to proving incompleteness is the construction of a formula that states that it *itself* not provable in $T$. Such a formula can be constructed as a solution to bi-implication, which can be thought of as equation between formulas, by a technique called **diagonalisation**. This technique appears, for example, in proofs that not all functions $\mathbb{N} \to \mathbb{N}$ are computable, that $\mathbb{R}$ has more elements than $\mathbb{N}$ and many other impossibility proofs.

**Theorem 13.12: Diagonalisation**

Let $\varphi$ be a $\mathcal{L}_{\text{PRA}}$-formula with $\text{fv}(\varphi) = \{x\}$. There is a closed formula $\psi$ and a derivation for

$$\text{PRA} \vdash \psi \leftrightarrow \varphi(\ulcorner\psi\urcorner) \tag{13.13}$$

*Proof.* We define the diagonalisation $\delta$ of $\varphi$ by $\delta = \varphi(\text{sub}(x, x))$. Let then $t = \ulcorner\delta\urcorner$ and $\psi = \delta(t)$. It remains to show eq. (13.13).

By definition

$$\psi = \delta(t) = \varphi(\text{sub}(t, t)) = \varphi(\text{sub}(\ulcorner\delta\urcorner, t))$$

and, using the property of sub and replacement (see exercise 61),

$$\mathrm{PRA} \vdash \varphi(\mathrm{sub}(\ulcorner\delta\urcorner, t)) \leftrightarrow \varphi(\ulcorner\delta(t)\urcorner)\,.$$

Finally, $\varphi(\ulcorner\delta(t)\urcorner) = \varphi(\ulcorner\psi\urcorner)$ by definition and we thus obtain

$$\mathrm{PRA} \vdash \psi \leftrightarrow \varphi(\ulcorner\psi\urcorner)$$

as required. □

Recall that $T$ is consistent if it has a model or, equivalently, if $T \vdash \bot$ is not derivable. We call $T$ **$\omega$-consistent** if for all $\varphi$ with $\mathrm{fv}(\varphi) = \{x\}$, whenever $T \vdash \neg\varphi(\underline{n})$ for all $n \in \mathbb{N}$, then $T \vdash \exists x.\, \neg\varphi$ is *not* derivable.

**Theorem 13.14: First Incompleteness Theorem**

Suppose that $T$ is consistent and let $\psi$ be the diagonalisation of $\neg\mathrm{Pr}_T(x)$:

$$\mathrm{PRA} \vdash \psi \leftrightarrow \neg\mathrm{Pr}_T(\ulcorner\psi\urcorner)$$

Then we have

1. $T \vdash \psi$ is *not* derivable, and
2. if $T$ is $\omega$-consistent, then also $T \vdash \neg\psi$ is *not* derivable.

*Proof.* To prove the first statement, suppose that $T \vdash \psi$ was derivable. The derivability condition D1 implies that in this case also $\mathrm{PRA} \vdash \mathrm{Pr}_T(\ulcorner\psi\urcorner)$ is derivable. However, by definition of $\psi$, also $\mathrm{PRA} \vdash \mathrm{Pr}_T(\ulcorner\psi\urcorner)$ is derivable. These two together allow us to derive $\bot$, which contradicts consistency of $T$. Thus, $T \vdash \psi$ cannot be derivable.

For the second part, assume that $T$ is $\omega$-consistent and that $T \vdash \neg\psi$ is derivable. By definition of $\psi$, we have $\mathrm{PRA} \vdash \neg\psi \leftrightarrow \neg\neg\mathrm{Pr}_T(\ulcorner\psi\urcorner)$. Thus, we obtain from the assumption $\mathrm{PRA} \vdash \neg\neg\mathrm{Pr}_T(\ulcorner\psi\urcorner)$ and, by using double negation elimination, $\mathrm{PRA} \vdash \mathrm{Pr}_T(\ulcorner\psi\urcorner)$. $\omega$-Consistency yields the reverse of D1, see [Bus98, p. 119], for $\psi$, which means that we obtain $T \vdash \psi$. This, however, contradicts the consistency of $T$ because $\psi$ asserts that it is not provable in $T$ and thus $T \vdash \neg\psi$ cannot be derivable. □

Recall from definition 12.9 that $T$ is complete, if for all $\varphi$ either $T \vdash \varphi$ or $T \vdash \neg\varphi$ holds. Theorem 13.14 shows that there is a formula that is not derivable if $T$ is consistent. Thus $T$ is incomplete. Note that this is different from, and does not contradict, the completeness of $\mathbf{cND}_1^=$!

Theorem 13.14 provides us with *some* formula that is not provable, but not necessarily one that is practically relevant. However, it turns out that $T$ cannot prove important mathematical statements, such that $T$ is consistent.

**Theorem 13.15: Second Incompleteness Theorem**

If $T$ is consistent, then $T \vdash \neg \mathrm{Pr}_T(\ulcorner \bot \urcorner)$ is *not* derivable.

*Proof.* In order to make the proof more transparent, let us use the notation introduced after theorem 13.11. This means that we have to show that $\neg\Box\bot$ is not derivable. Let $\psi$ be given by PRA $\vdash \psi \leftrightarrow \neg\Box\psi$, as in theorem 13.14. We show that PRA $\vdash \psi \leftrightarrow \neg\Box\bot$, from which it follows that $\neg\Box\bot$ cannot be derivable by theorem 13.14 because $\psi$ is not derivable. Using introduction of bi-implication, we have to prove both directions separately. The proof of $T \vdash \neg\Box\bot \to \psi$ can be found in fig. 13.1. We leave the proof of the other direction as an exercise. □

## Further Reading

- $\omega$-consistency is not a very natural condition, but it gives a simple proof. However, it can be dispensed with by a result of Rosser, see [Bus98, Sec. II.2.2.2].
- $\omega$-consistency can also be stated as formulas and then the first incompleteness theorem can proven entirely in $\mathbf{cND}_1$, see [Smo77, Sec. 4.2-4.3].
- Buss [Bus98, Sec. II.2]
- Smith [Smi13]
- Kirst and Peters [KP23]
- Provability logic [AB05; SV82; Sol76]
- Systems that can prove their own completeness [Vis82] (extensions of Heyting arithmetic [TvD88a])
- In section 13.1, we have seen first germs of so-called category theory [Awo10; Mac98; Rie16]. In particular, one could phrase lemma 13.3 as Term and $\mathrm{Form}^{=}$ being *functors* from a category of signatures and signature maps. The pullback could be formulated as functor, but also as fibration [Jac99; Str23] of models.

# Exercises

## Exercise 54

Prove lemma 13.3 by induction on terms and formulas, respectively.

## Exercise 55

Prove lemma 13.4 by induction on terms and formulas, respectively.

## Exercise 56

Prove that pullbacks preserve identities and compositions in the following sense.

a) Let $\mathrm{id}_{\mathcal{L}} \colon \mathcal{L} \to \mathcal{L}$ be given by $\mathrm{id}_{\mathcal{L}}(s) = s$ for all $s \in \mathcal{F} \cup \mathcal{R}$. Show that $\mathrm{id}_{\mathcal{L}}^* = \mathrm{id}_{\mathcal{L}\text{-}\mathbf{Mod}}$, where $\mathrm{id}_{\mathcal{L}\text{-}\mathbf{Mod}}$ is the identity map on $\mathcal{L}$-**Mod**.

b) Let $h_k \colon \mathcal{L}_k \to \mathcal{L}_{k+1}$ be signature maps for $k \in \{1, 2\}$. The composition $h_2 \circ h_1$ yields a signature map $\mathcal{L}_1 \to \mathcal{L}_3$. Show that $(h_2 \circ h_1)^* = h_1^* \circ h_2^*$.

## Exercise 57

Let $g \colon \mathcal{M}_1 \to \mathcal{M}_2$ be a $\mathcal{L}'$-model morphism and $h \colon \mathcal{L} \to \mathcal{L}'$ a signature morphism. Show that there is a model morphism $h^* g \colon h^* \mathcal{M}_1 \to h^* \mathcal{M}_2$.

## Exercise 58

Given two signature $\mathcal{L}_1$ and $\mathcal{L}_2$, show that there is signature coproduct $\mathcal{L}_1 \oplus \mathcal{L}_2$ with signature maps $\mathcal{L}_k \to \mathcal{L}_1 \oplus \mathcal{L}_2$ by using the universal property of disjoint unions (lemma B.2).

## Exercise 59

Prove the sequents in eqs. (13.9) and (13.10).

## Exercise 60

Let $\mathcal{L}_{\text{PRA}} \subseteq \mathcal{L}$ and $T_{\text{PRA}} \subseteq T$. Assume that $T \vdash \text{Pr}_T(\ulcorner \bot \urcorner)$ gives a proof for $T \vdash \bot$, and prove that $T \vdash \text{Pr}\, T(\ulcorner \neg\varphi \urcorner) \to \neg \text{Pr}\, T(\ulcorner \varphi \urcorner)$ is derivable for all $\mathcal{L}$-formulas $\varphi$ in $\mathbf{cND}_1^=$. (Use theorem 13.11)

## Exercise 61

Prove that the following rule is admissible in $\mathbf{ND}_1^=$ (and thus also $\mathbf{cND}_1^=$), where $\varphi(s)$ substituted $s$ in $\varphi(x)$ for $x$, see eq. (13.7).

$$\frac{\Delta \mid \Gamma \vdash s \doteq t}{\Delta \mid \Gamma \vdash \varphi(s) \leftrightarrow \varphi(t)}$$

*Used in theorem 13.12.*

## Exercise 62

Complete the proof of theorem 13.15 by showing that $T \vdash \psi \to \neg\Box\bot$ follows from the derivability conditions for the formula $\psi$ from theorem 13.14.

| | | |
|---|---|---|
| 1 | $\psi \to \neg\Box\psi$ | def. $\psi$ and ($\wedge$E$_1$) |
| 2 | $\neg\Box\psi \to \psi$ | def. $\psi$ and ($\wedge$E$_2$) |
| 3 | $\quad \Box\psi$ | |
| 4 | $\quad \Box\psi \to \Box\Box\psi$ | D2 |
| 5 | $\quad \Box\Box\psi$ | $\to$E, 4, 3 |
| 6 | $\quad \Box\psi \to \neg\psi$ | example 2.8 and DNE, 1 |
| 7 | $\quad \Box(\Box\psi \to \neg\psi)$ | D1, 6 |
| 8 | $\quad \Box\Box\psi \to \Box(\Box\psi \to \neg\psi) \to \Box\neg\psi$ | D3 |
| 9 | $\quad \Box\neg\psi$ | $\to$E, 8, 5, 7 |
| 10 | $\quad \Box(\psi \to \neg\psi \to \psi \wedge \neg\psi)$ | exercise 9.e) and D1 |
| 11 | $\quad \Box\psi \to \Box(\psi \to \neg\psi \to \psi \wedge \neg\psi)$ | |
| | $\quad\quad \to \Box(\neg\psi \to \psi \wedge \neg\psi)$ | D3 |
| 12 | $\quad \Box(\neg\psi \to \psi \wedge \neg\psi)$ | $\to$E, 11, 9, 10 |
| 13 | $\quad \Box\neg\psi \to \Box(\neg\psi \to \psi \wedge \neg\psi)$ | |
| | $\quad\quad \to \Box(\psi \wedge \neg\psi)$ | D3 |
| 14 | $\quad \Box(\psi \wedge \neg\psi)$ | $\to$E, 13, 12, 9 |
| 15 | $\quad \psi \wedge \neg\psi \to \bot$ | ($\wedge$E$_1$), ($\wedge$E$_2$),($\neg$E), ($\to$I) |
| 16 | $\quad \Box(\psi \wedge \neg\psi) \to \Box(\psi \wedge \neg\psi \to \bot) \to \Box\bot$ | D3 |
| 17 | $\quad \Box\bot$ | $\to$E, 16, 14, 15 |
| 18 | $\Box\psi \to \Box\bot$ | $\to$I, 3–17 |
| 19 | $\neg\Box\bot \to \neg\Box\psi$ | example 2.8, 18 |
| 20 | $\neg\Box\bot \to \psi$ | exercise 9.d), 2, 19 |

Figure 13.1.: Proof of $T \vdash \neg\Box\bot \to \psi$ in **cND$_1$** that is used in theorem 13.15

# Appendix

# A. Greek Letters

| Letter | Capitalised | Name | Command | Command (capitalised) |
|---|---|---|---|---|
| $\alpha$ | $A$ | alpha | `\alpha` | A |
| $\beta$ | $B$ | beta | `\beta` | B |
| $\gamma$ | $\Gamma$ | gamma | `\gamma` | `\Gamma` |
| $\delta$ | $\Delta$ | delta | `\delta` | `\Delta` |
| $\varepsilon$ | $E$ | epsilon | `\epsilon` | E |
| $\zeta$ | $Z$ | zeta | `\zeta` | Z |
| $\theta$ | $\Theta$ | theta | `\theta` | `\Theta` |
| $\iota$ | $I$ | iota | `\iota` | I |
| $\kappa$ | $K$ | kappa | `\kappa` | K |
| $\lambda$ | $\Lambda$ | lambda | `\lambda` | `\Lambda` |
| $\mu$ | $M$ | mu | `\mu` | M |
| $\nu$ | $N$ | nu | `\nu` | N |
| $\xi$ | $\Xi$ | xi | `\xi` | `\Xi` |
| $o$ | $O$ | omicron | `\omicron` | O |
| $\pi$ | $\Pi$ | pi | `\pi` | `\Pi` |
| $\rho$ | $\mathrm{P}$ | rho | `\rho` | `\Rho` |
| $\sigma$ | $\Sigma$ | sigma | `\sigma` | `\Sigma` |
| $\tau$ | $T$ | tau | `\tau` | T |
| $\upsilon$ | $\Upsilon$ | upsilon | `\upsilon` | `\Upsilon` |
| $\varphi$ | $\Phi$ | phi | `\varphi` | `\Phi` |
| $\chi$ | $X$ | chi | `\chi` | X |
| $\psi$ | $\Psi$ | psi | `\psi` | `\Psi` |
| $\omega$ | $\Omega$ | omega | `\omega` | `\Omega` |

Table A.1.: Greek Letters and Their Pronunciation

You may find all Greek letters with their respective pronunciation and the LaTeX-macro to typeset them in table A.1. The capital Greek letters P and the small letter $o$ require being set up with the `mathspec` package or the following two declarations.

```
\DeclareMathSymbol{\Rho}{\mathalpha}{operators}{"50}
```

```
\DeclareMathSymbol{\omicron}{\mathord}{letters}{"6F}
```

More information on this can be found in the `symbols-a4` documentation.

Moreover, by LaTeX uses different versions of some small Greek letters. To obtain the result in table A.1, one has to add the following to the document.

```
\renewcommand*{\phi}{\varphi}
\renewcommand*{\epsilon}{\varepsilon}
\renewcommand*{\theta}{\vartheta}
\renewcommand*{\kappa}{\varkappa}
\renewcommand*{\rho}{\varrho}
```

The reason for using these is that $\varphi$, $\varepsilon$, $\theta$, $\kappa$ and $\rho$ are easier to distinguish, to write or are just more common then their respective standard LaTeX-versions $\phi$, $\epsilon$, $\theta$, $\kappa$ and $\rho$. For instance, $\kappa$, $\rho$ and $\theta$ are difficult to distinguish from $k$, $p$ and $\Theta$. Phi written as $\varphi$ can be written in one movement, while $\phi$ needs two. Epsilon is common in both forms, but $\varepsilon$ is easier to write than $\epsilon$ because it also just needs one movement, and $\epsilon$ can be easily confused with the membership relation $\in$ and the Euro symbol €.

# B. Tools

## B.1. Maps

You will be familiar with functions on real numbers, like sine, cosine and the exponential function. Many textbooks abuse notation and make statements like "consider the function $\frac{1}{x} + \sin(x)$", making the assumption that the expression is read as "map every $x$ to $\frac{1}{x} + \sin(x)$". However, not only is this confusing because the mapping process is left implicit, but the important information of the nature of $x$ is left out. We do not know whether $x$ can be 0 (it probably should not), a real number or perhaps some other kind of number.

To overcome these issues, we only have to make one small step: consider functions as mathematical objects in their own right. This leads to possibly the single most important abstraction in mathematics, whether applied or pure: that of a **map**. Underlying this abstraction is the simple idea that a map can be applied to an **input**, processes this input and returns an **output**. To make maps useful, they have to obey two rules. First,

> for *every possible input* a map has to be able to produce an output.

This makes it vital that a map is not just the process but also comes with an **input type** and an **output type**, the former is usually called the **domain** and the latter **codomain**.

$$\text{Domain} \xrightarrow{\text{input}} \boxed{\text{map}} \xrightarrow{\text{output}} \text{Codomain}$$

For instance, it makes a big difference whether we define a map with all real numbers $\mathbb{R}$ or only on the non-zero ones $\mathbb{R}_{\neq 0}$ as domain. As an example of this, consider the equation $f(x) = \frac{1}{x}$. This equation makes no sense if we wish to consider $f$ as a map with domain $\mathbb{R}$ because $1/0$ is not well-defined, but it makes perfect sense with $\mathbb{R}_{\neq 0}$ as domain. Similarly, $f$ cannot have,

say, $\mathbb{R}_{\neq 0}$ as domain and the positive real number $\mathbb{R}_{>0}$ as codomain because $1/(-5)$ is certainly not positive. To express that $f$ is a map with domain $\mathbb{R}_{\neq 0}$ and codomain $\mathbb{R}$, we use the notation $f\colon \mathbb{R}_{\neq 0} \to \mathbb{R}$. However, we need to realise that the same expression, like $\frac{1}{x}$, can be part of the definition of many different maps. For example, even though we may define maps

$$f\colon \mathbb{R}_{\neq 0} \to \mathbb{R}\,, \quad g\colon \mathbb{R}_{\neq 0} \to \mathbb{R}_{\neq 0} \quad \text{and} \quad h\colon \mathbb{R}_{>0} \to \mathbb{R}_{>0}$$

by the same expression, that is,

$$f(x) = \frac{1}{x}\,, \quad g(x) = \frac{1}{x} \quad \text{and} \quad h(x) = \frac{1}{x}$$

these are different maps because they differ in domain and codomain. We may even be as bold as defining a map $k$ that admits 0 as input by declaring it to have type $k\colon \mathbb{R} \to \mathbb{R} \cup \{\infty\}$, which means that it can output a real number or infinity and thereby enabling the following definition.

$$k(x) = \begin{cases} 1/x, & x \neq 0 \\ \infty, & x = 0 \end{cases}$$

The second defining feature of maps is that

> the output of a map has to be *deterministic*.

In our example above, we know exactly what the output of $f$ is for any particular value and we can compute, perhaps with more or less effort, a definite output. On the other hand, consider the following attempt to define a map $m\colon \mathbb{R}_{\neq 0} \to \mathbb{R}$, which may reflect laziness after the weekend.

$$m(x) = \begin{cases} x, & \text{if it is a Monday} \\ 1/x, & \text{on any other weekday} \end{cases}$$

Clearly, this definition does not yield a definite output, unless the computation for $m$ can actually determine the weekday. This can be achieved by refining the domain and adding the weekday as an input, and we can rescue this example by defining an actual map $m'\colon \mathbb{R}_{\neq 0} \times W \to \mathbb{R}$, where $W$ is the set of

weekdays {Monday, … , Sunday}, as follows.

$$m'(x, w) = \begin{cases} x, & \text{if } w = \text{Monday} \\ 1/x, & \text{otherwise} \end{cases}$$

The notation $\mathbb{R}_{\neq 0} \times W$ stands for the **product** of $R_{\neq 0}$ and $W$, which consists of pairs $(x, w)$ of non-zero real numbers and weekdays. We will look at sets and products in appendix B.2 below. Finally, let us note that the determinism of maps can be more subtle than in our example above, where there was a clear strategy to resolve the non-determinism. An example is the following definition of a "map" $p\colon \mathbb{R} \to \mathbb{R}$, where "sin" is the trigonometric sine function.

$$p(x) = \begin{cases} y, & \sin(y) = x \\ 42, & \text{otherwise} \end{cases}$$

At first sight one may say that, yes, this is a perfectly fine definition. After all, $\sin(\pi) = 0$ so $p(0) = \pi$, right? No! The problem is that also $\sin(0) = 0$, $\sin(2\pi) = 0$ etc. In other words, the definition of $p$ does not determine the output as the equation $\sin(y) = 0$ does not have a unique solution. This makes $p$ not a well-defined map.

?

Can you salvage the definition of $p$ by changing its domain?

One last word on terminology. We used everywhere above the word *map*, except that we referred to sin as the sine *function*. In this book, we follow the convention that **map** refers to the general concept and reserve **function** for the special case when a map has some set of numbers as codomain [Lan71]. For instance, the maps $f, g, h, k, m'$ from above would also be called functions. In contrast, a map $s\colon W \to W$ on weekdays would not be considered a function because its codomain is not a set of numbers. This is not an uncommon convention but the reader should be aware that other authors may use the word *function* as designation for maps in general.

Given two maps $f\colon A \to B$ and $g\colon B \to C$, we denote by $\boxed{g \circ f}$ the **composition** of $g$ and $f$ with $(g \circ f)(x) = g(f(x))$. The composition can be pronounced as "$g$ composed with $f$" or "$g$ after $f$". For every set $A$, there is also a map $\boxed{\mathrm{id}_A}$ called the **identity** and defined as $\mathrm{id}_A\colon A \to A$ with $\mathrm{id}_A(x) = x$.

Composition and identity maps together give us a very visual way of thinking in terms of **diagrams**. For instance, the map $g\colon \mathbb{R}_{\neq 0} \to \mathbb{R}_{\neq 0}$ can be composed

with itself, which can be visualised as in the following diagram.

$$\mathbb{R}_{\neq 0} \xrightarrow{g} \mathbb{R}_{\neq 0} \xrightarrow{g} \mathbb{R}_{\neq 0}$$

This diagram is in fact a graph, in which the three nodes are labelled by the set $\mathbb{R}_{\neq 0}$ and the edges are labelled by $g$. We can of course calculate for all $x \in \mathbb{R}_{\neq 0}$ that

$$(g \circ g)(x) = \frac{1}{1/x} = x = \mathrm{id}_{\mathbb{R}_{\neq 0}}(x) \tag{B.1}$$

and thus $g \circ g = \mathrm{id}_{\mathbb{R}_{\neq 0}}$. This can be expressed as the following **commutative diagram**.

$$\begin{tikzcd} \mathbb{R}_{\neq 0} \arrow[r, "g"] \arrow[dr, "\mathrm{id}_{\mathbb{R}_{\neq 0}}"'] & \mathbb{R}_{\neq 0} \arrow[d, "g"] \\ & \mathbb{R}_{\neq 0} \end{tikzcd}$$

That a diagram commutes means that, whenever we compose the maps on two parallel paths in the diagram, then the resulting maps are equal. In the above diagram there are exactly two parallel paths: $\mathbb{R}_{\neq 0} \xrightarrow{g} \mathbb{R}_{\neq 0} \xrightarrow{g} \mathbb{R}_{\neq 0}$ and $\mathbb{R}_{\neq 0} \xrightarrow{\mathrm{id}_{\mathbb{R}_{\neq 0}}} \mathbb{R}_{\neq 0}$. Composing the maps along those paths results in the maps $g \circ g$ and $\mathrm{id}_{\mathbb{R}_{\neq 0}}$. That the diagram commutes means thus that $g \circ g = \mathrm{id}_{\mathbb{R}_{\neq 0}}$, which is precisely what we proved with eq. (B.1).

## B.2. Sets

We suppose that the reader has a basic understanding of sets. The purpose of this section is mostly to establish the notation and terminology that are used in this book.

### Set Products and Mapping Sets

Given sets $A$ and $B$, we denote by $\boxed{A \times B}$ their **product** consisting of ordered pairs

$$A \times B = \{(a, b) \mid a \in A, b \in B\}$$

and $\boxed{[A, B]}$ the **set of maps** $A \to B$. The product comes with two **projection** maps $p_1 \colon A \times B \to A$ and $p_2 \colon A \times B \to B$ defined, respectively, by $p_1(a, b) = a$ and $p_2(a, b) = b$. Moreover, we can create maps into the product, if we have maps into each component: let $f_1 \colon C \to A$ and $f_2 \colon C \to B$ be maps, we define the **pairing** $\boxed{\langle f_1, f_2 \rangle}$ of $f_1$ and $f_2$ by

$$\langle f_1, f_2 \rangle(c) = (f_1(c), f_2(c))$$

as in the following diagram.

$$\begin{tikzcd}
 & C \arrow[dl, "f_1"'] \arrow[dr, "f_2"] \arrow[dd, dashed, "\langle f_1, f_2 \rangle"] & \\
A & & B \\
 & A \times B \arrow[ul, "p_1"] \arrow[ur, "p_2"'] &
\end{tikzcd}$$

This diagram **commutes**, which means that all ways possibilities of composite maps between the same are the same. In this diagram, this means concretely that

$$p_1 \circ \langle f_1, f_2 \rangle = f_1 \quad \text{and} \quad p_2 \circ \langle f_1, f_2 \rangle = f_2 \,.$$

There are no other possibilities of composing maps in the diagram that would yield two (differently named) maps that could be compared. More is true about the pairing: given any other map $g \colon C \to A \times B$ with $p_1 \circ g = f_1$ and $p_2 \circ g = f_2$, we have

$$g(c) = (p_1(g(c)), p_2(g(c))) = (f_1(c), f_2(c)) = \langle f_1, f_2 \rangle(c) \,,$$

which means that $g = \langle f_1, f_2 \rangle$ and the pairing is the unique map that makes the above diagram commute!

## Disjoint Union

Frequently, we need to put two sets together without any interference between them, that is, *disjointly*. This happens, for instance, in the construction of first-order logic models. For instance in example 10.5, we view databases as relations on a set that consists of natural numbers, ASCII string and an error element err. In order to ensure that these do not collide with each other, we need to make sure that in the resulting set, err is neither a number nor an

ASCII string. More drastically, suppose we wish to disjointly add to the natural numbers and the set $\{42\}$, which is clearly not achieved by taking the union, as we have $\mathbb{N} \cup \{42\} = \mathbb{N}$. Instead, we define the **disjoint union** or **coproduct** $\boxed{A + B}$ of two sets $A$ and $B$ by

$$A + B = (\{1\} \times A) \cup (\{2\} \times B)\,.$$

We then have

$$\mathbb{N} + \{42\} = \{(1, n) \mid n \in \mathbb{N}\} \cup \{(2, 42)\}\,,$$

which is not equal to $\mathbb{N}$.

Unfortunately, this definition of the disjoint union is somewhat unwieldy. Instead, we will usually view $A$ and $B$ as subsets of $A + B$, even though formally there are only maps $i_1 \colon A \to A + B$ and $i_2 \colon B \to A + B$ given by $i_k(x) = (k, x)$. These maps are injective, which means that they are almost like subset inclusions. Further, we work with $A + B$ by making case distinctions: In order to define a map $f \colon A + B \to C$, we would say "if $x \in A$, then define $f(x)$ by [...], otherwise define it by [...]". For instance, we may want to define $f \colon \mathbb{N} + \{42\} \to \mathbb{N}$ as follows.

$$f(x) = \begin{cases} x + 1, & x \in \mathbb{N} \\ 0, & x \in \{42\} \end{cases}$$

This approach is formally justified by the following result.

**Lemma B.2: Universal property of disjoint union**

Let $A$ and $B$ be sets. For all sets $C$ and maps $f_1 \colon A \to C$ and $f_2 \colon B \to C$, there is a unique map $f \colon A + B \to C$ with $f \circ i_k = f_k$, as in the following commuting diagram.

$$\begin{tikzcd}
 & A + B \arrow[dd, dashed, "f"] & \\
A \arrow[ur, "i_1"] \arrow[dr, "f_1"'] & & B \arrow[ul, "i_2"'] \arrow[dl, "f_2"] \\
 & C &
\end{tikzcd}$$

The map $f$ result from lemma B.2 is also called **copairing** and is the formalisation of our intended case distinction.

## Powerset and Subset Operations

We will also be using the **powerset** of sets $A$, denoted by $\boxed{\mathcal{P}(A)}$, which is the set of all subsets of $A$:

$$\mathcal{P}(A) = \{U \mid U \subseteq A\}$$

The elements of $\mathcal{P}(A)$ can also be characterised by maps. Given $U \subseteq A$, we define the **characteristic function** $\boxed{\chi_U}$ with $\chi_U \colon A \to \mathbb{B}$ as follows.

$$\chi_U(x) = \begin{cases} 1, & x \in U \\ 0, & x \notin U \end{cases}$$

It is quite easy to see that the sets $[A, \mathbb{B}]$ and $\mathcal{P}(A)$ are isomorphic, see exercise 64.

Using the powerset, we can also express the image and preimage of maps. Let $f \colon A \to B$ be a map, $U \subseteq A$ and $V \subseteq B$. We denote by $f^{\to}U$, $f^{\leftarrow}V$ and $\operatorname{im} f$, respectively, the **image** of $U$ under $f$, the **preimage** of $V$ under $f$ and the **(total) image** of $f$, which are given by the following equations.

$$\begin{aligned} f^{\to}U &= \{f(x) \in B \mid x \in U\} \\ f^{\leftarrow}V &= \{x \in A \mid f(x) \in V\} \\ \operatorname{im} f &= f^{\to}A = \{f(x) \in B \mid x \in A\} \end{aligned}$$

The **empty set** is denoted by $\boxed{\emptyset}$. Given two sets $A$ and $B$, we write $\boxed{A \setminus B}$ for their **set difference**, which is defined by

$$A \setminus B = \{x \in A \mid x \notin B\}\,.$$

The set of natural numbers starting at 0 is denoted by $\mathbb{N}$. Given a natural number $n \in \mathbb{N}$, we write $\boxed{[n\rangle}$ for the **set of the first $n$ natural numbers**:

$$[n\rangle = \{k \in \mathbb{N} \mid k < n\}\,.$$

In particular, we have $[0\rangle = \emptyset$, $[1\rangle = \{0\}$ etc. Generally, $[n\rangle$ has exactly $n$ elements.

### Exercise 63

Prove the universal property of the disjoint union stated in lemma B.2.

### Exercise 64

Show for any set $A$ that the sets $[A, \mathbb{B}]$ and $\mathcal{P}(A)$ are isomorphic, by showing that the map $f\colon \mathcal{P}(A) \to [A, \mathbb{B}]$ given by $f(U) = \chi_U$ has an inverse.

### Exercise 65

Show for any set $A$ that the sets $[[2\rangle, A]$ and $A \times A$ are isomorphic, that is, there is a map $[[2\rangle, A] \to A \times A$ with an inverse.

### Exercise 66

Show that there is exactly one map $\emptyset \to X$ for any set $X$.

### Exercise 67

Suppose that $X$ is a finite set with $m$ elements and let $n \in \mathbb{N}$. Show that $[[n\rangle, X]$ has $m^n$ elements.

## B.3. Families of Sets and Maps

In appendices B.1 and B.2, have considered individual sets and maps between sets. However, we often need to speak of several sets and maps between them as one mathematical object. One way to do this are **families**, which we briefly discuss now. Let $I$ be a set. By an $I$-indexed **set family** $X$, we mean a collection of sets $X_i$ for each $i \in I$. We denote such a family also by $(X_i)_{i \in I}$. Similarly, an $I$-indexed **map family** $f$ between $I$-indexed set families $X$ and $Y$, is a collection of maps $f_i\colon X_i \to Y_i$ for every $i \in I$. In this case, we either write $f\colon X \to Y$ or, if we want to be explicit about the indices, $(f_i\colon X_i \to Y_i)$.

It is important to note that every set $A$ yields the constant family $KA$, given by $(KA)_i = A$. Thus, when we say that $f$ is a family of maps $f_i \colon X_i \to A$, then we mean that it is a map family $f \colon X \to KA$.

## B.4. Induction on Natural Numbers

We denote by $\mathbb{N}$ the set of **natural numbers** starting at 0. But what is this set exactly? Naively, one could define it by saying $\mathbb{N} = \{0, 1, 2, \ldots\}$. However, making precise the meaning of the dots is quite difficult. A better way is to define the natural numbers as some set with a certain property, as follows.

**Definition B.3: Natural numbers and iteration principle**

The **natural numbers** are a set $\mathbb{N}$ with an element $0 \in \mathbb{N}$ and a map $\mathrm{suc} \colon \mathbb{N} \to \mathbb{N}$, such that for any set $A$ with an element $a_0$ and a map $f \colon A \to A$, there is a unique map $g \colon \mathbb{N} \to A$ such that $g(0) = a_0$ and $g(\mathrm{suc}(n)) = f(g(n))$ for all $n \in \mathbb{N}$. We say that $g$ is defined by iteration of $(a_0, f)$.

To make our life simple, we will write $n+1$ instead of $\mathrm{suc}(n)$ for the successor of the number $n$. We can the express any natural number by $1 = 0 + 1$, $2 = 1+1$, $3 = 2+1$ and so forth. When we wrote $\mathbb{N} = \{0, 1, 2, \ldots\}$ above, we intuitively understood that $\mathbb{N}$ should consist of exactly the numbers expressed in this way and that there should be no spurious elements like catfish $\in \mathbb{N}$. That this is the case is expressed by the iteration property because it tells us that every element in $\mathbb{N}$ is either of the form 0 or $n + 1$ for some $n \in \mathbb{N}$. This principle allows us to count from any number down to 0. One can say that

> elements of $\mathbb{N}$ are *static* representations of numbers, while iteration reflect the *dynamics* of counting.

If we consider a pair $(a_0, f)$ given as in definition B.3, then one can compare this to the following imperative program, in which a for-loop is used to repeatedly apply $f$ to the initial value $a_0$.

```
def g(n):
  res = a_0
  for i in [1, ..., n]:
    res = f(res)
  return res
```

Using the iteration principle can be a nuisance because we have to explicitly specify the pair $(a_0, f)$, which only gets worse for the more complicated structures that we encounter in, for example, chapter 4. For instance, suppose that we wish to define exponentiation in terms of multiplication. Our intuition would be to proceed with the following definition.

$$a^0 = 1 \quad \text{and} \quad a^{n+1} = a^n \cdot a \tag{B.4}$$

This definition arises by applying the iteration principle to the pair $(1, f_a)$ with $f_a(x) = x \cdot a$. However, defining exponentiation via the formal iteration principle is clearly inconvenient, and we will use the equational style in eq. (B.4) whenever possible and *we are sure that the equations can be reduced to the iteration principle.* Such a reduction is not always easy. For instance, the factorial function can be expressed by the following equations.

$$0! = 1 \quad \text{and} \quad (n+1)! = n! \cdot (n+1) \tag{B.5}$$

If we tried to use the iteration principle directly to define ! as map $\mathbb{N} \to \mathbb{N}$ via iteration, then we run into the problem that $n+1$ is not available for the multiplication. Instead, we have to define first a map $\mathrm{fac}\colon \mathbb{N} \to \mathbb{N} \times \mathbb{N}$ by iteration from $(1,1)$ and $f(k,r) = (k+1, r \cdot k)$. We then find that $\mathrm{fac}(n) = (n+1, n!)$. This shows that the factorial function can be obtained by iteration but this requires considerable overhead. Therefore, we typically prefer specifications like in eq. (B.5).

The iteration principle the natural numbers can be used to define most maps on the natural numbers that we may be interested in.[19] From the iteration principle, we can also derive the usual induction principle.[20]

**Theorem B.6: Induction principle for $\mathbb{N}$**

Let $P$ be a property of $\mathbb{N}$, where we write $P(n)$ for the property $P$ at $n \in \mathbb{N}$. To prove that $P(n)$ holds for all $n \in \mathbb{N}$, it suffices to show that

- $P(0)$ holds (**base case**), and
- for all $n \in \mathbb{N}$, assuming that $P(n)$ holds, that $P(n+1)$ holds (**induction step**).

*Proof.* The proof of the induction principle needs a concrete definition of what a property is. For simplicity, we assume that a property is a subset, that is, $P \subseteq \mathbb{N}$ where $P(n)$ means that $n \in P$. Our goal is then to show that all natural numbers are contained in $P$. In turn, we obtain this by iterating $(0, \mathrm{suc})$ to get a map $g\colon \mathbb{N} \to P$ because 0 is in $P$ and for any $n \in P$, also $n+1 \in P$.

Since $g$ is uniquely defined by $g(0) = 0$ and $g(n+1) = g(n) + 1$, we obtain that $\mathbb{N} \subseteq P$, as required. $\square$

The assumption $P(n)$ in the induction step is called the **induction hypothesis**, and it should be noted that it is only assumed for *each individual* $n$. Sometimes, the induction principle is mistakenly stated with the induction hypothesis separated as follows.

**Incorrect "induction principle"**: To prove that $P(n)$ holds for all $n \in \mathbb{N}$,

1. prove that $P(0)$ holds,
2. assume that $P(n)$ holds for all $n \in \mathbb{N}$, and
3. prove that $P(n+1)$ holds for all $n \in \mathbb{N}$.

This, however, is incorrect because the second point implies immediately the first and third, thereby allowing us to prove that any property! Clearly, we do not want this and theorem B.6 is the correct principle. In chapter 4, we will see how first order logic allows us to resolve such ambiguities of natural language.

## B.5. Formal Languages

Recall that $A^*$ denotes the set of *words* over an alphabet $A$. Concretely, the set of words is given by

$$\boxed{A^* = \{\varepsilon\} \cup \{a_0 a_1 \cdots a_n \mid n \in \mathbb{N}, a_k \in A\},}$$

where $\varepsilon$ is the empty word. For instance, if $A = \{a, b\}$, then $A^*$ contains the singleton words $a$ and $b$, and longer words like $abbaa$. The set of *languages* over $A$ is the powerset $\mathcal{P}(A^*)$, that is, the set of all subsets of $A^*$. Given two words $v, w \in A^*$, we denote by $vw$ or $v + w$ the **concatenation** of the words $v$ and $w$, that is, considering $v$ and $w$ as one word by reading their letters in order.

$$(v_0 \cdots v_n) + (w_0 \cdots w_m) = v_0 \cdots v_n w_0 \cdots w_m$$

We will often make use of **context-free grammars**, which generate languages. These grammars will be notated in so-called **Backus-Naur form**. Let us start with an example.

**Example B.7**

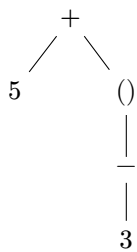
Suppose we want to define a language of arithmetic expressions, in which one can use numbers in $\mathbb{N}$, addition and negation. In this case, we would say that such expressions $e$ are generated by the following grammar.

$$e ::= n\,, n \in \mathbb{N} \mid e + e \mid -e \mid (e) \tag{B.8}$$

This grammar can be read as follows. The symbol $::=$ says that an expression $e$ can be generated by using any of the four options on the right-hand side, where the options are separated by the vertical bar. The first option is that $e$ can be any natural number $n$, which is the only case where we can start to generate an expression. To generate larger expressions, we have to use any of the other two options. For instance, if we have generated already expressions $e_1$ and $e_2$, then the second option allows us to generate the expression "$e_1 + e_2$", the third option gives us "$-e_1$" and the fourth introduces parentheses "$(e_1)$". It is important to realise that "$+$" and "$-$" have no meaning, they are just syntax. In the terminology of (context-free) grammars, "(", ")", "$+$", "$-$" and "$n$" are called **terminal symbols**, while $e$ in the grammar is called a **non-terminal symbol**.
We can now define the language generated by the grammar, call it $L$, as a subset of all words over the alphabet $A = \mathbb{N} \cup \{+, -, (, )\}$ by

$$E = \{e \in A^* \mid \text{ generated by eq. (B.8)}\}\,.$$

But what does "generated by" mean exactly? We can think of eq. (B.8) as a way of describing trees of a certain shape. For instance, the expression $5 + (-3)$ can be seen as a linear, textual description of the following tree.




In section 9.1, we have already seen how to describe such trees. The labels

are numbers and the operators, that is, we put

$$L = \mathbb{N} \cup \{+, -, ()\}$$

and the branching width $B$ is given by

$$B_n = \emptyset \qquad B_+ = [2\rangle \qquad B_- = [1\rangle \qquad B_{()} = [1\rangle\,,$$

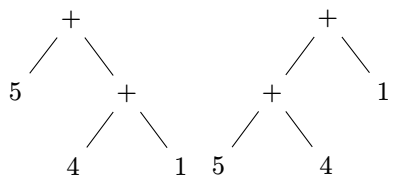
which indicates that the numbers are leaves, "+" has two children, and "−" and "()" have one. An expression can be seen as a tree of branching type $(L, B)$. To get a language, we define a map flat: $\mathcal{T}(L, B) \to A^*$ by iteration of the family $\{f_x\}_{x \in L} : [B_x, A^*] \to A^*$ defined by

$$\begin{aligned}
f_n(\alpha) &= n \\
f_+(\alpha) &= \alpha(0) \mathbin{+\!\!+} \text{"+"} \mathbin{+\!\!+} \alpha(1) \\
f_+(\alpha) &= \text{"}-\text{"} \mathbin{+\!\!+} \alpha(0) \\
f_{()}(\alpha) &= \text{"("} \mathbin{+\!\!+} \alpha(0) \mathbin{+\!\!+} \text{")"}
\end{aligned}$$

This gives us that expressions are given as the image of the map flat:

$$E = \{\text{flat}(t) \mid t \in \mathcal{T}(L, B)\}$$

There is something peculiar about trees compared to expressions: The latter need parentheses to disambiguate, as we do not know how to generate the word $5 + 4 + 1$ with our grammar and there are two different trees that flat maps to this word:




To resolve this ambiguity, we normally denote these expressions by, respectively, $5 + (4 + 1)$ and $(5 + 4) + 1$. This tells us that trees do not need the parentheses and all ambiguity is removed. In fact, we can see this already in the branching type, in which the parentheses do not add any branching and merely reflect the parentheses in the grammar. Often parentheses can be left out by constructing a grammar more cleverly than what we did, but we leave that for our specific uses of grammars.

**Definition B.9**

Let $A$ be an alphabet (a set). A **context-free grammar** $G$ over $A$ is a tuple $(V, R)$ where $V$ is a finite set of non-terminal symbols and $R \subseteq V \times (A \cup V)$ is a relation, the **production rules** of $G$.

**Example B.10**

Taking $V = \{e\}$ and

$$R = \{(e, n) \mid n \in \mathbb{N}\} \cup \{(e, e + e)\} \cup \{(e, -e)\} \cup \{(e, (e))\}$$

is the grammar given in example B.7.

# C. Three-Valued Logic

In what follows, we describe the so-called 3-valued Heyting logic or algebra. Let $\mathbb{T}$ be the set $\{0, X, 1\}$. Intuitively, we understand 0 and 1 as true and false, as in chapter 3, while the third element $X$ of $\mathbb{T}$ should be seen as an unknown truth value. This can occur, for example, in computer when the voltage of a logical signal is not high enough or fluctuates and thereby creates an undefined logic state. We will see that $\mathbb{T}$ can be used a domain for modelling propositional logic. First of all, we define an order $\sqsubseteq$ on $\mathbb{T}$ by

$$0 \sqsubseteq X\,, \quad X \sqsubseteq 1\,, \quad 0 \sqsubseteq 1\,, \quad 0 \sqsubseteq 0\,, \quad X \sqsubseteq X \quad \text{and} \quad 1 \sqsubseteq 1\,.$$

This order allows us to use min and max as usual, and if we use them to interpret conjunction and disjunction, then we will see that they conform to our expectation of an unknown value as input to logic gates:

$\min(a, b)$ (interpretation of $\wedge$) and $\max(a, b)$ (interpretation of $\vee$)

| $a$ \ $b$ | 0 | X | 1 |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| X | 0 | X | X |
| 1 | 0 | X | 1 |

| $a$ \ $b$ | 0 | X | 1 |
|---|---|---|---|
| 0 | 0 | X | 1 |
| X | X | X | 1 |
| 1 | 1 | 1 | 1 |

We can also define a semantic implication $\Longrightarrow_{\mathbb{T}}$ on $\mathbb{T}$ just as we did for the Boolean semantics:

$$a \Longrightarrow_{\mathbb{T}} b = \begin{cases} 1, & a \sqsubseteq b \\ b, & \text{otherwise} \end{cases}$$

The following table lists all the possibilities for $\Longrightarrow_{Tri}$ and the resulting negation:

Tables of $a \Longrightarrow_{\mathbb{T}} b$ and negation of $a$

| $a$ \ $b$ | 0 | X | 1 |
|---|---|---|---|
| 0 | 1 | 1 | 1 |
| X | 0 | 1 | 1 |
| 1 | 0 | X | 1 |

| a | $a \Longrightarrow_{\mathbb{T}} 0$ |
|---|---|
| 0 | 1 |
| X | 0 |
| 1 | 0 |

Putting this all together, we can define for valuations $v\colon \mathrm{PVar} \to \mathbb{T}$ a map $[\![-]\!]_v^{\mathbb{T}}\colon \mathrm{PForm} \to \mathbb{T}$ analogously to definition 3.6. We can use this map to also gives us an entailment relation $\vDash_{\mathbb{T}}$ by defining

$$\Gamma \vDash_{\mathbb{T}} \varphi \text{ if for all v we have } [\![\Gamma]\!]_v^{\mathbb{T}} \leq [\![\varphi]\!]_v^{\mathbb{T}} .$$

Also similarly to the Boolean model (theorem 3.13), one can prove the following soundness result.

**Theorem C.1**

If $\Gamma \vdash \varphi$ is derivable in **ND**, then $\Gamma \vDash_{\mathbb{T}} \varphi$.

However, the similarity with the Boolean model stops when we move to classical logic. Indeed, it is easy to see that $\nvDash_{\mathbb{T}} p \vee \neg p$. Let $v$ be the valuation that is equal to $X$ everywhere. We then have

$$[\![p \vee \neg p]\!]_v = \max\{[\![p]\!]_v, [\![\neg p]\!]_v\} = \max\{X, 0\} = X \neq 1 .$$

This shows that $\neg \vDash_{\mathbb{T}} p \vee \neg p$ and therefore the (Contra)-rule from definition 3.16 cannot be sound for this three-valued model.

There are other possibilities for interpreting the implication, see Łukasiewicz's or Kleene's three-valued logic [Kle74, § 64], but different proof systems than **ND** and **cND** are needed to handle those interpretations.

# D. De Bruijn Trees

This appendix presents an alternative to the approach in section 5.2 for handling variables, binding and substitution. The present approach is very suited to computerised implementations and is commonly used in proof checkers and proof assistants. It is based on representing terms and formulas as *de Bruijn trees*. The idea is that bound variables are represented by a number that points to the quantifier that binds this variables. All free variables keep their name.

Figure D.1 shows the (de Bruijn-) tree representation of $\forall x.\, \forall y.\, Q(x, f(y, z))$, where the left figure shows the actual tree and the right figure indicates to which quantifier the numerical name refers. We see that the bound variables

(a) The plain tree (b) Tree with indication of the variable references

Figure D.1.: de Bruijn-Tree Representations of $\forall x.\, \forall y.\, Q(x, f(y, z))$

are numbered starting from the *inner-most* quantifier. Also note that the variable $z$ is free and thus keeps its name in the tree representation.

Since the nesting depth of quantifiers is important, we also note that tree representations are *not closed under subtrees*! For instance, the tree in fig. D.2a is not a valid tree representation because $\langle 1 \rangle$ is a dangling reference. Instead, if we want to remove the outer quantifier, we need to pick a name that we replace $\langle 1 \rangle$ with, say $x$, and then obtain the tree representation in fig. D.2b.

This tree representation addresses the issues presented in section 5.1 as follows.

(a) Invalid de Bruijn-Tree (b) Correct tree for $\forall y.\, Q(x, f(y, z))$

Figure D.2.: Attempts for representing trees for subformula $\forall y.\, Q(x, f(y, z))$ of $\forall x.\, \forall y.\, Q(x, f(y, z))$

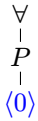



Figure D.3.: De Bruijn-Tree representing both formulas $\forall x.\, P(x)$ and $\forall y.\, P(y)$

1. The formulas $\forall x.\, P(x)$ and $\forall y.\, P(y)$ have the same tree representation, see fig. D.3.
2. Substitutions can be carried out without changing the binding of variables, see fig. D.4. Note that the *bound* variable $x$ is represented by $\langle 0 \rangle$ and thus there is no danger of substituting the *unbound* variable $x$ for the *unbound* variable $y$. Hence, the problem of accidentally binding a variable does not exist for the tree representation.

(a) Representation before substitution (b) Representation after substitution

Figure D.4.: Substitution of $x$ for the free variable $y$ in the tree of $\forall x.\, Q(x, y)$

# E. Proof Rondo Formalisations

## E.1. Train Crossing

```
# Train crossing model
1 TrainArrives → ClosingSignal
2 ¬CommunicationFailure ∧ ClosingSignal → BarrierClosed
3 ¬TrainStops ∧ TrainArrives → TrainOnCrossing
4 ¬BarrierClosed ∧ TrainArrives → TrainBrakes
5 ¬BrakeFailure ∧ TrainBrakes → TrainStops
6 TrainOnCrossing ∧ ¬BarrierClosed → Unsafe

# Closed world assumptions
7 CommunicationFailure ∨ (¬ClosingSignal) → (¬BarrierClosed)
8 BrakeFailure ∨ ¬TrainBrakes → ¬TrainStops

# Situation
9 TrainArrives
10 CommunicationFailure
11 BrakeFailure

# Proof that unsafe situation arises
12 CommunicationFailure ∨ ¬ClosingSignal : or i1 10
13 ¬BarrierClosed                        : -> e 7, 12
14 ¬BarrierClosed ∧ TrainArrives         : and i 13, 9
15 TrainBrakes                           : -> e 4, 14
16 BrakeFailure ∨ ¬TrainBrakes           : or i1 11
17 ¬TrainStops                           : -> e 8, 16
18 ¬TrainStops ∧ TrainArrives            : and i 17, 9
19 TrainOnCrossing                       : -> e 3, 18
20 TrainOnCrossing ∧ ¬BarrierClosed      : and i 19, 13
21 Unsafe                                : -> e 6, 20
```

## E.2. Drinker's Paradox

The following is a formalisation of the drinker's paradox in the Proof Rondo system.

```
~(Ax.P(x) -> D(x)) -> (Ex. P(x) & ~D(x))
Ex.P(x)
(Ax.P(x) -> D(x)) or ~(Ax.P(x) -> D(x))     : lem
| Ax.P(x) -> D(x)
|| with x0
|| P(x0)
||| P(x0) & D(x0)
||| Ax.P(x) -> D(x)                         : copy 4
|| (P(x0) & D(x0) -> (Ax.P(x) -> D(x)))    : -> i 6-7
|| (Ey. P(y) & D(y) -> (Ax.P(x) -> D(x)))  : E.y/x0 i 8
| (Ey. P(y) & D(y) -> (Ax.P(x) -> D(x)))   : E.x/x0 e 2, 5-9
------
| ~(Ax.P(x) -> D(x))
| (Ex. P(x) & ~D(x))                       : -> e 1, 11
|| with x0
|| P(x0) & ~D(x0)
||| P(x0) & D(x0)
||| ~D(x0)                                 : and e2 13
||| D(x0)                                  : and e2 14
||| _|_                                    : neg e 15,16
||| Ax.P(x) -> D(x)                        : bot e 17
|| (P(x0) & D(x0) -> (Ax.P(x) -> D(x)))    : -> i 14-18
|| (Ey. P(y) & D(y) -> (Ax.P(x) -> D(x)))  : E.y/x0 i 19
| (Ey. P(y) & D(y) -> (Ax.P(x) -> D(x)))   : E.x/x0 e 12, 13-20
------
(Ey. P(y) & D(y) -> (Ax.P(x) -> D(x)))     : or e 3, 4-10, 11-21
```

# F. Logic Programming

```
% :- table path(_,_,lattice(shortest/3))
% :- table conn/2

% Partial order of lists by length; used in tabled execution
shortest(P1, P2, P) :-
   length(P1, L1),
   length(P2, L2),
   (L1 < L2 -> P = P1; P = P2).

% Right
adjacent(pos(X1,Y1), pos(X2, Y1)) :- succ(X1, X2), X1 < 6.
% Down
adjacent(pos(X1,Y1), pos(X1, Y2)) :- succ(Y1, Y2), Y1 < 4.
% Left
adjacent(pos(X1,Y1), pos(X2, Y1)) :- succ(X2, X1).
% Up
adjacent(pos(X1,Y1), pos(X1, Y2)) :- succ(Y2, Y1).

% Can we go from U to V?
step(U, V) :- adjacent(U, V), free(V).

% conn(U, V) holds if two positions U and V connected.
conn(U, U).
conn(U, V) :-
  conn(W, V),
  step(U, W).

% Can our robot reach the goal?
connr :- robot(U), goal(V), conn(U, V).

% path(U, V, P) holds if P is a path from U to V. A path is here a
    list of positions.
path(U, U, [U]).
path(U, V, [U|P]) :-
  path(W, V, P),
  step(U, W).

% A path P with route(P) leads our robot from the initial position to
    the goal.
route(P) :- robot(U), goal(V), path(U, V, P).

```

```
% Initial position of robot.
robot(pos(2, 3)).

% Position of goal.
goal(pos(5,1)).

% All the positions that do not contain an obstacle.
free(pos(1,1)).
free(pos(1,4)).

free(pos(2,2)).
free(pos(2,3)).
free(pos(2,4)).

free(pos(3,1)).
free(pos(3,2)).
free(pos(3,4)).

free(pos(4,1)).
free(pos(4,2)).
free(pos(4,3)).
free(pos(4,4)).

free(pos(5,1)).
free(pos(5,3)).

free(pos(6,1)).
free(pos(6,2)).
free(pos(6,3)).
```

# Notes

1. Ken02; RL08.

2. Bre98; BP21.

3. Bat83.

4. "Upon this first, and in one sense this sole, rule of reason, that in order to learn you must desire to learn, and in so desiring not be satisfied with what you already incline to think, there follows one corollary which itself deserves to be inscribed upon every wall of the city of philosophy, **Do not block the way of inquiry.**" [Pei98].

5. "To begin with, every concept and every thought beyond immediate perception is a sign." [Pei92, Pragmatism (1907)]. See also Marr [Mar95]

6. This proof appeals somewhat informally to structural induction over terms. This can be justified in terms of induction on natural numbers, see appendix B.4, or terms of induction on trees in that will be discussed in chapter 9.

7. Wil95, p. 137.

8. Ste42.

9. Ste22.

10. See [Fre79], a translation of the small book and an introduction can be found in the collection edited by van Heijenoort [vHei66]. Frege's insights were crucial to the understanding of mathematical reasoning and instrumental in overcoming a crisis of doubt in mathematics at the turn of the 19th century.

11. In principle, one could rewrite definition 4.15 in such a way that the axioms result from that definition by using so-called nominal sets [GH08; Pit11], but that will takes us too far astray.

12. Persian: *Ignorant fool.* Al-Khwārizmī wrote his published texts in Arabic, but it is believed that he is of Persian origin. For the sake of exposition we let him use Persian when he becomes, let us say, informal.

13. Arm10; Dur88; Dur11.

14. Al-Khwārizmī lived in the 8th and 9th century in the Abbasid Caliphate during the so-called *Golden Age of Islam* [Han09]. It is believed that The House of Wisdom was a library and a vibrant working place for scholars, who produced novel science and translated and critiqued existing texts [Al-11].

15. Janina Hosiasson was a Polish philosopher and logician, mostly known for her work on induction [Kra22; Szn23] but also for various translations. Active research is still going into understanding her work [Szn24]. She was married to Adolf Lindenbaum [ZP14] and fled during the war from Warsaw. Her way is partially traceable via letters that she wrote [Szu18], but we know that she ended up in Vilnius where she was arrested in 1941 and was a few months later, in April 1942, executed [Bee+17; GW18]. It should be noted that the situation surrounding Vilnius, which was Polish for some time but then became part of Lithuania, and the relation between the German occupation and the Lithuanian citizens was complicated [Sužo6; Tau21].

16. [vHei66]

17. Terminology: Closed terms and formulas are often also called ground terms and formulas.

18. It would be more correct to say "category theoretical logic", but people tend to shorten the name.

19. Defining exact set of definable maps requires one to set up a formal theory, like primitive recursion, Peano arithmetic or Gödel's system T.

20. We do not specify formally at this point what a property is, but we can think of subsets $P \subseteq \mathbb{N}$.

# References


[ADW17] Samson Abramsky, Anuj Dawar and Pengming Wang. 'The Pebbling Comonad in Finite Model Theory'. In: *32nd Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2017, Reykjavik, Iceland, June 20-23, 2017*. IEEE Computer Society, 2017, pp. 1–12. DOI: 10.1109/LICS.2017.8005129.

[Aho+07] Alfred V. Aho, Monica S. Lam, Ravi Sethi and Jeffrey D. Ullman. *Compilers: Principles, Techniques, & Tools*. Second edition. Boston: Pearson/Addison Wesley, 2007. 1009 pp. ISBN: 978-0-321-48681-3.

[AMS07] Thorsten Altenkirch, Conor McBride and Wouter Swierstra. 'Observational Equality, Now!' In: *Proc. of PLPV '07*. Workshop on Programming Languages Meets Program Verification. ACM, 2007, pp. 57–68. ISBN: 978-1-59593-677-6. DOI: 10.1145/1292597.1292608.

[And02] Peter B. Andrews. *An Introduction to Mathematical Logic and Type Theory: To Truth Through Proof*. 2nd ed. Applied Logic Series. Springer Netherlands, 2002. ISBN: 978-1-4020-0763-7. DOI: 10.1007/978-94-015-9934-4.

[Ane12] Irving H. Anellis. 'Peirce's Truth-functional Analysis and the Origin of the Truth Table'. In: *History and Philosophy of Logic* 33.1 (2012), pp. 87–97. ISSN: 0144-5340. DOI: 10.1080/01445340.2011.621702.

[Arm10] Armeniapedia contributors. *Duduk - Armeniapedia*. 2010. URL: https://www.armeniapedia.org/wiki/Duduk (visited on 27/02/2025).

[AB05] Sergei N. Artemov and Lev D. Beklemishev. 'Provability Logic'. In: *Handbook of Philosophical Logic, 2nd Edition*. Ed. by D.M. Gabbay and F. Guenthner. Dordrecht: Springer Netherlands, 2005, pp. 189–360. ISBN: 978-1-4020-3521-0. DOI: 10.1007/1-4020-3521-7_3.

[Awo10] Steve Awodey. *Category Theory*. 2nd ed. Oxford Logic Guides 52. Oxford University Press, 2010. ISBN: 978-0-19-923718-0.

[BS01] Franz Baader and Wayne Snyder. 'Unification Theory'. In: *Handbook of Automated Reasoning*. Ed. by Alan Robinson and Andrei Voronkov. Amsterdam: North-Holland, 2001, pp. 445–533. ISBN: 978-0-444-50813-3. DOI: 10.1016/B978-044450813-3/50010-2.

[Bar77] Jon Barwise. *Handbook of Mathematical Logic*. Studies in Logic and the Foundations of Mathematics 90. North-Holland, 1977. ISBN: 0-7204-2285-X.

[Bat83] Penelope Hartshorne Batcheler. *Historic Structure Report: Architectural Data Section, Charles S. Peirce House, Delaware Water Gap National Recreation Area, Pennsylvania*. Denver Service Center, Mid-Atlantic/North Atlantic Team, Branch of Cultural Resources, National Park Service, U. S. Dept. of the Interior, 1983, p. 189. URL: https://www.cspeirce.com/faqs/menu/library/aboutcsp/batcheler/Arisbe_NPS_Historic_Structure_Report_1983.pdf.

[Bee+17] Janet L. Beery, Sarah J. Greenwald, Jacqueline A. Jensen-Vallin and Maura B. Mast, eds. *Women in Mathematics*. Vol. 10. Association for Women in Mathematics Series. Cham: Springer International Publishing, 2017. ISBN: 978-3-319-66693-8. DOI: 10.1007/978-3-319-66694-5.

[Bre98] Joseph Brent. *Charles Sanders Peirce: A Life*. Vol. Rev. and enl. ed. Bloomington, Ind: Indiana University Press, 1998. ISBN: 978-0-253-21161-3.

[Bri94] Douglas S. Bridges. *Computability*. Graduate Texts in Mathematics 146. Springer, 1994. ISBN: 978-1-4612-0863-1. DOI: 10.1007/978-1-4612-0863-1.

[BP21] Robert Burch and Kelly A. Parker. 'Charles Sanders Peirce'. In: *The Stanford Encyclopedia of Philosophy*. Ed. by Edward N. Zalta and Uri Nodelman. Summer 2022. Metaphysics Research Lab, Stanford University, 2021. URL: https://plato.stanford.edu/archives/sum2022/entries/peirce/.

[Bus98] Samuel R. Buss, ed. *Handbook of Proof Theory*. Vol. 137. Studies in Logic and the Foundations of Mathematics. Elsevier, 1998. ISBN: 0-444-89840-9. DOI: 10.1016/S0049-237X(98)80061-X.

[CK90] C.C. Chang and H. Jerome Keisler, eds. *Model Theory*. Vol. 73. Studies in Logic and the Foundations of Mathematics. Elsevier, 1990. ISBN: 978-0-444-88054-3. DOI: 10.1016/S0049-237X(08)70073-9.

[Chu36] Alonzo Church. 'An Unsolvable Problem of Elementary Number Theory'. In: *American Journal of Mathematics* 58.2 (1936), pp. 345–363. ISSN: 00029327. DOI: 10.2307/2371045. JSTOR: 2371045.

[CH08] J. R. B. Cockett and P. J. W. Hofstra. 'Introduction to Turing Categories'. In: *Annals of Pure and Applied Logic* 156.2 (2008), pp. 183–209. ISSN: 0168-0072. DOI: 10.1016/j.apal.2008.04.005.

[Cod72] Edgar F. Codd. 'Relational Completeness of Data Base Sublanguages'. In: *Research Report / RJ / IBM / San Jose, California* RJ987 (1972).

[Cod83] Edgar F. Codd. 'A Relational Model of Data for Large Shared Data Banks'. In: *Commun. ACM* 26.1 (1983), pp. 64–69. ISSN: 0001-0782. DOI: 10.1145/357980.358007.

[Coh+16] Cyril Cohen, Thierry Coquand, Simon Huber and Anders Mörtberg. 'Cubical Type Theory: A Constructive Interpretation of the Univalence Axiom'. In: *CoRR* abs/1611.02108 (2016). URL: http://arxiv.org/abs/1611.02108.

[DG23] Harry Deutsch and Pawel Garbacz. 'Relative Identity'. In: *The Stanford Encyclopedia of Philosophy*. Ed. by Edward N. Zalta and Uri Nodelman. Fall 2023. Metaphysics Research Lab, Stanford University, 2023. URL: https://plato.stanford.edu/archives/fall2023/entries/identity-relative/.

[Dur88] Jean During. *BARBAṬ – Encyclopaedia Iranica*. 1988. URL: https://iranicaonline.org/articles/barbat (visited on 27/02/2025).

[Dur11] Jean During. *DRUMS – Encyclopaedia Iranica*. 2011. URL: https://iranicaonline.org/articles/drums (visited on 27/02/2025).

[EF95] Heinz-Dieter Ebbinghaus and Jörg Flum. *Finite Model Theory*. Springer Monographs in Mathematics. Berlin, Heidelberg: Springer, 1995. ISBN: 978-3-540-28787-2. DOI: 10.1007/3-540-28788-4.

[EFT21] Heinz-Dieter Ebbinghaus, Jörg Flum and Wolfgang Thomas. *Mathematical Logic*. 3rd edition. Graduate Texts in Mathematics. 2021. ISBN: 3-030-73839-6. DOI: 10.1007/978-3-030-73839-6.

[EFD05] Rüdiger Ebendt, Görschwin Fey and Rolf Drechsler. *Advanced BDD Optimization*. Berlin/Heidelberg: Springer-Verlag, 2005. ISBN: 978-0-387-25453-1. DOI: 10.1007/b107399.

[EE70] Samuel Eilenberg and Calvin Elgot. *Recursiveness*. Academic Press, 1970.

[Fit52] F. Fitch. *Symbolic Logic, An Introduction*. The Ronald Press Company, 1952.

[Fre79] Gottlob Frege. *Begriffsschrift: Eine Der Arithmetischen Nachgebildete Formelsprache Des Reinen Denkens*. Translated reprint and introduction in Jean van Heijenoort, "From Frege to Gödel", 1966. pp. 1–82. Lubrecht & Cramer, 1879.

[GH08] Murdoch James Gabbay and Martin Hofmann. 'Nominal Renaming Sets'. In: *Proceedings of Logic for Programming, Artificial Intelligence, and Reasoning, 15th International Conference, LPAR 2008*. Ed. by Iliano Cervesato, Helmut Veith and Andrei Voronkov. Vol. 5330. Lecture Notes in Computer Science. Springer, 2008, pp. 158–173. DOI: 10.1007/978-3-540-89439-1_11.

[Gal87] Jean H. Gallier. *Logic for Computer Science: Foundations of Automatic Theorem Proving*. Wiley, 1987. 511 pp. ISBN: ISBN 978-0-471-61546-0.

[Gan04] Jonardon Ganeri. 'Indian Logic'. In: *Handbook of the History of Logic*. Ed. by Dov M. Gabbay and John Woods. Vol. 1. North-Holland, 2004, pp. 309–395. DOI: 10.1016/S1874-5857(04)80007-4.

[GW18] Ángel Garrido and Urszula Wybraniec-Skardowska, eds. *The Lvov-Warsaw School. Past and Present*. Studies in Universal Logic. Cham: Springer International Publishing, 2018. ISBN: 978-3-319-65429-4. DOI: 10.1007/978-3-319-65430-0.

[Gen35] Gerhard Gentzen. 'Untersuchungen Über Das Logische Schließen. I'. In: *Mathematische Zeitschrift* 39.1 (1935), pp. 176–210. ISSN: 1432-1823. DOI: 10.1007/BF01201353.

[Han09] James Hannam. *God's Philosophers: How the Medieval World Laid the Foundations of Modern Science*. London: Icon Books Ltd, 2009. 1 p. ISBN: 978-1-84831-070-4.

[Hec03] Richard G. Heck. 'Frege on Identity and Identity-Statements: A Reply to Thau and Caplan'. In: *Canadian Journal of Philosophy* 33.1 (2003), pp. 83–102. ISSN: 0045-5091, 1911-0820. DOI: 10.1080/00455091.2003.10716536.

[Hod93] Wilfrid Hodges. *Model Theory*. Encyclopedia of Mathematics and Its Applications 42. Cambridge [England]: Cambridge University Press, 1993. ISBN: 978-0-521-30442-9. DOI: 10.1017/CBO9780511551574.

[HS98] Martin Hofmann and Thomas Streicher. 'The Groupoid Interpretation of Type Theory'. In: *Twenty-Five Years of Constructive Type Theory (Venice, 1995)*. Vol. 36. Oxford Logic Guides. Oxford Univ. Press, New York, 1998, pp. 83–111. ISBN: 978-0-19-850127-5. URL: http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.37.606.

[Hro04] Juraj Hromkovič. *Theoretical Computer Science*. Texts in Theoretical Computer Science. An EATCS Series. Springer, 2004. ISBN: 3-540-14015-8.

[HR04] Michael Huth and Mark Dermot Ryan. *Logic in Computer Science - Modelling and Reasoning about Systems*. 2nd ed. Cambridge University Press, 2004, p. 427. ISBN: 0-521-54310-X.

[Jac99] Bart Jacobs. *Categorical Logic and Type Theory*. Studies in Logic and the Foundations of Mathematics 141. Amsterdam: North Holland, 1999.

[Ken02] Hubert Kennedy. *Peano: Life and Works of Giuseppe Peano*. Definitive Edition. San Francisco: Peremptory Publications, 2002.

[Al-11] Jim Al-Khalili. *The House of Wisdom: How Arabic Science Saved Ancient Knowledge and Gave Us the Renaissance*. 1. American ed. A Penguin Book. New York: Penguin Books, 2011. 302 pp. ISBN: 978-1-59420-279-7.

[KP23] Dominik Kirst and Benjamin Peters. 'Gödel's Theorem Without Tears - Essential Incompleteness in Synthetic Computability'. In: *31st EACSL Annual Conference on Computer Science Logic (CSL 2023)*. Ed. by Bartek Klin and Elaine Pimentel. Vol. 252. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, 30:1–30:18. ISBN: 978-3-95977-264-8. DOI: 10.4230/LIPIcs.CSL.2023.30.

[Kle74] Stephen Cole Kleene. *Introduction to Metamathematics*. 7. repr. Bibliotheca Mathematica. Groningen: Wolters-Noordhoff and North-Holland, 1974. ISBN: 978-0-444-10088-7.

[Kow79] Robert A. Kowalski. 'Algorithm = Logic + Control'. In: *Commun. ACM* 22.7 (1979), pp. 424–436. DOI: 10.1145/359131.359136.

[Kra22] 'Polish Philosophers of Science and Nature in the 20th Century'. In: *Polish Philosophers of Science and Nature in the 20th Century*. Ed. by Władysław Krajewski. Brill, 2022. ISBN: 978-90-04-45779-9. URL: https://brill.com/display/title/30518.

[Lan21] Markus Land. *Introduction to Infinity-Categories*. Compact Textbooks in Mathematics. Cham: Springer International Publishing, 2021. ISBN: 978-3-030-61523-9. DOI: 10.1007/978-3-030-61524-6.

[Lan71] Serge Lang. *Basic Mathematics*. Addison-Wesley Series in Introductory Mathematics. Addison-Wesley, 1971. ISBN: 0-387-96787-7.

[LN15] John Longley and Dag Normann. *Higher-Order Computability*. Theory and Applications of Computability. Berlin, Heidelberg: Springer, 2015. ISBN: 978-3-662-47991-9. DOI: 10.1007/978-3-662-47992-6.

[Lon17] John R. Longley. 'Notions of Computability at Higher Types I'. In: Logic Colloquium 2000. Ed. by René Cori, Alexander Razborov, Stevo Todorčević and Carol Wood. Cambridge University Press, 2017, pp. 32–142. ISBN: 978-1-316-75588-4. DOI: 10.1017/9781316755884.004.

[Lop24] Aliaume Lopez. *Preservation Theorems Through the Lens of Topology*. Version 2. 2024. DOI: 10.48550/arXiv.2007.07879. arXiv: 2007.07879 [cs, math]. Pre-published.

[Mac98] Saunders Mac Lane. *Categories for the Working Mathematician*. 2nd ed. Graduate Texts in Mathematics 5. Springer, 1998. ISBN: 0-387-98403-8.

[Mar95] David Marr. 'Signs of C. S. Peirce'. In: *American Literary History* 7.4 (1995), pp. 681–699. ISSN: 0896-7148. JSTOR: 490069. URL: https://www.jstor.org/stable/490069.

[MT98] Christoph Meinel and Thorsten Theobald. *Algorithms and Data Structures in VLSI Design*. Available online: http://www.hpi.uni-potsdam.de/fileadmin/hpi/FG_ITS/books/OBDD-Book.pdf. Berlin, Heidelberg: Springer, 1998. ISBN: 978-3-642-58940-9. DOI: 10.1007/978-3-642-58940-9.

[MN12] Dale Miller and Gopalan Nadathur. *Programming with Higher-Order Logic*. Cambridge University Press, 2012. 306 pp. ISBN: ISBN 978-0-521-87940-8. URL: http://www.cambridge.org/de/academic/subjects/computer-science/programming-languages-and-applied-logic/programming-higher-order-logic?format=HB.

[Mil+91] Dale Miller, Gopalan Nadathur, Frank Pfenning and Andre Scedrov. 'Uniform Proofs as a Foundation for Logic Programming'. In: *Ann. Pure Appl. Logic* 51.1–2 (1991), pp. 125–157. DOI: 10.1016/0168-0072(91)90068-W.

[NG14] Rob Nederpelt and Professor Herman Geuvers. *Type Theory and Formal Proof: An Introduction*. 1st ed. New York, NY, USA: Cambridge University Press, 2014. ISBN: 1-107-03650-X.

[Pau92] Lawrence C Paulson. 'Designing a Theorem Prover'. In: *Handbook of Logic in Computer Science*. Ed. by Jane Spurr, S Abramsky and Dov M Gabbay. Oxford University Press, 1992. ISBN: 978-0-19-853761-8. DOI: 10.1093/oso/9780198537618.003.0004.

[Pea16] Guiseppe Peano. 'Sul Principio d'identità'. In: *Mathesis, Società Italiana di Matematica* Bollettino 8 (1916), p. 40.

[Pei92] Peirce Edition Project, ed. *The Essential Peirce, Volume 2: Selected Philosophical Writings (1893-1913)*. Vol. 2. Indiana University Press, 1992.

[Pei98] Peirce Edition Project. 'The First Rule of Logic (1898)'. In: *The Essential Peirce*. Vol. 1. United States: Indiana University Press / the Peirce Edition Project, 1998, pp. 42–56. ISBN: 978-0-253-21190-3.

[Pét67] Rózsa Péter. *Recursive Functions*. Trans. by István Földes. 3d, rev. ed. New York: Academic Press, 1967. 300 pp.

[PD01] Frank Pfenning and Rowan Davies. 'A Judgmental Reconstruction of Modal Logic'. In: *Math. Struct. Comput. Sci.* 11.4 (2001), pp. 511–540. DOI: 10.1017/S0960129501003322.

[Pit11] Andrew M. Pitts. 'Nominal Sets'. Lecture Notes. MGS, 2011. URL: http://www.cs.nott.ac.uk/~pszvc/mgs/MGS2011_nominal_sets.pdf.

[Pra06] Dag Prawitz. *Natural Deduction: A Proof-Theoretical Study*. Dover Books on Mathematics. Dover Publications, 2006. ISBN: 978-0-486-44655-4.

[PTW18] Graham Priest, Koji Tanaka and Zach Weber. 'Paraconsistent Logic'. In: *The Stanford Encyclopedia of Philosophy*. Ed. by Edward N. Zalta. Summer 2018. Metaphysics Research Lab, Stanford University, 2018. URL: https://plato.stanford.edu/archives/sum2018/entries/logic-paraconsistent/.

[Rie16] Emily Riehl. *Category Theory in Context*. Aurora: Dover Modern Math Originals. Dover Publications, 2016. 272 pp. ISBN: 978-0-486-80903-8. URL: http://www.math.jhu.edu/~eriehl/context/.

[RL08] Clara Silvia Roreo and Erika Luciano. *Giuseppe Peano: Matematico e Maestro*. Università di Torino e NerosuBianco: Dipartimento di Matematica, 2008. ISBN: 978-88-900876-6-0. URL: https://iris.unito.it/handle/2318/52823.

[Ros08] Benjamin Rossman. 'Homomorphism Preservation Theorems'. In: *J. ACM* 55.3 (2008), 15:1–15:53. ISSN: 0004-5411. DOI: 10.1145/1379759.1379763.

[SV82] Giovanni Sambin and Silvio Valentini. 'The Modal Logic of Provability. The Sequential Approach'. In: *Journal of Philosophical Logic* 11.3 (1982), pp. 311–342. ISSN: 1573-0433. DOI: 10.1007/BF00293433.

[Smi13] Peter Smith. *An Introduction to Gödel's Theorems*. 2nd ed. Cambridge Introductions to Philosophy. Cambridge: Cambridge University Press, 2013. DOI: 10.1017/CBO9780511800962.

[Smo77] Craig Smoryński. 'The Incompleteness Theorems'. In: *Handbook of Mathematical Logic*. Ed. by Jon Barwise. Vol. 90. Studies in Logic and the Foundations of Mathematics. Elsevier, 1977, pp. 821–865. ISBN: 0-7204-2285-X. DOI: 10.1016/S0049-237X(08)71123-6.

[Sol76] Robert M. Solovay. 'Provability Interpretations of Modal Logic'. In: *Israel Journal of Mathematics* 25.3 (1976), pp. 287–304. ISSN: 1565-8511. DOI: 10.1007/BF02757006.

[Sol13] Daniel Solow. *How to Read and Do Proofs: An Introduction to Mathematical Thought Processes*. 6th edition. Hoboken, New Jersey: Wiley, 2013. 336 pp. ISBN: 978-1-118-16402-0.

[Ste42] L. Susan Stebbing. *A Modern Introduction to Logic*. Third Edition. London: Methuen & Co. Ltd., 1942. URL: https://archive.org/details/in.ernet.dli.2015.190795.

[Ste22] Susan Stebbing. *Thinking to Some Purpose*. Originally published 1939 by Penguin Books. London: Routledge, 2022. 294 pp. ISBN: 978-1-00-329518-1. DOI: 10.4324/b22927.

[Str23] Thomas Streicher. *Fibered Categories à La Jean Bénabou*. Version 20. 2023. DOI: 10.48550/arXiv.1801.02927. arXiv: 1801.02927 [math]. Pre-published.

[Suž90] Saulius Sužiedėlis. 'The Military Mobilization Campaigns of 1943 and 1944 in German-occupied Lithuania: Contrasts in Resistance and Collaboration'. In: *Journal of Baltic Studies* (1990). DOI: 10.1080/01629778900000231.

[Suž06] Saulius Sužiedėlis. 'Lithuanian Collaboration during the Second World War: Past Realities, Present Perceptions'. In: *"Kollaboration" in Nordosteuropa: Erscheinungsformen Und Deutungen Im 20. Jahrhundert / Herausgegeben von Joachim Tauber*. Harrassowitz Verlag, 2006, pp. 164–173. URL: https://www.lituanistika.lt/content/38246.

[Szn23] Marta Sznajder. *Hosiasson Bibliography*. Marta Sznajder. 2023. URL: https://martasznajder.com/hosiassion-bibliography/ (visited on 05/02/2025).

[Szn24] Marta Sznajder. 'Janina Hosiasson-Lindenbaum on Analogical Reasoning: New Sources'. In: *Erkenn* 89.4 (2024), pp. 1349–1365. ISSN: 1572-8420. DOI: 10.1007/s10670-022-00586-y.

[Szu18] Tadeusz Szubka. 'List Janiny Hosiasson-Lindenbaum do George'a Edwarda Moore'a'. In: *Filozofia Nauki* 26.1 (1 2018), pp. 129–141. ISSN: 2657-5868. DOI: 10.14394/filnau.2018.0007.

[Tau21] Joachim Tauber. 'Collaboration in Lithuania'. In: *Complicated Complicity: European Collaboration with Nazi Germany during World War II*. Ed. by Martina Bitunjac and Julius H. Schoeps. De Gruyter Oldenbourg, 2021, pp. 121–134. ISBN: 978-3-11-067118-6. DOI: 10.1515/9783110671186-007.

[TS00] Anne Sjerp Troelstra and Helmut Schwichtenberg. *Basic Proof Theory*. 2nd ed. Cambridge Tracts in Theoretical Computer Science. Cambridge: Cambridge University Press, 2000. ISBN: 0-521-77911-1.

[TvD88a] Anne Sjerp Troelstra and Dirk van Dalen. *Constructivism in Mathematics: An Introduction, Volume I*. Studies in Logic and the Foundations of Mathematics 121. North-Holland, 1988. 384 pp. ISBN: 0-444-70506-6.

[TvD88b] Anne Sjerp Troelstra and Dirk van Dalen. *Constructivism in Mathematics: An Introduction, Volume II*. Studies in Logic and the Foundations of Mathematics 123. North-Holland, 1988. 384 pp. ISBN: 0-444-70358-6.

[Uni13] The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study, 2013. URL: http://homotopytypetheory.org/book.

[vHei66] Jean van Heijenoort. *From Frege to Gödel : A Source Book in Mathematical Logic, 1879-1931*. Source Books in the History of the Sciences. Cambridge: Harvard University Press, 1966. ISBN: 978-0-674-32449-7.

[Vis82] Albert Visser. 'On the Completenes Principle: A Study of Provability in Heyting's Arithmetic and Extensions'. In: *Annals of Mathematical Logic* 22.3 (1982), pp. 263–295. ISSN: 0003-4843. DOI: 10.1016/0003-4843(82)90024-9.

[Wil95] Morgan Grayce Willow. 'L. Susan Stebbing (1885–1943)'. In: *A History of Women Philosophers: Contemporary Women Philosophers, 1900-Today*. Ed. by Mary Ellen Waithe. A History of Women Philosophers. Dordrecht: Springer Netherlands, 1995, pp. 125–155. ISBN: 978-94-011-1114-0. DOI: 10.1007/978-94-011-1114-0_6.

[Yan22] Noson S. Yanofsky. *Theoretical Computer Science for the Working Category Theorist*. Elements in Applied Category Theory. Cambridge University Press, 2022. ISBN: 978-1-108-87234-8. DOI: 10.1017/9781108872348.

[ZP14] Jan Zygmunt and Robert Purdy. 'Adolf Lindenbaum: Notes on His Life, with Bibliography and Selected References'. In: *Log. Univers.* 8.3 (2014), pp. 285–320. ISSN: 1661-8300. DOI: 10.1007/s11787-014-0108-2.

# Symbols

| | | |
|---|---|---|
| **ND** | natural deduction for propositional logic | p. 27 |
| $\mathbf{ND}_1$ | natural deduction for first-order logic | p. 87 |
| $\mathbf{ND}_1^{=}$ | natural deduction for first-order logic with equality | p. 148 |
| $\bot$ | absurdity ("falsity") | p. 11, 67 |
| $t\,\sigma$ | application of the term $t$ to substitution $\sigma$ | p. 82 |
| $\varphi\,\sigma$ | application of the formula $\varphi$ to substitution $\sigma$ | p. 83 |
| ar | arity of function and predicate symbols in first-order signature | p. 63 |
| $\leftrightarrow$ | bi-implication ("if and only if" or "iff") | p. 12 |
| $[\![\varphi]\!](v)$ | Boolean semantics of $\varphi$ under valuation $v$ | p. 46 |
| $\exists x : P.\, \varphi$ | formula $\varphi$ should hold for some object "in" $P$ | p. 142 |
| $\forall x : P.\, \varphi$ | formula $\varphi$ should hold for all objects "in" $P$ | p. 142 |
| **cND** | classical natural deduction for propositional logic | p. 51 |
| $\mathbf{cND}_1$ | classical natural deduction for first-order logic | p. 91 |
| $\mathbf{cND}_1^{=}$ | classical natural deduction for first-order logic with equality | p. 148 |
| $\chi_U$ | characteristic function of the set $U \subseteq A$ | p. 255 |
| $g \circ f$ | composition of maps, read "$g$ after $f$" | p. 251 |
| $\wedge$ | conjunction ("and") | p. 11, 67 |
| $\Gamma, \varphi$ | extension of context $\Gamma$ by formula $\varphi$, i.e., the concatenation of the list $\Gamma$ with $\varphi$ | p. 27 |
| $A + B$ | disjoint union of the sets $A$ and $B$ | p. 254 |
| $\vee$ | disjunction ("or") | p. 11, 67 |
| $\uparrow$ | $\uparrow$ indicates a diverging computation | p. 216, *see also* $\overline{\mathbb{N}}$ |
| $\emptyset$ | empty set | p. 255 |
| $s \doteq t$ | syntactic equality ("s is provably equal to t") | p. 140 |
| $\exists x.\, \varphi$ | existential quantification ("there is $x$, such that $\varphi$") | p. 67 |
| $[n\rangle$ | set of first $n$ natural numbers | p. 255 |

| | | |
|---|---|---|
| $A^n$ | $n$-fold product of the set $A$ | p. 177 |
| $\mathcal{M}$ | $\mathcal{L}$-model for a first-order signature $\mathcal{L}$ | p. 178 |
| $\lvert\mathcal{M}\rvert$ | universe of an $\mathcal{L}$-model | p. 178 |
| $D \mid G \vDash \varphi$ | $G$ entails the first-order formula $\varphi$ | p. 190 |
| $D \mid G \vDash_{\mathcal{M}} \varphi$ | $G$ entails $\varphi$ in the model $\mathcal{M}$ | p. 190 |
| $\Delta \mid \Gamma \vdash \varphi$ | first-order sequent consisting of variable context $\Delta$, assumptions $\Gamma$ and an FOL formula $\varphi$ | p. 86 |
| $\mathrm{Form}(\mathcal{L})$ | set of FOL formulas over $\mathcal{L}$ | p. 67 |
| $[\![\varphi]\!]$ | Boolean semantics of FOL formula $\varphi$ in model $\mathcal{M}$ | p. 186 |
| $[\![t]\!]$ | Semantics of term $t$ in some model | p. 183 |
| $\mathrm{Val}_{\mathcal{M}}(D)$ | valuations in first-order model $\mathcal{M}$ for variables in $U$ | p. 182 |
| $\forall x.\, \varphi$ | universal quantification ("for all $x$, $\varphi$") | p. 67 |
| $\mathrm{Form}(\mathcal{L}^{=})$ | first-order formulas with equality $\doteq$ | p. 141 |
| $\mathrm{fv}(\varphi)$ | free variables appearing in formula $\varphi$ | p. 70 |
| $x \# \varphi$ | variable $x$ is fresh for formula $\varphi$ | p. 83 |
| $x \# \sigma$ | variable $x$ is fresh for substitution $\sigma$ | p. 83 |
| $\mathcal{F}$ | set of function symbols in a first-order signature | p. 63 |
| $\mathrm{id}_A$ | identity map on set $A$ | p. 251 |
| $\eta$ | the identity substitution maps a variable to itself | p. 81 |
| $f^{\to}U$ | image of $U$ under the map $f$ | p. |
| $\to$ | implication ("implies") | p. 11, 67 |
| $\varphi$ **in** $\Gamma$ | checks if formula $\varphi$ appears in context $\Gamma$ | p. 26 |
| $\mathcal{L}$-**Mod** | collection of all $\mathcal{L}$-models | p. 206 |
| $[A, B]$ | set of maps from $A$ to $B$ | p. 253 |
| $\mathcal{M}_1 \sqcup \mathcal{M}_2$ | Union of two models on a common universe | p. 200 |
| $\varphi \barwedge \psi$ | nand (not-and) connective | p. |
| $\mathbb{N}$ | set of natural numbers | p. 257 |
| $\mathrm{Pr}(\Delta \mid \Gamma \vdash \varphi)$ | set of proof trees of $\mathbf{ND}_1$ | p. 113 |
| $\neg$ | negation ("not") | p. 12 |
| $\langle f_1, f_2 \rangle$ | pairing of maps into a set product | p. 253 |
| $\overline{A}$ | set of partial elements over $A$ | p. 216 |
| $\overline{\mathbb{N}}$ | set of partial numbers $\mathbb{N} \cup \uparrow$ | p. 216 |
| $\Gamma \vdash \varphi$ | propositional sequent consisting of assumptions $\Gamma$ and a propositional formula $\varphi$ | p. 27 |
| $\mathcal{P}(A)$ | powerset of $A$ | p. 255 |

| | | |
|---|---|---|
| $\mathcal{R}$ | set of predicate symbols of a first-order signature ("$\mathcal{R}$" stands for "relation") | p. 63 |
| $f^{\leftarrow}V$ | preimage of $V$ under the map $f$ | p. |
| $\mathrm{PR}_n$ | set of primitive recursive functions of arity $n$ | p. 224 |
| $A \times B$ | product of the sets $A$ and $B$ | p. 252 |
| PForm | set of propositional formulas | p. 11 |
| $h^*\mathcal{M}$ | pullback of model $\mathcal{M}$ along signature map $h$ | p. 235 |
| $\varphi \equiv \psi$ | formulas $\varphi$ and $\psi$ are semantically equivlent | p. 49 |
| $\Longrightarrow$ | semantic implication | p. 45 |
| $T_1 \equiv T_2$ | theories $T_1$ and $T_2$ have the same models | p. 206 |
| $A \setminus B$ | set difference: elements of $A$ without those in $B$ | p. 255 |
| $\mathcal{L}$ | (first-order) signature consisting of function and predicate symbols | p. 63 |
| $\mathrm{Sub}(\varphi)$ | the formulas that appear anywhere is a given formula $\varphi$ | p. 16 |
| $\tau \bullet \sigma$ | composition that carries out the substitution $\tau$ after $\sigma$ | p. 102 |
| $\mathrm{supp}(\sigma)$ | support of the substitution $\sigma$ | p. 101 |
| $T$-**Mod** | collection of all $T$-models | p. 206 |
| $\mathrm{Term}(\mathcal{L})$ | terms over a signature $\mathcal{L}$ | p. 65 |
| $\mathrm{var}(t)$ | variables appearing in term $t$ | p. 66 |
| $T \vdash \varphi$ | deduction of $\varphi$ from theory $T$ | p. 221 |
| $\mathrm{im}\, f$ | (total) image the map $f$ | p. |
| tr | family of maps indexed by labels of a branching type to construct a tree | p. 166 |
| $\top$ | syntactic truth ("true" or "top") | p. 12 |
| $\mathbb{B}$ | set of Boolean truth values 1 ant 0 | p. 45 |
| $\Gamma \vdash_u \varphi$ | sequent $\Gamma \vdash_u \varphi$ is derivable by a uniform proof | p. 127 |
| $\mathrm{unique}_x(t, \varphi)$ | uniqueness predicate ("$t$ is unique with the property $\varphi(x)$") | p. 144 |
| $\exists! x.\, \varphi$ | uniqueness quantification ("there is a unique $x$, such that $\varphi$") | p. 144 |
| $\sigma[x := t]$ | updates the substitution $\sigma$ to assign $t$ to variable $x$ | p. 81 |
| $v[x \mapsto a]$ | updates the valuation $v$ to assign $a$ to variable $x$ | p. 185 |

# Index


absurdity, 11
Ackermann function, 229
application of formula, 83
arity, 63
artificial intelligence, 14
atom, 208
    predicate atom, 117
automated reasoning, 14
axiom, 221
axiomatisable, 221
axiomatisation, 221
bi-implication, 12
Boolean propositional semantics, 46
bounded quantifier, 142
    bounded existential quantifier
    bounded universal quantifier
categorical logic, 150
characteristic function, 216, 255
closed world assumption, 33
code, 239
codomain, 249
commute, 253
composition, 224, 251
computable, 216
conjunction, 11
constant, 64
context, 24, 27, 86
    context extension, 27
    context membership
contraposition, 31
copairing, 254
coproduct, 254, *see* disjoint union
cut rule, 113
database, 14
decidable, 216
decoding, 217
deduction, 27, 87, 161
deduction from a theory, 221
deduction rule, 24, *see* rule
detour, 113
diagonalisation, 240
diagram, 251
    commutative diagram, 252
direct subformula, 16
disjoint union, 254
disjunction, 11
diverging computation, 216, *see also* partial numbers
DNE, 91, *see* double negation elimination
domain, 216, 249
double negation elimination, 91
embedding, 207
empty set, 255
encoding, 217
entailment, 190
syntactic equality, 140
existential quantifier, 62
expert system, 14
fact, 14
family, 256
    map family, 256
    set family, 256

finite ordinal, 255
finite product, 177
first-order formula semantics, 186
$\mathcal{L}$-model, 178
universe, 178
$[\![\varphi]\!]_{\mathcal{M}}$, 186
$[\![t]\!]_{\mathcal{M}}$, 183
formula, 11, 63
    $\mathcal{L}$-, 67, *see* first-order formula
    $\mathcal{L}^{=}$-, 141, *see* formulas with equality
    atomic, 16
    closed, 117
    contingent, 49
    first-order formula, 67
    propositional formula, 11
    provable, 27, 87
    satisfiable, 49, 190
    valid, 49, 190
formulas with equality, 141
function, 216, 251
    partial function, 216
function symbol, 62, 63
    n-ary, 64
goal formula, 124
goal-oriented, 116
Horn clause, 117
    body, 117
Horn clause theory, 117
identity, 251
image, 255
implication, 11
inductive extension, 167
input, 249
input type, 249, *see also* domain
iteration, 167
iteration principle, 166
knowledge, 14
logic program, 117
map, 249, 251
modal logic, 15
$\mathcal{L}$-model collection, 206
model morphism, 207
model union, 200
name permutation, 103
nand, 49
natural numbers, 257
negation, 12
normal form, 113
object, 60
object variable, 60
output, 249
output type, 249, *see also* codomain
pairing, 253
parse tree, 16, 161
partial elements, 216
partial numbers, 216
partition, 64
powerset, 255
PRA, 236, *see* primitive recursive arithmetic
predicate symbol, 63
    n-ary, 64
preimage, 255
primitive iteration, 223
primitive recursive arithmetic, 236
primitive recursive function, 215, 224
principal of explosion, 27
probabilistic logic, 15
product, 251, 252
projection, 253
proof tree, 27, 87
proof trees of $\mathbf{ND}_1$
property, 168
    inductive, 168
proposition, 10
propositional variable, 10
provability-predicate, 240
pullback, 235
recursively enumerable, 216

reflexivity, 148, 149
relation symbol, 63, *see* predicate symbol
relative entailment, 190
replacement, 148, 149
representable, 217
routing problem, 14
rule, 24
    admissible, 29
    elimination, 25
    introduction, 25
satisfy, 190
scope, 60, 80
semantic equivalence, 49
semantic implication, 45
semantically equivalent, 206
semi-decidable, 216
sequent, 24
    first-order sequent, 86
    propositional sequent, 27
set difference, 70, 255
set of maps, 253
signature, 63
signature map, 233
simple graph, 203
strong morphism, 207
structural property, 205
subformula, 16
substitution, 80, 81
    application, 82
    closing substitution, 123
    composition, 102
    identity, 81, 174
    support, 101
    update, 81
system **ND**, 27
system $\mathbf{ND}_1$, 87
system $\mathbf{ND}_1^=$, 148
system **cND**, 51
system $\mathbf{cND}_1$, 91
system $\mathbf{cND}_1^=$, 148
tautology, 49, 190, *see also* valid
term, 63, 65
term semantics, 183
theorem, 27
theory, 206
    complete theory, 221
    consistent, 50, 209
$T$-model, 206
top-level connective, 16
total, 216
(total) image, 255
tree, 166
    parameterised, 172
tree constructor, 166
truth, 12
truth values, 44, 45
unary, 69
unification, 104
    unification problem, 101
    unification state, 104
unification variable, 101
unifier, 101
    most general unifier, 103
uniform proof, 127
uniqueness predicate, 144
uniqueness quantifier, 144
universal quantifier, 60
valuation, 45
    $D$-valuation, 182
    update, 185
variable
    bound variable, 69, 80
    free variable, 70
    fresh, 83
    fresh, 83
    term variable, 66
$\Pi_1$-formula, 208
$\omega$-consistent, 241

# Solutions

You will find here the answers to quizzes only, the solutions to exercises are provided separately.

## Answers to the Quizzes

**Answer to quiz on page 28** No, the requirement is redundant, because the first requirement already asks for the correct rule label and the (Assum)-rule has no further sequents that have to be proven.

**Answer to quiz on page 32** The truth proposition has, dually to absurdity, only an introduction rule.

**Answer to quiz on page 41** The first proof attempt is not correct because the ($\to$I)-rule uses a hypothesis but line 2 does not open a new flag. A correct proof would look as follows.

| | | |
|---|---|---|
| 1 | $p$ | |
| 2 | $\quad q$ | |
| 3 | $\quad p$ | Assum, 1 |
| 4 | $q \to p$ | $\to$I, 2–3 |

The second attempt is correct and proves the sequent $p \vdash r \to p \wedge r$.

**Answer to quiz on page 44** For each of the three variables, there are two possibilities 0 and 1. Thus, there are $2^3 = 8$ different combinations of truth values and each of them gets its own row.

**Answer to quiz on page 47** We must have that $[\![\varphi \leftrightarrow \psi]\!](v) = 1$ if and only if $[\![\varphi]\!](v) = [\![\psi]\!](v)$, which is equivalent to saying that $[\![\varphi]\!](v) - [\![\psi]\!](v) = 0$. Therefore, $[\![\varphi \leftrightarrow \psi]\!](v) = 1 - ([\![\varphi]\!](v) - [\![\psi]\!](v))$.

**Answer to quiz on page 62** In eq. (4.6), we used that $s(r, k)$ is supposedly uniquely defined for any $k$. But typically, the path would be of finite length, say $l$. If $k$ is larger than $l$, the map $s$ cannot be defined for $s(r, k)$.

There are several ways to solve this. A more elegant one will be presented in chapter 7, but that approach uses a different definition of paths and indexing entirely. For our present definition, we can introduce a function symbol len, which returns the **len**gth of a path, and a predicate symbol $B$, where $B(k, n)$ intuitively holds if $k$ is natural number below the **B**ound $n$. The formula (4.6) can then be improved to

$$\forall r.\, P(r) \to \forall k.\, B(k, \mathrm{len}(r)) \to \neg O(s(r, k))\,.$$

**Answer to quiz on page 82** If $t$ is any term, then it is easy to see that $t\,\eta = t$. Thus, the identity substitution is the identity for the application of terms to substitutions. Formally, this can be proven by induction over terms.

In fact, this goes even deeper. We can compose substitutions, just like we compose maps: For substitutions $\sigma$ and $\tau$, let us denote by $\tau \bullet \sigma$ their composition, which is given by applying to every term $\sigma(x)$ the substitution $\tau$:

$$(\tau \bullet \sigma)(x) = \sigma(x)\,\tau$$

Since $\sigma(x)\,\eta = \sigma(x)$, we thus obtain $\eta \bullet \sigma = \sigma$. The other way round, we also have $(\sigma \bullet \eta)(x) = x\sigma = \sigma(x)$. Hence, composition with $\eta$ does not change a substitution and thus the identity substitution acts like the identity map.

**Answer to quiz on page 83** The reason is that in the axiom (SP), we would otherwise exclude even the identity substitution because $x \in \mathrm{var}(\eta(x))$. However, the present definition gives us $x \# \eta$ and we could prove $\varphi\,\eta = \varphi$.

**Answer to quiz on page 119** The idea is that multiplication is given recursively by $0 \cdot n = 0$ and $(m + 1) \cdot n = (m \cdot n) + n$. This can be translated into the following two Horn clauses.

$$\begin{aligned}
&\forall n. && M(0, n, 0)\\
&\forall m.\, \forall n.\, \forall i.\, \forall k.\, M(m, n, i) \land P(i, n, k) \to && M(s(m), n, k)
\end{aligned}$$

**Answer to quiz on page 123** No, if a signature $\mathcal{L}$ has no constants, then it is not possible to obtain an $\mathcal{L}$-term without variables. For example, suppose $\mathcal{L}$ is a signature with only one unary function symbol $f$. Then the only terms we can build over this signature are of the form $x, f(x), f(f(x)), \dots$ for variables $x$. Thus, it is not possible to obtain a closed term without variables from $\mathcal{L}$. This can formally be proven by induction over first-order terms.

**Answer to quiz on page 143** Recall that we denote $s(\underline{0})$ by $\underline{1}$. Then the formula $\exists y : N.\, x \doteq p(p(y, y), \underline{1})$ states that $x$ is odd.

**Answer to quiz on page 150** To derive symmetry, we apply the replacement rule to the formula $x \doteq s$, which we refer to as $\varphi$, as follows. First, we note that $\varphi[x := t] = t \doteq s$, which means that we can prove symmetry by showing that $\varphi[x := t]$ is derivable from $s \doteq t$. Second, we note that $\varphi[x := s] = s \doteq s$, whence $\varphi[x := s]$ is provable by (=I). Putting this together, we obtain the following derivation for symmetry of $\doteq$.

$$\dfrac{\Delta \mid \Gamma \vdash s \doteq t \qquad \dfrac{}{\Delta \mid \Gamma \vdash \varphi[x := s]}\,(=\mathrm{I})}{\Delta \mid \Gamma \vdash \varphi[x := t]}\,(=\mathrm{E})$$

**Answer to quiz on page 178** The sets $A^n$ and $A^{[n\rangle}$ are isomorphic. Concretely, there are maps $f\colon A^n \to A^{[n\rangle}$ and $g\colon A^{[n\rangle} \to A^n$ given by $f(a_1, \dots, a_n)(k) = a_{k+1}$ and $g(\alpha) = (\alpha_0, \dots, \alpha_{n-1})$.

**Answer to quiz on page 179** There are exactly two possibilities because $\mathcal{M}(P)$ must be a map $A^0 \to \mathbb{B}$. Thus, either $\mathcal{M}(P)() = 1$ or $\mathcal{M}(P)() = 0$.

**Answer to quiz on page 189** The formula $\forall x.\, \exists y.\, L(x, y) \wedge \neg(x \doteq y)$ translates in the language model $\mathcal{M}_l$ to "for every language $U$ there is language $V$ that strictly contains $U$: $U \subset V$." This is not true because the total language $A^*$ is maximal. Indeed, formally we have for all valuations $v$ and $V \subseteq A^*$ with $w = v[x \mapsto A^*]$ that

$$[\![L(x, y) \wedge \neg(x \doteq y)]\!](w[y \mapsto V]) = \begin{cases} 1, & A^* \subseteq U \text{ and } A^* \neq V \\ 0, & \text{otherwise} \end{cases} = 0.$$

This gives $[\![\exists y.\, L(x,y) \wedge \neg(x \doteq y)]\!](w) = 0$ and thus

$$\begin{aligned}
&[\![\forall x.\, \exists y.\, L(x,y) \wedge \neg(x \doteq y]\!](v) \\
&= \min\{[\![\exists y.\, L(x,y) \wedge \neg(x \doteq y)]\!](v[x \mapsto U]) \mid U \in A^*\} \\
&\leq [\![\exists y.\, L(x,y) \wedge \neg(x \doteq y)]\!](w) \\
&= 0
\end{aligned}$$

**Answer to quiz on page 253** One possible way to salvage $p$ is as an actual map $p' \colon [0, 2\pi) \to \mathbb{R}$, where $[0, 2\pi)$ is the interval consisting of all real numbers from 0 up to, but excluding, $2\pi$. In this case, the equation $\sin(y) = x$ has a unique solution for any $x \in [0, 2\pi)$.